

苏州大学
SOOCHOW UNIVERSITY

博士学位论文



论文题目 BC 网络上独立生成树构造研究

研究生姓名 程宝雷

指导教师姓名 樊建席（教授）

专业名称 计算机应用技术

研究方向 并行与分布式系统

论文提交日期 2014 年 5 月

苏州大学学位论文使用授权声明

本人完全了解苏州大学关于收集、保存和使用学位论文的规定，即：学位论文著作权归属苏州大学。本学位论文电子文档的内容和纸质论文的内容相一致。苏州大学有权向国家图书馆、中国社科院文献信息情报中心、中国科学技术信息研究所（含万方数据电子出版社）、中国学术期刊（光盘版）电子杂志社送交本学位论文的复印件和电子文档，允许论文被查阅和借阅，可以采用影印、缩印或其他复制手段保存和汇编学位论文，可以将学位论文的全部或部分内容编入有关数据库进行检索。

涉密论文 ☐

本学位论文属 _____ 在 _____ 年 _____ 月解密后适用本规定。

非涉密论文 ☒

论文作者签名： 程军 日 期： 2014-3-20

导 师 签 名： 程军 日 期： 2014-3-20

苏州大学学位论文独创性声明

本人郑重声明：所提交的学位论文是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含其他个人或集体已经发表或撰写过的研究成果，也不含为获得苏州大学或其它教育机构的学位证书而使用过的材料。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式标明。本人承担本声明的法律责任。

论文作者签名： 程宝雷 日 期： 2014-3-20

BC网络上独立生成树构造研究

摘 要

高性能计算机在国防太空、石油勘探、生物制药、天气预报以及基础理论研究等领域发挥着日益重要的作用,其性能很大程度上取决于系统中处理器或者处理机之间连接的方式(互连网络)。在并行处理领域,互连网络及其性质的研究是一个重要课题。一个互连网络可以用一个简单图 $G = (V(G), E(G))$ 来表示,其中 $V(G)$ 代表 G 上的顶点集合,而 $E(G)$ 代表 G 上的边集合。

超立方体是常用的互连网络之一,其许多优越性质如低直径、高连通性、对称性、递归构造性等受到众多研究者的青睐。同时,超立方体具有性能优越的变型网络如交叉立方体、莫比乌斯立方体、扭立方体等。研究者依据一一对应连接和可递归构造性质,将超立方体及其若干变型归结为一类互连网络——BC网络。在互连网络中,独立生成树在信息的可靠传输、并行传输、安全分发及故障处理器的诊断等方面具有重要的作用。本文采用从特殊到一般的方法研究BC网络上独立生成树(IST)的存在性及其构造问题,包括以下内容:

1. 交叉立方体上以任一顶点为根的独立生成树的串行构造

(1) 证明了 n 维交叉立方体 CQ_n 满足一个重要的地址变换性质,依据该地址变换性质给出了 CQ_{n-1}^0 和 CQ_{n-1}^1 上同构树的构造方法。

(2) 给出了 CQ_n 上以任一顶点为根的 n 棵IST的递归构造方法,证明了其正确性,并设计了时间复杂度为 $O(N\log^2 N)$ 的构造算法,这里 N 是 CQ_n 的顶点个数。

2. 莫比乌斯立方体上以任一顶点为根的独立生成树的串行构造

与交叉立方体不一样的是, n 维莫比乌斯立方体 M_n 是由两个不同构的子莫比乌斯立方体 M_{n-1}^0 和 M_{n-1}^1 组成,从而交叉立方体上地址变换的方法不再适用于在莫比乌斯立方体上构造IST。我们从顶点之间维邻接关系的角度进行研究,取得了如下成果:

(1) 结合 M_n 的构造规律,从顶点之间维邻接关系的角度提出了适合于在 M_{n-1}^0 和 M_{n-1}^1 上构造同构树的方法。

(2) 基于顶点之间的维邻接关系,设计了 M_n 上独立生成树的构造过程中扩展顶点集合和扩展顶点集合划分的选择方法,并引入非负向量来证明其上存在IST的正确性。

(3) 设计了时间复杂度为 $O(N\log N)$ 的递归构造算法,这里 N 是 M_n 的顶点个数。

其中, M_n 上基于顶点之间的维邻接关系构造 M_{n-1}^0 和 M_{n-1}^1 中同构树的方法同样适合在 CQ_{n-1}^0 和 CQ_{n-1}^1 中构造同构树, 是 CQ_n 上地址变换方法的改进方法。另外, 在 M_n 上所设计的IST的递归构造算法的效率比 CQ_n 上的高。

3. 交叉立方体上独立生成树的并行构造

从交叉立方体和莫比乌斯立方体上IST的串行构造方法来看, 其对应算法的时间复杂度高且IST在构造过程中存在相互依赖的缺点, 如第 n 棵树在构造过程中依赖于前面 $n-1$ 棵树, 因此, 寻找交叉立方体的新的性质并基于其设计独立生成树的并行构造方法以提高相应算法的效率是一个很值得研究的内容。在这方面我们取得了如下研究结果:

(1) 证明了交叉立方体上满足一个重要性质——维扩散性质。

(2) 提出一个具有UUID成员和VALUE成员的树存储结构, 确保树中顶点的UUID成员总是具有唯一性。

(3) 基于交叉立方体上维扩散性质给出了其上IST的并行构造算法。

4. 莫比乌斯立方体上独立生成树的并行构造

在对交叉立方体上IST的串行和并行构造方法、莫比乌斯立方体上IST的串行构造方法总结的基础上, 我们进一步研究了莫比乌斯立方体上IST的并行构造问题, 取得了如下研究结果:

(1) 从排列组合的角度, 将圆排列的概念引入到IST的构造方法中。

(2) 首先证明了递减的圆排列能够用于并行构造莫比乌斯立方体上以任一顶点为根的IST。其次, 进一步证明了任一圆排列能够用来并行构造莫比乌斯立方体和超立方体上以任一顶点为根的IST。

(3) 指出利用不同的圆排列构造的多组IST能够构造莫比乌斯立方体上任意两个顶点之间多组顶点不相交路径。

(4) 探讨了基于IST的高效诊断算法。

5. BC网络上IST的构造

特殊BC网络如超立方体、交叉立方体、局部扭立方体、莫比乌斯立方体中IST的构造及其证明依赖于其特有的性质。我们研究了基于它们的共性来构造其上的IST的一个一般性方法, 取得了如下结果:

(1) 给出了条件BC网络的定义, 证明了超立方体、交叉立方体、局部扭立方体、莫比乌斯立方体等均属于条件BC网络。同时, 指出了存在不属于条件BC网络的BC网络。

(2) 证明了利用递增的圆排列能够在 $O(N)$ 时间内构造 n 维条件BC网络 XC_n 上以

任一顶点为根的 n 棵同构于 n -级类二项树的IST，这里 $n \geq 2$ 且 $N = 2^n$ 是 XC_n 的顶点个数。

(3) 针对任一BC网络，提出了维的 V 排列并证明了 V 排列能够用来构造任一 n 维BC网络 X_n 上以任一顶点为根的生成树且证明这些生成树均同构于 n 级二项树。同时，指出基于 V 排列能够构造 X_n 上多组两两独立的生成树。

综上所述，本文首先给出了特殊BC网络交叉立方体、莫比乌斯立方体上IST的串行构造方法。然后，在总结相关串行构造方法的基础上，设计了其上IST的并行构造方法，并进一步研究了条件BC网络和BC网络上IST的构造问题。

关键词： 互连网络，地址变换，维邻接关系，圆排列，独立生成树

作 者：程宝雷

指导教师：樊建席

Research on Construction of Independent Spanning Trees on BC Networks

Abstract

High performance computers play more and more important roles in the areas of space defense, oil exploration, biological drugs manufacture, weather forecast, and basic theory research. The performance of such computers depends on the connection pattern(interconnection network) of processors or processing machines in the parallel computer system to a large extent. In the parallel computing area, the study of the interconnection networks and their properties is a key subject. An interconnection network can be represented by a simple graph $G = (V(G), E(G))$, where $V(G)$ represents the vertex set and $E(G)$ represents the edge set of G .

Hypercubes are a class of interconnection networks among the common used interconnection networks, many advantageous properties of which such as lower diameter, higher connectivity, symmetry, and recursive construction etc. have attracted the interest from many researchers. At the same time, hypercubes have variants such as crossed cubes, Möbius cubes, twisted cubes, etc. Researchers ascribe hypercubes and many of their variants into a class of interconnection networks—BC networks based on the bijective connection and recursive construction properties. Independent spanning trees (ISTs for short) play important roles in reliable communication, parallel communication, secure distribution of messages and diagnosis of the faulty processors. This thesis adopts the methods from particular to general to study the existence and construction of ISTs rooted at any vertex on BC networks, including the following contents:

1. The serial construction of ISTs rooted at any vertex on an arbitrary crossed cube

(1) We prove that the n -dimensional crossed cube CQ_n possesses an important property—vertex-address-mapping, based on which, an method to construct the isomorphic spanning tree on CQ_{n-1}^1 based on the tree rooted at any vertex on CQ_{n-1}^0 is provided.

(2) We give a recursive method to construct n ISTs rooted at any vertex on CQ_n , prove its correctness, and design an algorithm with the time complexity $O(N\log^2 N)$, where N is the number of vertices in CQ_n .

2. The serial construction of ISTs rooted at any vertex on an arbitrary Möbius cube

Noting that the n -dimensional Möbius cube M_n is composed of two sub-Möbius cubes M_{n-1}^0 and M_{n-1}^1 , which are not isomorphic, as a result, the constructive method of ISTs on CQ_n is not adequate for M_n . We focus on the dimensional relations between any adjacent two vertices in M_n and obtained the following results:

(1) We propose the constructive method of two isomorphic trees on M_{n-1}^0 and M_{n-1}^1 , based on the dimensional-adjacent relations between any two vertices in the trees.

(2) We design the select method of extensive vertex set and extensive vertex set partition during the construction of ISTs, based on the dimensional-adjacent relations between any two vertices in the trees. We also adopt non-negative vectors to prove the correctness of ISTs on M_n .

(3) A recursive algorithm with the time complexity $O(N\log N)$ is proposed, where N is the number of vertices in M_n .

The method to construct isomorphic trees on M_{n-1}^0 and M_{n-1}^1 is also adequate for the construction of isomorphic trees on CQ_{n-1}^0 and CQ_{n-1}^1 . Thus, the method is improved edition of the method used in crossed cubes. In addition, the efficiency of the recursive algorithm designed for ISTs on M_n is higher than that for CQ_n .

3. Parallel construction of ISTs on crossed cubes

The time complexity of the recursive construction of ISTs on crossed cubes and Möbius cube is relatively large. At the same time, during the construction of ISTs, there exists dependence of ISTs, for example, the construction of the n th tree depends on the other $n - 1$ trees. Finding new properties of crossed cubes based on which to design parallel construction method to improve the efficiency of corresponding algorithm becomes an important research content. We obtain the following results:

(1) We prove that crossed cubes possess an important property—dimensional expanding.

(2) A storage structure contains the UUID domain and the VALUE domain is proposed, which can make sure that the UUID domain of each vertex is unique.

(3) We provide a parallel algorithm of ISTs on crossed cubes based on the dimensional expanding property.

4. Parallel construction of ISTs on Möbius cubes

Based on the recursive constructive methods for crossed cubes, Möbius cubes and the parallel construction method for crossed cubes, we further study the parallel construction of ISTs on Möbius cubes and obtain the following results:

(1) From the perspective of permutation and combination, we adopt the concept of circular-dimensional permutation into the construction of ISTs.

(2) We first prove that the descending circular-dimensional permutation can be used to construct ISTs rooted at any vertex on an arbitrary Möbius cube, then prove that any circular permutation can be used to construct ISTs rooted at any vertex on an arbitrary Möbius cube and an arbitrary hypercube.

(3) We point out that different circular-dimensional permutations can be used to construct multiple disjoint paths between any two vertices in Möbius cubes.

(4) We study the diagnosis algorithm based on ISTs.

5. Construction of ISTs on BC networks

The construction and proof of ISTs on the special BC networks such as hypercubes, crossed cubes, locally twisted cubes, Möbius cubes are based on their special properties. We study the general method to construct ISTs based on the common property of such cubes and obtain the following results:

(1) We give the definition of conditional BC networks and prove that hypercubes, crossed cubes, locally twisted cubes, Möbius cube, etc. belong to condition BC networks. At the same time, we also point out there exist networks which do not belong to conditional BC networks.

(2) We prove that the ascending circular-dimensional permutation can be used to construct ISTs rooted at any vertex on n -dimensional conditional BC network XC_n in $O(N)$ time, where each IST is isomorphic to n -level binomial-like tree and $N = 2^n$ is the number of vertices in XC_n , $n \geq 2$.

(3) For any BC network, we propose the definition of V -dimensional permutation and prove that any V -dimensional permutation can be used to construct a spanning tree isomorphic to n -level binomial tree rooted at any vertex on any n -dimensional BC network X_n . We point out that there exist multiple sets of two ISTs on X_n based on V -dimensional permutations.

In summary, this thesis first gives the serial construction methods of ISTs on special BC networks such as crossed cubes and Möbius cubes. Then, based on such serial construction methods, the parallel construction methods of ISTs on the above two networks are designed. Furthermore, the construction problems of ISTs on conditional BC networks

and BC networks are studied.

Keywords: Interconnection Network, Address Mapping, Dimensional-Adjacent Relation, Circular-Dimensional Permutation, Independent Spanning Trees

Written by Baolei Cheng

Supervised by Jianxi Fan

目 录

第一章 绪论	1
1.1 引言	1
1.2 互连网络研究概述	2
1.3 独立生成树构造研究概述	4
1.3.1 独立生成树的研究意义	4
1.3.2 独立生成树的研究现状	5
1.4 研究内容	7
1.5 文章组织结构	9
第二章 相关知识	11
2.1 基本概念和符号表示	11
2.2 特殊BC网络	14
2.2.1 超立方体及其基本性质	14
2.2.2 超立方体的几种主要变型	15
2.2.3 超立方体变型的性质	21
2.3 BC网络及其相关性质	22
2.3.1 BC网络的定义	22
2.3.2 BC网络的相关性质	23
2.4 本章小结	24
第三章 交叉立方体上IST的串行构造	25
3.1 地址变换方法	26
3.1.1 CQ_{n-1}^0 和 CQ_{n-1}^1 上同构树构造分析	26
3.1.2 地址变换方法	27
3.1.3 依据地址变换方法构造同构树	29
3.2 IST的构造	30
3.3 IST优化构造问题的探讨	43

3.3.1	IST的高度与结构	43
3.3.2	IST的地址长度	45
3.4	本章小结	47
第四章	莫比乌斯立方体上IST的串行构造	49
4.1	M_{n-1}^0 和 M_{n-1}^1 上同构树构造分析	49
4.2	莫比乌斯立方体的性质	49
4.3	IST的构造	54
4.3.1	0- M_n 上IST的构造	54
4.3.2	1- M_n 上IST的构造	67
4.4	算法MIST和MIST_UNION构造的IST的性质	67
4.5	基于顶点之间的维邻接关系构造同构树的讨论	71
4.6	满足一定特征的网络上独立生树构造问题的讨论	72
4.7	本章小结	73
第五章	交叉立方体上IST的并行构造	74
5.1	预备知识	74
5.1.1	维邻接关系途径和维邻接关系树	74
5.1.2	交叉立方体上顶点之间维邻接关系的重要性质	76
5.2	基于顶点之间维邻接关系的维扩散性质构造IST	79
5.2.1	一个构造 $CQ_n(u_{n-1}u_{n-2}\dots u_k)$ 上生成树的算法	79
5.2.2	构造通用维邻接关系树的算法	81
5.2.3	基于顶点之间维邻接关系构造IST的一个算法	84
5.3	基于顶点之间维邻接关系的IST的证明	84
5.3.1	路径和生成树的证明	86
5.3.2	IST的证明	90
5.4	本章小结	93
第六章	莫比乌斯立方体上IST的并行构造	94
6.1	预备知识	94
6.2	维邻接关系途径和维邻接关系树	94
6.3	IST的构造	96

6.3.1	基于一个圆排列来构造IST的算法	96
6.3.2	算法PMCIST的正确性证明	97
6.3.3	基于圆排列的IST的构造算法	103
6.4	基于任一圆排列构造以任一顶点为根的IST	104
6.4.1	构造算法	104
6.4.2	基于IST的不相交路径	109
6.5	IST在诊断上的应用	111
6.6	IST的模拟实验和IST在诊断上的应用	114
6.7	本章小结	117
第七章	BC网络上IST的构造	119
7.1	BC互连网络	119
7.2	条件BC网络	121
7.2.1	条件BC网络的定义	121
7.2.2	条件BC网络族	121
7.2.3	圆排列和V排列	123
7.3	条件BC网络上IST构造	124
7.3.1	条件BC网络上IST的构造算法	124
7.3.2	条件BC网络上IST的正确性证明	125
7.4	任意BC网络上的生成树和IST	130
7.5	本章小结	132
第八章	总结与展望	134
8.1	总结	134
8.2	展望	136
	参考文献	137
	攻读博士学位期间取得的科研成果及参与的学术交流情况	150
	致谢	154

第一章 绪论

1.1 引言

近年来，在国防太空、石油勘探、生物制药、天气预报以及基础理论研究等领域出现了需要庞大计算量的应用问题，这些问题对计算性能提出了极高的需求。单个微处理器的性能已经远远不能满足需要，尽管自20世纪80年代以来，处理器性能提高的速度基本遵循摩尔定律——每18个月翻一番 [1]。具有多处理器的并行计算机的出现提高了处理速度，从而为实现高性能计算提供了解决方案，它可以满足人们对计算机处理速度和性能不断增长的需求 [2]。由于任何高性能计算和超级计算都离不开并行技术，因此高性能计算（High Performance Computing）等同于并行计算（Parallel Computing）或超级计算（Supercomputing），并被定义为并行计算机上所做的计算。相应地，并行计算机又称为超级计算机或者高性能计算机 [3]。早期的并行计算机就是指由多个处理单元（也称为处理器，或简称为CPU）组成的计算机系统，这些处理单元相互通信和协作，能快速、高效地求解大型复杂问题 [4]。而目前其概念得到进一步拓展，工作站机群也加入并行计算机的大家庭，从而使得各种并行体系结构并存发展。高性能计算机已经成为衡量一个国家综合国力的重要标志，它也是国家信息化建设的根本保证。

在我国，2006年国务院发布的《国家长期科学和技术发展规划纲要（2006-2020）》中明确指出加速发展高性能计算对提高我国国防建设与国家安全、国民经济建设、国家重点工程和基础科学研究等尖端科技领域的核心支撑能力具有十分重要的战略意义。目前我国高性能计算机硬件的研制已经走在了世界的前列。世界超级计算机TOP500组织于2013年6月17日在德国莱比锡举行的“2013国际超级计算大会”上，正式发布第41届世界超级计算机500强排名，其中，由中国国防科技大学开发的超级计算机天河二号以持续计算速度每秒3.39亿亿次的优越性能位居榜首。李克强总理在2014年的政府工作报告中也肯定了我国在超级计算方面取得的突破。

从世界超级计算机Top500的排名上看，发达国家占据明显优势。这也表明世界上的发达国家对高性能计算机的研制是非常重视的。美国、日本和欧洲各国都相继提出关于发展高性能计算和高性能计算机的计划，如美国科学技术联邦协调理事会提出的著名的HPCC（High Performance Computer and Communication）重大

挑战计划，由美国能源部联合美国三大核武器实验室共同提出的ASCI（Accelerated Strategic Computing Initiative）计划，美国国防先进研究计划局启动的HPCS（High Productivity Computing System）计划 [3]，日本的真实世界计算（RWC）计划，欧洲的千万亿次超级计算机合作发展计划等。如今，并行计算机已经是世界公认的高新技术制高点和21世纪最重要的科学领域之一。

接下来，我们简要介绍并行计算机的发展历史。并行计算机的萌芽阶段始于20世纪60年代初，著名的CDC 6600计算机采用一种非对称的共享存储结构，其中央处理器采用双CPU，且连接多个外部处理器。到了20世纪70年代，出现的向量计算机代表真正意义上的第一代高性能计算机，它在计算机中加入向量流水部件，大大提高了科学计算中向量运算的速度。在随后的十几年里，向量计算机得到了快速发展，在此期间推出了多个著名的向量计算机，如CDC Cyber205、Fujitsu VP1000/VP2000、NEC SX1/SX2、CRAY-2、CRAY-3和我国研发的银河一号等。大规模并行处理（MPP，Massively Parallel Processing）系统在20世纪90年代得到蓬勃发展，Intel Paragon、IBM SP2、Intel TFLOPS和我国的曙光-1000等都是这种类型的机器。MPP由多个微处理器通过高速互连网络构成，处理器间通过消息传递的方式进行通信和协调。到了90年代的中期，对称多处理机（SMP: Symmetrical Multi-Processors）在中、低档市场上以其更优的性能/价格比取代了MPP。SMP虽然扩展能力有限，但是由于其与单机系统很好的兼容性，被广泛应用在商业计算领域，如IBM R50、SGI Power Challenge、DEC Alpha服务器8400和我国曙光1号等都是这种类型的机器 [4]。此时，并行向量处理机（PVP: Parallel Vector Processor）、MPP、SMP、DSM（Distributed Shared Memory）多处理机、工作站机群（COW: Cluster of Workstations）等各种体系结构并存发展。2000年以来，工作站机群成为了主流并行机。工作站机群是将一群工作站或者高档微机采用某种结构的互连网络连接起来，利用各个计算机的资源，统一调度、协调处理，以达到较好的计算性能。纵观并行计算机的发展历史，自90年代以来，并行计算机采用的体系结构都需要用到互连网络。互连网络是并行计算机的核心，并行计算机的性能依赖于其内部的互连网络。因此，对互连网络拓扑结构及其性质的研究具有重要的理论和实际意义 [5-19]。

1.2 互连网络研究概述

并行计算机中有多个处理器（处理机）单元，它可以执行任务的并行计算，即将一个任务分割成多个子任务，分配给不同的处理器（处理机）来并行地执行。

并行计算机内部处理器（处理机）按照一定方式连接起来的网络称为多处理器互连网络，简称为互连网络。互连网络一般可以分成静态互连网络（Static Network）和动态互连网络（Dynamic Network）。典型的静态互连网络有：一维线性阵列、二维网孔、树、超立方体和蝶形网络等 [3]。动态互连不采用固定连接，而是在连接路径的交叉点处放置电子开关、选路器或仲裁器等，以提供动态连接的特性。本文中研究的互连网络指的是静态互连网络。

近30年以来，互连网络的研究在并行计算领域中占有重要地位。研究者提出了很多不同的互连网络拓扑结构，例如超立方体、树型网络（Tree）、网孔（Mesh）、环绕（Torus）、奇图（Odd Graph）、偶图（Even Graph）、星图（Star）、De Bruijn图、蝶形网络等。超立方体是一种非常流行的多处理器互连网络拓扑结构。它具有许多优越的性质，例如低直径、高连通性、对称性等 [20]。超立方体具有很好的常用网络模拟能力，很多常用网络可以嵌入超立方体中 [21–27]。由于超立方体具有这些优越性质，且适合并行算法，已经有多种并行机如nCUBE、CM-5、iPSC-860及SGI Origin 2000等采用超立方体体系结构。超立方体除了在互连网络中的应用，最近它也被应用到了其它领域中。它可以用于P2P覆盖网络（Overlay Network）的设计 [28–32]和无线传感器网络的设计（Wireless Sensor Network） [33, 34]。Yang等将超立方体用于Ad-Hoc网络上，提出了一种基于超立方体的多址接入协议（Multiple Access Protocol） [35]。2008年Lee等研究了超立方体及其变型在纳米计算机中应用的可能性，他们提出了一种称为“M-超立方体”的超立方体变型，它能提供更高维的布局，并且可以支持三维集成电路。他们证明了包括“M-超立方体”在内的超立方体具有适合于并行计算和通信网络设计的独特的、理想的内在结构 [36]。最近，Wu等还将超立方体应用在人类联系网络（Human Contact Networks）上 [37]，提出了两种特殊的基于特征匹配过程的多路径路由算法。

虽然超立方体是一种在实践中得到广泛应用的互连网络之一，然而对于一个具有 2^n 个顶点，连通度是 n 的网络拓扑结构来说，超立方体的网络直径并不是最优的。也就是说，在保证连通度和顶点总数不变的前提下，还可以设计出直径更小的互连网络。于是，超立方体的几种变型陆续被提出：扭立方体（Twisted Cube）于1991年被提出 [38]；交叉立方体（Crossed Cube）于1992年被提出 [39]；莫比乌斯立方体（Möbius Cube）于1995年被提出 [40]；局部扭立方体（Locally Twisted Cube）于2005年被提出 [41]。这几种超立方体变型都是研究者通过改变超立方体的某些连接而提出的，其直径约为超立方体直径的一半。Fan等归纳出超立方体及其上述变型

具有两个共性——一一对应连接和可递归构造性质，并根据这两个性质，提出了一一对应连接网络（Bijective Connection Network，简称为BC网络，也称为类超立方体网络（Hypercube-Like Network [42]））的定义，它不仅包括超立方体、交叉立方体、莫比乌斯立方体、局部扭立方体、扭立方体等，而且包括除这些网络之外的许多尚未定义的网络。迄今为止，研究者在BC网络上已经取得了一系列研究成果 [43–49]。

1.3 独立生成树构造研究概述

互连网络的一个重要性能指标是其上能否模拟性能较好的常用网络。网络模拟在移植为一种互连网络开发的并行算法至另一种互连网络上有重要的应用，并且在分配并发的进程给互连网络中的处理器上有很好的应用。互连网络中的常用网络如树、圈、网格等结构的构造问题一直是国内外学者的关注焦点 [8, 50–68]，其中独立生成树的构造问题近年来尤为突出。

1.3.1 独立生成树的研究意义

一个互连网络可以用一个图 G 来表示。 G 上的 n 条路径如果具有相同的起点和终点，并且这 n 条路径中任意两条路径上的边都不相同，则称这 n 条路径为边不相交路径； G 上的 n 条路径如果具有相同的起点和终点，并且这 n 条路径中任意两条路径上的顶点都不相同（除起点和终点）且边也都不相同，则称这 n 条路径为顶点不相交路径。 G 上的两棵生成树如果具有相同的根，且这两棵生成树中由根到任一顶点的两条路径（每棵生成树各一条）为顶点（边）不相交路径，则称这两棵树顶点（边）相互独立。若 n 棵生成树两两顶点（边）相互独立，则称之为 n 棵顶点（边）独立生成树（Vertex-Independent Spanning Trees，简称IST（Edge-Independent Spanning Trees简称EIST））。容易看出，若 G 的一组生成树是顶点-独立生成树，那么它们一定是边-独立生成树。在本文中，我们将重点讨论顶点-独立生成树。

若 G 中存在以顶点 r 为根的 n 棵IST $T_0, T_1, T_2, \dots, T_{n-1}$ ，根据定义，可得出 r 与 G 中任一其它顶点 s 之间存在 n 条顶点不相交路径。在互连网络中，IST在信息的可靠传输、并行传输、安全分发及故障处理器的诊断等方面具有重要的作用。

1) 信息的可靠传输

在互连网络中，常常基于树型结构传输信息，但若树中一个顶点产生拥塞，则由该顶点引出的子树中所有顶点将都产生拥塞。Bao等提出了一种用IST来设计可靠

广播协议的方法 [69,70]。设图 G 中顶点 r 要广播一组数据包,若 G 上存在以顶点 r 为根的 n ($n \geq 2$) 棵IST,则 r 将沿每棵IST广播每个数据包的一份拷贝。从而在正常情况下, G 中除 r 之外的每个顶点将收到每个数据包的 n 份拷贝。显然,若 n 充分大,则这 n 份拷贝同时丢失的可能性将是很小的,从而达到信息可靠传输的目的。

另一方面,由于在并行计算机系统中有大量处理器和链路,处理器或链路发生故障的情形是不可避免的,而经过故障处理器或故障链路传递的数据将是不可靠的。若 G 上存在以顶点 r 为根的 n 棵IST,且 G 中故障顶点(边)的数目不超过 $n-1$,则 r 与任意其它无故障顶点之间总存在至少一条可靠路径(即所经过的所有顶点(边)都是无故障的路径)。

2) 信息的并行传输与安全分发

设 G 中的顶点 r 要传输数据 m 给 G 中任一目标顶点 s ($s \neq r$),若 G 上存在 n 棵以 r 为根的IST $T_1, T_2, \dots, T_n$,那么 r 和 s 之间存在 n 条顶点不相交路径 $P_1, P_2, \dots, P_n$ 。可以在发送时将数据 m 分成 n 个数据包: $m_1, m_2, \dots, m_n$,然后 r 将每一个数据包 m_i 沿路径 P_i 进行并行传输, $i = 1, 2, \dots, n$ 。在正常情况下, s 将收到所有的数据包,这些数据包被合并后,就可以得到完整的数据 m 。这一传输模式可利用 n 条顶点不相交路径来分解流量,从而提高了传输速度。同时,路径 P_i 中的所有中间顶点只传输数据包 m_i ,无需传输其它信息,因此也可以同时在多条路径中传输互不干扰的机密信息,达到安全传输信息的目的 [70]。

3) 故障处理器的并行诊断

如前所述,在一个具有大规模处理器和链路的并行计算机系统中,处理器或链路发生故障的概率是较高的。出现了故障,我们需要定位故障并进行修复。若 G 中故障顶点的数目不超过 $n-1$,且 G 上存在以顶点 r 为根的 n 棵IST的并行构造算法,则可以借助于该算法,设计出关于故障处理器的高效并行诊断算法 [71]。故障边的情况也可采用类似方法进行诊断。

因此,互连网络上以任一顶点为根的多棵IST的存在性及构造也是衡量其性能的重要指标。

1.3.2 独立生成树的研究现状

如前所述,IST在信息的可靠传输、并行传输、安全分发及故障处理器的诊断等方面具有重要的意义。但是,关于图上IST的存在性有下述两个猜想 [72,73]:

猜想1: 设 G 是一个 n -顶点连通图, 则 G 中存在以任一顶点为根的 n 棵IST。

猜想2: 设 G 是一个 n -边连通图, 则 G 中存在以任一顶点为根的 n 棵EIST。

Khuller等证明了一个结论: 若 n -顶点连通图存在 n 棵IST, 则 n -边连通图中存在 n 棵边独立生成树 [74]。但是, 在2013年, Gopalan等举出了一个反例, 指出文献 [74]的证明不正确 [75]。当 $n \leq 4$ 时, 猜想1是成立的 [72, 73, 76, 77]。但当 $n \geq 5$ 时, 猜想1仍然没有得到解决 (Open)。依据Gopalan等的结论, 我们可以推出当 $n \geq 4$ 时, 猜想2仍然没有得到解决 (Open) [75, 78]。因此, 人们主要致力于在一些特殊图, 如平面图 (Planar Graph) [79–82]、乘积图 (Product Graph) [64, 83, 84]、奇图 [65]、偶图 [66]、弦环图 (Chordal Ring Graph) [85, 86]、De Bruijn和Kautz图 [87]、折叠超级星图 (Folded Hyper-Star) [60]、多维环绕网络 (Multidimensional Torus Network) [61]、递归流图 (Recursive Circulant Graph) [58]、超立方体及其变型网络上IST的研究。其中, 近年来, 超立方体及其变型上IST的成果不断涌现。

一、超立方体

由于 n 维超立方体 Q_n 是一个乘积图 [83], 因此Obokata等提出的构造方法同样能用来构造其上以任一顶点为根的 n 棵IST, 但这种构造方法没有考虑到IST的性能 (如树的高度)。Tang等修改了 [88]中的构造方法, 提出了一个在 $O(N \log N)$ 时间内构造 Q_n 上以顶点0为根的 n 棵优化的IST的算法, 其中 $N = 2^n$ 为 Q_n 上顶点的个数 [88]。由于 Q_n 是一个凯莱图 (Cayley Graph) [89], 因此它是顶点对称的和边对称的, 从而Tang等很好地解决了超立方体上以任一顶点为根的IST的构造问题。Yang等基于海明距离拉丁方 (Hamming Distance Latin Square) 给出了 Q_n 上 n 棵IST的并行构造算法 [90]。Werapun等指出Yang等算法的时间复杂度为 $O(n^2)$, 进一步引入双跳指针 (Double Pointer Jumping), 给出了 Q_n 上IST的时间复杂度为 $O(n)$ 的高效并行构造算法 [91]。

二、超立方体变型

Hsieh等提出了一个在 n 维局部扭立方体 LTQ_n 上构造以顶点0为根的 n 棵边-独立生成树的算法, 但未给出时间复杂度的分析 [92]。Lin等证明了Hsieh等给出的算法所构造的 LTQ_n 上的边-独立生成树实际上也是顶点-独立生成树 [93]。但由于 LTQ_n 不是顶点对称的, 因此该算法并没有完全解决在 LTQ_n 上构造以任一顶点为根的 n 棵IST的问题。Liu等证明了 LTQ_n 满足奇偶转换 (Odd-Even-Transitive) 性质, 从而将 LTQ_n 中的所有顶点划分为两类: 以顶点0为代表的类和以顶点1为代表的类 (这里

的0和1是十进制的) [94]。他们证明了 LTQ_n 上的以任一顶点为根的 n 棵IST的构造问题可以归结为以0或1为根的IST的构造问题并给出了相应的构造算法。进一步,他们证明了该算法不仅适用于 LTQ_n 而且适用于 Q_n ,但未分析算法的时间复杂度与树的结构。

Yang等提出了一个在 TQ_n 上构造以0为根的 n 棵边独立生成树的时间复杂度为 $O(N\log N)$ 的算法。他们还证明了其中 $\lfloor n/2 \rfloor$ 棵是IST,但并没有解决 TQ_n 上以任一顶点为根的 n 棵IST的存在性及构造问题 [95]。Wang等结合扭立方体和扭曲立方体的构造规律,扩展了其定义,给出了其上以任一顶点为根的 n 棵IST的时间复杂度为 $O(N\log N)$ 的构造算法 [96,97]。

但是,文献中对超立方体的几种变型上IST的存在性及构造问题的研究都是根据这些变型的特殊性质,采用特殊的方法进行的,缺乏一般性。超立方体还存在其它变型如交叉立方体、莫比乌斯立方体等,对这些变型上IST构造的研究很少;进一步,在包含这些变型的BC网络上IST的一般构造方法和相应构造算法的研究更是缺乏。

1.4 研究内容

针对BC网络上IST的一般构造方法及相应的通用构造算法缺乏的问题,本文采用由特殊到一般的研究方法,首先研究交叉立方体、莫比乌斯立方体上IST的串行构造方法,然后解决IST在串行构造过程中的依赖关系,进一步设计了适合于交叉立方体、莫比乌斯立方体上IST的并行构造方法。进一步,在总结交叉立方体、莫比乌斯立方体上IST并行构造方法的基础上,提取出一类BC网络的共性,给出了条件BC网络的定义,并设计了适合于在其上构造IST的通用方法。最后,探讨了适合于任一BC网络的IST的构造方法。

一、交叉立方体上独立生成树的串行构造

n 维交叉立方体 CQ_n 上存在 $2^{\lceil (n-4)/2 \rceil}$ 个顶点等价类,其特定的构造规律使得现有IST构造方法不再适用于在其上构造以任一顶点为根的IST。针对 CQ_n 上IST的构造问题,我们取得了如下研究成果:

(1) 结合 CQ_n 的构造规律,提出了适合于在子交叉立方体 CQ_{n-1}^0 和 CQ_{n-1}^1 中构造同构树的地址变换方法。

(2) 基于地址变换方法的扩展顶点集合和扩展顶点集合划分,设计了 CQ_n 上以

任一顶点为根构造 n 棵IST的方法并证明了其正确性。

(3) 设计了一个时间复杂度为 $O(N\log^2 N)$ 的递归构造算法。

二、莫比乌斯立方体上独立生成树的串行构造

与交叉立方体不一样的是, n 维莫比乌斯立方体 M_n 是由两个不同构的子莫比乌斯立方体 M_{n-1}^0 和 M_{n-1}^1 组成,从而交叉立方体上地址变换的方法不再适用于在莫比乌斯立方体上构造IST。我们从顶点之间维邻接关系的角度研究,取得了如下成果:

(1) 结合 M_n 的构造规律,从顶点之间维邻接关系的角度提出了适合于在不同子莫比乌斯立方体 M_{n-1}^0 和 M_{n-1}^1 上构造同构树的方法。

(2) 基于顶点之间的维邻接关系,设计了 M_n 上独立生成树的构造过程中扩展顶点集合和扩展顶点集合划分的选择方法,并引入非负向量来证明其上IST存在的正确性。

(3) 设计了时间复杂度为 $O(N\log N)$ 的递归构造算法。

M_n 上基于顶点之间的维邻接关系构造 M_{n-1}^0 和 M_{n-1}^1 中同构树的方法同样适合在 CQ_{n-1}^0 和 CQ_{n-1}^1 中构造同构树,该方法是 CQ_n 上地址变换方法的改进。另外,在 M_n 上所设计的IST的构造算法的效率比 CQ_n 上的构造算法高。

三、交叉立方体上独立生成树的并行构造

从交叉立方体和莫比乌斯立方体上IST的串行构造方法来看,其对应算法的时间复杂度高且IST在构造过程中存在相互依赖的缺点,如第 n 棵树在构造过程中依赖于前面 $n-1$ 棵树。我们研究了交叉立方体的新的性质并基于其设计独立生成树的并行构造方法,取得了如下研究结果:

(1) 证明了交叉立方体上满足一个重要性质——维扩散性质。

(2) 提出一个具有UUID成员和VALUE成员的树存储结构,确保树中顶点的UUID成员总是具有唯一性。

(3) 基于交叉立方体上维扩散性质给出了其上IST的并行构造算法。

四、莫比乌斯立方体上独立生成树的并行构造

在对交叉立方体上IST的串行和并行构造方法、莫比乌斯立方体上IST的串行构造方法总结的基础上,我们进一步研究了莫比乌斯立方体上IST的并行构造问题,取得了如下研究结果:

(1) 从排列组合的角度,将圆排列的概念引入到IST的构造方法中。

(2) 首先证明了递减的圆排列能够用于并行构造莫比乌斯立方体上以任一顶点为根的IST。其次, 进一步证明了任一圆排列能够用来并行构造莫比乌斯立方体上以任一顶点为根的IST。

(3) 指出能够利用不同的圆排列构造的多组IST能够用来构造莫比乌斯立方体上任意两个顶点之间多组顶点不相交路径。

(4) 探讨了基于IST的高效诊断算法。

(5) 采用JUNG技术对IST的构造进行了模拟。

五、BC网络上IST的构造

到目前为止, 在特殊BC网络如超立方体、交叉立方体、局部扭立方体、莫比乌斯立方体等上IST的构造问题已经被解决。但是, 特殊BC网络中IST的构造及其证明依赖于其特有的性质。因此, 这些特殊BC网络如超立方体、交叉立方体、局部扭立方体、莫比乌斯立方体等上是否具有共同的性质, 根据该性质能否设计一个一般性的算法来构造其上的IST? 对于所有的BC网络, 是否存在一个共同的方法来构造其上类似于二项树的生成树和同构于类二项树的IST?

针对上述问题, 我们提出了一个一般的方法构造BC网络上的生成树和IST, 取得结果如下:

(1) 我们给出了条件BC网络的定义, 证明了超立方体、交叉立方体、局部扭立方体、莫比乌斯立方体等均属于条件BC网络。同时, 我们证明存在不属于条件BC网络的BC网络。

(2) 我们证明了利用递增的圆排列能够在 $O(N)$ 时间内构造 n 维条件BC网络 XC_n 上以任一顶点为根的同构于 n -级类二项树的IST, 这里 $n \geq 2$ 且 $N = 2^n$ 是 XC_n 的顶点个数。

(3) 针对任一BC网络, 我们提出了维的 V 排列。我们证明了 V 排列能够用来构造任一 n 维BC网络 X_n 上以任一顶点为根的生成树且证明这些生成树均同构于 n 级二项树。我们指出, 基于 V 排列, 能够在 X_n 上构造多组两两独立的生成树。

1.5 文章组织结构

全文的组织结构如下:

第一章介绍了课题的相关背景知识、独立生成树的研究意义、研究内容以及文章的组织结构。

第二章首先介绍了和本文相关的图论基本概念和符号表示，引入了超立方体及其几种主要变型的概念和定义，给出了BC网络的定义及其相关性质。

第三章和第四章分别针对特殊BC网络交叉立方体和莫比乌斯立方体给出了其上以任一顶点为根的IST的存在性证明，并给出了串行构造算法。

第五章和第六章分别针对交叉立方体和莫比乌斯立方体研究了其上以任一顶点为根的IST的并行构造问题并给出了相应算法。第六章还探讨了独立生成树在互连网络中故障处理器的诊断技术上的应用问题。

第七章研究了条件BC网络及任意BC网络上以任一顶点为根的IST的构造问题。

第八章对本课题所做的工作进行了总结，并对未来工作进行了展望。

第二章 相关知识

本章首先介绍本文用到的一些图论基本概念和符号表示,然后介绍几种特殊互连网络如超立方体、交叉立方体、莫比乌斯立方体、扭立方体、局部扭立方体的定义及其性质,最后介绍了包含上述特殊立方体的BC网络的概念及其性质。

2.1 基本概念和符号表示

由于不同的文献中对于一些图论中概念的描述和符号的表示会有一些区别,为了不产生混淆,下面列出本文将用到的与互连网络相关的一些图论的基本概念和符号表示 [2,98–100]。

图 (Graph): 图 G 是一个有序的二元组 $G(V(G), E(G))$,简写为 $G(V, E)$ 。其中, $V(G)$ 是图 G 的顶点集合 (Vertex Set),而 $E(G)$ 是图 G 的边集合 (Edge Set)。 $V(G)$ 中的元素称为 G 的顶点 (Vertex), $E(G)$ 中的元素称为 G 的边 (Edge),并且 $E(G) \subseteq V(G) \times V(G)$ 。

无向图 (Undigraph): 如果 $V(G) \times V(G)$ 是无序顶点对集合,则称图 G 为无向图。

有向图 (Digraph): 如果 $V(G) \times V(G)$ 是有序顶点对集合,则称图 G 为有向图。

简单图 (Simple Graph): 如果图 G 中任意两个顶点之间至多存在一条边且每个顶点都没有到它自身的边,则称图 G 为简单图。本文中的图都是简单图。

子图 (Subgraph): 对于两个图 $G_1 = (V(G_1), E(G_1))$ 和 $G_2 = (V(G_2), E(G_2))$,如果 $V(G_1) \subseteq V(G_2)$ 并且 $E(G_1) \subseteq E(G_2)$,则称 G_1 是 G_2 的子图。

顶点度 (Vertex Degree): 图中与某个顶点相关联的边的数目称为这个顶点的顶点度,简称为度 (Degree)。

d -正则 (d -Regular): 如果图 G 中每个顶点的度都为 d ,那么则称图 G 为 d -正则图。

端点: 对图 G 中的边 (x, y) ,我们称顶点 x 和 y 为该边的端点。

相邻 (Adjacent): 如果两个顶点之间有边相连,那么称这两个顶点之间是相邻的。如果两条边有一个共同的端点,则称这两条边是相邻的。

途径 (Walk): 图 G 中两个顶点 $x^{(0)}$ 与 $x^{(n)}$ 之间的一条途径 (Walk) $W = x^{(0)} e^{(1)} x^{(1)} e^{(2)} \dots x^{(n-1)} e^{(n)} x^{(n)}$ 是一个顶点和边的非空有限交替序列, 其中 $x^{(0)}$ 和 $x^{(n)}$ 分别称为该途径的起点和终点。且每条边 $e^{(j)} = x^{(j-1)}x^{(j)}$, $1 \leq j \leq n$, W 中的顶点和边可重复出现。途径所经过的边的数目称为 W 的长度。我们用 $V(W)$ 表示 W 经过的所有顶点的集合, $E(W)$ 表示 W 经过的所有边的集合。 W 也可简写为 $x^{(0)}, x^{(1)}, \dots, x^{(n)}$ 。

路径 (Path): 若途径 $W : x^{(0)}, x^{(1)}, \dots, x^{(n)}$ 的顶点各不相同, 则称 W 为路径, 其中 $x^{(0)}$ 和 $x^{(n)}$ 分别称为该路径的起点和终点。起点为 $x^{(0)}$ 和终点为 $x^{(n)}$ 的路径也可表示为 $\langle x^{(0)}, x^{(n)} \rangle$ -路径。

边不相交的 (Edge-Disjoint): 设 P 和 Q 是顶点 x 与顶点 y 之间的两条不同的路径, 若 $E(P) \cap E(Q) = \emptyset$, 则称 P 和 Q 是边不相交的。

顶点不相交的 (Vertex-Disjoint): 设 P 和 Q 是顶点 x 与顶点 y 之间的两条不同的路径, 若 $E(P) \cap E(Q) = \emptyset$ 且 $V(P) \cap V(Q) = \{x, y\}$, 则称 P 和 Q 是顶点不相交的。

生成树 (Spanning Tree): 设 G 的子图 T 是 G 上以顶点 r 为根的一棵树, 若 T 包含 G 的所有顶点, 则称 T 是 G 上以顶点 r 为根的一棵生成树。

边-相互独立的 (Edge-Independent): 设 T_1 和 T_2 是 G 上以顶点 r 为根的两棵不同的生成树, 若对于 G 中的任一顶点 $s \in V(G) \setminus \{r\}$ (显然, s 也是 T_1 和 T_2 中的顶点), T_1 中 r 与 s 之间的路径 P 和 T_2 中 r 与 s 之间的路径 Q 满足 $E(P) \cap E(Q) = \emptyset$, 则称 G 的两棵生成树 T_1 和 T_2 是边-相互独立的 (Edge-Independent)。

顶点-相互独立的 (Vertex-Independent): 若对于 G 中的任一顶点 $s \in V(G) \setminus \{r\}$, T_1 中 r 与 s 之间的路径 P 和 T_2 中 r 与 s 之间的路径 Q 满足 $V(P) \cap V(Q) = \{r, s\}$ 且 $E(P) \cap E(Q) = \emptyset$, 则称 G 的两棵生成树 T_1 和 T_2 是顶点-相互独立的 (Vertex-Independent)。

距离 (Distance): 任意顶点 u 和 v 之间的最短路径的长度就是它们之间的距离, 记为 $\text{dist}(G, u, v)$, 简写为 $\text{dist}(u, v)$ 。

直径 (Diameter): 图的直径定义为 $D(G) = \max\{\text{dist}(G, u, v) | u, v \in V(G) \text{ 并且 } u \neq v\}$, 本文中图 G 的直径表示为 $D(G)$ 。

导出子图: 设 V' 是图 G 顶点集 V 的一个非空子集。以 V' 为顶点集, 以两端点均在 V' 中的全部边作为边集的子图, 称为 G 中由非空顶点子集 V' 导出的子图, 记 $G[V']$ 。

连通 (Connected): 若图中任意两个顶点之间存在一条路径, 则称该图是连通的。

边连通度 (Edge Connectivity): 图 G 的边连通度就是使 G 变成不连通或平凡图

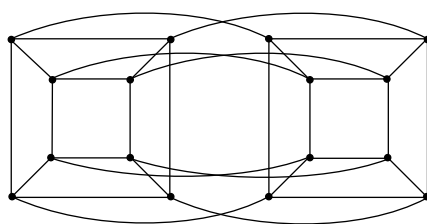


图 2.1 点可迁图示例

(仅有一个顶点的图称为平凡图)，所需去掉的最少的边数。本文中图 G 的边连通度表示为 $\kappa(x_n)$ 。

顶点连通度 (Vertex Connectivity): 图 G 的顶点连通度就是使 G 变成不连通或平凡图，所需去掉的最少的顶点数。本文中图 G 的顶点连通度表示为 $\lambda(x_n) = n$ 。顶点连通度和边连通度统称为连通度。连通度为 k 的图称为 k -连通图。

同构 (Isomorphism): 两个简单图 G_1 和 G_2 为同构的，记为 $G_1 \cong G_2$ 当且仅当存在一个双射 $\psi: V(G_1) \rightarrow V(G_2)$ ，有 $(x, y) \in E(G_1) \iff (\psi(x), \psi(y)) \in E(G_2)$ 。图到自身的同构被称做自同构。如果图 G 存在自同构 ψ ，使得对于任意两个顶点 x 和 y ，都有 $\psi(x) = y$ ，则称 G 为点可迁的。点可迁的例子如图2.1所示。

哈密尔顿圈 (Hamiltonian Cycle): 包含图 G 中的所有顶点一次且仅一次的圈称为 G 的哈密尔顿圈。含有哈密尔顿圈的图称为哈密尔顿图。

哈密尔顿连通的 (Hamiltonian Connected): 若 G 的任意两个不同顶点之间存在一条哈密尔顿路径，则称 G 是哈密尔顿连通的。

k -泛圈 (k -Pancyclic): 如果图 G 中存在从 k 到 $|V(G)|$ 之间每个长度的圈，那么称图 G 为 k -泛圈的。如果图 G 中存在从3到 $|V(G)|$ 之间每个长度的圈，那么称图 G 为泛圈的。

k -点泛圈 (k -Vertex-Pancyclic) 和 k -边泛圈 (k -Edge-Pancyclic): 如果图 G 中的每个顶点 (每条边) 包含在从 k 到 $|V(G)|$ 之间每个长度的圈中，则称 G 为 k -点泛圈的 (k -边泛圈的)。一个图是 k -边泛圈的必然是 k -点泛圈的。

笛卡尔乘积图 (Cartesian Product Graph): 设 G_1 和 G_2 为两个无向图， G_1 和 G_2 的笛卡尔乘积记为 $G_1 \times G_2$ 也是无向图，其中 $V(G_1 \times G_2) = V(G_1) \times V(G_2)$ ，对于不同的顶点 x_1x_2 和 y_1y_2 (这里 $x_1y_1 \in V(G_1)$ ， $x_2y_2 \in V(G_2)$) 相邻当且仅当 $x_1 = y_1$ 且 $(x_2, y_2) \in E(G_2)$ 或者 $x_2 = y_2$ 且 $(x_1, y_1) \in E(G_1)$ 。有向图的笛卡尔乘积图的定义类似。图2.2给出了笛卡尔乘积图的示例。

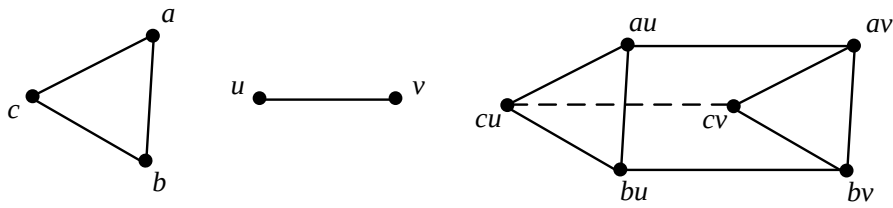


图 2.2 笛卡尔乘积图示例

2.2 特殊BC网络

BC网络是具有一一对应连接和可递归构造特性的网络，它不仅包括超立方体、交叉立方体、莫比乌斯立方体、局部扭立方体、扭立方体等，而且包括除这些网络之外的许多尚未定义的网络。我们将超立方体及其现有变型交叉立方体、莫比乌斯立方体、局部扭立方体和扭立方体等称为特殊BC网络。本节将首先介绍超立方体的定义及其基本性质。其次，介绍其几种特殊变型交叉立方体、扭立方体、莫比乌斯立方体、扭曲立方体及局部扭立方体的定义并简要阐述了这些变型的性质。最后，介绍了BC网络的定义及其性质。

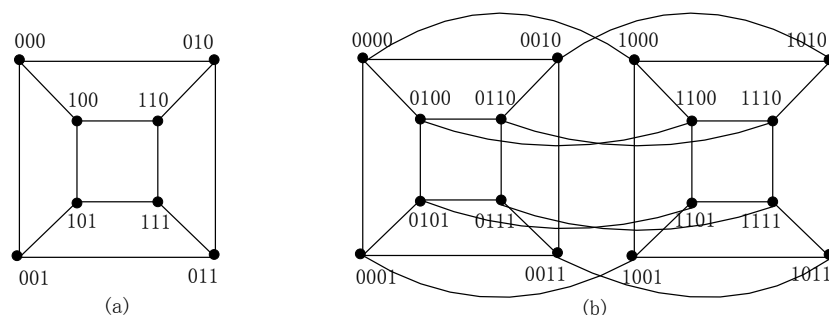
2.2.1 超立方体及其基本性质

一个长度为 n 的二进制字符串 x 可以表示为 $x_{n-1}x_{n-2}\dots x_0$ ，这里 x_{n-1} 是最高位， x_0 是最低位，且 $x_i \in \{0, 1\}$ ，对于 $0 \leq i \leq n-1$ 。 x_i 的补集表示为 $\bar{x}_i$ ($\bar{0} = 1$ 和 $\bar{1} = 0$)。假设 z 是一个长度为 k 的二进制串。 z^i 表示将字符串 z 重复 i 次后所得到的新串。如果 $i = 0$ ， z^i 代表空串。给定一个二进制串集合 W ，如果存在一个子串 $s \in \{0, 1\}^m$ ，使得 s 是 W 集合中所有顶点的前缀和整数 m ， $1 \leq m \leq n-1$ 。那么， s 被称为集合 W 中所有二进制串的前缀。令 $x = x_{m-1}x_{m-2}\dots x_0 \in \{0, 1\}^m$ ， $y = y_{n-1}y_{n-2}\dots y_0 \in \{0, 1\}^n$ 。我们用 xy 来表示 $x_{m-1}x_{m-2}\dots x_0y_{n-1}y_{n-2}\dots y_0$ 和 xW 表示集合 $\{xw | w \in W\}$ 。

通常按顶点连接方式或利用笛卡尔集来定义超立方体，其按顶点连接方式定义如下 [101]：

定义 2.1. n 维超立方体记为 Q_n ， Q_n 中两个顶点 $u = u_{n-1}u_{n-2}\dots u_1u_0$ 和 $v = v_{n-1}v_{n-2}\dots v_1v_0$ 之间有边相连当且仅当顶点 u 和顶点 v 有且仅有一位坐标不同，即：

$$\sum_{i=0}^{n-1} |u_i - v_i| = 1. \quad (2.1)$$

图 2.3 (a)3维超立方体 Q_3 ; (b)4维超立方体 Q_4

利用笛卡儿乘积的递归定义如下:

定义 2.2. $Q_1 = K_2$, $Q_n = Q_{n-1} \times Q_1 = K_2 \times K_2 \times \dots \times K_2, n \geq 2$ 。

其中, K_2 是二阶的完全图。

我们进一步给出其另外一种递归定义。

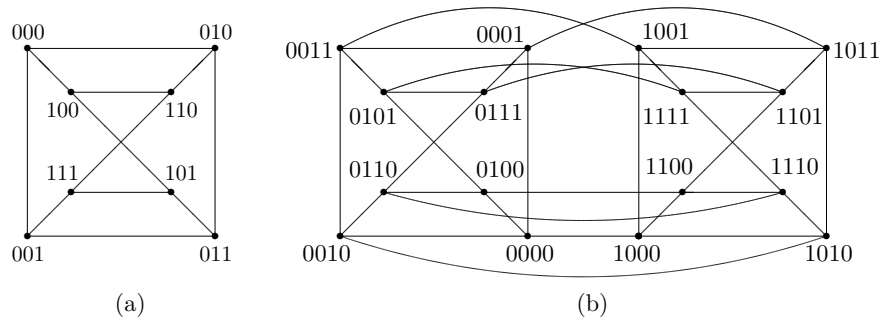
定义 2.3. n 维超立方体 Q_n 递归定义如下。 Q_1 是两个顶点0和1的完全(无向)图。 Q_n 由两个子立方体 Q_{n-1}^0 和 Q_{n-1}^1 组成。 Q_{n-1}^0 和 Q_{n-1}^1 中所有顶点地址的最高位分别为0和1。顶点 $u = u_{n-1}u_{n-2} \dots u_0 \in CQ_{n-1}^0$ 和 $v = v_{n-1}v_{n-2} \dots v_0 \in CQ_{n-1}^1$, 这里 $u_{n-1} = 0$ 和 $v_{n-1} = 1$, 在 Q_n 中相邻接当且仅当 $u_{n-2}u_{n-3} \dots u_0 = v_{n-2}v_{n-3} \dots v_0$ 。

图2.3分别示出了3维和4维超立方体。

对于任意整数 $n \geq 1$, Q_n 的基本性质 [101]有四点: (1) Q_n 是一个 n -正则图。(2) Q_n 的连通度 $\kappa(Q_n) = n$ 。(3) 若 $n \geq 2$, 则 Q_n 不是哈密顿连通的。(4) 超立方体的直径 $D(Q_n) = n$ 。

2.2.2 超立方体的几种主要变型

虽然超立方体具有很多优越性质, 并已在实际中得到广泛的应用, 但这并不意味着它的所有性质都是最优的。例如在连通度相同的情况下, 超立方体的直径相对于其顶点规模来说并不是最优的, 通过改变超立方体中顶点之间的某些边, 可以构造出多种直径近似于超立方体直径一半的超立方体的变型(维数充分大), 如交叉立方体(Crossed Cube) [39]、莫比乌斯立方体(Möbius Cube) [40]、扭立方体(Twisted Cube) [38, 102]、局部扭立方体(Locally Twisted Cube) [41]等。

图 2.4 (a) 3维交叉立方体 CQ_3 ; (b) 4维交叉立方体 CQ_4

一、交叉立方体

交叉立方体是由K. Efe在1992年正式提出的，其定义如下：

定义 2.4. [39] [103] 两个二进制串 $x = x_1x_0$ 和 $y = y_1y_0$ 被称为关联对（表示为 $x \sim y$ ）当且仅当 $(x, y) \in \{(00, 00), (10, 10), (01, 11), (11, 01)\}$ 。

定义 2.5. [39] n 维交叉立方体 CQ_n 递归定义如下。 CQ_1 是两个顶点0和1的完全（无向）图。 CQ_n 由两个子立方体 CQ_{n-1}^0 和 CQ_{n-1}^1 组成。 CQ_{n-1}^0 和 CQ_{n-1}^1 中所有顶点地址的最高位分别为0和1。顶点 $u = u_{n-1}u_{n-2} \dots u_0 \in CQ_{n-1}^0$ 和 $v = v_{n-1}v_{n-2} \dots v_0 \in CQ_{n-1}^1$ ，这里 $u_{n-1} = 0$ 和 $v_{n-1} = 1$ ，在 CQ_n 中相邻接当且仅当

- (1) $u_{n-2} = v_{n-2}$ ，当 n 是偶数，且
- (2) $u_{2i+1}u_{2i} \sim v_{2i+1}v_{2i}$ ，对于 $\lfloor (n-1)/2 \rfloor > i \geq 0$ 。

由定义可知道， CQ_2 是长度为4的无向圈， CQ_3 和 CQ_4 分别如图2.4(a)和(b)所示。

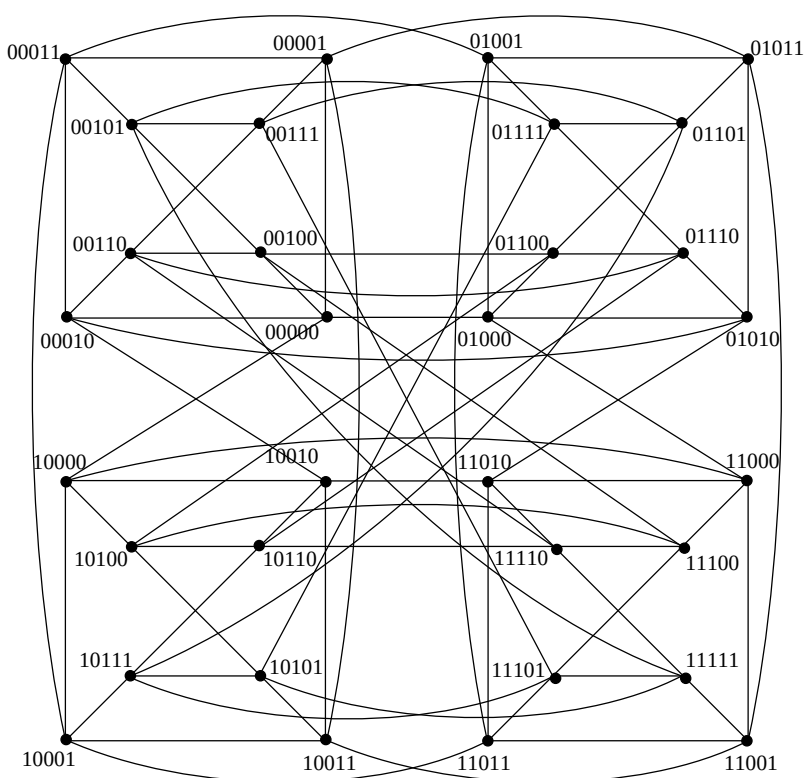
图2.5示出了5维交叉立方体。

二、扭立方体

Hilbers等人于1987年提出的扭立方体（Twisted Cube）[104]。这里我们介绍Fan等在2005年给出的递归定义[105]。在扭立方体的定义中，对于任意整数 $n \geq 1$ ，一个长度为 n 的二进制串 u 被表示为 $u_{n-1}u_{n-2} \dots u_0$ ，这里 $u_i \in \{0, 1\}$ 且 $i \in \{0, 1, \dots, n-1\}$ 。我们定义 $\theta(u, i) = u_i \oplus u_{i-1} \oplus \dots \oplus u_0$ ，这里 $\oplus$ 是异或运算。

扭立方体的递归定义如下：

定义 2.6. [105] 一维的扭立方体 TQ_1 被定义为一个有2个顶点0和1的完全图。对于任意奇数 $n \geq 3$ ， TQ_n 由4个子立方体 TQ_{n-2}^{00} ， TQ_{n-2}^{01} ， TQ_{n-2}^{10} ， TQ_{n-2}^{11} 组成，其中，

图 2.5 5维交叉立方体 CQ_5

TQ_{n-2}^{ab} 和 TQ_{n-2} 同构并且对于 $a, b \in \{0, 1\}$, $V(TQ_{n-2}^{ab}) = \{abx | x \in V(TQ_{n-2})\}$ 并且 $E(TQ_{n-2}^{ab}) = \{(abx, aby) | (x, y) \in E(TQ_{n-2})\}$ 。 $V(TQ_n) = \bigcup_{a,b \in \{0,1\}} V(TQ_{n-2}^{ab})$, 并且 $E(TQ_n) = \bigcup_{a,b \in \{0,1\}} E(TQ_{n-2}^{ab}) \cup E'$, 在这里, $u = u_{n-1}u_{n-2} \dots u_1u_0, v = v_{n-1}v_{n-2} \dots v_1v_0 \in V(TQ_n)$, 如果满足下面三个条件中的一个, 那么 $(u, v) \in E'$ 就成立。

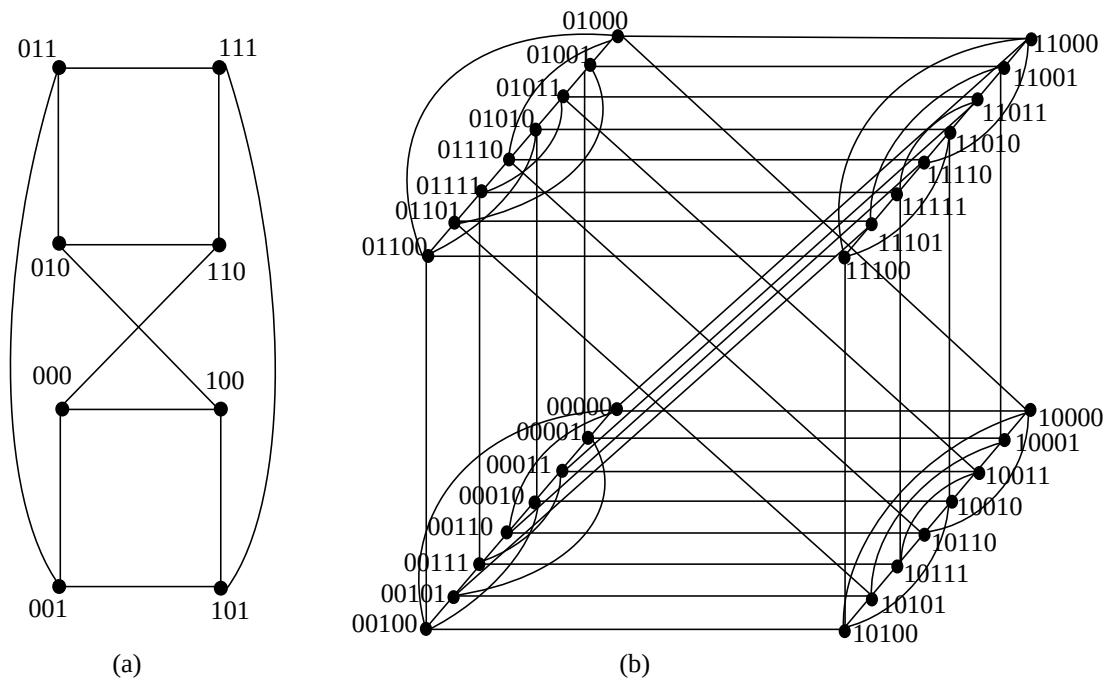
- (1) $u = \overline{v_{n-1}v_{n-2} \dots v_1v_0}$;
- (2) $u = \overline{v_{n-1}v_{n-2}v_{n-3} \dots v_1v_0}$ 并且 $\theta(u, n-3) = 0$;
- (3) $u = v_{n-1}\overline{v_{n-2}v_{n-3} \dots v_1v_0}$ 并且 $\theta(u, n-3) = 1$ 。

3维和5维的扭立方体如图2.6所示。

三、莫比乌斯立方体

n 维莫比乌斯立方体 (Möbius Cube) M_n 有两种类型: 一种是0型- n 维莫比乌斯立方体, 记为 $0-M_n$; 一种是1型- n 维莫比乌斯立方体, 记为 $1-M_n$ 。

定义 2.7. [40] $0-M_1$ 和 $1-M_1$ 都是具有两个顶点0和1的完全图。对于任意整数 $n \geq 2$, $0-M_n$ 和 $1-M_n$ 都包含一个0-型 $(n-1)$ 维的子莫比乌斯立方体 M_{n-1}^0 和一个1-型 $(n-1)$ 维的

图 2.6 (a) TQ_3 ; (b) TQ_5

子莫比乌斯立方体 M_{n-1}^1 , M_{n-1}^0 中所有顶点的最高位为0, M_{n-1}^1 中所有顶点的最高位为1。对于 $x_{n-1} = \overline{y_{n-1}} = 0$ 的两个顶点 $x = x_{n-1}x_{n-2} \dots x_0 \in V(M_{n-1}^0)$ 和 $y = y_{n-1}y_{n-2} \dots y_0 \in V(M_{n-1}^1)$, 存在

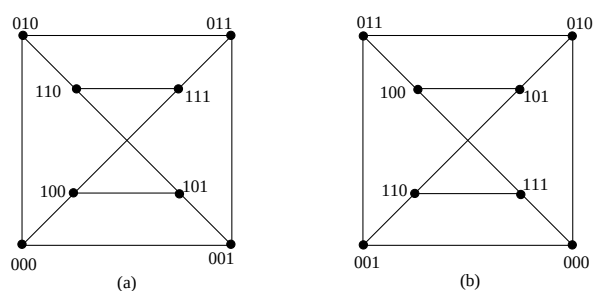
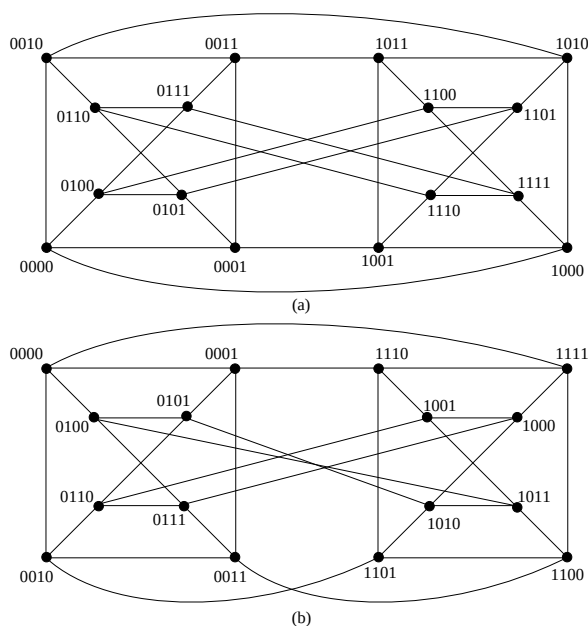
- (1) $(x, y) \in E(0-M_n)$ 当且仅当 $x_i = y_i, i = 0, 1, \dots, n-2$;
- (2) $(x, y) \in E(1-M_n)$ 当且仅当 $x_i = \overline{y_i}, i = 0, 1, \dots, n-2$ 。

0型-3维莫比乌斯立方体 $0-M_3$ 和1型-3维莫比乌斯立方体 $1-M_3$ 分别如图2.7(a)和(b)所示。容易看出, $0-M_3$ 和 $1-M_3$ 是同构的。0型-4维莫比乌斯立方体 $0-M_4$ 和1型-4维莫比乌斯立方体 $1-M_4$ 分别如图2.8(a)和(b)所示。

四、扭曲立方体

Wang等于1999年提出了扭曲立方体 (Twisted-Cube) [106]。为了和扭立方体 (Twisted Cube) 区分, 我们称Twisted-Cube为扭曲立方体, 并将 n 维扭曲立方体在文献中一般记作 TN_n 。它是一个具有 n 个顶点的 n -正则图。对于任意整数 $1 \leq i \leq n$, 令 $e_{n,i}$ (简写为 e_i) 表示一个长度为 n 的二进制串, 其中第 i 位为1, 其它位为0。扭曲立方体的递归定义如下。

- (1) 当 $n = 1, 2$ 时, $u = v + e_i$ (此时也有 $v = u + e_i$), 这里的加法指的是模2加。

图 2.7 (a) $0-M_3$; (b) $1-M_3$ 图 2.8 (a) $0-M_4$; (b) $1-M_4$

(2) 当 $n \geq 3$ 时, TN_n 由两个 $n-1$ 维的扭曲立方体 TN_{n-1}^0 和 TN_{n-1}^1 相互连接而成。

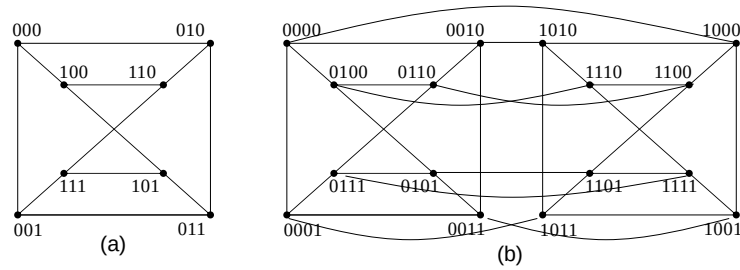
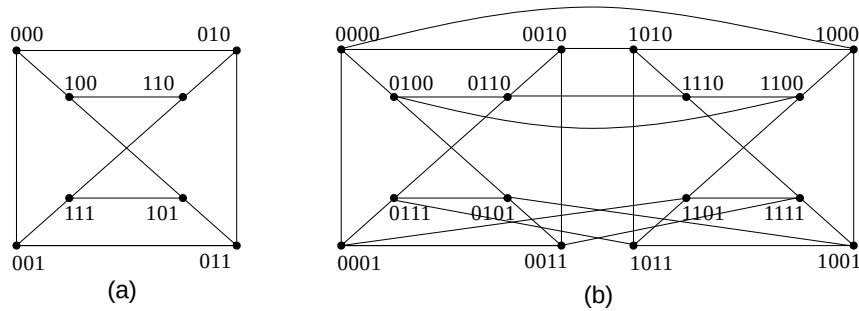
其中 $TN_{n-1}^0 \cong TN_{n-1}^1 \cong TN_{n-1}$ 。对于 TN_{n-1}^0 中的每个顶点 $u = u_n u_{n-1} \dots u_2 u_1$ 和 TN_{n-1}^1 中的对应顶点 $v = v_n v_{n-1} \dots v_2 v_1$ 按下面的方式相连接, 这里的乘法是指的模2乘。

$$v = u + e_n + \sum_{k=1}^{\lceil \frac{n}{2} \rceil - 1} u_{2k-1} \cdot e_{2k} + \sum_{k=3}^{n-1} u_k \cdot e_2. \quad (2.2)$$

3维和4维的扭曲立方体分别如图2.9(a)和(b)所示。

五、局部扭立方体

局部扭立方体 LTQ_n 是 Q_n 的一种重要变型, 由Yang等 [41] 提出。与 n 维超立方体类似, n 维局部扭立方体 LTQ_n 是一个有 2^n 个顶点的 n -正则图。 LTQ_n 的每一个顶点用

图 2.9 (a) TN_3 ; (b) TN_4 图 2.10 (a) 3维局部扭立方体 LTQ_3 ; (b) 4维局部扭立方体 LTQ_4

一个唯一的长度为 n 的二进制串来标识。 LTQ_n 的递归定义如下。

定义 2.8. [41] 对任意整数 $n \geq 2$ ，一个 n 维局部扭立方体 LTQ_n ，被递归定义如下：

(1) LTQ_2 是一个图，由4个顶点组成，标号分别为00, 01, 10 和11，且这4个顶点由4条边相连：(00, 01)，(00, 10)，(01, 11) 和(10, 11)。

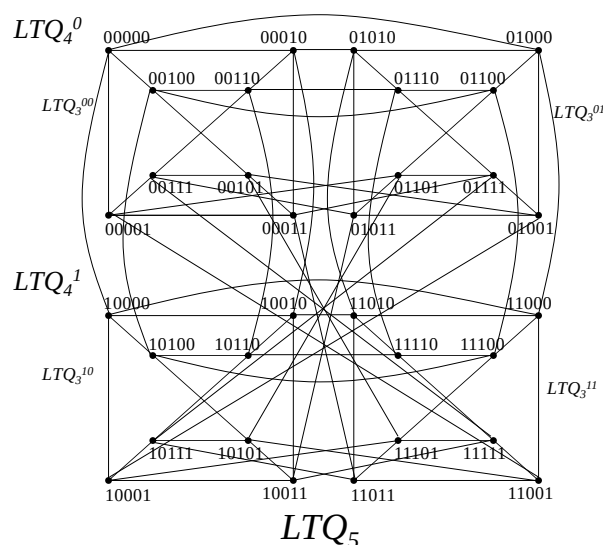
(2) 对于任意整数 $n \geq 3$ ， LTQ_n 由两个 LTQ_{n-1} 的拷贝构造。令 LTQ_{n-1}^0 表示在其中一个拷贝中的每个顶点标号前加0，而 LTQ_{n-1}^1 表示在另一个拷贝中的每个顶点标号前加1所得到的图。将 LTQ_{n-1}^0 的每个顶点 $x = 0x_{n-2}x_{n-3} \dots x_0$ 和 LTQ_{n-1}^1 的顶点 $y = 1(x_{n-2} + x_0)x_{n-3} \dots x_0$ 用边连接，其中‘+’代表了模2加。

LTQ_3 ， LTQ_4 和 LTQ_5 分别如图2.10(a)，图2.10(b)和图2.11所示。

除了上述递归定义之外，局部扭立方体还有另一个等价的非递归定义，具体描述如下：

定义 2.9. [41] 对任意整数 $n \geq 2$ ，一个 n 维局部扭立方体， LTQ_n ，可以被定义如下：

LTQ_n 是一个顶点集为 $\{0, 1\}^n$ 的图。 LTQ_n 的两个顶点 $x = x_{n-1}x_{n-2} \dots x_0$ 和 $y = y_{n-1}y_{n-2} \dots y_0$ 是相邻的当且仅当：

图 2.11 5维局部扭立方体 LTQ_5

(1) 对于某个 $2 \leq k \leq n-1$, $x_k = \overline{y_k}$ 且 $x_{k-1} = y_{k-1} + x_{n-1}$, 并且, 对于其它剩余的位 $x_r = y_r$ 。或者,

(2) 对于某个 $k \in \{0, 1\}$, $x_k = \overline{y_k}$, 并且对于所有其它剩余位 $x_r = y_r$ 。

从上述 LTQ_n 的两个定义, 我们可以得知, 在 LTQ_n 中, 任意两个相邻的顶点最多只有两位不同。

2.2.3 超立方体变型的性质

如前所述, 交叉立方体、莫比乌斯立方体、扭立方体、局部扭立方体等都是超立方体的特殊变型 (在扭曲立方体上的研究成果较少, 故不作介绍)。 CQ_n, M_n, TQ_n 和 LTQ_n 与 Q_n 一样, 都是具有 2^n 个顶点和 $n2^{n-1}$ 条边的 n -连通 n -正则图。 CQ_n, M_n, TQ_n, LTQ_n 还有比超立方体优越的其它性质如哈密顿性质、路径构造、圈构造、宽直径和故障直径等。

在哈密顿性质方面, Park 等证明了一类包括 CQ_n, M_n, TQ_n 等在内的若干的变型都是哈密顿的和哈密顿连通的 [107]。 Hsieh 等证明, 若 LTQ_n 的每个顶点关联至少两条无故障边, 则 LTQ_n 是 $(2n-5)$ -哈密顿的 ($n \geq 3$) [108]。 Park 等证明: 一类包括 CQ_n, M_n, TQ_n 等在内的若干 Q_n 的变型都是 $(n-2)$ -哈密顿的和 $(n-3)$ -哈密顿连通的 ($n \geq 3$) [109]。 而超立方体是一个二分图, 它不是哈密顿连通的, 因此它的哈密顿性质不如这些变型。

在路径构造和圈构造方面,文献[110–113]中证明:对任意两个顶点 $x, y \in V(H)$ 和任意整数 l , $\text{dist}(H, x, y) + 2 \leq l \leq 2^n - 1$, H 中能够构造长度为 l 的路径,其中 $H \in \{CQ_n, M_n, TQ_n, LTQ_n\}$ ($n \geq 3$)。需要指出的是,由于 Q_n 的任何两个顶点之间或者只能构造奇长路径(若这两个顶点间的距离为奇数)或者只能构造偶长路径(若这两个顶点间的距离为偶数)。因此, CQ_n, M_n, TQ_n, LTQ_n 在路径构造方面的性质优于 Q_n 。[114]证明了包括 CQ_n, M_n, TQ_n, LTQ_n 在内的若干超立方体的变型都是4-边泛圈的 ($n \geq 2$)。因为 Q_n 不包含奇长圈,所以 Q_n 的变型 CQ_n, M_n, TQ_n, LTQ_n 在圈构造方面的性质也优于 Q_n 。

关于完全二叉树构造,文献[52]和[115]分别证明了 CQ_n 和 TQ_n 中分别能够构造一棵完全二叉树。根据文献[116]和[117]分别在 Q_n 和 LTQ_n 上得到的完全二叉树的构造结果, Q_n 和 LTQ_n 模拟树网络的能力不如 CQ_n 和 TQ_n 。

在宽直径和故障直径方面,文献[38]和[118]分别证明了 TQ_n 和 CQ_n 的宽直径和故障直径大约是 Q_n 的一半。进一步,[119]证明了 M_n 的宽直径至多是 $\lceil (n+2)/2 \rceil + 2$ 。但是, Q_n 的宽直径和故障直径都是 $n+1$, 相对较大[101]。

2.3 BC网络及其相关性质

2.3.1 BC网络的定义

一个 n 维BC网络,表示为 X_n ,是一个具有 2^n 个顶点的 n -正则图。我们将 X_n 的每一个顶点表示为长度为 n 的二进制串 $x_{n-1}x_{n-2}\dots x_0$ 。 x 的第 i 位 x_i 能够表示为 $\text{bit}(x, i)$ 。我们引入一一对应连接和BC网络的定义如下。

定义 2.10. [15] 令 G 为一个网络,满足 $V(G) = V_1 \cup V_2$, 这里 $V_1 \neq \emptyset$, $V_2 \neq \emptyset$, 且 $V_1 \cap V_2 = \emptyset$ 。如果下面两个条件成立:

- (1) 对于任一顶点 $u \in V_1$, 存在唯一顶点 $v \in V_2$ 使得 $(u, v) \in E(G)$ 和
- (2) 对于任一顶点 $v' \in V_2$, 存在一个唯一顶点 $u' \in V_1$ 使得 $(u', v') \in E(G)$,

那么在集合 V_1 和 V_2 之间存在一个一一对应连接 (Bijective Connection, 简写为BC), 表示为 $V_1 \xleftrightarrow{G} V_2$ 。

定义 2.11. [15] 一维BC网络 X_1 是基于两个顶点0和1的完全图。1维BC网络族被定义为 $\mathcal{L}_1 = \{X_1\}$ 。令 G 是一个网络。 G 是一个 n 维BC网络,表示为 X_n , 如果存在 $V_0, V_1 \subset V(G)$ 使得如下条件成立:

(1) $V_0 = 0V'_0$ 和 $V_1 = 1V'_1$, 这里 $V'_0 = V'_1 = \{0, 1\}^{n-1}$;

(2) $V(G) = V_0 \cup V_1$, $V_0 \neq \emptyset$, $V_1 \neq \emptyset$, $V_0 \cap V_1 = \emptyset$ 且

(3) $V_0 \xleftrightarrow{G} V_1$, $G[V_0] \in \mathcal{L}_{n-1}$ 和 $G[V_1] \in \mathcal{L}_{n-1}$ 。

n 维BC网络族被定义为 $\mathcal{L}_n = \{G | G \text{ 是一个 } n \text{ 维 BC 网络}\}$ 。

2.3.2 BC网络的相关性质

近年来, BC网络受到了研究者的关注, 其已被证明的主要性质和一些相关的研究成果如下:

(1) 令 $X_n \in \mathcal{L}_n$, $D(X_n) \leq n$, $\kappa(x_n) = n$, $\lambda(x_n) = n$ [15]。

(2) X_n 是 n -可诊断的 [43]。

(3) X_n 是 $(2n-2)/(2n-2)$ -可诊断的 [43]。

(4) X_n 是一个 Hamilton 图 [43]。

(5) 对于 $n \geq 4$, $x, y \in V(X_n)$ 且 $x \neq y$, X_n 中能够构造长度为 $\text{dist}(X_n, x, y) + 2 \leq l \leq 2^n - 1$ 的圈。

(6) X_n 是边泛圈的。

(7) 假设 X_n 上每一个顶点至少有一个无故障邻接边且故障边的数量 $|F|$ 不超过 $2n-3$, 文献 [45] 给出一个算法, 能在 $O(n)$ 时间内找到 X_n 上任意两个顶点之间长度至多 $n + 3\lceil \log_2 |F| \rceil$ 的无故障的路径。

(8) 文献 [46] 提出了一类新型立方体——螺旋立方体 (Spined Cube), 其直径仅为 $\lceil n/3 \rceil + 3$, 对于任意整数 $n \geq 14$ 。该结构首次证明了超立方体的一类变型具有如此小的直径。

(9) 文献 [120] 证明了 BC 网络的故障直径的上界为 $n + \lceil \log(|F_n| + 1) \rceil$ 。

(10) X_n 是点泛圈的 [44]。

(11) X_n 是超级连通的 (Super-Connected) [44]。

(12) X_n 是超级边-连通的 (Super-Edge-Connected) [44]。

(13) 文献 [47] 给出了一个高效算法, 能够在 $O(N)$ 时间内诊断出 X_n 上所有的故障处理器, 这里 $N = 2^n$ 且 $n \geq 4$ 。

(14) 文献 [48] 证明了 X_n 的超级连通度是 $2n-2$ ($n \geq 3$), 条件连通度为 $4n-7$ ($n \geq 5$)。

(15) 文献 [49]证明了 X_n 的 h -额外连通度 (h -Extra Connectivity) 是 $n(h+1)-h(h+3)/2$ 。

2.4 本章小结

本章首先介绍了本文需要用到的一些图论基本概念和符号表示, 然后引入了超立方体及其几种主要变型的概念和定义。最后, 给出了BC网络的定义以及其性质。

第三章 交叉立方体上IST的串行构造

作为并行计算中一种重要的网络拓扑结构,交叉立方体在哈密尔顿性质、路径、圈、完全二叉树构造及宽直径和故障直径构造方面具有比超立方体更优越的性质。近年来,其又有若干新的性质被发现,如在路由方面表现出较大的优越性——Chang等提出了 CQ_n 上保持距离关联对的定义,并且根据该定义所设计的方法,给定任意两个顶点的地址,可以快速计算出这两个顶点之间的距离 [118]。喻昕等基于Chang等的算法,在构造路径的每一步时进一步考虑包含全部可进行最短路径路由的边,提出了时间复杂度为 $O(n^2)$ 最短路由算法 [5]。Chang等证明了 CQ_n 的条件故障直径仅仅比其故障直径大1 [121]。Wang给出了 CQ_n 上一个改进的哈密尔顿圈构造算法 [122]以及 CQ_n 中包含所有顶点和最少边数的 $\kappa(1 \leq \kappa \leq n)$ 连通子网络的构造算法 [123]。Dong等、Lai等和Yang等分别给出了 CQ_n 上 $N \times N$ 网格树 (Mesh Tree)、网格和多维环绕的构造算法 [68, 124, 125]。

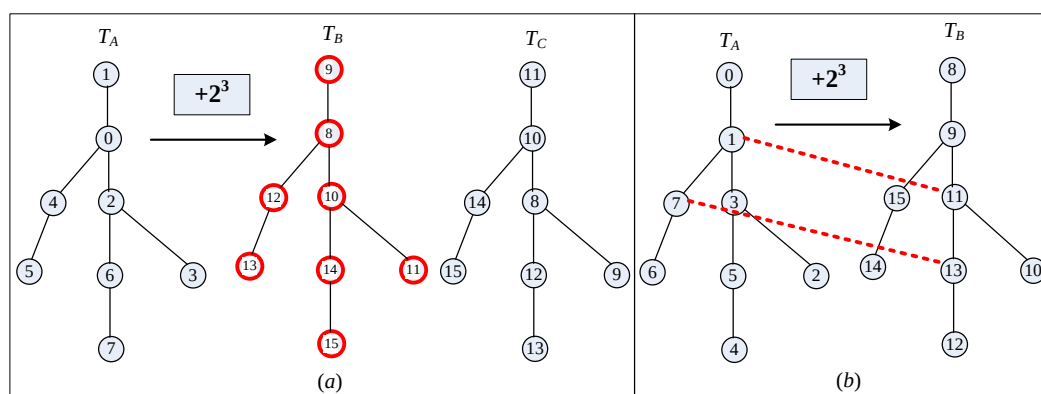
n 维超立方体是点可迁图,其上任一顶点都是等价的,从而超立方体上独立生成树的构造问题转换为其上以顶点0为根的IST构造问题。但是,交叉立方体又被称为多重扭立方体 [126, 127], n 维交叉立方体 CQ_n 上具有 $2^{\lceil (n-4)/2 \rceil}$ 个顶点等价类,这里 $n \geq 4$ [127, 128],从而使得其上IST的构造比超立方体更加复杂。在本章中,我们研究交叉立方体上IST的串行构造方法,主要取得如下结果:

(1) 作为超立方体的变型,交叉立方体不能采用超立方体上相同的地址变换方法。我们根据交叉立方体特有的构造方法,证明了其满足一个重要的地址变换性质,依据该地址变换性质给出了交叉立方体上同构树的构造方法。

(2) 给出基于地址变换的扩展顶点集合和扩展顶点集合划分的定义。指出了扩展顶点集合和扩展顶点集合划分的不唯一性并指出不同的扩展顶点集合划分将有可能导致不同的IST的构造。

(3) 证明了交叉立方体上以任一顶点为根的IST的存在性,设计了时间复杂度为 $O(N \log^2 N)$ 的算法。

此外,我们探讨了交叉立方体上的IST的优化构造问题,如能够构造出同构的且最大高度较小的一组IST。

图 3.1 Q_n 上地址变换方法不适用于 CQ_n (a) 1 为根; (b) 0 为根

3.1 地址变换方法

3.1.1 CQ_{n-1}^0 和 CQ_{n-1}^1 上同构树构造分析

我们首先来分析 Q_n , 其由 Q_{n-1}^0 和 Q_{n-1}^1 组成。给定 Q_{n-1}^0 上以 1 为根的生成树 T_A , 将 T_A 中的每个顶点加上值 2^n , 即可得到 Q_{n-1}^1 上以 1 的 n 维邻接点为根的生成树 T_B 。 T_A 中每一棵树的顶点与其在 T_B 中的邻接点均处于与其对称 (我们称到根的距离相同的两个顶点为对称顶点) 的位置。如给定 Q_3^0 上以 1 为根的一棵生成树 T_A , 将 T_A 中的每个顶点加上值 $2^3 = 8$, 即可得到 Q_3^1 上以 9 为根的一棵生成树 T_B 。但是, 在 CQ_n 上, 顶点等价类具有 $2^{\lceil (n-4)/2 \rceil}$ 个顶点等价类, 这里 $n \geq 4$ [127, 128]。若采用同样的方法, 将 CQ_{n-1}^0 上树 T_A 中每个顶点加 2^n 得到 T_B 。经验证, 有如下两个问题:

(1) 对某些顶点等价类而言, T_A 的根与 T_B 的根不再是邻接点。

若给定 CQ_3^0 上以顶点 1 为根的一棵生成树, 将 T_A 中的每个顶点加上值 $2^3 = 8$, 不能得到 Q_3^1 上以 11 为根的一棵生成树 T_B , 且 T_B 的根顶点 9 不再是 T_A 的根顶点 1 的邻接点, 如图 3.1(a) 所示。但若给定 CQ_3^0 上以顶点 0 为根的一棵生成树, 将 T_A 中的每个顶点加上值 $2^3 = 8$, 即可得到 Q_3^1 上以 8 为根的一棵生成树 T_B , 如图 3.1(b) 所示。

(2) T_A 中任一顶点的邻接点将有可能不在 T_B 中与之对称的位置上。

如图 3.1(b) 中, 顶点 0 与其邻接点 8 在对称的位置上, 但顶点 1 与其邻接点 11 却不在对称的位置。顶点 1 到根顶点的距离为 1, 而其邻接点 11 到根顶点的距离为 2。在该情况下, 如何选择一个顶点集合, 使得其在 T_A 和 T_B 中的构成的子树处于对称的位置是我们需要探讨的问题。

因此, 我们拟结合交叉立方体的构造规律, 设计新的适合于其上任一顶点的地

址变换方法。

3.1.2 地址变换方法

根据定义 2.4, 下面的推论成立。

推论 3.1. 令 x_1x_0 和 y_1y_0 是两个长度为 2 的二进制字符串。如果 $x_1x_0 \sim y_1y_0$, 那么 $\overline{x_1}x_0 \sim \overline{y_1}y_0$ 。

证明. 如果 $x_1x_0 \sim y_1y_0$, 那么根据定义 2.4, $(x_1x_0, y_1y_0) \in \{(00, 00), (10, 10), (01, 11), (11, 01)\}$ 。我们容易验证 $(\overline{x_1}x_0, \overline{y_1}y_0) \in \{(00, 00), (10, 10), (01, 11), (11, 01)\}$ 。因此, $\overline{x_1}x_0 \sim \overline{y_1}y_0$ 。□

定义 3.1. [52] 对于任意整数 m ($1 \leq m \leq n-1$) 和二进制串 $s \in \{0, 1\}^m$, $CQ_n(s)$ 被定义为 CQ_n 中由具有共同前缀 s 的顶点导出的子图。

引理 3.2. [52] $CQ_n(s)$ 同构于 $CQ_{n-|s|}$, 这里 $|s|$ 代表二进制串 s 的长度。

根据引理 3.2, 对于任意整数 $n \geq 2$, 如果 $u = u_{n-1}u_{n-2} \dots u_0$ 是 CQ_n 中的任一顶点, 那么 u 包含于 CQ_n 的子图: $CQ_n(u_{n-1}u_{n-2} \dots u_{n-i})$, 对于任意整数 i ($1 \leq i \leq n-1$)。

下面的定理, 最早出现于文献 [39], 用于描述如何检查 CQ_n 中给定的一对顶点是否相邻接。在本文中, 该定理可以被认为是交叉立方体的非递归定义。

引理 3.3. [39] 对于任意整数 $n \geq 1$, $(u_{n-1}u_{n-2} \dots u_0, v_{n-1}v_{n-2} \dots v_0)$ 是 CQ_n 中的一条边当且仅当存在一个整数 l , 使得

- (1) $u_{n-1}u_{n-2} \dots u_l = v_{n-1}v_{n-2} \dots v_l$,
- (2) $u_{l-1} \neq v_{l-1}$,
- (3) $u_{l-2} = v_{l-2}$ 如果 l 是偶数, 且
- (4) 对于 $0 \leq i < \lfloor (l-1)/2 \rfloor$, $u_{2i+1}u_{2i} \sim v_{2i+1}v_{2i}$ 。

当引理 3.3 的条件 1 和条件 2 同时成立时, 我们称 u 和 v 具有一个最左不同位 $l-1$, 记为 $LDF(u, v) = l-1$ 。当两个邻接点 u 和 v 具有一个最左不同位 d , 我们说 v 是 u 的 d 维邻接点或边 (u, v) 是 d 维边。我们用 $N(u, d)$ 表示 u 的 d 维邻接点。进一步, 对于集合 $W \subseteq V(CQ_n)$, $N(W, d) = \{N(w, d) | w \in W\}$ 表示 W 中所有顶点的 d 维邻接点的集合。

假设 CQ_n 中有 n 棵以顶点 u 为根的 IST $T_1, T_2, \dots, T_n$ 。对于 T_i 中的顶点 $v \neq u$ ($1 \leq i \leq n$), 我们定义 $\text{parent}(v, T_i)$ 为顶点 v 的父顶点, $\text{ancestor}(v, T_i)$ 为 T_i 中 v 的祖先

顶点的集合, 该集合为 T_i 中从顶点 $\text{parent}(v, T_i)$ 到 u 这条路径中所有顶点的集合。我们把 ancestor 的定义进一步扩展到路径中, 令路径 $P: u = x^{(0)}, x^{(1)}, \dots, x^{(i-1)}, x^{(i)} = v, x^{(i+1)}, x^{(i+2)}, \dots, x^{(k)} (0 \leq i \leq k)$ 。顶点 v 在路径 P 中的祖先集合被定义为 $\text{ancestor}(v, P)$, 是顶点集合 $\{u = u^{(0)}, u^{(1)}, \dots, u^{(i-1)}\}$ 。

需要指出的是, 最左不同位、 d 维邻接点、 d 维边、 $N(u, d)$ 、 $N(W, d)$ 、 $\text{parent}(v, T_i)$ 及 $\text{ancestor}(v, T_i)$ 等定义不仅适用于交叉立方体, 也适用于任意BC网络, 在后面的章节中将不再重复定义。

定义 3.2. [52] CQ_n 的保持维的自同构是 $V(CQ_n)$ 上的一个映射 ψ , 使得 (x, y) 是 CQ_n 上的 d 维边当且仅当 $(\psi(x), \psi(y))$ 是 CQ_n 上的 d 维边, 对于 $x, y \in V(CQ_n)$ 。如果 CQ_n 上存在一个保持维的自同构能够将顶点 u 映射到顶点 v , 那么顶点 u 和 v 被称为相似顶点。

接下来的定理给出了 CQ_n 上的一个保持维的自同构。

引理 3.4. 对于整数 $k (0 \leq k \leq \lfloor (n-2)/2 \rfloor)$, 从 $V(CQ_n)$ 到 $V(CQ_n)$ 的映射被定义如下:

$\psi(x) = x_{n-1}x_{n-2}x_{n-3}x_{n-4} \dots x_{2k+3}x_{2k+2}\overline{x_{2k+1}}x_{2k}x_{2k-1}x_{2k-2} \dots x_1x_0$, 对于任意顶点 $x \in V(CQ_n)$,

那么, ψ 是 CQ_n 上保持维的自同构, x 和 $\psi(x)$ 为相似顶点。

证明. 显然, ψ 是 $V(CQ_n)$ 上的映射。对于 CQ_n 上的 d 维边 (a, b) , $0 \leq d \leq n-1$, 令 $a = a_{n-1}a_{n-2} \dots a_0$ 和 $b = b_{n-1}b_{n-2} \dots b_0$ 。根据引理 3.3, 下面四个条件成立。

$$a_{n-1}a_{n-2} \dots a_{d+1} = b_{n-1}b_{n-2} \dots b_{d+1} \quad (2.1)$$

$$a_d \neq b_d \quad (2.2)$$

$$a_{d-1} = b_{d-1} \text{ 如果 } d+1 \text{ 是偶数} \quad (2.3)$$

$$\text{对于 } 0 \leq i < \lfloor d/2 \rfloor, a_{2i+1}a_{2i} \sim b_{2i+1}b_{2i} \quad (2.4)$$

令 $p = p_{n-1}p_{n-2} \dots p_0 = \psi(a) = a_{n-1}a_{n-2} \dots \overline{a_{2k+1}}a_{2k} \dots a_0$ 和 $q = q_{n-1}q_{n-2} \dots q_0 = \psi(b) = b_{n-1}b_{n-2} \dots \overline{b_{2k+1}}b_{2k} \dots b_0$ 。我们有如下情形:

情形1. $d \leq 2k+1$ 。那么下面四个条件成立。

$$p_{n-1}p_{n-2} \dots p_{d+1} = q_{n-1}q_{n-2} \dots q_{d+1} \quad (2.5)$$

$$p_d \neq q_d \quad (2.6)$$

$$p_{d-1} = q_{d-1} \text{ 如果 } d+1 \text{ 是偶数} \quad (2.7)$$

$$\text{对于 } 0 \leq i < \lfloor d/2 \rfloor, p_{2i+1}p_{2i} \sim q_{2i+1}q_{2i} \quad (2.8)$$

情形2. $d \geq 2k + 2$ 。显然, (2.5) 到(2.7) 仍然成立。既然 $a_{2k+1}a_{2k} \sim b_{2k+1}b_{2k}$, 根据推论 3.1, 我们有 $\overline{a_{2k+1}a_{2k}} \sim \overline{b_{2k+1}b_{2k}}$, 即 $p_{2k+1}p_{2k} \sim q_{2k+1}q_{2k}$ 。因此, (2.8) 仍然成立。

综上所述, 根据引理 3.3, (p, q) 是 d 维边。引理成立。

3.1.3 依据地址变换方法构造同构树

接下来, 我们使用 $u = u_{n-1}u_{n-2} \dots u_0$ 表示 CQ_n 上任意选中的顶点且我们也能够用十进制表示例子中涉及 CQ_n 上顶点的地址。进一步, 我们总是令 u 表示 CQ_n 上 IST 的根顶点。

对于整数 $n \geq 2$, 给定 $CQ_{n-1}^{u_{n-1}}$ 上以顶点 u 为根的一棵生成树, 依据算法 CIOT, 我们将得到 $CQ_{n-1}^{u_{n-1}}$ 中一棵以顶点 $N(u, n-1)$ 为根的树 T_B 且 T_B 同构于 T_A 。

算法 1 CIOT

输入: $CQ_{n-1}^{u_{n-1}}$ 上一棵以顶点 $u = u_{n-1}u_{n-2} \dots u_0$ 为根的生成树, 这里 $n \geq 2$;

输出: 一棵树 T_B ;

开始

步骤 1:

1: 将 T_A 中每一个顶点的第 $(n-1)$ 位转换成其补集, 我们得到一棵树, 表示为 T_A' ;

步骤 2:

2: **for** $k = 0$ to $\lfloor (n-1)/2 \rfloor - 1$ **do**

3: **if** $u_{2k} = 1$ **then**

4: 对于任一顶点 $v \in V(T_A')$, 将其第 $(2k+1)$ 位转换成其补集;

5: **end if**

6: **end for**

我们得到一棵树 T_B ;

结束

在图 3.2 中, (a) 和 (b) 分别示出了 CQ_3^1 上对应于 CQ_3^0 上以 0000 和 0001 为根顶点的 T_A 的 T_B 的构造。

表示法 3.1. 我们用 η 来表示由算法 CIOT 定义的从 T_A 到 T_B 的映射。

根据算法 CIOT 和引理 3.4, 我们能够证明下面的引理。

引理 3.5. 对于任意整数 $n \geq 2$, 令 T_A 和 u 为算法 CIOT 的输入, T_B 为算法 CIOT 的输出。那么, 下列结论成立:

(1) $\eta(u) = N(u, n-1)$ 。

(2) T_B 是 $CQ_{n-1}^{u_{n-1}}$ 上一棵以顶点 $N(u, n-1)$ 为根的同构于 T_A 的生成树。

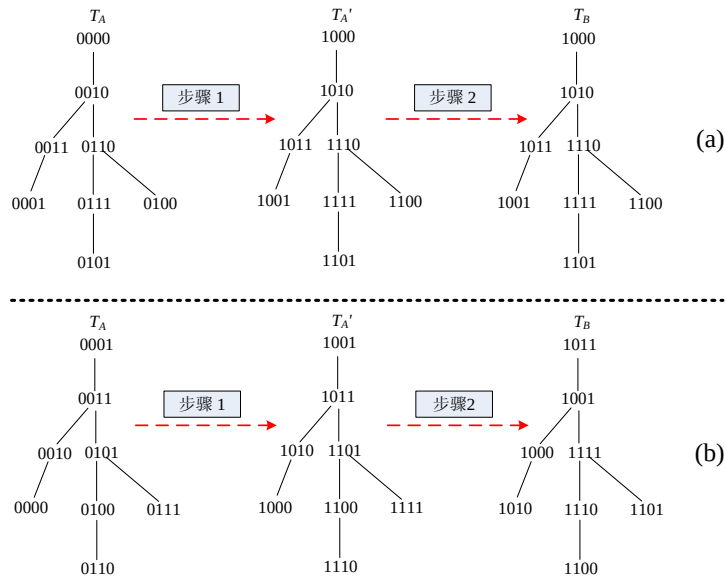


图 3.2 CQ_3^1 上根据算法CIOT构造出一棵生成树 T_B , 这里(a) T_A 是 CQ_3^0 上以顶点0000为根的生成树; (b) T_A 是 CQ_3^0 上以顶点0001为根的生成树

3.2 IST的构造

在本节中, 依据上节的研究成果, 我们首先给出一些关于 CQ_n 上以任一顶点为根的 n 棵IST的预备成果, 然后, 我们提供一个递归算法来构造 CQ_n 上以任一顶点为根的 n 棵IST并且分析算法的时间复杂度。

为方便构造 CQ_n 上以 u 为根的 n 棵IST, 我们给出如下两个定义和两个推论。

定义 3.3. 对于整数 $n \geq 2$, 假设输入 T_A 是 $CQ_{n-1}^{u_{n-1}}$ 上一棵以 u 为根的生成树, η 是由算法CIOT定义的同构映射。如果存在一个非空集合 $W \subseteq V(T_A) \setminus \{u\}$ 使得

- (1) 对于 $v \in W$, $\text{ancestor}(v, T_A) \setminus \{u\} \subseteq W$ 且
- (2) $\eta(W) = N(W, n-1)$, 这里 $\eta(W) = \{\eta(v) | v \in W\}$,

那么我们称 W 为一个对应于生成树 T_A 的扩展顶点集合 (Extendable Vertex Set, 简称EVS)。

需要指出的是, 对于任一生成树 T_A , 有可能存在多个与其对应的EVS。在图 3.3中, $\{0, 2\}$, $\{0, 2, 3\}$, $\{0, 2, 4, 6\}$, $\{0, 2, 3, 4, 6\}$, $\{0, 2, 4, 5, 6\}$, $\{0, 2, 4, 6, 7\}$, $\{0, 2, 3, 4, 6, 7\}$, $\{0, 2, 3, 4, 5, 6\}$, $\{0, 2, 4, 5, 6, 7\}$ 和 $\{0, 2, 3, 4, 5, 6, 7\}$ 皆是 CQ_3^0 上对应于以顶点1为根的树 T_A 的EVS, 这里 T_A 和 T_B 分别是算法CIOT的输入和输出。

显然, 下面的推论成立。

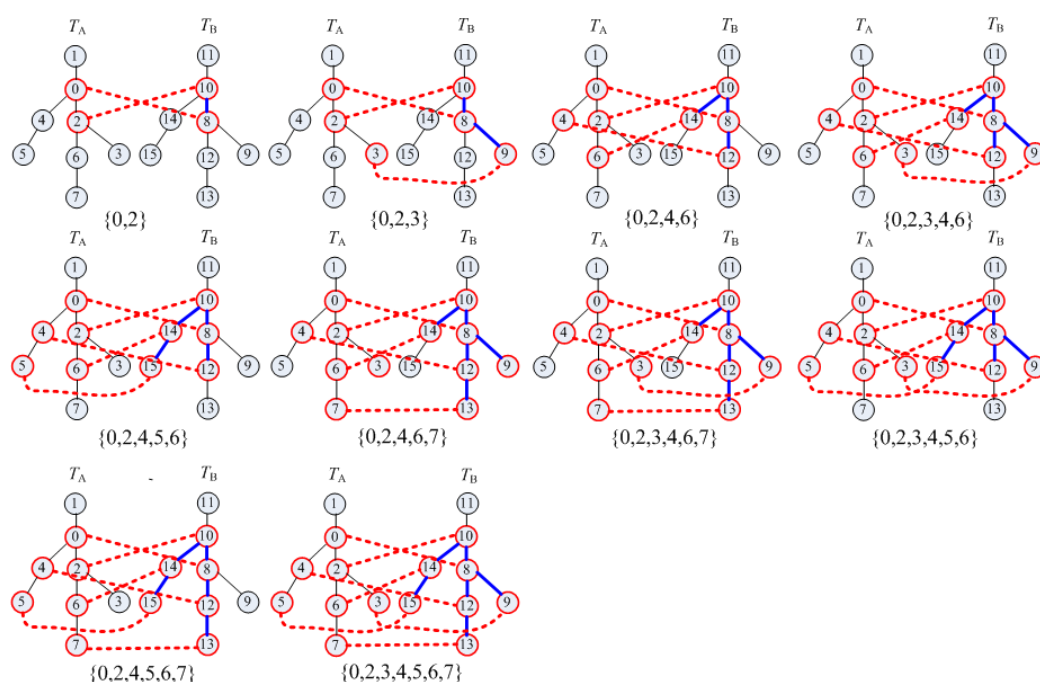


图 3.3 一些对应于 CQ_3^0 上以顶点1为根的 T_A 的 EVS, 这里 T_A 和 T_B 分别是算法 CIOT 的输入和输出

推论 3.6. 令 T_A 是 $CQ_{n-1}^{u_{n-1}}$ 上以顶点 u 为根的一棵生成树。如果存在一个对应于 T_A 的 EVS W , 那么 $T_A[W \cup \{u\}]$ 是以顶点 u 为根的一棵树。

定义 3.4. 对于任意整数 $n \geq 2$, 假设 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 是 $CQ_{n-1}^{u_{n-1}}$ 上 $n-1$ 棵以顶点 u 为根的 IST, $W_1, W_2, \dots, W_{n-1}$ 是分别对应于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 的 EVS。如果下面两个条件成立:

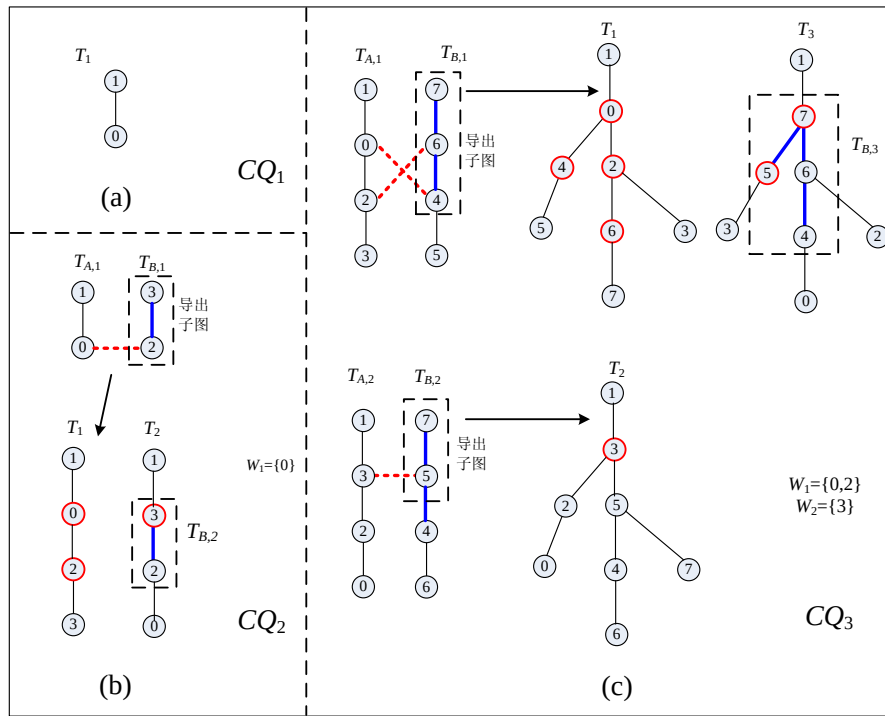
- (1) 对于任意两个整数 i 和 j ($1 \leq i < j \leq n-1$), $W_i \cap W_j = \emptyset$ 且
- (2) $\bigcup_{i=1}^{n-1} W_i \cup \{u\} = V(CQ_{n-1}^{u_{n-1}})$,

那么我们称 $\{\{u\}, W_1, W_2, \dots, W_{n-1}\}$ 是 $V(CQ_{n-1}^{u_{n-1}})$ 上的一个扩展顶点集合划分 (Extendable Vertex Set Partition, 简称 EVSP)。

例如, 在图 3.4(b) 中, $\{\{1\}, \{0\}\}$ 是 $V(CQ_1^0)$ 上的一个 EVSP, 这里 $\{0\}$ 是以 CQ_1^0 上以 1 为根的 $T_{A,1}$ 的一个 EVS。在图 3.4(c) 中, $\{\{1\}, \{0, 2\}, \{3\}\}$ 是 $V(CQ_2^0)$ 上的一个 EVSP, 这里 $\{0, 2\}$ 和 $\{3\}$ 分别是对应于 CQ_2^0 上以 1 为根的树 $T_{A,1}$ 和 $T_{A,2}$ 的 EVS。

基于上述结果, 我们有如下推论。

推论 3.7. 对于任意整数 $n \geq 2$, 令 $CQ_{n-1}^{u_{n-1}}$ 上一棵以 u 为根的生成树 T_A 是算法 CIOT 的

图 3.4 CQ_n 上以顶点 $0^{n-1}1$ 为根的 IST $T_1, T_2, \dots, T_n$, 对于 $n = 1, 2$ 和 3

输入, T_B 是算法 $CIOT$ 的输出。若 W 是对应于 T_A 的一个 EVS , 那么 $T_B[N(W \cup \{u\}, n - 1)]$ 是 $CQ_{n-1}^{\overline{u_{n-1}}}$ 上一棵以顶点 $N(u, n - 1)$ 为根的树。

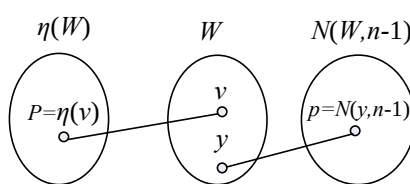
证明. 根据推论 3.6, $T_A[W \cup \{u\}]$ 是 $CQ_{n-1}^{\overline{u_{n-1}}}$ 上一棵以顶点 u 为根的树。根据引理 3.5, $\eta(u) = N(u, n - 1)$ 且 T_B 是 $CQ_{n-1}^{\overline{u_{n-1}}}$ 上一棵以顶点 $N(u, n - 1)$ 为根的同构于 T_A 的树。根据定义 3.3, $\eta(W) = N(W, n - 1)$ 。从而 $\eta(W \cup \{u\}) = \eta(W) \cup \{\eta(u)\} = N(W \cup \{u\}, n - 1)$ 。因此, $T_B[\eta(W \cup \{u\})] = T_B[N(W \cup \{u\}, n - 1)]$ 是 $CQ_{n-1}^{\overline{u_{n-1}}}$ 上以顶点 $N(u, n - 1)$ 为根的一棵树。

令 $u = 1$ (十进制表示), CQ_3 上的 $T_{B,1}[N(W_1 \cup \{1\}, 2)]$ 和 $T_{B,2}[N(W_2 \cup \{1\}, 2)]$ 是以顶点 $N(1, 2) = 7$ 为根的两棵树, 如图 3.4 (c) 中虚线包围的子图所示。

接下来, 我们给出一个关键引理。该引理提供一个方法来验证一类顶点的集合是否是一个 $CQ_{n-1}^{\overline{u_{n-1}}}$ 上一棵生成树的 EVS , 这里 $n \geq 2$ 。

引理 3.8. 对于任意整数 $n \geq 2$, 如果 T_A 是 $CQ_{n-1}^{\overline{u_{n-1}}}$ 上以 u 为根的一棵生成树且存在一个顶点集合 $W \subseteq V(T_A) \setminus \{u\}$, 使得如下两个条件成立:

- (1) 对于顶点 $v \in W$, $\text{ancestor}(v, T_A) \setminus \{u\} \subseteq W$ 且
- (2) 对于任一顶点 $x_{n-1}x_{n-2} \dots x_0 \in W$, 整数 j ($0 \leq j < \lfloor (n-1)/2 \rfloor$), 且 $x_{2j} = \overline{u_{2j}}$,

图 3.5 $\eta(W) \subseteq N(W, n-1)$ 的证明

$x_{n-1}x_{n-2} \dots x_{2j+3}x_{2j+2}\overline{x_{2j+1}}x_{2j}x_{2j-1}x_{2j-2} \dots x_1x_0 \in W$,

那么, W 是对应于 T_A 的一个 EVS。

证明. 根据定义 3.3, 我们只需证明 $\eta(W) = N(W, n-1)$ 。根据定义 2.5, $CQ_{n-1}^{u_{n-1}}$ 中的每一个顶点有且仅有一个 $(n-1)$ 维邻接点。注意到 η 是由算法 CIOT 提供的从 T_A 到 T_B 的同构映射。那么, 我们有 $|\eta(W)| = |N(W, n-1)| = |W|$, 意味着如果 $\eta(W) \subseteq N(W, n-1)$, 那么 $\eta(W) = N(W, n-1)$ 。结论 $\eta(W) \subseteq N(W, n-1)$ 的证明如下。

我们只需证明对于任一顶点 $p \in \eta(W)$, 存在一个顶点 $y \in W$, 使得 $p = N(y, n-1)$ 。注意到 η 是由算法 CIOT 提供的从 T_A 到 T_B 的同构映射, 存在一个顶点 $v = v_{n-1}v_{n-2} \dots v_0 \in W$ 使得 $\eta(v) = p$, 如图 3.5 所示。根据该引理中的条件, 我们有 $v_{n-1} = u_{n-1}$ 。构造 $y = u_{n-1}y_{n-2} \dots y_0$ 如下:

对于任意整数 j ($0 \leq j < \lfloor (n-1)/2 \rfloor$), 令

$$y_{2j+1}y_{2j} = \begin{cases} \overline{v_{2j+1}}v_{2j}, & \text{若 } v_{2j} = \overline{u_{2j}}, \\ v_{2j+1}v_{2j}, & \text{若 } v_{2j} = u_{2j}. \end{cases}$$

如果 n 是偶数, 令 $y_{n-2} = u_{n-2}$ 。

那么, 我们能够验证 $y \in W$ 。根据定义 2.5 和算法 CIOT, $p = \eta(v) = N(y, n-1)$ 。

因此, 引理成立。

我们给出算法 CIT 如下。

基于第二章中给出的独立生成树的定义, 我们给出引理 3.9 如下:

引理 3.9. 令 T_i 和 T_j ($i \neq j$) G 上以顶点 u 为根的两棵生成树。 T_i 和 T_j 是相互独立的当且仅当对于 G 上的任一顶点 v , $v \neq u$, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) = \{u\}$ 且 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) \supset \{u\}$ 。

证明. 首先, 我们证明“当”的正确性。

算法 2 CIT

输入: $CQ_{n-1}^{u_{n-1}}$ 上 $n-1$ 棵以顶点 u 为根的 IST $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}, W_1, W_2, \dots, W_{n-1}$ 使得 $W_1, W_2, \dots, W_{n-1}$ 是分别对应于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 的 EVS, $\{u, W_1, W_2, \dots, W_{n-1}\}$ 是 $V(CQ_{n-1}^{u_{n-1}})$ 的 EVSP, 这里 $n \geq 2$;

输出: n 棵树 $T_1, T_2, \dots, T_n$;

开始

步骤 1:

- 1: 基于以顶点 u 为根的 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 及算法 CIOT, 构造 $CQ_{n-1}^{u_{n-1}}$ 上 $n-1$ 棵以顶点 $N(u, n-1)$ 为根的树 $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$;

步骤 2:

构造 CQ_n 上的树 $T_1, T_2, \dots, T_{n-1}$ 如下:

- 2: **for** $i = 1$ to $n-1$ **do**

- 3: $V(T_i) = V(CQ_n)$;

- 4: $E(T_i) = E(T_{A,i}) \cup E(T_{B,i}) \cup \{(w, N(w, n-1)) | w \in W_i\} \setminus E(T_{B,i}[N(W_i, n-1)])$;

- 5: **end for**

步骤 3:

- 6: $T_{B,n} = \bigcup_{i=1}^{n-1} T_{B,i}[N(W_i \cup \{u\}, n-1)]$;

- 7: $V(T_n) = V(CQ_n)$;

- 8: $E(T_n) = E(T_{B,n}) \cup \{(z, N(z, n-1)) | z \in V(T_{B,n})\}$;

结束

我们分别将 $\text{path}(T_i, u, v)$ 和 $\text{path}(T_j, u, v)$ 表示为 P 和 Q 。根据第二章中独立生成树和顶点不相交的定义, 如果图 G 上以顶点 u 为根的树 T_i 和 T_j 是相互独立的, 那么 P 和 Q 是顶点不相交的, 对于任一顶点 $v \in V(G) \setminus \{u\}$, 这意味着 $E(P) \cap E(Q) = \emptyset$ 和 $V(P) \cap V(Q) = \{u, v\}$ 。进一步, 我们有 $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) = \{u\}$ 。既然 $E(P) \cap E(Q) = \emptyset$, 我们有 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) \supset \{u\}$ 。

接下来, 我们证明“仅当”的正确性。

如果结论不成立, 那么 $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) \neq \{u\}$ 或 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) \not\supset \{u\}$ 。显然, $u \in \text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j)$ 和 $u \in \text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j)$ 。我们有如下两种情形:

情形1. $(v, T_i) \cap \text{ancestor}(v, T_j) \neq \{u\}$ 。那么, $(v, T_i) \cap \text{ancestor}(v, T_j) \supset \{u\}$, 这意味着存在一个顶点 $w \in (\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j)) \setminus \{u\}$ 。因此, 与 P 和 Q 是相互独立的这一前提矛盾。

情形2. $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) \not\supset \{u\}$ 。那么 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) = \{u\}$, 这意味着边 (u, v) 分别是 T_i 和 T_j 中的边, 这与 P 和 Q 是相互独立的这

一前提矛盾。

基于上述讨论, 引理成立。

现在, 我们采用定理 3.10证明算法CIT得出的 n 棵树 $T_1, T_2, \dots, T_n$ 是 CQ_n 上以顶点 u 为根的 n 棵IST。

定理 3.10. 对于任意整数 $n \geq 2$, 假设输入 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 是 $CQ_{n-1}^{u_{n-1}}$ 上 $n-1$ 棵以顶点 u 为根的IST且输入 $W_1, W_2, \dots, W_{n-1}$ 为分别对应于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 的EVS, 使得 $\{u, W_1, W_2, \dots, W_{n-1}\}$ 是 $V(CQ_{n-1}^{u_{n-1}})$ 上的EVSP。那么, 算法CIT输出的 $T_1, T_2, \dots, T_n$ 是 CQ_n 上 n 棵以 u 为根的IST。

证明. 显然, $T_1, T_2, \dots, T_{n-1}$ 是 CQ_n 上 $n-1$ 棵以顶点 u 为根的生成树。接下来, 我们首先证明 $T_{B,n}$ 是 $CQ_{n-1}^{u_{n-1}}$ 上以顶点 $N(u, n-1)$ 为根的生成树和 T_n 是 CQ_n 上以顶点 u 为根的生成树。然后, 我们将证明 $T_1, T_2, \dots, T_n$ 相互独立。

(1) 根据算法CIOT和引理 3.5, 我们能够验证 $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$ 是 $CQ_{n-1}^{u_{n-1}}$ 上 $n-1$ 棵以顶点 $N(u, n-1)$ 为根的IST。根据定义 3.4, $\bigcup_{i=1}^{n-1} W_i \cup \{u\} = V(CQ_{n-1}^{u_{n-1}})$ 。进一步, 我们有 $\bigcup_{i=1}^{n-1} N(W_i \cup \{u\}, n-1) = V(CQ_{n-1}^{u_{n-1}})$ 和 $T_{B,n} = \bigcup_{i=1}^{n-1} T_{B,i}[N(W_i \cup \{u\}, n-1)]$ 。因此, $V(T_{B,n}) = \bigcup_{i=1}^{n-1} N(W_i \cup \{u\}, n-1) = V(CQ_{n-1}^{u_{n-1}})$ 。根据推论 3.7, $T_{B,n} = \bigcup_{i=1}^{n-1} T_{B,i}[N(W_i \cup \{u\}, n-1)]$ 是 $CQ_{n-1}^{u_{n-1}}$ 上一棵以顶点 $N(u, n-1)$ 为根的树。因此, $T_{B,n}$ 是 $CQ_{n-1}^{u_{n-1}}$ 上一棵以顶点 $N(u, n-1)$ 为根的生成树。

既然 $V(T_n) = V(CQ_n)$ 和 $E(T_n) = E(T_{B,n}) \cup \{(z, N(z, n-1)) | z \in V(T_{B,n})\}$, T_n 是 CQ_n 上一棵以顶点 u 为根的生成树。

(2) 接下来, 我们将首先证明 $T_1, T_2, \dots, T_{n-1}$ 是相互独立的。然后, 我们将证明对于任意整数 i ($1 \leq i \leq n-1$), T_n 和 T_i 是相互独立的。

对于任意两个整数 i 和 j ($1 \leq i < j \leq n$)和任一顶点 $v \in V(CQ_n)$ 且 $v \neq u$, 接下来, 我们总是定义 $X_i = \text{ancestor}(v, T_i) \cap V(CQ_{n-1}^{u_{n-1}})$, $Y_i = \text{ancestor}(v, T_i) \cap V(CQ_{n-1}^{u_{n-1}})$, $X_j = \text{ancestor}(v, T_j) \cap V(CQ_{n-1}^{u_{n-1}})$ 和 $Y_j = \text{ancestor}(v, T_j) \cap V(CQ_{n-1}^{u_{n-1}})$ 。显然 $X_i \cap Y_j = \emptyset$ 和 $X_j \cap Y_i = \emptyset$ 。因此,

$$\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) = (X_i \cup Y_i) \cap (X_j \cup Y_j) = (X_i \cap X_j) \cup (Y_i \cap Y_j). \quad (3.1)$$

进一步, 我们证明如下两个结论(i)和(ii):

(i) $T_1, T_2, \dots, T_{n-1}$ 是相互独立的。

对于任意两个整数 i 和 j ($1 \leq i < j \leq n-1$) 和对于任一顶点 $v \in V(CQ_n)$ 且 $v \neq u$, 我们有如下情形:

情形1*. $v \in V(CQ_{n-1}^{u_{n-1}})$. 显然, $Y_i = \emptyset$ 且 $Y_j = \emptyset$. 因此,

$$(X_i \cap X_j) \cup (Y_i \cap Y_j) = X_i \cap X_j. \quad (3.2)$$

既然 $T_{A,i}$ 和 $T_{A,j}$ 是相互独立的, 根据引理 3.9, $X_i \cap X_j = \{u\}$ 且

$$X_i \cup X_j \supset \{u\}. \quad (3.3)$$

根据等式(3.1)和(3.2), 我们有 $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) = (X_i \cap X_j) \cup (Y_i \cap Y_j) = X_i \cap X_j = \{u\}$. 根据等式(3.3), 我们有 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) = (X_i \cup Y_i) \cup (X_j \cup Y_j) = X_i \cup X_j \supset \{u\}$.

情形2*. $v \in V(CQ_{n-1}^{\overline{u_{n-1}}})$. 根据算法CIT, $X_i \subseteq W_i \cup \{u\}$ 且 $X_j \subseteq W_j \cup \{u\}$. 既然 $T_{A,i}$ 和 $T_{A,j}$ 是相互独立的, 根据定义 3.4 有 $W_i \cap W_j = \emptyset$ 且根据引理 3.9 有 $X_i \cap X_j = \{u\}$. 既然 $T_{B,i}$ 和 $T_{B,j}$ 是相互独立的, 根据算法CIT, $Y_i \cap Y_j = \emptyset$ 且 $Y_i \cup Y_j \neq \emptyset$. 根据算法CIT, $u \notin Y_i \cup Y_j$, 这意味着 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) = (X_i \cup Y_i) \cup (X_j \cup Y_j) \supset \{u\}$. 进一步, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) = (X_i \cap X_j) \cup (Y_i \cap Y_j) = \{u\}$.

根据引理 3.9, T_i 和 T_j 是相互独立的。

(ii) T_n 和 T_i 是相互独立的, 对于任意整数 i ($1 \leq i \leq n-1$).

我们将证明 $X_i \cap X_n = \{u\}$ 且 $Y_i \cap Y_n = \emptyset$, 对于任意整数 i ($1 \leq i \leq n-1$). 对于任一顶点 $v \in V(CQ_n)$ 且 $v \neq u$, 我们有如下情形:

情形1. $v \in V(CQ_{n-1}^{u_{n-1}})$. 显然, $Y_i = \emptyset$ 且 $X_n = \{u\}$. 因此,

$$X_i \cap X_n = \{u\} \quad (3.4)$$

且

$$Y_i \cap Y_n = \emptyset. \quad (3.5)$$

我们容易验证 $Y_n \neq \emptyset$ 且 $u \notin Y_n$, 这意味着 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) \supset \{u\}$. 根据等式(3.1), (3.4) 和(3.5), 我们有 $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_n) = (X_i \cap X_n) \cup (Y_i \cap Y_n) = X_i \cap X_n = \{u\}$.

情形2. $v \in V(CQ_{n-1}^{u_{n-1}})$ 。显然, $X_n = \text{ancestor}(v, T_n) \cap V(CQ_{n-1}^{u_{n-1}}) = \{u\}$, $Y_i = \text{ancestor}(v, T_i) \cap V(CQ_{n-1}^{u_{n-1}})$ 且 $X_i \cap X_n = \{u\}$ 。那么, 根据算法CIT, 必定存在一个整数 k ($1 \leq k \leq n-1$), 使得 $Y_n \subseteq N(W_k \cup \{u\}, n-1)$ 。容易验证 $N(u, n-1) \notin Y_i$ 和 $N(u, n-1) \in Y_n$ 。因此, $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_n) = (X_i \cup Y_i) \cup (X_n \cup Y_n) \supseteq \{u\} \cup \{N(u, n-1)\} \supset \{u\}$ 。进一步, 我们通过如下子情形来证明 $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_n) = \{u\}$ 。

情形2.1. $i \neq k$ 。显然, $Y_i \cap N(\{u\}, n-1) = \emptyset$ 。既然 $T_{B,i}$ 和 $T_{B,k}$ 是相互独立的, 那么 $Y_i \cap Y_n \subseteq Y_i \cap N(W_k \cup \{u\}, n-1) = (Y_i \cap N(W_k, n-1)) \cup (Y_i \cap N(\{u\}, n-1)) = Y_i \cap N(W_k, n-1) = \emptyset$ 。因此, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_n) = (X_i \cap X_n) \cup (Y_i \cap Y_n) = \{u\}$ 。

情形2.2. $i = k$ 。显然, $Y_i \cap N(\{u\}, n-1) = \emptyset$ 。根据算法CIT, 我们能够验证 $v \in N(W_i \cup \{u\}, n-1)$, 使得 $Y_i = \emptyset$ 。因此, $Y_i \cap Y_n = \emptyset$ 。相应地, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_n) = (X_i \cap X_n) \cup (Y_i \cap Y_n) = \{u\}$ 。

根据引理 3.9, T_i 和 T_n 是相互独立的。

综上所述, $T_1, T_2, \dots, T_n$ 是相互独立的。

需要指出的是 $V(CQ_{n-1}^{u_{n-1}})$ 上可能有多组EVSP。例如, 图 3.6(a) 示出了 $V(CQ_3^0)$ 上的一个EVSP $\{\{1\}, \{0,2,4,6\}, \{3\}, \{5,7\}\}$, 这里 $\{0,2,4,6\}$, $\{3\}$ 和 $\{5,7\}$ 分别是对应于以顶点1为根的 $T_{A,1}$, $T_{A,2}$ 和 $T_{A,3}$ 的EVS。图 3.6(b) 示出了 $V(CQ_3^0)$ 上的另一个EVSP $\{\{1\}, \{0,2\}, \{3\}, \{4,5,6,7\}\}$, 这里 $\{0,2\}$, $\{3\}$ 和 $\{4,5,6,7\}$ 分别是对应于以顶点1为根的 $T_{A,1}$, $T_{A,2}$ 和 $T_{A,3}$ 的EVS。我们注意到, 图 3.6(a) 中的 T_4 不同于图 3.6(b) 中的 T_4' 。

接下来, 我们将给出一个算法, 称为CIST, 来构造 CQ_n 上以顶点 u 为根的 n 棵IST, 这里 $n \geq 1$ 。在算法CIST中, 我们将提供选取 $V(CQ_{n-1}^{u_{n-1}})$ 上EVSP的规则 (参见算法CIST上的第9-21行)。

在算法CIST中, CQ_n 上的生成树 T_i 由 $T_{A,i}$ 和 $T_{B,i}$ 递归地构造, 对于整数 i ($1 \leq i \leq n-1$)。进一步, 令 $T_{B,n} = \bigcup_{i=1}^{n-1} T_{B,i}[N(W_i \cup \{u\}, n-1)]$ 。 T_n 是由 $T_{B,n}$ 中的每个顶点 z 连接 $N(z, n-1)$ 而成。

例如, 在图 3.4 中, (a), (b) 和 (c) 示出了 n 棵 CQ_n 上以顶点 $0^{n-1}1$ 为根的IST $T_1, T_2, \dots, T_n$ 的构造, 对于 $n = 1, 2$ 和 3 。在图 3.4(a) 中, CQ_1 上的 T_1 是以顶点1为根, 这里 $V(T_1) = \{1, 0\}$ 且 $E(T_1) = \{(1, 0)\}$ 。在图 3.4(b) 中, CQ_1^0 上的 $T_{A,1}$ 来自于 CQ_1 上的 T_1 , $W_1 = \{0\}$ 是对应于 CQ_1^0 上 $T_{A,1}$ 的一个EVS。 CQ_1^1 上以顶点 $N(1, 1) = 3$ 为根的 $T_{B,2}$ 由 $N(W_1 \cup \{1\}) = \{2, 3\}$ 的导出子图组成且用虚线画出。 CQ_2 中的 T_1 将集合 $W_1 = \{0\} \subset V(CQ_1^0)$ 中的每一个顶点 w 连接 $T_{B,1}$ 中的 $N(w, 1)$ 而得到的。 CQ_2 上的 T_2 是通过将 $T_{B,2}$ 中的每一个

算法 3 $CIST(n, u_{n-1}u_{n-2} \dots u_0)$ **输入:** 整数 n ($n \geq 1$) 和 $u = u_{n-1}u_{n-2} \dots u_0$;**输出:** CQ_n 上 n 棵以顶点 $u_{n-1}u_{n-2} \dots u_0$ 为根的树 $T_1, T_2, \dots, T_n$, $CQ_n(u_{n-1})$ 上 $n-1$ 棵以 $u_{n-1}u_{n-2} \dots u_0$ 为根的树 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 和 $n-1$ 个顶点集合 $W_1, W_2, \dots, W_{n-1}$;**开始** $W_i = \emptyset$, 对于 $i = 1, 2, \dots, n-1$;**步骤 1:**1: **if** ($n == 1$) **then**2: 返回以顶点 u_0 为根的树, 满足 $V(T_1) = \{0, 1\}$ 且 $E(T_1) = \{(0, 1)\}$;3: **else**4: 调用 $CIST(n-1, u_{n-2}u_{n-3} \dots u_0)$;5: **end if****步骤 2:**6: **for** $i = 1$ to $n-1$ **do**7: $T_{A,i} = u_{n-1}T_i$;8: **end for****步骤 3:**9: **if** ($n == 2$) **then**10: $W_1 = \{u_1\bar{u}_0\}$;11: **else**12: $W_{n-1} = \{u_{n-1}N(u_{n-2}u_{n-3} \dots u_0, n-2)\}$; /* 初始化 W_{n-1} */13: **for** $i = 1$ to $n-2$ **do**14: **if** ($i \% 2 == 1$) **then**15: $W_i = u_{n-1}(W_i \cup N(W_i, n-2))$;16: **else**17: $W_i = u_{n-1}W_i$;18: $W_{n-1} = W_{n-1} \cup N(W_i, n-2)$;19: **end if**20: **end for**21: **end if****步骤 4:**22: 基于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 和 $W_1, W_2, \dots, W_{n-1}$ 使得 $W_1, W_2, \dots, W_{n-1}$ 是分别对应于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 的EVS 且 $\{u\}, W_1, W_2, \dots, W_{n-1}$ 是 $V(CQ_{n-1}^{u_{n-1}})$ 上的一个EVSP, 根据算法CIT, 构造以 $u_{n-1}u_{n-2} \dots u_0$ 为根的 $T_1, T_2, \dots, T_n$;**结束**

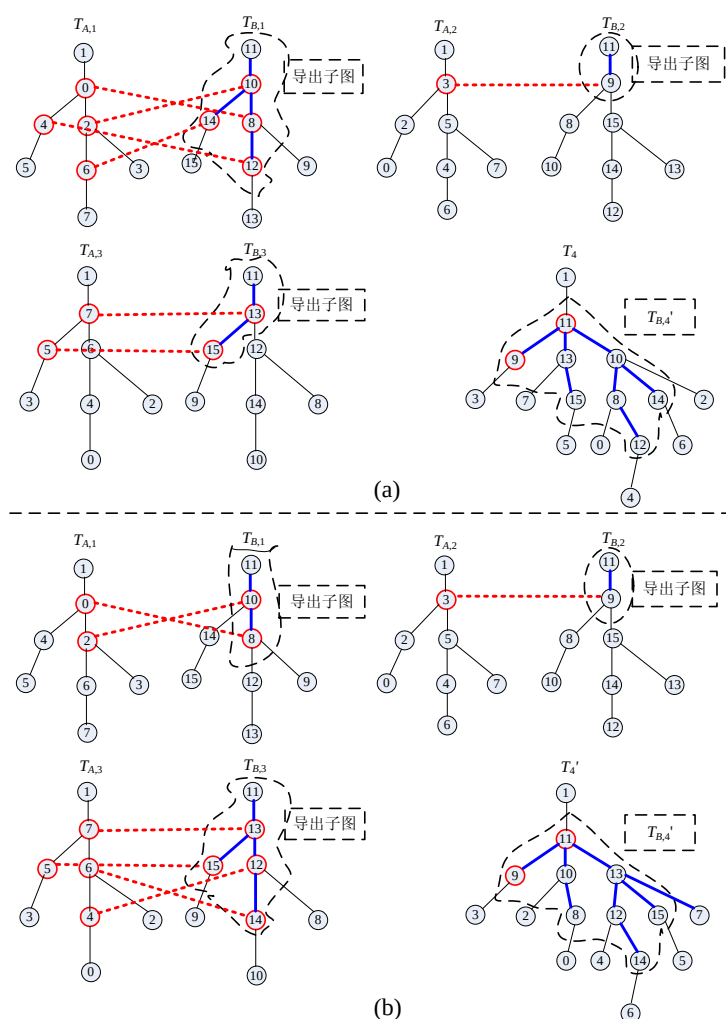


图 3.6 (a) $V(CQ_3^0)$ 上的一个 EVSP $\{\{1\}, \{0, 2, 4, 6\}, \{3\}, \{5, 7\}\}$, 这里 $\{0, 2, 4, 6\}$, $\{3\}$ 和 $\{5, 7\}$ 是分别对应于以 1 为根的 $T_{A,1}$, $T_{A,2}$ 和 $T_{A,3}$ 的 EVS; (b) $V(CQ_3^0)$ 上的另一个 EVSP $\{\{1\}, \{0, 2\}, \{3\}, \{4, 5, 6, 7\}\}$, 这里 $\{0, 2\}$, $\{3\}$ 和 $\{4, 5, 6, 7\}$ 分别是以顶点 1 为根的 $T_{A,1}$, $T_{A,2}$ 和 $T_{A,3}$ 的 EVS

顶点 z 连接到 $N(z, 1)$ 而得到。在图 3.4(c) 中, CQ_2^0 上的 $T_{A,1}$ 和 $T_{A,2}$ 分别来自于 CQ_2 上以 1 为根的 IST T_1 和 T_2 。这里, $\{\{1\}, \{0, 2\}, \{3\}\}$ 是 $V(CQ_2^0)$ 上的一个 EVSP, 使得 $W_1 = \{0, 2\}$ 和 $W_2 = \{3\}$ 是分别对应于以 CQ_2^0 上以顶点 1 为根的 $T_{A,1}$ 和 $T_{A,2}$ 的 EVS。 CQ_3 上的 T_1 是通过将 $W_1 = \{0, 2\} \subset V(CQ_2^0)$ 中的每个顶点 w 连接 $T_{B,1}$ 中的 $N(w, 2)$ 而得到。 CQ_3 上的 T_2 通过将 $W_1 = \{3\} \subset V(CQ_2^0)$ 中的每个顶点 w 连接 $T_{B,2}$ 上的 $N(w, 2)$ 而得到。 $T_{B,3}$ 是由 CQ_2^1 上以顶点 $N(1, 2) = 7$ 为根的 $N(W_1 \cup \{1\}) = \{4, 6, 7\}$ 的导出子图和 CQ_2^1 上以顶点 $N(1, 2) = 7$ 为根的 $N(W_2 \cup \{1\}) = \{5, 7\}$ 的导出子图组成。 CQ_3 上的 T_3 是将 $T_{B,3}$ 中的每一个顶点 z 连接 $N(z, 2)$ 而得到。

为了证明算法CIST的正确性, 我们给出如下的表示法。

表示法 3.2. 如果 $G' = (V', E')$ 是 CQ_n 的子图, 对于 $n \geq 2$ 和 $s \in \{0, 1\}$, 那么我们用 sG' 来表示图 (sV', sE') , 这里 $sE' = \{(sx, sy) | (x, y) \in E'\}$ 。

定理 3.11. 对于 $n \geq 1$ 和 CQ_n 上的任一顶点 $u = u_{n-1}u_{n-2} \dots u_0$, 下面结论成立:

(i) 算法CIST 得到 $CQ_{n-1}^{u_{n-1}}$ 上以顶点 $u = u_{n-1}u_{n-2} \dots u_0$ 为根的 $n-1$ IST $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 和 $W_1, W_2, \dots, W_{n-1}$ 使得 $W_1, W_2, \dots, W_{n-1}$ 是分别对应于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 的EVS, $\{u, W_1, W_2, \dots, W_{n-1}\}$ 是 $V(CQ_{n-1}^{u_{n-1}})$ 上的一个EVSP, 对于任意整数 $i, 1 \leq i \leq n-1$,

$$W_i = \begin{cases} \{u_{n-1}u_{n-2} \dots u_i \overline{u_{i-1}} x_{i-2} x_{i-3} \dots x_0 \in V(CQ_{n-1}^{u_{n-1}}) | x_{i-2p} = u_{i-2p} \\ \text{对于 } p \in \{1, 2, \dots, i/2\}, \text{ 若 } i \text{ 是偶数,} \\ \{u_{n-1}x_{n-2}x_{n-3} \dots x_i \overline{u_{i-1}} x_{i-2} x_{i-3} \dots x_0 \in V(CQ_{n-1}^{u_{n-1}}) | x_{i-2p-1} = u_{i-2p-1} \\ \text{对于 } p \in \{1, 2, \dots, (i-1)/2\}, \text{ 否则。} \end{cases}$$

(ii) 由算法CIST构造的 $T_1, T_2, \dots, T_n$ 是 CQ_n 上 n 棵以顶点 u 为根的IST。

证明. 我们采用关于 n 的数学归纳法进行证明。

基础: 显然该定理对于 $n = 1$ 成立。我们首先证明该定理对于 $n = 2$ 成立。

(i) 根据算法CIST, T_1 是 CQ_1 上以顶点 u_0 为根的生成树, 满足 $V(T_1) = \{0, 1\}$ 和 $E(T_1) = \{(0, 1)\}$ 。根据算法CIST, $T_{A,1} = u_1 T_1$ 和 $W_1 = \{u_1 \overline{u_0}\}$ 。显然, $T_{A,1}$ 是 $CQ_1^{u_1}$ 上一棵以顶点 $u_1 u_0$ 为根的生成树。 W_1 是对应于以 $u_1 u_0$ 为根的 $T_{A,1}$ 的EVS。 $\{u, W_1\}$ 是 $V(CQ_1^{u_1})$ 上的一个EVSP。显然 W_1 满足该定理中的条件。

(ii) 根据算法CIST, 我们得到 CQ_2 上以顶点 $u_1 u_0$ 为根的两棵生成树 T_1 和 T_2 。根据定理 3.10, T_1 和 T_2 是 CQ_2 上以顶点 $u_1 u_0$ 为根的两棵IST。

因此, 该定理对于 $n = 2$ 成立。

归纳: 假设该定理对于 $n = \tau$ ($\tau \geq 2$) 成立。即,

(i) 算法CIST 得到 $CQ_{\tau-1}^{u_{\tau-1}}$ 上以顶点 $u_{\tau-1}u_{\tau-2} \dots u_0$ 为根的 $\tau-1$ 棵IST $T'_{A,1}, T'_{A,2}, \dots, T'_{A,\tau-1}$ 和 $W'_1, W'_2, \dots, W'_{\tau-1}$ 使得 $W'_1, W'_2, \dots, W'_{\tau-1}$ 分别对应于 $T'_{A,1}, T'_{A,2}, \dots, T'_{A,\tau-1}$ 的EVS, $\{u_{\tau-1}u_{\tau-2} \dots u_0, W'_1, W'_2, \dots, W'_{\tau-1}\}$ 是 $V(CQ_{\tau-1}^{u_{\tau-1}})$ 上的一个EVSP, 对于

任意整数 i , $1 \leq i \leq \tau - 1$,

$$W'_i = \begin{cases} \{u_{\tau-1}u_{\tau-2} \dots u_i \overline{u_{i-1}}x_{i-2}x_{i-3} \dots x_0 \in V(CQ_{\tau-1}^{u_{\tau-1}}) | x_{i-2p} = u_{i-2p} \\ \text{对于 } p \in \{1, 2, \dots, i/2\}, \text{ 若 } i \text{ 是偶数,} \\ \{u_{\tau-1}x_{\tau-2}x_{\tau-3} \dots x_i \overline{u_{i-1}}x_{i-2}x_{i-3} \dots x_0 \in V(CQ_{\tau-1}^{u_{\tau-1}}) | x_{i-2p-1} = u_{i-2p-1} \\ \text{对于 } p \in \{1, 2, \dots, (i-1)/2\}, \text{ 否则。} \end{cases} \quad (3.6)$$

(ii) 由算法CIST构造的 $T'_1, T'_2, \dots, T'_\tau$ 是 CQ_τ 上 τ 棵以顶点 $u_{\tau-1}u_{\tau-2} \dots u_0$ 为根的IST。

我们将证明该定理对于 $n = \tau + 1$ 成立。

(i) 根据算法CIST, $T_{A,1} = u_\tau T'_1, T_{A,2} = u_\tau T'_2, \dots, T_{A,\tau} = u_\tau T'_\tau$ 。显然, $T_{A,1}, T_{A,2}, \dots, T_{A,\tau}$ 是 $CQ_\tau^{u_\tau}$ 上 τ 棵以顶点 $u_\tau u_{\tau-1} \dots u_0$ 为根的IST。

进一步, 根据算法CIST, 对于任意整数 i , $1 \leq i \leq \tau - 1$,

$$W_i = \begin{cases} u_\tau W'_i & , \text{ 若 } i \text{ 是偶数,} \\ u_\tau (W'_i \cup N(W'_i, \tau - 1)) & , \text{ 否则。} \end{cases} \quad (3.7)$$

$$W_\tau = \{u_\tau N(u_{\tau-1}u_{\tau-2} \dots u_0, \tau - 1)\} \cup \bigcup_{j=1}^{\lfloor (\tau-1)/2 \rfloor} N(u_\tau W'_{2j}, \tau - 1) \quad (3.8)$$

根据等式(3.6)和(3.7), 对于任意整数 i , $1 \leq i < \tau$, 我们有:

$$W_i = \begin{cases} \{u_\tau u_{\tau-1} \dots u_i \overline{u_{i-1}}x_{i-2}x_{i-3} \dots x_0 \in V(CQ_\tau^{u_\tau}) | x_{i-2p} = u_{i-2p} \\ \text{对于 } p \in \{1, 2, \dots, i/2\}, \text{ 若 } i \text{ 是偶数,} \\ \{u_\tau x_{\tau-1}x_{\tau-2} \dots x_i \overline{u_{i-1}}x_{i-2}x_{i-3} \dots x_0 \in V(CQ_\tau^{u_\tau}) | x_{i-2p-1} = u_{i-2p-1} \\ \text{对于 } p \in \{1, 2, \dots, (i-1)/2\}, \text{ 否则。} \end{cases} \quad (3.9)$$

根据等式(3.6)和(3.8), 我们有:

$$W_\tau = \begin{cases} \{u_\tau \overline{u_{\tau-1}}x_{\tau-2}x_{\tau-3} \dots x_0 \in V(CQ_\tau^{u_\tau}) | x_{\tau-2p} = u_{\tau-2p} \\ \text{对于 } p \in \{1, 2, \dots, \tau/2\}, \text{ 若 } \tau \text{ 是偶数,} \\ \{u_\tau \overline{u_{\tau-1}}x_{\tau-2}x_{\tau-3} \dots x_0 \in V(CQ_\tau^{u_\tau}) | x_{\tau-2p-1} = u_{\tau-2p-1} \\ \text{对于 } p \in \{1, 2, \dots, (\tau-1)/2\}, \text{ 否则。} \end{cases} \quad (3.10)$$

根据等式(3.9)和(3.10), 对于整数 i , $1 \leq i \leq \tau$, 我们有:

$$W_i = \begin{cases} \{u_\tau x_{\tau-1} x_{\tau-2} \dots u_i \overline{u_{i-1}} x_{i-2} x_{i-3} \dots x_0 \in V(CQ_\tau^{u_\tau}) | x_{i-2p} = u_{i-2p} \\ \text{对于 } p \in \{1, 2, \dots, i/2\}, \text{ 若 } i \text{ 是偶数,} \\ \{u_\tau x_{\tau-1} x_{\tau-2} \dots x_i \overline{x_{i-1}} x_{i-2} x_{i-3} \dots x_0 \in V(CQ_\tau^{u_\tau}) | x_{i-2p-1} = u_{i-2p-1} \\ \text{对于 } p \in \{1, 2, \dots, (i-1)/2\}, \text{ 否则.} \end{cases}$$

那么, W_i 满足引理 3.8 中的条件(2) 对于 $i = 1, 2, \dots, \tau$.

现在我们证明 W_i 满足引理 3.8 中的条件(1). 根据算法 CIST, W_i' 中的每一个顶点 v 邻接于 $T_{A,i}$ 中的 $N(v, \tau)$, 对于 $i = 1, 2, \dots, \tau - 1$. 既然 $W_1', W_2', \dots, W_{\tau-1}'$ 是分别对应于 $T'_{A,1}, T'_{A,2}, \dots, T'_{A,\tau-1}$ 的 EVS, 根据等式(3.7), 我们能够验证 $W_1, W_2, \dots, W_{\tau-1}$ 满足引理 3.8 中的条件(1). 根据推论 3.7, $T'_{B,i}[\{N(u_{\tau-1}u_{\tau-2} \dots u_0, \tau - 1)\} \cup N(W_i', \tau - 1)]$ 是 $CQ_{\tau-1}^{u_{\tau-1}}$ 上一棵以顶点 $N(u_{\tau-1}u_{\tau-2} \dots u_0, \tau - 1)$ 为根的树, 对于 $i = 1, 2, \dots, \tau - 1$. 根据等式(3.8)和算法 CIST, 我们进一步验证 $T_{A,\tau}[\{u_\tau u_{\tau-1} \dots u_0\} \cup W_\tau]$ 是 $CQ_\tau^{u_\tau}$ 上一棵以顶点 $u_\tau u_{\tau-1} \dots u_0$ 为根的树, 这意味着 W_τ 满足引理 3.8 中的条件(1). 因此, $W_1, W_2, \dots, W_\tau$ 是分别是对应于 $CQ_\tau^{u_\tau}$ 上以顶点 $u_\tau u_{\tau-1} \dots u_0$ 为根的 $T_{A,1}, T_{A,2}, \dots, T_{A,\tau}$ 的 EVS.

进一步, 对于任意两个整数 i 和 j , $1 \leq i < j \leq \tau$, $W_i \cap W_j = \emptyset$ 和 $\bigcup_{i=1}^{\tau} W_i \cup \{u_\tau u_{\tau-1} \dots u_0\} = V(CQ_\tau^{u_\tau})$. 根据定义 3.4, $\{\{u_\tau u_{\tau-1} \dots u_0\}, W_1, W_2, \dots, W_\tau\}$ 是 $V(CQ_\tau^{u_\tau})$ 上的一个 EVSP.

(ii) 根据算法 CIST, 我们得到 $CQ_{\tau+1}$ 上以顶点 $u_\tau u_{\tau-1} \dots u_0$ 为根的 $\tau + 1$ 生成树 $T_1, T_2, \dots, T_{\tau+1}$. 根据定理 3.10, $T_1, T_2, \dots, T_{\tau+1}$ 是 $CQ_{\tau+1}$ 上以顶点 $u_\tau u_{\tau-1} \dots u_0$ 为根的 $\tau + 1$ IST.

综上所述, 我们已经证明了该定理满足 $n = \tau + 1$.

例如, 图 3.4 (b) 和(c) 分别示出了 CQ_n 上根据算法 CIST 得出的以 1 (十进制表示) 为根的 n 棵 IST 的构造过程, 对于 $n = 2, 3$.

定理 3.12. 对于任意整数 $n \geq 1$, 算法 CIST 在 $O(M \log^2 N)$ 时间内构造出 CQ_n 上以 u 为根的 n 棵 IST, 这里 $N = 2^n$ 是 CQ_n 中的顶点个数.

证明. 根据定理 3.11, 算法 CIST 构造出 CQ_n 上以 u 为根的 n 棵 IST $T_1, T_2, \dots, T_n$. 我们分析算法的时间复杂度如下.

令 $t_1(N)$, $t_2(N)$ 和 $t(N)$ 分别表示算法 CIOT, CIT 和 CIST 的运行时间, 这里 $N = 2^n$ 是 CQ_n 中的顶点个数. 根据算法 CIOT, 最坏时间复杂度为

$$t_1(N) = N/2 + \lfloor (n-1)/2 \rfloor * (N/2) = \lfloor (n+1)/2 \rfloor * (N/2)。$$

根据算法CIT,

$$\begin{aligned} t_2(N) &= O((n-1) * t_1(N)) + O(\sum_{i=1}^{n-1} |W_i| + \sum_{i=1}^{n-1} (|W_i| - 1) + N/2) \\ &= O((n-1) * (\lfloor (n+1)/2 \rfloor * (N/2))) + O((N/2 - 1) + ((N/2 - 1) - (n-1)) + N/2) \\ &= O((n-1) * (\lfloor (n+1)/2 \rfloor * N/2)) + O(3N/2 - n - 1)。 \end{aligned}$$

根据算法CIST,

$$\begin{aligned} t(N) &= t(N/2) + O((n-1) * (N/2)) + O(\sum_{i=1}^{n-1} 2|W_i|) + t_2(N) \\ &= t(N/2) + O((n-1) * N/2) + O(2 * (N/2 - 1)) + O((n-1) * (\lfloor (n+1)/2 \rfloor * N/2)) + \\ &\quad O(3N/2 - n - 1) \\ &= t(N/2) + O(n^2 N) \\ &= O(n^2 2^n + (n-1)^2 2^{n-1} + \dots + 1^2 * 2^1) \\ &\leq O(n^2 * (2^n + 2^{n-1} + \dots + 2^1)) \\ &= O(n^2 2^n) \\ &= O(N \log^2 N) \end{aligned}$$

因此, 算法CIST的时间复杂度为 $O(N \log^2 N)$ 。

3.3 IST优化构造问题的探讨

3.3.1 IST的高度与结构

依据算法CIST, 给定 $V(CQ_3^0)$ 上的一个EVSP $\{\{1\}, \{0, 2, 4, 6\}, \{3\}, \{5, 7\}\}$, 这里 $\{0, 2, 4, 6\}, \{3\}$ 和 $\{5, 7\}$ 分别是对应于以顶点1为根的 $T_{A,1}$, $T_{A,2}$ 和 $T_{A,3}$ 的EVS。我们可构造出 $V(CQ_4^0)$ 上的EVSP $\{\{1\}, \{0, 2, 4, 6, 8, 10, 12, 14\}, \{3\}, \{5, 7, 13, 15\}, \{9, 11\}\}$, 这里 $\{0, 2, 4, 6, 8, 10, 12, 14\}, \{3\}, \{5, 7, 13, 15\}$ 和 $\{9, 11\}$ 分别是对应于以顶点1为根的 $T_{A,1}$, $T_{A,2}$, $T_{A,3}$ 和 $T_{A,4}$ 的EVS。我们可以看出其中 T_1, T_2, T_4 和 T_5 的高度均为6, 而 T_3 的高度为7, 所构造出的IST并不是全部相互同构, 且高度不一, 如图3.7所示。

那么, 能否在 CQ_n 上能否采用串行方法构造出性能更为优越的一组生成树呢? 答案是肯定的。如基于图3.6(b)中的四棵IST及相应的EVS, 我们构造出 $V(CQ_4^0)$ 上的EVSP $\{\{1\}, \{0, 2, 8, 10\}, \{3, 9\}, \{4, 5, 6, 7, 12, 13, 14, 15\}, \{11\}\}$, 这里 $\{0, 2, 8, 10\}, \{3, 9\}, \{4, 5, 6, 7, 12, 13, 14, 15\}$ 和 $\{11\}$ 分别是对应于以顶点1为根的 $T_{A,1}$, $T_{A,2}$, $T_{A,3}$ 和 $T_{A,4}$ 的EVS, 如图3.8和图3.9所示。

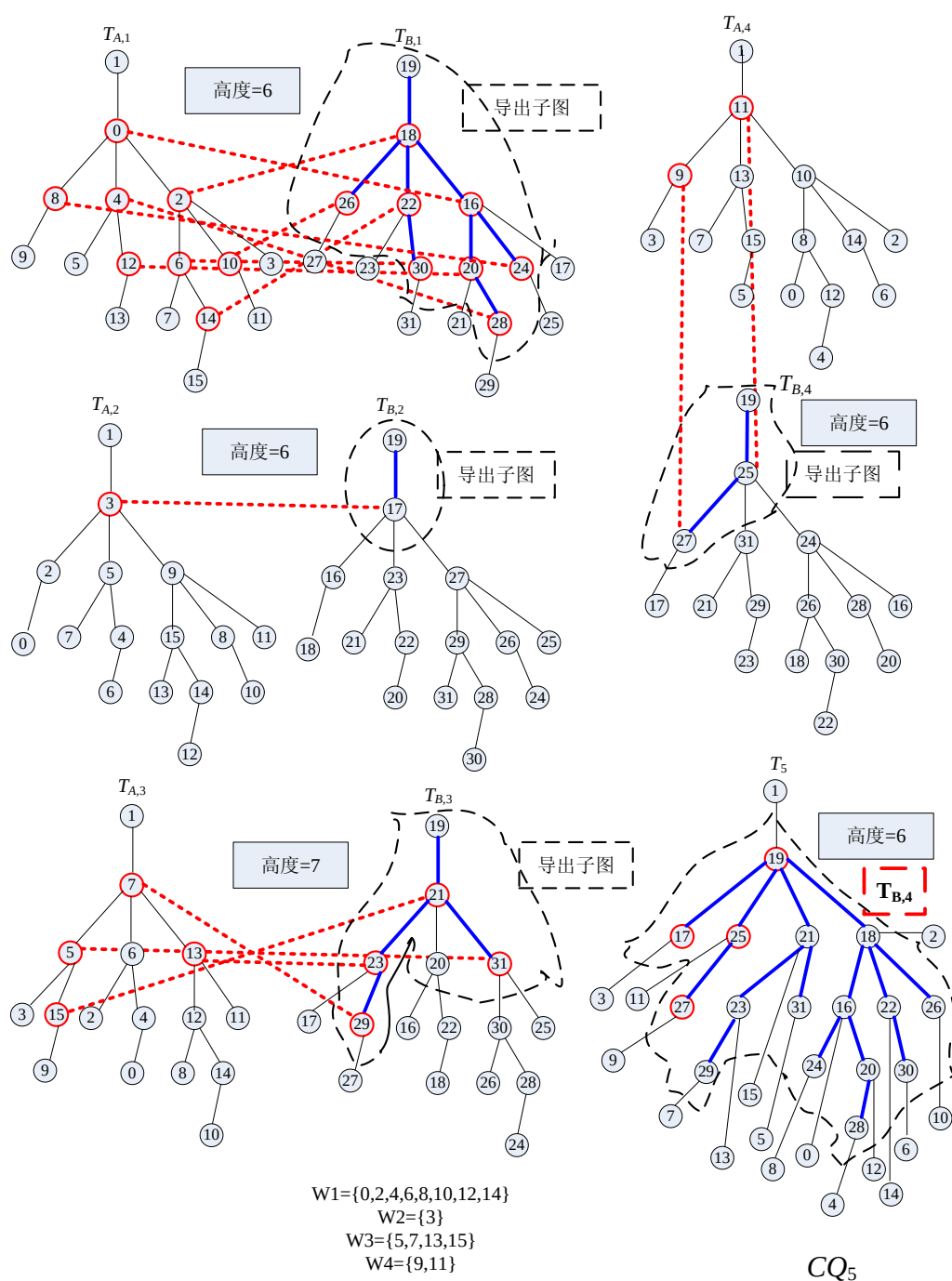
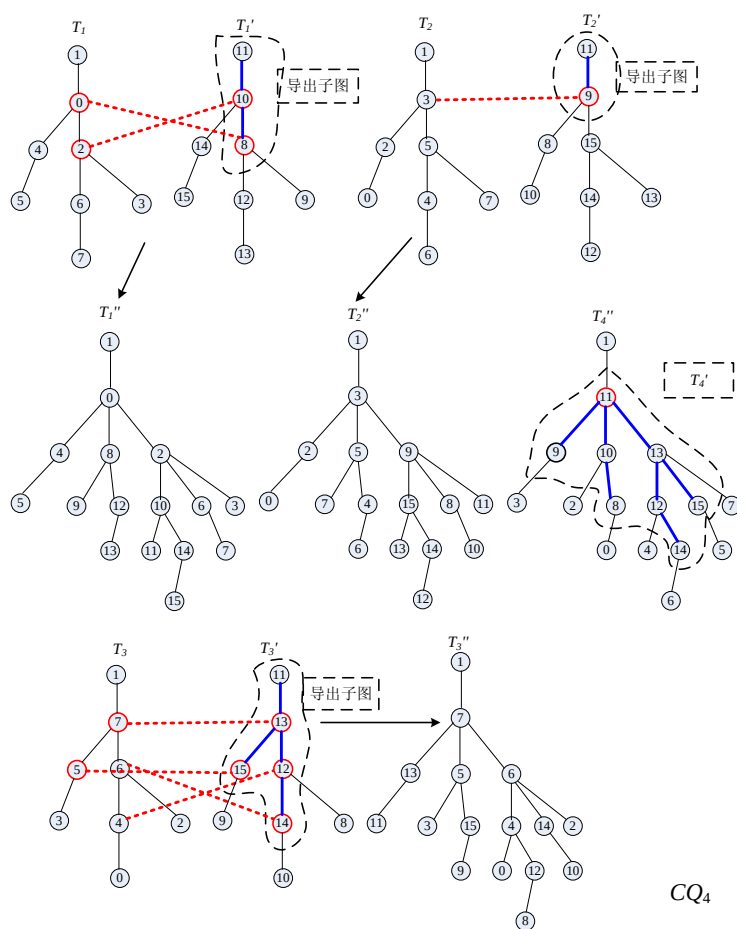


图 3.7 依据算法CIST在 CQ_5 上构造的以1为根的4棵IST

图 3.8 在 CQ_4 上构造的以1为根的4棵IST

我们可以看出其中 T_1, T_2, T_3, T_4 和 T_5 的高度均为6且相互同构，性能优于图3.7所示的一组IST，该组IST依据算法CIST构造。这里，我们只需要将算法CIST的扩展顶点集合选择方法作相应修改即可。因证明过程类似，不再赘述。在后面的第五章中，我们将给出更高效的并行方法来构造交叉立方体上以任一顶点为根的IST。

3.3.2 IST的地址长度

我们也可对算法CIST中IST中的顶点地址做些讨论。需要注意的是，在算法CIST的递归构造方法中，每棵树中顶点地址的长度也是从1位依次变换到 n 位。这里，顶点地址均以二进制表示。下面以构造 CQ_4 上以顶点1011为根的四棵IST加以说明：

- (1) 在第一轮迭代中，构造的是 CQ_1 上以顶点1为根的1棵IST。

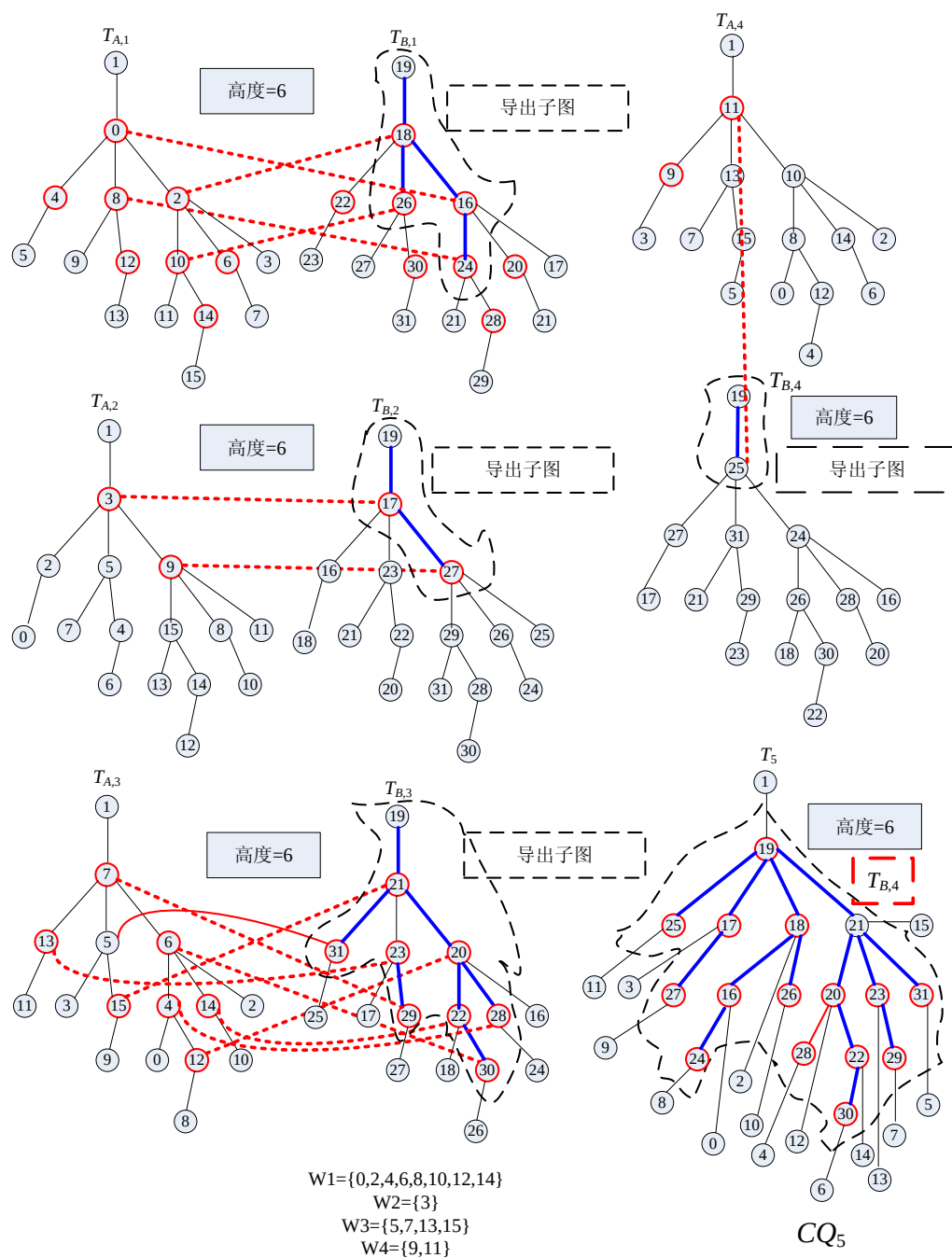


图 3.9 在 CQ_5 上构造的以 1 为根的 5 棵 IST

(2) 在第二轮迭代中, 构造的是 CQ_2 上以顶点11为根的2棵IST。

(3) 在第三轮迭代中, 构造的是 CQ_3 上以顶点011为根的3棵IST。

(4) 在第四轮迭代中, 构造的是 CQ_4 上以顶点1011为根的4棵IST。

我们也可以让顶点地址保持不变, 使得在IST的构造过程中顶点1011始终保持不变。

(1) 在第一轮迭代中, 构造的是 CQ_4 的导出子图 $CQ_4(101)$ 上以顶点1011为根的一棵IST。

(2) 在第二轮迭代中, 构造的是 CQ_4 的导出子图 $CQ_4(10)$ 上以顶点1011为根的二棵IST。

(3) 在第三轮迭代中, 构造的是 CQ_4 的导出子图 $CQ_4(1)$ 上以顶点1011为根的三棵IST。

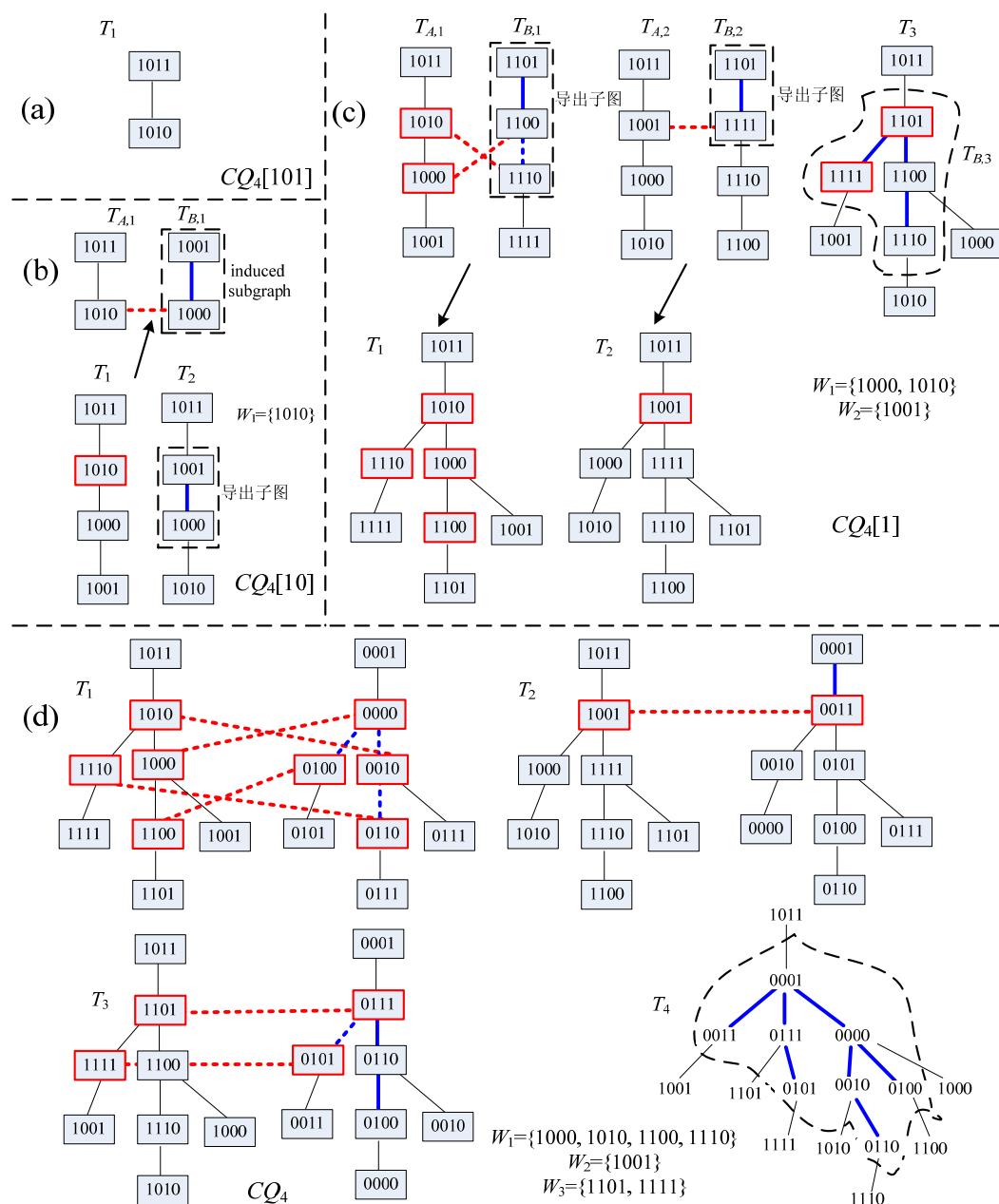
(4) 在第四轮迭代中, 构造的是 CQ_4 的导出子图 CQ_4 上以顶点1011为根的四棵IST。

迭代效果如图 3.10所示。只需相应地修改算法CIST即可, 这里不再详述。

3.4 本章小结

本章首先证明交叉立方体满足一个重要的地址变换性质, 依据该性质能够构造不同子立方体中同构的生成树。然后, 给出基于地址变换的扩展顶点集合和扩展顶点集合划分的定义。接着, 设计了时间复杂度为 $O(N\log^2 N)$ 的IST的构造算法, 并证明了其正确性。

本章关于交叉立方体上IST构造的内容已经发表在SCI源期刊《Information Sciences》上(第一作者)。

图 3.10 在 CQ_4 上构造的以1为根的4棵IST

第四章 莫比乌斯立方体上IST的串行构造

与交叉立方体不同的是, n 维莫比乌斯立方体是由两个不同构的 $n-1$ 维子莫比乌斯立方体组成, 从而交叉立方体上地址变换的方法不再适用于在莫比乌斯立方体上构造IST。本章将研究莫比乌斯立方体上IST的串行构造算法, 主要取得如下结果:

(1) 我们提出一个完全从树中顶点之间邻接关系的角度研究同构树的构造的新方法。

(2) 提出用非负向量的方法来对IST中不相交路径的特点进行分析, 从而证明了莫比乌斯立方体上IST的存在性。

(3) 给出了时间复杂度为 $O(M \log N)$ 的以任一顶点为根的IST的构造算法, 相对于第三章的算法, 性能有所提高。

(4) 对满足一定特征的网络上的IST的构造问题做了在一定条件下的探讨。

4.1 M_{n-1}^0 和 M_{n-1}^1 上同构树构造分析

由于 M_n 是由两个不同构的子莫比乌斯立方体 M_{n-1}^0 和 M_{n-1}^1 组成, 给定 M_{n-1}^0 上的一棵生成树, 依据 Q_n 或 CQ_n 上的地址变换方法, 能否构造 M_{n-1}^1 上的一棵生成树吗? 答案是否定的。

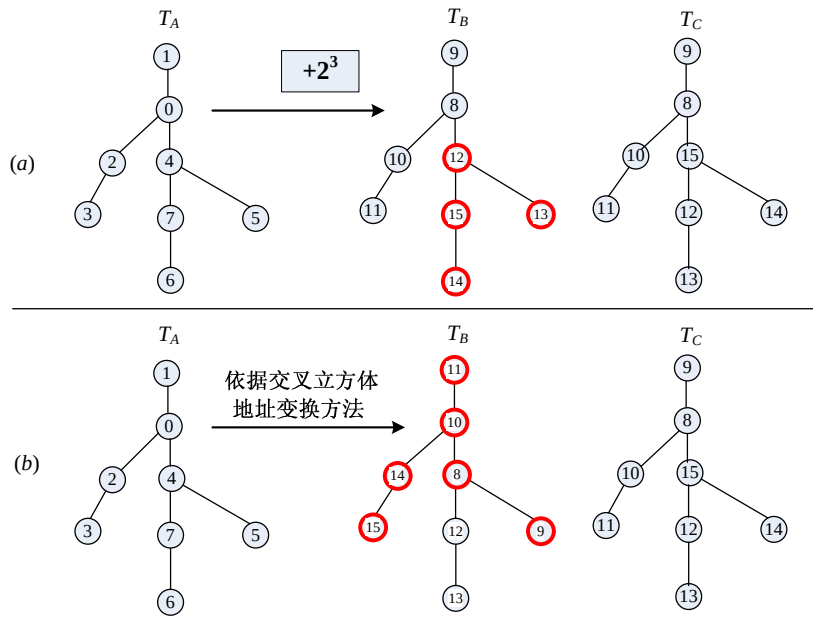
如图 4.1(a) 所示, 对 T_A 中所有顶点按 Q_n 上地址变换方法变换后, 所得到的树 T_B 不是 M_3^1 中的生成树。同样, 对 T_A 中所有顶点按 CQ_n 上地址变换方法变换后, 所得到的树 T_B 也不是 M_3^1 中的生成树, 如图 4.1(b) 所示。

4.2 莫比乌斯立方体的性质

一个长度为 n 的二进制字符串 x 可以表示为 $x_{n-1}x_{n-2}\dots x_0$, 这里 x_{n-1} 是最高位, x_0 是最低位。令 $x_i \in \{0, 1\}$ 为 x 的第 i 位, 且用 $\text{bit}(x, i)$ 来表示。

定义 4.1. 对于任意整数 m , $1 \leq m \leq n-1$ 和二进制串 $s \in \{0, 1\}^m$, $M_n(s)$ 被定义为 M_n 上具有共同前缀 s 的顶点集合导出的子图。

引理 4.1. $M_n(s)$ 同构于 $k \cdot M_{n-|s|}$, 这里 $k = \text{bit}(s, 0)$ 。

图 4.1 Q_n 和 CQ_n 上地址变换方法不适用于 M_n

证明. 根据定义 4.1, $V(M_n(s))$ 中的任一顶点能够表示为 sw , 这里 w 是一个长度为 $n - |s|$ 的二进制串。显然, w 代表 $V(k-M_{n-|s|})$ 中的一个顶点, 这里 $k = \text{bit}(s, 0)$ 。考虑映射 $\psi: V(M_n(s)) \rightarrow V(k-M_{n-|s|})$, 这里 $\psi(sw) = w$ 对于任意 $sw \in V(M_n(s))$ 。显然 ψ 是一个双射。令 sw_1 和 sw_2 是 $V(M_n(s))$ 中的两个顶点, 则 (sw_1, sw_2) 是 $M_n(s)$ 中的一条边当且仅当 (w_1, w_2) 是 $k-M_{n-|s|}$ 中的一条边。因此, $M_n(s)$ 和 $k-M_{n-|s|}$ 是同构的。

对于整数 $n \geq 2$, 如果 $u = u_{n-1}u_{n-2} \dots u_0$ 是 M_n 中的任一顶点, 那么 u 属于 M_n 的导出子图 $M_n(u_{n-1}u_{n-2} \dots u_i)$, 对于任意整数 i , $1 \leq i \leq n-1$, 这里 $M_n(u_{n-1}u_{n-2} \dots u_i)$ 同构于 $u_i-M_{n-|u_{n-1}u_{n-2} \dots u_i|}$ 。

下面的引理描述了如何判断莫比乌斯立方体中一对给定的顶点是否相邻。该引理可以被认为是莫比乌斯立方体的非递归定义。

引理 4.2. 对于任意整数 $n \geq 1$, $(u_{n-1}u_{n-2} \dots u_0, v_{n-1}v_{n-2} \dots v_0)$ 是 M_n 中的一条边当且仅当存在整数 l , 使得

- (1) $u_{n-1}u_{n-2} \dots u_l = v_{n-1}v_{n-2} \dots v_l$,
- (2) $u_{l-1} \neq v_{l-1}$,
- (3) $u_{l-2}u_{l-3} \dots u_0 = v_{l-2}v_{l-3} \dots v_0$ 若 $u_l = 0$, 且
- (4) $u_{l-2}u_{l-3} \dots u_0 = \overline{v_{l-2}v_{l-3} \dots v_0}$ 若 $u_l = 1$ 。

证明. 根据定义 2.7, 该引理成立。

为了简化讨论, 我们也使用十进制来表示 M_n 以任一顶点 u 为根的生成树 T 中的顶点。图 4.2(a) 示出了 T 中顶点之间的邻接关系。例如, 4 是 5 的 0 维邻接点, $4 = N(5, 0)$ 。

引理 4.3. 对于任意两个顶点 $u = u_{n-1}u_{n-2}\dots u_0, v \in V(M_{n-1}^{u_{n-1}})$ 和任意整数 $d, 0 \leq d \leq n-2$, 如果 u 和 v 有一个最左不同位 d , 那么 $N(v, n-1)$ 和 $N(u, n-1)$ 在 $M_{n-1}^{u_{n-1}}$ 中具有一个最左不同位 d 。

证明. 令 $v = u_{n-1}u_{n-2}\dots u_{d+1}\overline{u_d}v_{d-1}v_{d-2}\dots v_0$ 。我们有如下情形:

情形1. $u, v \in V(0-M_n)$ 。那么,

$$N(v, n-1) = \overline{u_{n-1}}u_{n-2}u_{n-3}\dots u_{d+1}\overline{u_d}v_{d-1}v_{d-2}\dots v_0 \in V(M_{n-1}^{u_{n-1}}) \text{ 且}$$

$$N(u, n-1) = \overline{u_{n-1}}u_{n-2}u_{n-3}\dots u_0 \in V(M_{n-1}^{u_{n-1}})。$$

显然, 顶点 $N(v, n-1)$ 和 $N(u, n-1)$ 在 $M_{n-1}^{u_{n-1}}$ 中有一个最左不同位 d 。

情形2. $u, v \in V(1-M_n)$ 。那么,

$$N(v, n-1) = \overline{u_{n-1}}\overline{u_{n-2}}\dots\overline{u_{d+1}}u_d\overline{v_{d-1}}\overline{v_{d-2}}\dots\overline{v_0} \in V(M_{n-1}^{u_{n-1}}) \text{ 且}$$

$$N(u, n-1) = \overline{u_{n-1}}\overline{u_{n-2}}\dots\overline{u_0} \in V(M_{n-1}^{u_{n-1}})。$$

显然, $N(v, n-1)$ 和 $N(u, n-1)$ 在 $M_{n-1}^{u_{n-1}}$ 中具有一个最左不同位 d 。

基于以上讨论, 引理成立。

下面的三个引理将用于证明算法 MIST 构造的树是一棵生成树 (参见 4.3 节)。

引理 4.4. 给定 M_n 上的一条途径 $P: x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$, 对于任意整数 $k, 1 \leq k \leq n$, 如果 $m_1, m_2, \dots, m_k$ 各不相同且 $0 \leq m_i \leq n-1$ 对于 $i = 1, 2, \dots, k$, 那么 P 是一条路径。

证明. 对于任意两个整数 i 和 $j, 0 \leq i < j \leq k$, 令 $l = \max\{m_{i+1}, m_{i+2}, \dots, m_j\}$ 。根据定义 2.7, 我们能够验证 $x^{(i)}$ 和 $x^{(j)}$ 有一个最左不同位 l , 这意味着 $x^{(i)} \neq x^{(j)}$, 从而引理成立。

根据引理 4.4 和定义 2.7, 容易验证引理 4.5。

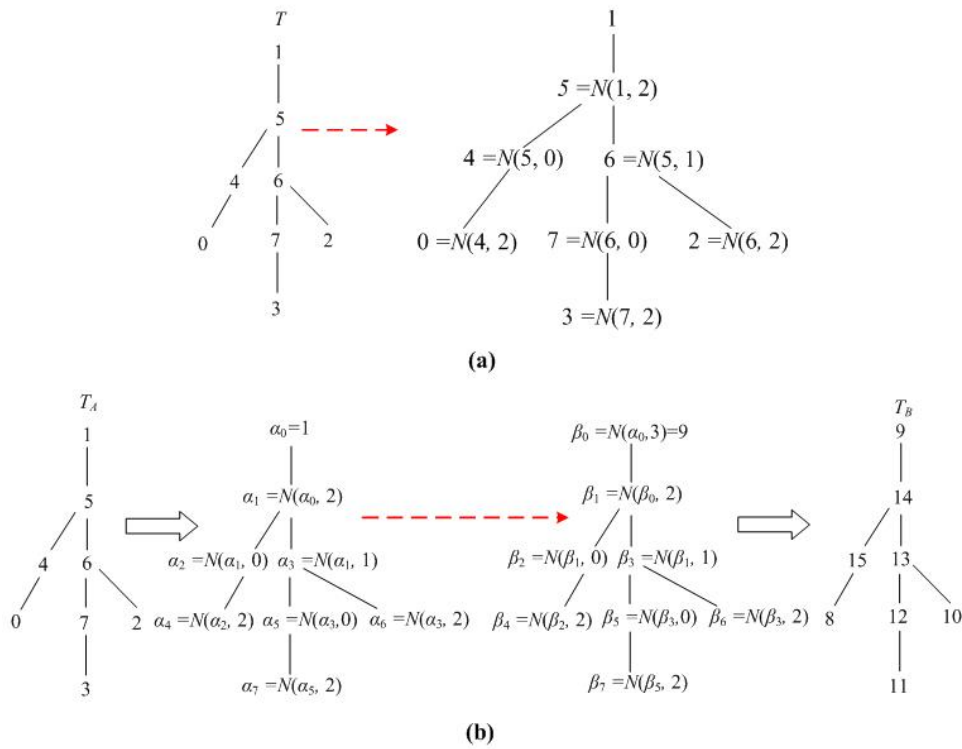


图 4.2 (a) 树 T 中顶点之间的邻接关系; (b) 依据算法 MIOT, 对应于 M_3^0 上以 1 为根的生成树 T_A , M_3^1 上图 T_B 的构造, 这里 T_A 和 T_B 中的顶点皆用十进制数表示。

引理 4.5. 给定 M_n 上的一条途径 $P: x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k), x^{(k+1)} = N(x^{(k)}, m_1)$, 对于任意整数 $k, 2 \leq k \leq n$, 如果 $m_1, m_2, \dots, m_k$ 各不相同且 $0 \leq m_i \leq n-1$ 对于 $i = 1, 2, \dots, k$, 那么 P 是一条路径。

例如, 考虑图 4.3(c) 中 $T_{B,2}$ 上的路径 $P: 5, 6=N(5, 1), 7=N(6, 0), 4=N(7, 1)$ 。

我们称向量 $\langle \alpha_1, \alpha_2, \dots, \alpha_n \rangle$ 为非负向量当 $\alpha_1, \alpha_2, \dots, \alpha_n \geq 0$ 。

引理 4.6. 令非负向量 $\langle \alpha_1, \alpha_2, \dots, \alpha_n \rangle = \langle q-1, q-2, \dots, 0, n-1, n-2, \dots, q \rangle$ 对于任意整数 $q, 0 \leq q \leq n-1$ 。给定 M_n 上的两条路径 $P: x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$ 和 $P': x^{(0)}, y^{(1)} = N(x^{(0)}, m_1'), y^{(2)} = N(y^{(1)}, m_2'), \dots, y^{(t)} = N(y^{(t-1)}, m_t')$, 对于任意两个整数 k 和 $t, 1 \leq k \leq n$ 和 $1 \leq t \leq n$, 这里 $0 \leq m_1, m_2, \dots, m_k, m_1', m_2', \dots, m_t' \leq n-1$, 如果下面条件成立:

- (1) $\langle \alpha_{i_1}, \alpha_{i_2}, \dots, \alpha_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$,
- (2) $\langle \alpha_{j_1}, \alpha_{j_2}, \dots, \alpha_{j_t} \rangle = \langle m_1', m_2', \dots, m_t' \rangle$, 这里 $1 \leq j_1 < j_2 < \dots < j_t \leq n$ 且
- (3) $m_1 \neq m_1'$,

那么 $x^{(k)} \neq y^{(t)}$ 。

证明. 令 $A = \{m_1, m_2, \dots, m_k\}$ 和 $B = \{m_1', m_2', \dots, m_t'\}$ 。假设 $l = \max((A \cup B) \setminus (A \cap B))$ 。根据定义 2.7, 我们能够验证 $x^{(k)}$ 的第 l 位不同于 $y^{(t)}$ 中的对应位。因此, $x^{(k)} \neq y^{(t)}$ 。

例如, 考虑 $T_{B,1}$ 中的路径 $P: 8, 15=N(8, 2), 12=N(15, 1), 13=N(12, 0)$ 和 $T_{B,1}$ 中的路径 $P': 8, 10=N(8, 1), 11=N(10, 0)$, 如图 4 所示。显然, $13 \neq 11$ 。

现在我们给出如下引理, 该引理用来证明算法 MIST_UNION 构造的 n 棵树 (参见第 4.3 节) 是 n 棵 M_n 上以任一顶点为根的 IST。

引理 4.7. 令向量 $\langle \alpha_1, \alpha_2, \dots, \alpha_{n-q-1} \rangle = \langle n-2, n-3, \dots, q \rangle$ 和向量 $\langle \beta_1, \beta_2, \dots, \beta_{n-r-1} \rangle = \langle n-2, n-3, \dots, r \rangle$, 这里 $0 \leq q < r \leq n-2$ 。给定 M_n 上的两条路径 $P: x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k-1)} = N(x^{(k-2)}, m_{k-1}), x^{(k)} = N(x^{(k-1)}, m_k)$ 和 $P': y^{(0)}, y^{(1)} = N(y^{(0)}, m_1'), y^{(2)} = N(y^{(1)}, m_2'), \dots, y^{(t-1)} = N(y^{(t-2)}, m_{t-1}'), y^{(t)} = N(y^{(t-1)}, m_t')$, 对于任意两个整数 k 和 t , $1 \leq k \leq n$ 和 $1 \leq t \leq n$, 这里 $0 \leq m_1, m_2, \dots, m_k, m_1', m_2', \dots, m_t' \leq n-1$, 如果下面条件成立:

- (1) $x^{(0)}$ 和 $y^{(0)}$ 具有一个最左不同位 r ,
- (2) $\langle \alpha_{i_1}, \alpha_{i_2}, \dots, \alpha_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n-q+1$,
- (3) $\langle \beta_{j_1}, \beta_{j_2}, \dots, \beta_{j_t} \rangle = \langle m_1', m_2', \dots, m_t' \rangle$, 这里 $1 \leq j_1 < j_2 < \dots < j_t \leq n-r+1$,
- (4) $m_1 < m_1'$, 且
- (5) $v \in V(P) \cap V(P')$ 和 $v \neq u$,

那么 $\text{ancestor}(v, P) \cap \text{ancestor}(v, P') = \emptyset$ 。

证明. 假设 $v = N(x^{(c-1)}, m_c)$ 和 $v = N(y^{(d-1)}, m_d')$, 这里 $1 \leq c \leq k$ 和 $1 \leq d \leq t$ 。我们有如下两种情形:

情形 1. $m_c = m_d'$ 。我们有 $m_c \geq r$ 和 $m_1' > r$ 。令 $A = \{m_1, m_2, \dots, m_c\}$, $B = \{m_1', m_2', \dots, m_d'\}$ 和 $C = (A \cup B) \setminus (A \cap B)$ 。

情形 1.1. $C = \emptyset$ 。根据定义 2.7, $x^{(0)} = y^{(0)}$ 。这是一个矛盾。

情形 1.2. $C \neq \emptyset$ 。我们有 $\max(C) > r$ 。根据定义 2.7, 我们能够验证 $x^{(0)}$ 和 $y^{(0)}$ 有一个最左不同位 $\max(C)$ 大于 r 。这是一个矛盾。

情形 2. $m_c \neq m_d'$ 。假设路径 $x^{(0)}, x^{(1)}, x^{(2)}, \dots, x^{(c-1)}, v$ 和路径 $y^{(0)}, y^{(1)}, y^{(2)}, \dots, y^{(d-1)}$ 中存在另外一个顶点 $v' \neq v$ 。那么, v 和 v' 是顶点 $x^{(g-1)}$ 的 m_g 维邻接点, v' 是 $y^{(h-1)}$ 的 m_h' 维邻接点, 这里 $1 \leq g \leq c$ 和 $1 \leq h \leq d$ 。

令 $A = \{m_g, m_{g+1}, \dots, m_c\}$ 和 $B = \{m_h', m_{h+1}', \dots, m_d'\}$ 。那么, $(A \cup B) \setminus (A \cap B) \supseteq \min\{m_c, m_d'\}$ 。假设 $l = \max((A \cup B) \setminus (A \cap B))$ 。根据定义 2.7, 我们能够验证 P 中 v 的第 l 位不同于 P' 中 v 的第 l 位。这是一个矛盾。

综上所述, 引理成立。

例如, 考虑 $T_{B,2}$ 中的 $T_{B,1}$ 中的路径 $P: 15, 12, 13$ 和路径 $P': 12, 15$, 如图 4.4 所示。我们有 $\text{ancestor}(15, P) \cap \text{ancestor}(15, P') = \emptyset$ 。

4.3 IST的构造

根据上节的结论, 我们首先讨论 $0-M_n$ 上以任一顶点为根的 IST 的构造。然后, 我们讨论 $1-M_n$ 上以任一顶点为根的 IST 的构造。最后, 我们设计一个算法来构造 M_n 上以任一顶点为根的 IST, 并且分析算法的时间复杂度。接下来, 我们总是使用 $u = u_{n-1}u_{n-2} \dots u_0$ 来表示 M_n 上 IST 的根。

4.3.1 $0-M_n$ 上 IST 的构造

由于 $0-M_n$ ($1-M_n$) 是由两个不同构的 $0-M_{n-1}$ 和 $1-M_{n-1}$ 连接而成, 上一章交叉立方体上的地址变换的方法在 $0-M_n$ ($1-M_n$) 将不再适用, 如在 $0-M_3$ 中存在一条边 $(1,5)$, 1 的邻接点为 9, 依据交叉立方体上的地址变换方法, 将得到边 $(9,13)$, 而边 $(9,13)$ 不是 $1-M_3$ 上的一条边, $(9,14)$ 才是 $1-M_3$ 上的一条边。我们接下来将设计适合于 $0-M_n$ 上独立生成树的构造方法。

对于任意整数 $n \geq 2$, 给定 $M_{n-1}^{u_{n-1}}$ 上以顶点 u 为根的生成树, 我们将提供算法 MIOT, 该算法能够构造图 T_B 。需要指出的是, 算法 MIOT 实际上是算法 MIST 调用的一个子算法, 输入 T_A 将在算法 MIST 中被递归构造。

显然, 给定 $M_{n-1}^{u_{n-1}}$ 上的 T_A , 基于算法 MIOT, T_B 将被唯一地构造出来。

图 4.2(b) 示出了对应于 M_3^0 上以 1 为根的 T_A 的 M_3^1 上以 $N(u, n-1) = 9$ 为根的图 T_B 的构造过程。

根据引理 4.3 和算法 MIOT, 我们有如下结论。

引理 4.8. 对于任意整数 $n \geq 2$, 如果 T_A 是 $M_{n-1}^{u_{n-1}}$ 上的生成树, 那么算法 MIOT 构造的 T_B 是 $M_{n-1}^{u_{n-1}}$ 的子图。

算法 4 MIOT

输入: $M_{n-1}^{u_{n-1}}$ 上以顶点 $u = \alpha_0 = u_{n-1}u_{n-2} \dots u_0$ 为根的生成树 T_A , 这里 $\alpha_0, \alpha_1, \dots, \alpha_{2^{n-1}-1}$ 是 T_A 的顶点

集合且 $\alpha_0, \alpha_1, \dots, \alpha_{2^{n-1}-1}$ 是树 T_A 按广度优先遍历得到的结果;

输出: 一个图 T_B ;

开始

步骤 1:

将树 T_A 复制成树 T_B ;

将 T_B 的根顶点 α_0 转换成 $N(\alpha_0, n-1)$, 表示为 β_0 ;

步骤 2:

for $k = 1$ to $2^{n-1} - 1$ **do**

对于 T_A , 令 $i = \text{LDF}(\alpha_k, \text{parent}(\alpha_k))$;

/*根据定义 2.7, 我们有 $0 \leq i \leq n-1$. */

将 T_B 中的 α_k 替换成 $N(\text{parent}(\alpha_k), i)$, 表示为 β_k ;

end for

结束

定义 4.2. 对于整数 $i = 0, 1, 2, \dots, 2^{n-1} - 1$, 我们定义映射 $\psi : V(T_A) \rightarrow V(T_B)$, 使得 $\psi(\alpha_i) = \beta_i$.

如下引理成立。

引理 4.9. 假设算法 MIOT 构造的 T_B 中的 $\beta_0, \beta_1, \dots, \beta_{2^{n-1}-1}$ 各不相同。那么, T_B 是 $\overline{M_{n-1}^{u_{n-1}}}$ 上一个以 $N(u, n-1)$ 为根的生成树且 ψ 是 T_A 和 T_B 之间的一个同构映射。

类似于第三章 EVS 和 EVSP 的定义, 我们针对算法 MIOT 给出适合于莫比乌斯立方体的 EVS 和 EVSP 的定义。

定义 4.3. 对于整数 $n \geq 2$, 假设 T_A 是 $M_{n-1}^{u_{n-1}}$ 上一棵以顶点 u 为根的生成树, 图 T_B 是算法 MIOT 构造的 $\overline{M_{n-1}^{u_{n-1}}}$ 上以顶点 $N(u, n-1)$ 为根的生成树。如果存在一个非空集合 $W \subseteq V(T_A) \setminus \{u\}$, 使得

(1) 如果 $v \in W$, 那么 $\text{ancestor}(v, T_A) \setminus \{u\} \subseteq W$ 且

(2) $\psi(W) = N(W, n-1)$, 这里 $\psi(W) = \{\psi(v) | v \in W\}$,

那么我们称 W 是一个关于生成树 T_A 的扩展顶点集合 (Extendable Vertex Set), 简写为 EVS。

需要指出的是, 对于任意生成树 T_A , 有可能有多个对应于 T_A 的 EVS。根据定义 4.3, 我们有如下的推论。

推论 4.10. 令 T_A 是 $M_{n-1}^{u_{n-1}}$ 上一棵以顶点 u 为根的生成树, 算法MIOT构造的图 T_B 是 $\overline{M_{n-1}^{u_{n-1}}}$ 上一棵以顶点 $N(u, n-1)$ 为根的生成树。如果存在一个对应于 T_A 的EVS W , 那么 $T_A[W \cup \{u\}]$ 是以 u 为根的树。

定义 4.4. 对于任意整数 $n \geq 2$, 令 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 是 $M_{n-1}^{u_{n-1}}$ 上以顶点 u 为根的 $n-1$ IST, $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$ 是算法MIOT分别基于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 构造出来的树。如果下列条件成立:

(1) 算法MIOT构造的 $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$ 是 $\overline{M_{n-1}^{u_{n-1}}}$ 上 $n-1$ 棵以顶点 $N(u, n-1)$ 为根的IST,

(2) $W_1, W_2, \dots, W_{n-1}$ 是分别对应于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 的EVS,

(3) 对于任意两个整数 $i, j, 1 \leq i < j \leq n-1, W_i \cap W_j = \emptyset$ 且

(4) $\bigcup_{i=1}^{n-1} W_i \cup \{u\} = V(M_{n-1}^{u_{n-1}}),$

那么, 我们称 $\{\{u\}, W_1, W_2, \dots, W_{n-1}\}$ 为 $V(M_{n-1}^{u_{n-1}})$ 上的一个扩展顶点集合划分 (Extendable Vertex Set Partition), 简称EVSP。

图 4.3(c)示出了 $V(M_2^0)$ 上的一个EVSP $\{\{1\}, \{0\}, \{2, 3\}\}$, 使得 $W_1 = \{0\}$ 和 $W_2 = \{2, 3\}$ 是分别对应于 M_2^0 上以顶点1为根的 $T_{A,1}$ 和 $T_{A,2}$ 的EVS。图 4.4 示出了 $V(M_3^0)$ 上的一个EVSP $\{\{1\}, \{0\}, \{2, 3\}, \{4, 5, 6, 7\}\}$, 使得 $W_1 = \{0\}, W_2 = \{2, 3\}$ 和 $W_3 = \{4, 5, 6, 7\}$ 是分别对应于 M_3^0 上以顶点1为根的 $T_{A,1}, T_{A,2}$ 和 $T_{A,3}$ 的EVS。

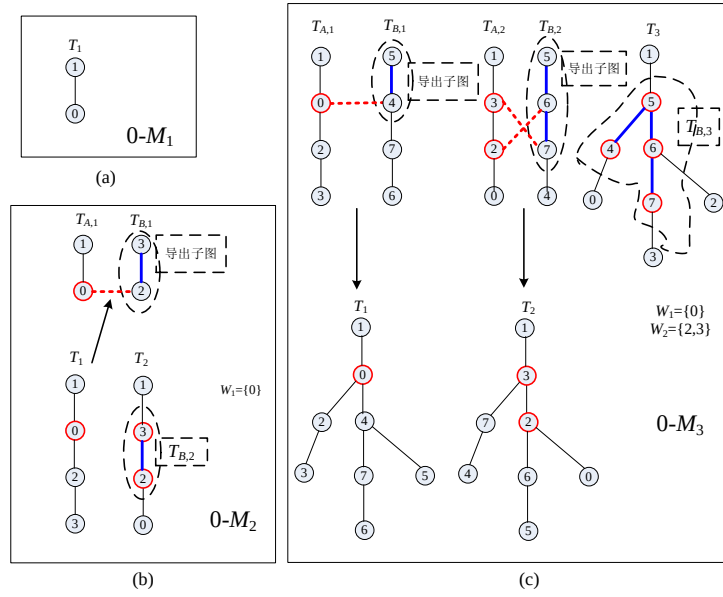
基于如上结论, 我们有如下推论。

推论 4.11. 对于任意整数 $n \geq 2$, 假设 T_A 是 $M_{n-1}^{u_{n-1}}$ 上一棵以顶点 u 为根的生成树, 算法MIOT构造的图 T_B 是 $\overline{M_{n-1}^{u_{n-1}}}$ 上一棵以顶点 $N(u, n-1)$ 为根的生成树。如果 W 是对应于 T_A 的EVS, 那么 $T_B[N(W, n-1) \cup \{N(u, n-1)\}]$ 是 $\overline{M_{n-1}^{u_{n-1}}}$ 上一棵以顶点 $N(u, n-1)$ 为根的生成树。

例如, $0-M_3$ 上以虚线标出的 $T_{B,1}[N(W_1, 2) \cup \{N(u, 2)\}]$ 是一棵以顶点 $N(1, 2) = 5$ 为根的树, 如图 4.3(c)所示。虚线标出的 $0-M_3$ 上的 $T_{B,2}[N(W_2, 2) \cup \{N(u, 2)\}]$ 是一棵以顶点 $N(1, 2) = 5$ 为根的树, 如图 4.3所示。

下面的引理给出了一个决定EVS的方法。

引理 4.12. 对于任意整数 $n \geq 2$, 如果 T_A 是 $M_{n-1}^{u_{n-1}}$ 上一棵以顶点 $u = u_{n-1}u_{n-2} \dots u_0$ 为根的生成树, 算法MIOT构造的图 T_B 是 $\overline{M_{n-1}^{u_{n-1}}}$ 上一棵以顶点 $N(u, n-1)$ 为根的生成树, 且存在集合 $W_A \subseteq V(T_A) \setminus \{u\}$, 使得如下两个条件成立:

图 4.3 $0-M_n$ 上以顶点 $0^{n-1}1$ 为根的 IST $T_1, T_2, \dots, T_n$, 对于 $n = 1, 2$ 和 3

(1) 若 $v \in W$, 那么 $\text{ancestor}(v, T_A) \setminus \{u\} \subset W$, 且

(2) $W = \{u_{n-1}u_{n-2} \dots u_i \overline{u_{i-1}} x_{i-2} x_{i-3} \dots x_0 \in V(M_{n-1}^{u_{n-1}}) | \text{存在一个整数 } i, \text{ 使得 } 1 \leq i \leq n-1\}$,

那么 W 是对应于 T_A 的 EVS。

证明. 根据定义 4.2, 我们只需证明 $\psi(W) = N(W, n-1)$ 。对于任一顶点 $v \in W$, u 和 v 具有一个最左不同位 $i-1$ 。根据引理 4.3, $N(v, n-1)$ 和 $N(u, n-1)$ 具有一个最左不同位 $i-1$ 。根据算法 MIOT, 容易验证 $N(u, n-1)$ 和 $\psi(v)$ 具有最左不同位 d 。我们首先证明 $\psi(W) \subseteq N(W, n-1)$ 。

$\psi(W) \subseteq N(W, n-1)$ 的证明:

我们只需证明对于任一顶点 $p \in \psi(W)$, 存在一个顶点 $x \in W$ 使得 $p = N(x, n-1)$ 。注意到 ψ 是由算法 MIOT 提供的 T_A 和 T_B 之间的同构映射, 存在一个顶点 $v = u_{n-1}u_{n-2} \dots u_i \overline{u_{i-1}} v_{i-2} v_{i-3} \dots v_0 \in W$ 使得 $\psi(v) = p$ 。

根据引理 4.3, 存在一个顶点 $x = u_{n-1}u_{n-2} \dots u_i \overline{u_{i-1}} x_{i-2} x_{i-3} \dots x_0$, 使得 $N(x, n-1) = p$ 。显然, $x \in W$ 。

那么, 类似地, 我们容易证明 $\eta(W) \supseteq N(W, n-1)$ 。

基于如上讨论, 引理成立。

例如, 考虑图 4.4 中 $V(M_3^0)$ 上的 EVSP $\{\{1\}, \{0\}, \{2, 3\}, \{4, 5, 6, 7\}\}$, 使得 $W_1 = \{0\}$ ($W_1 = \{0000\}$), $W_2 = \{2, 3\}$ ($W_2 = \{0010, 0011\}$) 和 $W_3 = \{4, 5, 6, 7\}$ ($W_3 = \{0100, 0101, 0110, 0111\}$) 是 M_3^0 上分别对应于 $T_{A,1}$, $T_{A,2}$ 和 $T_{A,3}$ 的 EVS。

然后, 我们给出算法 MIT。基于该算法, 我们将 $0-M_n$ 上 IST 的构造问题转换成 $M_{n-1}^{u_{n-1}}$ 上 IST 的构造问题和 $M_{n-1}^{\overline{u_{n-1}}}$ 上 IST 的构造问题。注意到算法 MIT 是算法 MIST 调用的一个子算法。

算法 5 MIT

输入: $M_{n-1}^{u_{n-1}}$ 上以 u 为根的 $n-1$ 棵 IST $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$;

算法 MIOT 分别基于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 构造的 $M_{n-1}^{\overline{u_{n-1}}}$ 上 $n-1$ 棵以顶点 $N(u, n-1)$ 为根的 IST

$T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$;

分别对应于 $M_{n-1}^{u_{n-1}}$ 上以顶点 u 为根的 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 的 EVS $W_1, W_2, \dots, W_{n-1}$ 和 $\{u, W_1, W_2, \dots, W_{n-1}\}$ 是 $V(M_{n-1}^{u_{n-1}})$ 的一个 EVSP。

输出: n 棵树 $T_1, T_2, \dots, T_n$;

开始

构造 $0-M_n$ 上的 $T_1, T_2, \dots, T_{n-1}$ 如下:

1: **for** $i = 1$ to $n-1$ **do**

2: $E_i = E(T_{A,i}) \cup E(T_{B,i});$

3: **end for**

4: **for** $i = 1$ to $n-1$ **do**

5: 对于任一 $w \in W_i$, $E_i = E_i \cup \{(w, N(w, n-1))\};$

6: **end for**

7: **for** $i = 1$ to $n-1$ **do**

8: $E_i = E_i \setminus E(T_{B,i}[N(W_i, n-1)]);$

9: **end for**

10: **for** $i = 1$ to $n-1$ **do**

11: $V(T_i) = V(T_{A,i}) \cup V(T_{B,i});$

12: $E(T_i) = E_i;$

13: **end for**

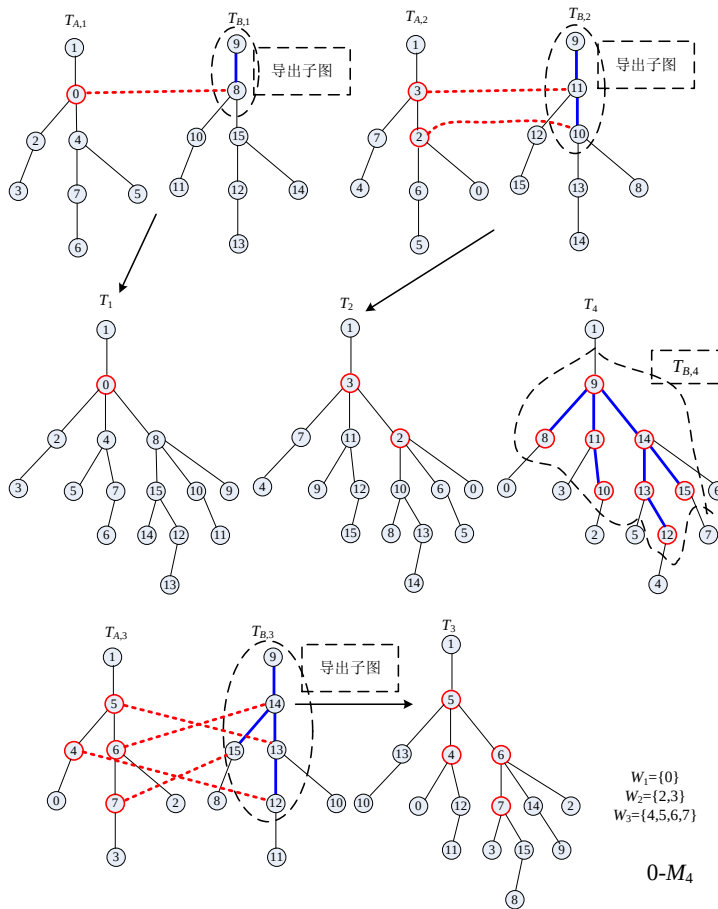
14: $T_{B,n} = \bigcup_{i=1}^{n-1} T_{B,i}[N(W_i \cup \{u\}, n-1)];$

15: $V(T_n) = V(T_{B,n}) \cup N(V(T_{B,n}), n-1);$

16: $E(T_n) = E(T_{B,n}) \cup \{(z, N(z, n-1)) | z \in V(T_{B,n})\};$

结束

图 4.3 示出了 $0-M_n$ 上以顶点 $0^{n-1}1$ 为根的 n 棵生成树 $T_1, T_2, \dots, T_n$ 的构造, 对于 $n = 1, 2$ 和 3 。 $0-M_1$ 上的 T_1 以顶点 1 为根, $V(T_1) = \{1, 0\}$ 和 $E(T_1) = \{(1, 0)\}$ 。 M_1^0 上的 $T_{A,1}$ 来自于 $0-M_1$ 上的 T_1 , $W_1 = \{0\}$ 是关于 $T_{A,1}$ 的 EVS。 M_1^1 上的 $T_{B,2}$ 由 $T_{B,1}$ 上以顶

图 4.4 0-M₄上以顶点1为根的4棵IST T_1, T_2, T_3, T_4

点 $N(1, 1) = 3$ 为根的导出子图组成，以虚线标出。0-M₂上的 T_1 通过将集合 $W_1 = \{0\} \subset V(T_{A,1})$ 中的每个顶点 w 连接 $T_{B,1}$ 中的 $N(w, 1)$ 构造而成。0-M₂上的 T_2 将 $T_{B,2}$ 中的每个顶点连接 $N(z, 1)$ 构造而出。 M_2^0 上的 $T_{A,1}$ 和 $T_{A,2}$ 分别来自于0-M₂上的 T_1 和 T_2 。 $\{\{1\}, \{0\}, \{2, 3\}\}$ 是 $V(M_2^0)$ 上的EVSP，使得 $W_1 = \{0\}$ 和 $W_2 = \{2, 3\}$ 分别是对应于 M_2^0 上 $T_{A,1}$ 和 $T_{A,2}$ 的EVS。 $T_{B,3}$ 由 M_2^1 中根为 $N(1, 2) = 5$ 的 $T_{B,1}$ 上的虚线表示的导出子图和 M_2^1 上根为 $N(1, 2) = 5$ 的 $T_{B,2}$ 上的虚线表示的导出子图组成。类似于上述讨论，我们可以构造0-M₃上以顶点1为根的 T_1, T_2 和 T_3 。

图 4.4示出了如何基于0-M₃上以顶点1为根的二棵IST $T_{A,1}, T_{A,2}$ 和 $T_{A,3}$ ，1-M₃上以顶点9为根的二棵IST $T_{B,1}, T_{B,2}$ 和 $T_{B,3}$ ，以及0-M₃上的一个EVSP $\{\{1\}, \{0\}, \{2, 3\}, \{4, 5, 6, 7\}\}$ 来构造0-M₄上四棵以顶点1为根的IST T_1, T_2, T_3 和 T_4 。

例如，在图 4.4中，我们有 $\text{ancestor}(6, T_1) \cap \text{ancestor}(6, T_2) = \{1\}$ 且 $\text{ancestor}(6, T_1) \cup \text{ancestor}(6, T_2) = \{0, 1, 2, 3, 4, 7\} \supset \{1\}$ 。

接下来,我们将证明算法MIT能够构造 $0-M_n$ 上以 u 为根的 n 棵IST。

定理 4.13. 对于任意整数 $n \geq 2$, 令 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 是 $M_{n-1}^{u_{n-1}}$ 上 $n-1$ 棵以顶点 u 为根的IST, $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$ 是算法MIOT分别基于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 构造出来的图。假设下列条件成立:

- (1) 算法构造的 $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$ 是 $\overline{M_{n-1}^{u_{n-1}}}$ 上 $n-1$ 棵以顶点 $N(u, n-1)$ 为根的IST和
- (2) $W_1, W_2, \dots, W_{n-1}$ 是分别对应于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 的EVS, 使得 $\{\{u\}, W_1, W_2, \dots, W_{n-1}\}$ 是 $V(M_{n-1}^{u_{n-1}})$ 上的EVSP,

那么, 算法MIT构造的 $T_1, T_2, \dots, T_n$ 是 $0-M_n$ 上 n 棵以顶点 u 为根的IST。

证明. 首先, 我们将证明 $T_1, T_2, \dots, T_{n-1}$ 是 $0-M_n$ 上以顶点 u 为根的 $n-1$ 棵生成树。接下来, 我们将在证明 $T_{B,n}$ 是 $u_{n-1}-M_{n-1}$ 上以顶点 $N(u, n-1)$ 为根的一棵生成树的基础上, 进一步证明 T_n 是 $0-M_n$ 上以顶点 u 为根的生成树。最后, 我们将证明 $T_1, T_2, \dots, T_n$ 是相互独立的。

(1) 根据算法MIOT, 容易验证 $T_1, T_2, \dots, T_{n-1}$ 是 $0-M_n$ 上 $n-1$ 棵以顶点 u 为根的生成树。

(2) 根据定义 4.4, 我们有 $\bigcup_{i=1}^{n-1} W_i \cup \{u\} = V(M_{n-1}^{u_{n-1}})$ 。进一步, 我们有 $\bigcup_{i=1}^{n-1} N(W_i \cup \{u\}, n-1) = V(\overline{M_{n-1}^{u_{n-1}}})$ 和 $T_{B,n} = \bigcup_{i=1}^{n-1} T_{B,i}[N(W_i \cup \{u\}, n-1)]$ 。根据推论 4.11, $T_{B,n} = \bigcup_{i=1}^{n-1} T_{B,i}[N(W_i \cup \{u\}, n-1)]$ 是 $\overline{M_{n-1}^{u_{n-1}}}$ 上一棵以顶点 $N(u, n-1)$ 为根的树。因此, $T_{B,n}$ 是 $\overline{M_{n-1}^{u_{n-1}}}$ 上一棵以顶点 $N(u, n-1)$ 为根的生成树。

既然 $V(T_n) = V(T_{B,n}) \cup N(V(T_{B,n}), n-1)$, $E(T_n) = E(T_{B,n}) \cup \{(z, N(z, n-1)) | z \in V(T_{B,n})\}$, T_n 是 $0-M_n$ 上的一棵生成树。

(3) 接下来, 我们将证明 $T_1, T_2, \dots, T_{n-1}$ 是相互独立的, 然后证明 T_n 和 T_i 是相互独立的, 对于任意整数 $i, 1 \leq i \leq n-1$ 。

对于任意整数 $i, j, 1 \leq i < j \leq n$ 和任一顶点 $v \in 0-M_n$ 且 $v \neq u$, 令 $X_i = \text{ancestor}(v, T_i) \cap V(M_{n-1}^{u_{n-1}})$, $Y_i = \text{ancestor}(v, T_i) \cap V(\overline{M_{n-1}^{u_{n-1}}})$, $X_j = \text{ancestor}(v, T_j) \cap V(M_{n-1}^{u_{n-1}})$, $Y_j = \text{ancestor}(v, T_j) \cap V(\overline{M_{n-1}^{u_{n-1}}})$ 。显然, $X_i \cap Y_i = \emptyset$, $X_j \cap Y_j = \emptyset$, $X_i \cap Y_j = \emptyset$, 且 $X_j \cap Y_i = \emptyset$ 。那么, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) = (X_i \cup Y_i) \cap (X_j \cup Y_j) = (X_i \cap X_j) \cup (Y_i \cap Y_j)$ 。

进一步, 我们证明如下两个结论(i) 和(ii):

结论(i) $T_1, T_2, \dots, T_{n-1}$ 是相互独立的。证明如下:

对于任意整数 $i, j, 1 \leq i < j \leq n-1$ 和任一顶点 $v \in V(0-M_n), v \neq u$, 我们有如下情形:

情形1*. $v \in V(M_{n-1}^{u_{n-1}})$ 。显然, $Y_i = \emptyset$ 且 $Y_j = \emptyset$ 。因此, $(X_i \cap X_j) \cup (Y_i \cap Y_j) = X_i \cap X_j$ 。既然 $T_{A,i}$ 和 $T_{A,j}$ 是相互独立的, 根据引理 3.9, $X_i \cap X_j = \{u\}$ 和 $X_i \cup X_j \supset \{u\}$ 。因此, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) = (X_i \cap X_j) \cup (Y_i \cap Y_j) = X_i \cap X_j = \{u\}$ 和 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) = (X_i \cup Y_i) \cup (X_j \cup Y_j) = X_i \cup X_j \supset \{u\}$ 。

情形2*. $v \in V(M_{n-1}^{\overline{u_{n-1}}})$ 。根据算法MIT, $X_i \subseteq W_i \cup \{u\}$ 且 $X_j \subseteq W_j \cup \{u\}$ 。根据定义 4.4, $W_i \cap W_j = \emptyset$ 。既然 $T_{A,i}$ 和 $T_{A,j}$ 是相互独立的, 根据引理 3.9, $X_i \cap X_j = \{u\}$ 。既然 $T_{B,i}$ 和 $T_{B,j}$ 是相互独立的, 根据算法MIT, $Y_i \cap Y_j = \emptyset$ 和 $Y_i \cup Y_j \neq \emptyset$ 。根据算法MIT, $u \notin Y_i \cup Y_j$, 这意味着 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) = (X_i \cup Y_i) \cup (X_j \cup Y_j) \supset \{u\}$ 。进一步, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) = \{u\}$ 。

根据引理 3.9, T_i 和 T_j 是相互独立的。

结论(ii) T_n 和 T_i 是相互独立的, 对于任意整数 $i, 1 \leq i \leq n-1$ 。证明如下:

我们将证明 $X_i \cap X_n = \{u\}$ 和 $Y_i \cap Y_n = \emptyset$ 对于任意整数 $i, 1 \leq i \leq n-1$ 。对于任一顶点 $v \in V(0-M_n)$ 且 $v \neq u$, 我们有如下情形:

情形1. $v \in V(M_{n-1}^{u_{n-1}})$ 。显然, $Y_i = \emptyset$ 且 $X_n = \{u\}$ 。我们能够验证 $Y_n \neq \emptyset$ 和 $u \notin Y_n$, 这意味着 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_n) \supset \{u\}$ 。进一步, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_n) = (X_i \cap X_n) \cup (Y_i \cap Y_n) = \{u\}$ 。

情形2. $v \in V(M_{n-1}^{\overline{u_{n-1}}})$ 。显然, $X_n = \text{ancestor}(v, T_n) \cap V(M_{n-1}^{u_{n-1}}) = \{u\}$, $Y_i = \text{ancestor}(v, T_i) \cap V(M_{n-1}^{\overline{u_{n-1}}})$ 和 $X_i \cap X_n = \{u\}$ 。那么, 根据算法MIT, 存在整数 $k, 1 \leq k \leq n-1$, 使得 $Y_n \subseteq N(W_k, n-1)$ 。容易验证 $N(u, n-1) \notin Y_i$ 和 $N(u, n-1) \in Y_n$ 。因此, $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_n) = (X_i \cup Y_i) \cup (X_n \cup Y_n) \supseteq \{u\} \cup \{N(u, n-1)\} \supset \{u\}$ 。进一步, 我们将证明 $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_n) = \{u\}$, 分为如下子情形:

情形2.1. $i \neq k$ 。显然, $Y_i \cap N(\{u\}, n-1) = \emptyset$ 。既然 $T_{B,i}$ 和 $T_{B,k}$ 是相互独立的, $Y_i \cap Y_n \subseteq Y_i \cap N(W_k \cup \{u\}, n-1) = (Y_i \cap N(W_k, n-1)) \cup (Y_i \cap N(\{u\}, n-1)) = Y_i \cap N(W_k, n-1) = \emptyset$ 。因此, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_n) = (X_i \cap X_n) \cup (Y_i \cap Y_n) = \{u\}$ 。

情形2.2. $i = k$ 。显然, $Y_i \cap N(\{u\}, n-1) = \emptyset$ 。根据算法MIT, 我们容易验证 $v \in N(W_i \cup \{u\}, n-1)$, 使得 $Y_i = \emptyset$ 和 $Y_i \cap Y_n = \emptyset$ 。相应地, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_n) = (X_i \cap X_n) \cup (Y_i \cap Y_n) = \{u\}$ 。

根据引理 3.9, T_i 和 T_n 是相互独立的。

综上所述, $T_1, T_2, \dots, T_n$ 是相互独立的。

需要指出的是, 基于上述讨论, 在 $V(M_{n-1}^{u_{n-1}})$ 上有可能存在多个 EVSP。如对于图 4.4, 我们给出表 4.1, 表中的每一行的第1个单元格内容均可是一个 EVSP, 第2, 3, 4个单元格内容分别是对应于 $T_{A,1}$, $T_{A,2}$ 和 $T_{A,3}$ 的 EVS。

表 4.1 基于图 4.4 的 EVSP

EVSP	$T_{A,1}$ 的 EVS	$T_{A,2}$ 的 EVS	$T_{A,3}$ 的 EVS
$\{\{1\}, \{0\}, \{3\}, \{2,4,5,6,7\}\}$	$\{0\}$	$\{3\}$	$\{2,4,5,6,7\}$
$\{\{1\}, \{0\}, \{2,3\}, \{4,5,6,7\}\}$	$\{0\}$	$\{2,3\}$	$\{4,5,6,7\}$
$\{\{1\}, \{0,2\}, \{3\}, \{4,5,6,7\}\}$	$\{0,2\}$	$\{3\}$	$\{4,5,6,7\}$
$\{\{1\}, \{0,4,7\}, \{3\}, \{2,5,6\}\}$	$\{0,4,7\}$	$\{3\}$	$\{2,5,6\}$
$\{\{1\}, \{0\}, \{3,4,7\}, \{2,5,6\}\}$	$\{0\}$	$\{3,4,7\}$	$\{2,5,6\}$
$\{\{1\}, \{0,2\}, \{3,4,7\}, \{5,6\}\}$	$\{0,2\}$	$\{3,4,7\}$	$\{5,6\}$
$\{\{1\}, \{0,4,7\}, \{2,3\}, \{5,6\}\}$	$\{0,4,7\}$	$\{2,3\}$	$\{5,6\}$
$\{\{1\}, \{0,2,4,7\}, \{3\}, \{5,6\}\}$	$\{0,2,4,7\}$	$\{3\}$	$\{5,6\}$
$\{\{1\}, \{0\}, \{2,3,4,7\}, \{5,6\}\}$	$\{0\}$	$\{2,3,4,7\}$	$\{5,6\}$

为构造 $0-M_n$ 上以任一顶点为根的 n 棵 IST, 我们将提供一个特定的规则来选择 $V(M_{n-1}^{u_{n-1}})$ 上的 EVSP (参见算法 MIST 中的第 15–20 行)。接下来, 我们将给出一个算法, 称为 MIST, 来构造 $0-M_n$ 上以顶点 $u = u_{n-1}u_{n-2} \dots u_0$ 为根的 n 棵 IST, 这里 $n \geq 2$ 。

基于算法 MIST, $0-M_n$ 上的生成树 T_i 通过成对结合 $T_{A,i}$ 和 $T_{B,i}$ 递归构造, 对于 i , $1 \leq i \leq n-1$ 。进一步, 令 $T_{B,n} = \bigcup_{i=1}^{n-1} T_{B,i}[N(W_i \cup \{u\}, n-1)]$, $T_{B,n}$ 是 $M_{n-1}^{u_{n-1}}$ 上的一棵生成树。 T_n 是通过连接 $T_{B,n}$ 中的每一个顶点 z 和 $N(z, n-1)$ 构成。

图 4.3 和图 4.4 事实上示出了算法 MIST 在低维莫比乌斯立方体 $0-M_n$ 的递归构造以 1 为根的 IST 过程, 这里 $n = 1, 2, 3, 4$ 。

表示法 4.1. 如果 $G' = (V', E')$ 是 $0-M_n$ 的子图, 对于任意整数 $n \geq 2$ 和 $s \in \{0, 1\}$, 那么我们用 sG' 来表示图 (sV', sE') , 这里 $sE' = \{(sx, sy) | (x, y) \in E'\}$ 。

定理 4.14. 对于任意整数 $n \geq 2$ 和 $0-M_n$ 上的任一顶点 u , 如下结论成立:

(1) 算法 MIST 构造 $M_{n-1}^{u_{n-1}}$ 上以顶点 $u = u_{n-1}u_{n-2} \dots u_0$ 为根的 $n-1$ 棵 IST $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 和 $V(M_{n-1}^{u_{n-1}})$ 上的一个 EVSP $\{\{u\}, W_1, W_2, \dots, W_{n-1}\}$, 使得 $W_1, W_2, \dots, W_{n-1}$ 是分别对应于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 的 EVS, 和

算法 6 MIST($n, u_{n-1}u_{n-2} \dots u_0$)

输入: 整数 $n \geq 2$ 和 $0-M_n$ 上的 $u = u_{n-1}u_{n-2} \dots u_0$;

输出: $0-M_n$ 上 n 棵以顶点 u 为根的树 $T_1, T_2, \dots, T_n$;

开始

1: $W_i = \emptyset$, 对于 $i = 1, 2, \dots, n-1$;

步骤 1:

2: **if** ($n == 1$) **then**

3: 返回以 u_0 为根的树 T_1 , $V(T_1) = \{u_0, N(u_0, 0)\}$ 且 $E(T_1) = \{(u_0, N(u_0, 0))\}$;

4: **else**

5: 调用 MIST($n-1, u_{n-2}u_{n-3} \dots u_0$);

6: **end if**

步骤 2:

7: **for** $i = 1$ to $n-1$ **do**

8: 若 $(u_{n-1} == 0)$, 则 $T_{A,i} = u_{n-1}T_i$; 否则 $T_{B,i} = \overline{u_{n-1}}T_i$;

9: **end for**

10: **if** ($u_{n-1} == 0$) **then**

11: 使用算法 MIOT 分别基于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 构造 $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$;

12: **else**

13: 使用算法 MIOT 分别基于 $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$ 构造 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$;

14: **end if**

步骤 3:

15: **if** ($n == 2$) **then**

16: $W_1 = \{u_1 \overline{u_0}\}$;

17: **else**

18: $W_i = u_{n-1}W_i$, 对于 $i = 1, 2, \dots, n-2$;

19: $W_{n-1} = \bigcup_{i=1}^{n-2} N(W_i \cup \{N(u, n-2)\}, n-2)$;

20: **end if**

步骤 4:

21: 通过使用算法 MIT, 基于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}, T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$ 和 $W_1, W_2, \dots, W_{n-1}$ 使得 $W_1, W_2, \dots, W_{n-1}$ 是分别对应于 $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ 的 EVS, $\{u, W_1, W_2, \dots, W_{n-1}\}$ 是 $V(M_{n-1}^{u_{n-1}})$ 上的 EVSP, 构造 $0-M_n$ 上以顶点 u 为根的 $T_1, T_2, \dots, T_n$;

/*注意到我们总是令 $T_1, T_2, \dots, T_n$ 表示 $0-M_n$ 上 n 棵以顶点 $u_{n-1}u_{n-2} \dots u_0$ 为根的 IST。*/

结束

$W_i = \{u_{n-1}u_{n-2} \dots u_i \overline{u_{i-1}} x_{i-2} x_{i-3} \dots x_0 \in V(M_{n-1}^{u_{n-1}})\}$ 对于任意整数 $i, 1 \leq i \leq n-1$ 。

(2) 算法MIST构造出的 $T_1, T_2, \dots, T_n$ 是 $0-M_n$ 上 n 棵以顶点 u 为根的IST。

(3) 假设 P 是 T_i 上的任一 $\langle u, v \rangle$ -路径, 对于整数 $i = 1, 2, \dots, n$, 这里 u 表示根顶点, v 表示 T_i 中的任意叶子顶点。令 $\langle u, v \rangle$ -路径为序列: $u, x^{(1)} = N(u, m_0), x^{(2)} = N(x^{(1)}, m_1), \dots, N(x^{(k)}, m_k) = v$, 这里 $1 \leq k \leq n$ 和 $0 \leq m_0, m_1, \dots, m_k \leq n-1$ 。那么, 如下子结论成立:

(i) $m_k = m_0 = i-1$ 。

(ii) 令非负向量 $\langle \alpha_1, \alpha_2, \dots, \alpha_n \rangle = \langle i-2, i-3, \dots, 0, n-1, n-2, \dots, i-1 \rangle$ 。那么 $\langle \alpha_{i_1}, \alpha_{i_2}, \dots, \alpha_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$ 。

证明. 我们采用关于 n 的数学归纳法进行证明。

基础: 我们容易验证该定理对于 $n=2$ 成立。

归纳: 假设定理对于 $n=\tau (\tau \geq 2)$ 成立, 即:

(1) 算法MIST构造出 $M_{\tau-1}^{u_{\tau-1}}$ 上以 $u_{\tau-1}u_{\tau-2} \dots u_0$ 为根的 $\tau-1$ IST $T_{A,1}', T_{A,2}', \dots, T_{A,\tau-1}'$ 和 $V(M_{\tau-1}^{u_{\tau-1}})$ 上的一个EVSP $\{u_{\tau-1}u_{\tau-2} \dots u_0, W_1', W_2', \dots, W_{\tau-1}'\}$, 使得 $W_1', W_2', \dots, W_{\tau-1}'$ 分别是对应于 $T_{A,1}', T_{A,2}', \dots, T_{A,\tau-1}'$ 的EVS, 和

$W_i' = \{u_{\tau-1}u_{\tau-2} \dots u_i \overline{u_{i-1}} x_{i-2} x_{i-3} \dots x_0 \in V(M_{\tau-1}^{u_{\tau-1}})\}$, 对于任意整数 $i, 1 \leq i \leq \tau-1$ 。

(2) 算法MIST构造出的 $T_1', T_2', \dots, T_\tau'$ 是 $0-M_\tau$ 上 τ 棵以 $u_{\tau-1}u_{\tau-2} \dots u_0$ 为根的IST。

(3) 假设 P 是 T_i' 上的任一 $\langle u_{\tau-1}u_{\tau-2} \dots u_0, v \rangle$ -路径, 对于整数 $i = 1, 2, \dots, \tau$, 这里 $u_{\tau-1}u_{\tau-2} \dots u_0$ 表示根顶点, v 表示 T_i' 中的任意叶子顶点。令 $\langle u_{\tau-1}u_{\tau-2} \dots u_0, v \rangle$ -路径为序列: $u_{\tau-1}u_{\tau-2} \dots u_0, x^{(1)} = N(u_{\tau-1}u_{\tau-2} \dots u_0, m_0), x^{(2)} = N(x^{(1)}, m_1), \dots, N(x^{(k)}, m_k) = v$ 对于整数 $k, 1 \leq k \leq \tau$ 和 $0 \leq m_0, m_1, \dots, m_k \leq n-1$ 。那么, 如下两个子结论成立:

(i) $m_k = m_0 = i-1$ 。

(ii) 令非负向量 $\langle \alpha_1, \alpha_2, \dots, \alpha_\tau \rangle = \langle i-2, i-3, \dots, 0, \tau-1, \tau-2, \dots, i-1 \rangle$ 。那么 $\langle \alpha_{i_1}, \alpha_{i_2}, \dots, \alpha_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq \tau$ 。

我们将证明该定理对于 $n=\tau+1$ 成立。

基于算法MIST, 我们有如下子情形:

情形1. $u_\tau = 0$ 。那么, $T_{A,1} = u_\tau T_1', T_{A,2} = u_\tau T_2', \dots, T_{A,\tau} = u_\tau T_\tau'$ 。显然, $T_{A,1}, T_{A,2}, \dots, T_{A,\tau}$ 是 $M_\tau^{u_\tau}$ 上 τ 以顶点 $u_\tau u_{\tau-1} \dots u_0$ 为根的IST。

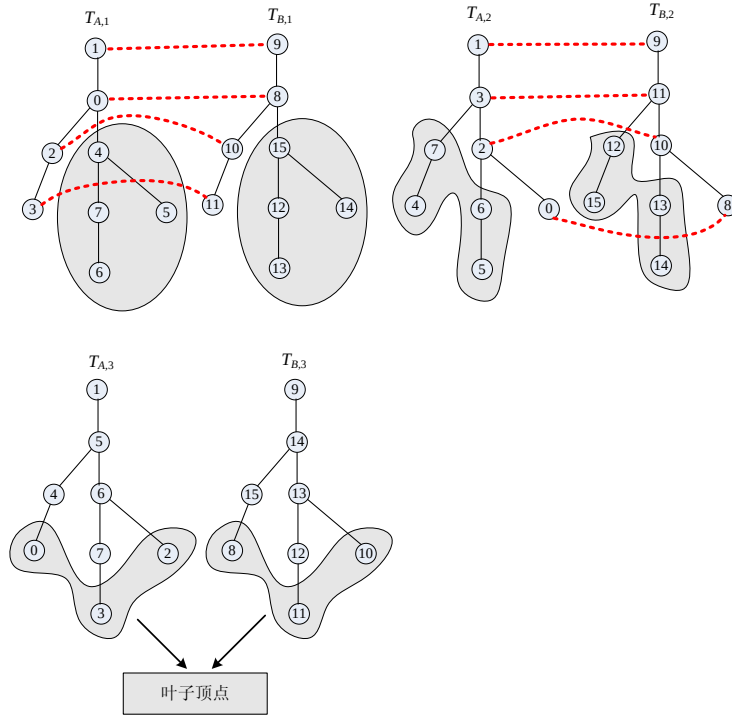


图 4.5 双射: $V(M_4(00) \cap V(T_{A,i})) \rightarrow V(M_4(10) \cap V(T_{B,i}))$ 对于 $1 \leq i \leq 2$, $V(M_4(00)) \setminus \{0\}$ 中的顶点和 $V(M_4(10)) \setminus \{9\}$ 中的顶点分别是 $T_{A,3}$ 和 $T_{B,3}$ 中的叶子顶点

依据算法MIST (参见算法MIST中的第10–13行), 我们得到 τ 个子图 $T_{B,1}, T_{B,2}, \dots, T_{B,\tau}$ 。根据引理 4.8, 我们能够验证对于 $T_{B,i}$ 上的任一 $\langle N(u_\tau u_{\tau-1} \dots u_0, \tau), v' \rangle$ -路径满足定理 4.14中的结论(3), 对于 $i = 1, 2, \dots, \tau$, 这里 $N(u_\tau u_{\tau-1} \dots u_0, \tau)$ 表示根顶点和 v' 表示树 $T_{B,i}$ 中的任意叶子顶点。根据引理 4.4, 4.5 和 4.6, 由算法MIOT构造的 $T_{B,1}, T_{B,2}, \dots, T_{B,\tau}$ 是 $M_\tau^{\overline{u_\tau}}$ 上 τ 棵以顶点 $N(u_\tau u_{\tau-1} \dots u_0, \tau)$ 为根的生成树。

根据定义 2.7 和引理 4.9, 对于任意顶点 $v \in V(M_{\tau+1}(\overline{u_\tau} u_{\tau-1}))$, 在 $T_{B,i}$ 和 $T_{B,j}$ 中, 我们有 $\psi(v) = N(v, \tau)$, 这里 $1 \leq i < j \leq \tau$ 。 $V(M_{\tau+1}(u_\tau u_{\tau-1})) \setminus \{u\}$ 和 $V(M_{\tau+1}(\overline{u_\tau} u_{\tau-1})) \setminus \{N(u_\tau u_{\tau-1} \dots u_0, \tau)\}$ 中的顶点分别是 $T_{A,\tau}$ 和 $T_{B,\tau}$ 上的叶子顶点, 如图 4.5所示。既然在 $T_{A,i}$ 和 $T_{A,j}$ 中 $\text{ancestor}(N(v, \tau), T_i) \cap \text{ancestor}(N(v, \tau), T_j) = \{u\}$ 和 $\text{ancestor}(N(v, \tau), T_i) \cup \text{ancestor}(N(v, \tau), T_j) \supset \{u\}$, 在 $T_{B,i}$ 和 $T_{B,j}$ 中, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) = \{N(u, \tau)\}$ 和 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) \supset \{N(u, \tau)\}$ 。因此, $V(M_{\tau+1}(\overline{u_\tau} u_{\tau-1})) \setminus \{N(u_\tau u_{\tau-1} \dots u_0, \tau)\}$ 中的顶点满足引理 3.9中的条件。根据引理 4.7, 我们能够验证 $V(M_{\tau+1}(\overline{u_\tau} \overline{u_{\tau-1}})) \setminus \{N(u_\tau u_{\tau-1} \dots u_0, \tau)\}$ 中的顶点满足引理 3.9中的条件。因此, $T_{B,1}, T_{B,2}, \dots, T_{B,\tau}$ 是 $M_\tau^{\overline{u_\tau}}$ 上 τ 棵以顶点 $N(u_\tau u_{\tau-1} \dots u_0, \tau)$ 为根的IST。

根据算法MIST, 对于任意整数 i , $1 \leq i \leq \tau - 1$,

$$W_i = u_\tau W_i'. \quad (4.1)$$

进一步,

$$W_\tau = \bigcup_{j=1}^{\tau-1} N(W_j' \cup \{u_\tau u_{\tau-1} \dots u_0\}, \tau - 1). \quad (4.2)$$

基于上述等式(4.1)和(4.2), 我们有如下结论:

对于任意整数 i , $1 \leq i \leq \tau$,

$$W_i = \{u_\tau u_{\tau-1} u_{\tau-2} \dots u_i \overline{u_{i-1} x_{i-2} x_{i-3} \dots x_0} \in V(M_\tau^{u_\tau})\}.$$

那么, 对于任意整数 i, j , $1 \leq i < j \leq \tau$, 我们有 $W_i \cap W_j = \emptyset$ 和 $\bigcup_{i=1}^{\tau} W_i \cup \{u_\tau u_{\tau-1} \dots u_0\} = V(M_\tau^{u_\tau})$. 根据引理 4.12, 我们能够验证 $W_1, W_2, \dots, W_\tau$ 是分别对应于 $M_\tau^{u_\tau}$ 上以顶点 $u_\tau u_{\tau-1} \dots u_0$ 为根的 $T_{A,1}, T_{A,2}, \dots, T_{A,\tau}$ 的EVS. 根据定义 4.4, $\{u_\tau u_{\tau-1} \dots u_0\}, W_1, W_2, \dots, W_\tau$ 是 $V(M_\tau^{u_\tau})$ 上的一个EVSP. 根据算法MIST, 我们得到 $0-M_{\tau+1}$ 上以顶点 $u_\tau u_{\tau-1} \dots u_0$ 为根的 $\tau + 1$ 树 $T_1, T_2, \dots, T_{\tau+1}$. 根据定理 4.13, $T_1, T_2, \dots, T_{\tau+1}$ 是 $0-M_{\tau+1}$ 上以顶点 $u_\tau u_{\tau-1} \dots u_0$ 为根的 $\tau + 1$ 棵IST.

进一步, 基于上述讨论, 容易验证如下结论:

假设 P 是 T_i 中的任一 $\langle u_\tau u_{\tau-1} \dots u_0, v \rangle$ -路径, 对于整数 $i = 1, 2, \dots, \tau + 1$, 这里 $u_\tau u_{\tau-1} \dots u_0$ 表示根顶点, v 表示 T_i 中的任意叶子顶点. 令 $\langle u_\tau u_{\tau-1} \dots u_0, v \rangle$ -路径为序列: $u_\tau u_{\tau-1} \dots u_0, x^{(1)} = N(u_\tau u_{\tau-1} \dots u_0, m_0), x^{(2)} = N(x^{(1)}, m_1), \dots, N(x^{(k)}, m_k) = v$, 这里 $1 \leq k \leq \tau + 1$ 和 $0 \leq m_0, m_1, \dots, m_k \leq n - 1$. 那么, 如下子结论成立:

(i) $m_k = m_0 = i - 1$.

(ii) 令非负向量 $\langle \alpha_1, \alpha_2, \dots, \alpha_\tau \rangle = \langle i - 2, i - 3, \dots, 0, \tau, \tau - 1, \dots, i - 1 \rangle$. 那么 $\langle \alpha_{i_1}, \alpha_{i_2}, \dots, \alpha_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq \tau + 1$.

情形2. $u_\tau = 1$. 类似于情形, 我们可以证明.

因此, 定理对于 $n = \tau + 1$ 成立.

如下结论是显然的.

定理 4.15. 算法MIST在 $O(N \log N)$ 时间内构造出 $0-M_n$ 上 n 棵任一顶点 u 为根的IST, 这里 $N = 2^n$.

4.3.2 $1-M_n$ 上IST的构造

因此, 我们给出一个时间复杂度为 $O(N\log N)$ 的完整算法, 称为MIST_UNION, 如下:

算法 7 MIST_UNION($n, u_{n-1}u_{n-2}\dots u_0$)

输入: 整数 $n \geq 2$ 和 M_n 中的顶点 $u = u_{n-1}u_{n-2}\dots u_0$;

输出: M_n 上 n 棵以顶点 u 为根的树 $T_1, T_2, \dots, T_n$;

开始

1: **if** ($u \in V(0-M_n)$) **then**

2: 调用算法MIST(n, u);

3: **else**

4: 调用算法MIST(n, u);

5: 令 $T_1'', T_2'', \dots, T_n''$ 表示算法MIST构造的 $0-M_n$ 上以顶点 u 为根的树;

6: 令 $T_1' = 0T_1'', T_2' = 0T_2'', \dots, T_n' = 0T_n''$ 表示算法MIST构造的 M_n^0 上以顶点 u 为根的 n 棵树;

7: 通过算法MIOT, 基于 $0-M_n$ 上以 u 为根的 $T_1', T_2', \dots, T_n'$ 分别构造 M_n^1 上以顶点 u 为根的树 $T_1, T_2, \dots, T_n$;

8: 删除 $V(T_1), V(T_2), \dots, V(T_n)$ 中的公共前缀'1';

9: **end if**

结束

基于引理 4.8, 定理 4.14 和定理 4.15, 我们有如下定理。

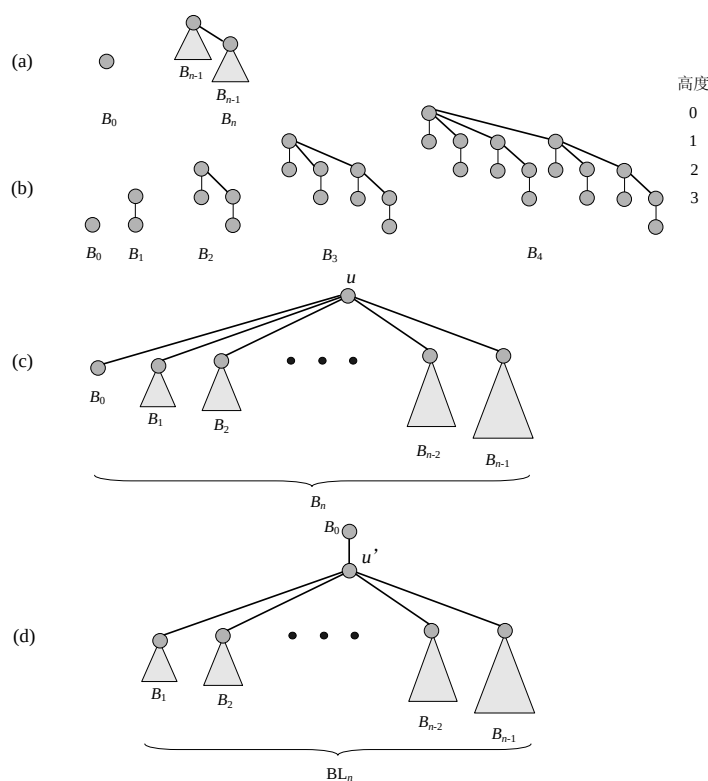
定理 4.16. 对于顶点 $u \in 1-M_n$, 基于算法MIST构造的 $0-M_n$ 上以顶点 u 为根的 n 棵IST $T_1', T_2', \dots, T_n'$, 如下结论成立:

- (1) 算法MIOT_UNION构造的 $T_1, T_2, \dots, T_n$ 是 $1-M_n$ 上 n 棵以任一顶点 u 为根的IST。
- (2) $T_1, T_2, \dots, T_n$ 分别同构于 $T_1', T_2', \dots, T_n'$ 。

4.4 算法MIST 和MIST_UNION构造的IST的性质

为了证明算法MIST构造的 $0-M_n$ 上以任一顶点为根的IST是相互同构的, 我们给出如下关于二项树的表示法。

一棵0-级二项树 (B_0) 有一个顶点。一棵 n -级二项树 (B_n) 由两棵 $(n-1)$ -级二项树通过在两棵树的根顶点之间增加一条公共边并且令任意一棵树的根为新的根顶点组成, 如图 4.6(a)所示。一棵 B_n 也能够基于 $B_{n-1}, B_{n-2}, \dots, B_0$ 通过使用顶点 u (根顶点) 来连接这些树的根顶点构成, 如图 4.6(b)和(c)所示。为简便起见, 我们

图 4.6 B_n 和 BL_n

使用 $\text{children_number}(v)$ 来表示树中一个顶点 v 的孩子顶点数量。显然，在树 B_n 中， $\text{children_number}(u)$ 值为 n ，如图 4.6(c) 所示。

定义 4.5. 类二项树 (Binomial-Like Tree) 被定义为：

- (1) 一棵 0-级类二项树 BL_0 有一个顶点。
- (2) 一棵 n -级类二项树 BL_n 能够由 $B_{n-1}, B_{n-2}, \dots, B_0$ 通过使用顶点 u' 来连接这些树的顶点构成。 BL_n 的根顶点为 B_0 。

图 4.6(d) 示出了 BL_n 。

根据定义 4.5， n 级类二项树 BL_n 表示为 T ，根顶点 u 只有一个孩子顶点 u' ，称为虚根 (Virtual Root)。接下来，我们总是假设 T 是以 u' 为根计算树 T 中孩子顶点的个数。根据定义 4.5，我们有如下引理：

引理 4.17. 给定一个 BL_n ，表示为 T ，令 u' 是 T 中根顶点 u 的唯一孩子顶点。如下结论成立：

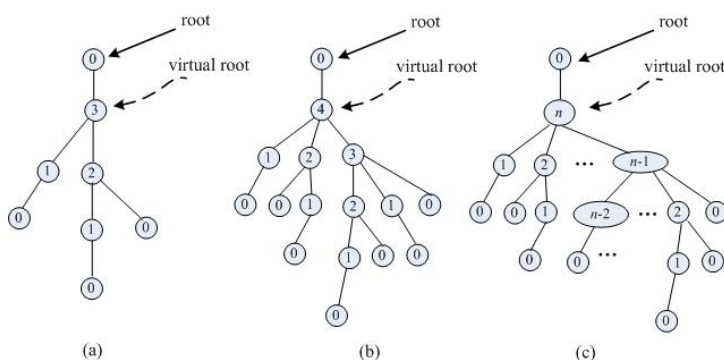


图 4.7 BL_3 , BL_4 和 BL_n , 这里每个顶点都标记为它的孩子顶点的个数

(1) 如果 T 以 u' 为根, 那么 $children_number(u) = 0$, $children_number(u') = n$, 且 u' 具有 n 个孩子顶点 $v_0, v_1, \dots, v_{n-1}$, 分别具有的孩子顶点的个数为 $0, 1, \dots, n-1$ 。

(2) 对于任一顶点 $v \in V(T) \setminus \{u, u'\}$, 令 $d = children_number(v)$ 。如果 $d > 0$, 那么 v 具有 d 个孩子顶点 $v_0, v_1, \dots, v_{d-1}$, 分别具有孩子顶点的个数为 $0, 1, \dots, d-1$ 。否则 v 是叶子顶点。

接下来, 我们使用引理 4.17 来给出一棵树中顶点的孩子顶点个数。例如, 图 4.7 (a), (b), (c) 分别示出了 BL_3 , BL_4 , BL_n 。各个顶点的值代表了其从虚根顶点观察得到的孩子顶点个数。

根据定义 4.5, 我们有如下引理。

引理 4.18. 对于 BL_n , 顶点个数为 2^n 。

引理 4.19. BL_n 的高度为 $n+1$, 对于任意整数 $n \geq 2$ 。

引理 4.20. 对于任意整数 $n \geq 2$ 和 $1 \leq i \leq n$, 如果 T 同构于 BL_n , 存在集合 $W \subset V(T)$, 使得如下条件成立:

- (1) $W = \{v \in V(T) | children_number(v) \geq i\}$,
- (2) 树 T' 通过将集合 W 中的每个顶点 z 链接 B_i (i -级二项树) 构成。

那么, T' 同构于 BL_{n+1} 。

图 4.4 示出了 $0-M_4$ 上依据算法 MIST 的 4 棵 IST T_1, T_2, T_3 和 T_4 的构造。我们能够验证 T_i 同构于 BL_4 , 对于 $i = 1, 2, 3, 4$ 。观察图 4.4 中的 T_2 , 容易验证 $T_{A,2}$ 同构于 BL_3 且存在扩展顶点集合 $W = \{2, 3\}$ 。显然, $W = \{v \in V(T_{A,2}) | children_number(v) \geq 2\}$ 。顶点 2 和 3 被连接到 B_2 。因此, T_2 同构于 BL_4 。

引理 4.21. 对于任意整数 $n \geq 2$, 集合 $S = \{T_1, T_2, \dots, T_n\}$ 中的每一棵由算法 *MIST* 构造的 $0-M_n$ 上以顶点 u 为根的 *IST* T_i 同构于 BL_n , 使得

$$W_i = \{v \in V(T_{A,i}) | \text{children_number}(v) \geq n - i\} \text{ 对于 } i = 1, 2, \dots, n - 1.$$

证明. 我们采用关于 n 的数学归纳法证明。

基础: 我们容易验证引理对于 $n = 2$ 成立, 如图 4.3(c) 所示。

归纳: 假设引理对于 $n = \tau (\tau \geq 2)$ 成立, 即:

对于任意整数 $\tau, \tau \geq 2$, 集合 $S = \{T_1', T_2', \dots, T_\tau'\}$ 中的每一棵由算法 *MIST* 构造的 *IST* T_i 同构于 BL_τ , 使得 $W_i' = \{v \in (T_{A,i}') | \text{children_number}(v) \geq \tau - i\}$, 对于 $i = 1, 2, \dots, \tau - 1$ 。

根据定理 4.14, 我们有如下结论。

(1) 算法 *MIST* 构造 $M_{\tau-1}^{u_{\tau-1}u_{\tau-2}\dots u_0}$ 上以顶点 $u_{\tau-1}u_{\tau-2}\dots u_0$ 为根的 $\tau - 1$ 棵 *IST* $T_{A,1}', T_{A,2}', \dots, T_{A,\tau-1}'$ 和 $V(M_{\tau-1}^{u_{\tau-1}u_{\tau-2}\dots u_0})$ 上的一个 *EVSP* $\{\{u_{\tau-1}u_{\tau-2}\dots u_0\}, W_1', W_2', \dots, W_{\tau-1}'\}$, 使得 $W_1', W_2', \dots, W_{\tau-1}'$ 分别是对应于 $T_{A,1}', T_{A,2}', \dots, T_{A,\tau-1}'$ 的 *EVS*, 对于任意整数 $i, 1 \leq i \leq \tau - 1$,

$$W_i' = \{u_{\tau-1}u_{\tau-2}\dots u_i \overline{u_{i-1}}x_{i-2}x_{i-3}\dots x_0 \in V(M_{\tau-1}^{u_{\tau-1}u_{\tau-2}\dots u_0})\}.$$

(2) 算法 *MIST* 构造的 $T_1', T_2', \dots, T_\tau'$ 是 $0-M_\tau$ 上 τ 棵以顶点 $u_{\tau-1}u_{\tau-2}\dots u_0$ 为根的 *IST*。

我们接下来证明该引理对于 $n = \tau + 1$ 成立。根据算法 *MIST*, 我们有如下情形:

情形1. $u_\tau = 0$. $T_{A,1} = u_\tau T_1', T_{A,2} = u_\tau T_2', \dots, T_{A,\tau} = u_\tau T_\tau'$ 。显然, $T_{A,1}, T_{A,2}, \dots, T_{A,\tau}$ 是 $M_\tau^{u_\tau}$ 上 τ 棵以顶点 $u_\tau u_{\tau-1}\dots u_0$ 为根的 *IST*。

根据算法 *MIST*, 我们有:

对于任意整数 $i, 1 \leq i \leq \tau - 1$ 。

$$W_i = u_\tau W_i'. \quad (4.3)$$

进一步,

$$W_\tau = \bigcup_{i=1}^{\tau-1} N(W_i \cup \{N(u_\tau u_{\tau-1}u_{\tau-2}\dots u_0, \tau - 1)\}, \tau - 1). \quad (4.4)$$

根据引理 4.12, $W_1, W_2, \dots, W_\tau$ 是分别对应于 $M_\tau^{u_\tau}$ 上以顶点 $u_\tau u_{\tau-1}\dots u_0$ 为根的 $T_{A,1}, T_{A,2}, \dots, T_{A,\tau}$ 的 *EVS*。

根据算法 *MIT* (参见算法 *MIT* 的第8行), 对于任意整数 $i, 1 \leq i \leq \tau, E(T_{B,i}[N(W_i, \tau)])$ 中的所有边被删除。对于 $N(W_i, \tau)$ 中的任意整数 $z, 1 \leq i \leq \tau$, 我们有:

$\text{children_number}(z) = \tau + 1 - i$, 对于 $i = 1, 2, \dots, \tau$ 。

根据引理 4.20, $T_1, T_2, \dots, T_\tau$ 是 τ 棵同构于 $BL_{\tau+1}$ 的树。

我们能够验证 $T_{B,i}[N(W_i, \tau)]$ 同构于 B_{i-1} , 对于 $1 \leq i \leq \tau$ 。根据二项树的定义, $T_{B,\tau+1} = \bigcup_{i=1}^{\tau} N(W_i \cup \{u\}, \tau)$ 导出的子图同构于 B_τ 。通过将 $V(T_{B,\tau+1})$ 中的每一个顶点 z 连接 $N(z, \tau)$ 构造 $T_{\tau+1}$ 。根据定义 4.5 和引理 4.20, $T_{\tau+1}$ 同构于 $BL_{\tau+1}$ 。根据算法 MIST, 等式(4.3)和等式(4.4), 容易验证:

$W_i = \{v \in (T_{A,i}) | \text{children_number}(v) \geq \tau - i + 1\}$ 对于 $i = 1, 2, \dots, \tau$ 。

情形2. $u_\tau = 1$ 。类似于情形1可以证明。

因此, 我们已经证明了该引理对于 $n = \tau + 1$ 成立。

基于引理 4.9 和引理 4.21, 我们有如下引理。

引理 4.22. 对于任意整数 $n \geq 2$, 集合 $S = \{T_1, T_2, \dots, T_n\}$ 中的 M_n 上由算法 MIST_UNION 构造的每一棵以顶点 u 为根的 IST T_i 同构于 BL_n , 这里 $1 \leq i \leq n$ 。

综合定理 4.14, 定理 4.15, 引理 4.19 和引理 4.22, 我们有如下定理。

定理 4.23. 算法 MIST_UNION 在 $O(N \log N)$ 时间内构造出 M_n 上以顶点 u 为根的 n 棵 IST $T_1, T_2, \dots, T_n$, 这里 $N = 2^n$ 。

- (1) T_i 同构于 BL_n , 对于 $i = 1, 2, \dots, n$ 。
- (2) T_i 的高度为 $n + 1$, 对于 $i = 1, 2, \dots, n$ 。

图 4.3 和图 4.4 实际上示出了 $0-M_n$ 以 1 为根的 n 棵 IST 的构造过程, 对于 $n=1, 2, 3, 4$ 。

4.5 基于顶点之间的维邻接关系构造同构树的讨论

需要注意的是, 容易验证, 上一章中交叉立方体上的地址变换方法并没有改变树中顶点之间的维邻接关系。因此, 在莫比乌斯立方体上, 基于顶点之间的维邻接关系, 依据 M_n^0 上的一棵生成树构造 M_n^1 上一棵同构的生成树的方法同样适用于交叉立方体, 即基于顶点之间的维邻接关系依据 CQ_n^0 上的一棵生成树构造 CQ_n^1 上一棵同构的生成树。

图 4.8(a) 示出了依据 M_3^0 上的一棵生成树构造 M_3^1 上的一棵生成树的构造过程; 图 4.8(b) 示出了依据 CQ_3^0 上的一棵生成树构造 CQ_3^1 上的一棵生成树的构造过程。因此,

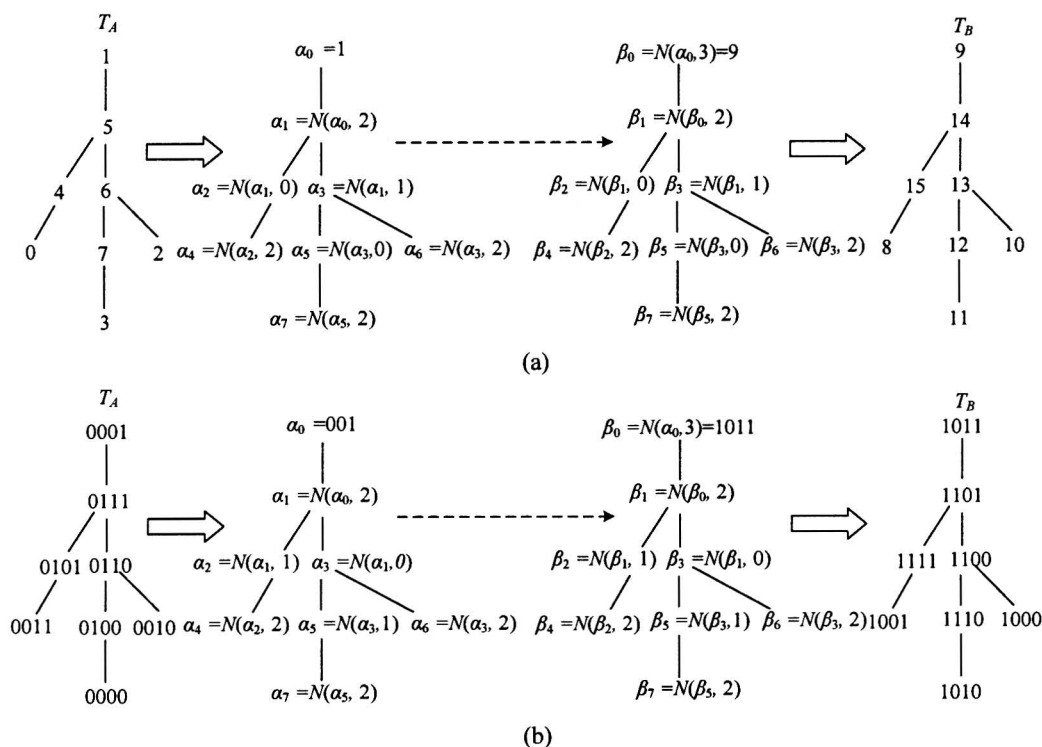


图 4.8 (a)依据 M_3^0 上的以1为根的生成树 T_A 构造 M_3^1 上以9为根的生成树 T_B ; (b)依据 CQ_3^0 上的以0001为根的生成树 T_A 构造 CQ_3^1 上以1011为根的生成树 T_B

对于交叉立方体来说，本章基于顶点之间的维邻接关系构造同构树的方法是顶点地址变换方法在一定程度上的优化。

4.6 满足一定特征的网络上独立生树构造问题的讨论

依据前面一章和本章的结论，我们能够将相关成果进一步推广。首先对于EVS和EVSP的定义，我们可以做进一步推广，假设 n 连通图 G 由两个 $n-1$ 连通子图 G_0 和 G_1 组成，且满足如下条件：

- (1) G_0 中存在 $n-1$ 棵以任一顶点为根的IST $T_{A,0}, T_{A,1}, \dots, T_{A,n-1}$ 。
- (2) G_1 中存在 $n-1$ 棵以任一顶点为根的IST $T_{B,0}, T_{B,1}, \dots, T_{B,n-1}$ 。

定义 4.6. 对于整数 $n \geq 2$ ，假设 T_A 是 G_0 上一棵一顶点 u 为根的生成树， T_B 是 G_1 上以顶点 $N(u, n-1)$ 为根的生成树。如果存在一个非空集合 $W \subseteq V(T_A) \setminus \{u\}$ ，使得如果 $v \in W$ ，那么 $\text{ancestor}(v, T_A) \setminus \{u\} \subseteq W$ 且 $\text{ancestor}(N(v, n-1), T_B) \setminus \{u\} \subseteq N(W, n-1)$ ，我们称 W 是一个关于生成树 T_A 的修正的扩展顶点集合。

根据修正的扩展顶点集合的定义,可以看出其只需要满足 $T_A[W \cup \{u\}]$ 是一棵以 u 为根的树且 $T_B[N(v, n-1) \cup \{N(v, n-1)\}]$ 是一棵以 $N(v, n-1)$ 为根的树,不再要求同构于子树 $T_B[N(W, n-1) \cup \{N(v, n-1)\}]$ 。EVSP的定义类似,基于任一顶点,都可以找到其对应的EVSP。那么,在 G 上都可以找到以该顶点为根的IST。

4.7 本章小结

本章给出了一个 $O(N \log N)$ 算法MIST_UNION来构造 n 维莫比乌斯立方体 M_n 上 n 棵以任一顶点为根的IST的构造。我们的主要贡献如下:

1. 给出了 M_n 上 n 棵以任一顶点为根的IST的构造性证明。
2. 我们设计了一个时间复杂度为 $O(N \log N)$ 的递归构造算法来构造 M_n 上 n 棵以任一顶点 u 为根的IST $T_1, T_2, \dots, T_n$, 对于 $n \geq 2$ 和任一顶点 u 。
3. 我们证明了 M_n 上以 u 为根的每棵IST同构于 n -级类二项树,其高度为 $n+1$,对于整数 $n \geq 2$ 。

在交叉立方体上IST构造方法不适用的情况下,设计了适合于莫比乌斯立方体上IST的递归构造算法,算法的时间复杂度为 $O(N \log N)$,性能比第三章交叉立方体上的算法好,相关成果发表在SCI源期刊《The Computer Journal》上(第一作者)。

第五章 交叉立方体上IST的并行构造

第三章介绍了交叉立方体上IST的串行构造算法，其时间复杂度略高且串行构造方法具有IST在构造过程中相互依赖构造的缺点，如第 n 棵树在构造过程中依赖于前面 $n-1$ 棵树，如何解除这样的依赖关系，是我们探讨的要点。在论文的第四章中，介绍了莫比乌斯立方体上所构造的IST皆同构于类二项树。本章我们研究交叉立方体上同构于类二项树的IST的并行构造，主要取得如下结果：

- (1) 给出交叉立方体上关于顶点之间维邻接关系的一个重要性质——维扩散性质。
- (2) 提出一个具有UUID成员和VALUE成员的树存储结构，确保树中顶点的UUID成员总是具有唯一性。
- (3) 基于交叉立方体上维扩散性质给出了其上IST的并行构造算法。
- (4) 在 n 维交叉立方体上并行构造 n 棵以任一顶点为根且同构于类二项树的IST。

5.1 预备知识

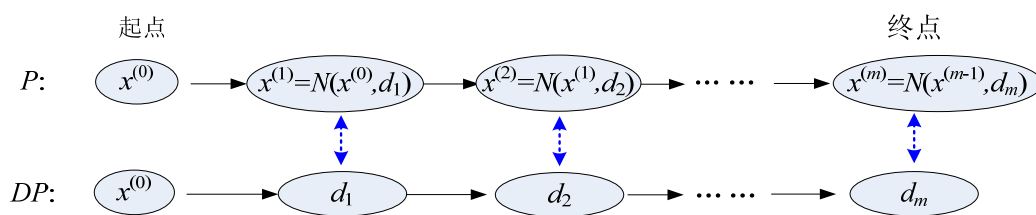
5.1.1 维邻接关系途径和维邻接关系树

根据 CQ_n 的定义，我们有如下引理。

引理 5.1. 给定顶点 $u = u_{n-1}u_{n-2} \dots u_0 \in V(CQ_n)$ ，我们有 $N(u, k) \in V(CQ_n(u_{n-1}u_{n-2} \dots u_{k+1}\overline{u_k}))$ ，对于 $0 \leq k \leq n-1$ 。

在本节，我们首先介绍树中顶点方面的一些知识，然后，我们讨论维邻接关系途径的定义和维邻接关系树的定义，其对 CQ_n 上以任一顶点为根的 n 棵IST的并行构造有着重要作用。

备注 5.1. 接下来，我们将每一个顶点用两个成员属性表示：UUID和VALUE，前者仅仅表示一棵树中一个全局唯一的标记符，后者存储 CQ_n 上顶点的二进制地址或两个顶点之间的维邻接关系，该关系将被用于维邻接关系树（Dimension-Adjacent Tree）（参见定义 5.2）。由于有UUID成员区分，我们允许一棵树中一个顶点的VALUE成员等于另一个顶点的VALUE成员。

图 5.1 对应于路径 P 的维邻接关系途径 DP

当一棵树中每个顶点有一个唯一的VALUE成员，我们将不区分顶点和顶点的VALUE成员。

定义 5.1. 令 P 表示 CQ_n 中起点为顶点 $x^{(0)}$ 和终点为 $x^{(m)}$ 的路径，对于 $n \geq 2$ 。

维邻接关系途径（Dimension-Adjacent Walk）被定义如下（假设我们采用 $x^{(0)}$ 到 $x^{(m)}$ 的方向）：

(1) 将 P 复制为 DP ；

(2) 对于每条边 $(x, y) \in E(P)$ ，若 (x, y) 是 d 维边，将 y 替换为 d （显然， $0 \leq d \leq n-1$ ）。那么，我们得到一条途径。令 DP 表示 P 的维邻接关系途径。

在图 5.1 中，基于路径 $P: x^{(0)}, x^{(1)} = N(x^{(0)}, d_1), x^{(2)} = N(x^{(1)}, d_2), \dots, x^{(m)} = N(x^{(m-1)}, d_m)$ ，我们能得到维邻接关系途径 $DP: x^{(0)}, d_1, d_2, \dots, d_m$ 。

基于定义 5.1，我们给出如下定义。

定义 5.2. 假设 T 是 CQ_n 上一棵以顶点 u 为根的树。按如下两步骤处理：

(1) 将树 T 复制为 DT ；

(2) 对于任一顶点 $v \in V(T) \setminus \{u\}$ ，如果 $(\text{parent}(v, T), v)$ 是树 T 中的一条 d 维边，我们将数 DT 中的 v 替换为 d 。

那么，我们得到一棵关于树 T 的维邻接关系树（Dimension-Adjacent Tree），简称为 DT 。

显然，给定 CQ_n 上以 u 为根的一棵树 T ， DT 将被唯一地构造，反之亦然（给定根顶点）。在图 5.2(a)， T_1 是 CQ_3 上以顶点111为根的生成树， DT_1 是关于 T_1 的维邻接关系树。给定根顶点111，我们容易构造出基于 DT_1 的 T_1 。在图 5.2(b)中， T_2 不是 CQ_3 上以111为根的生成树，因为有两个顶点具有相同的VALUE成员111。 DT_2 是 T_2 的维邻接关系树。

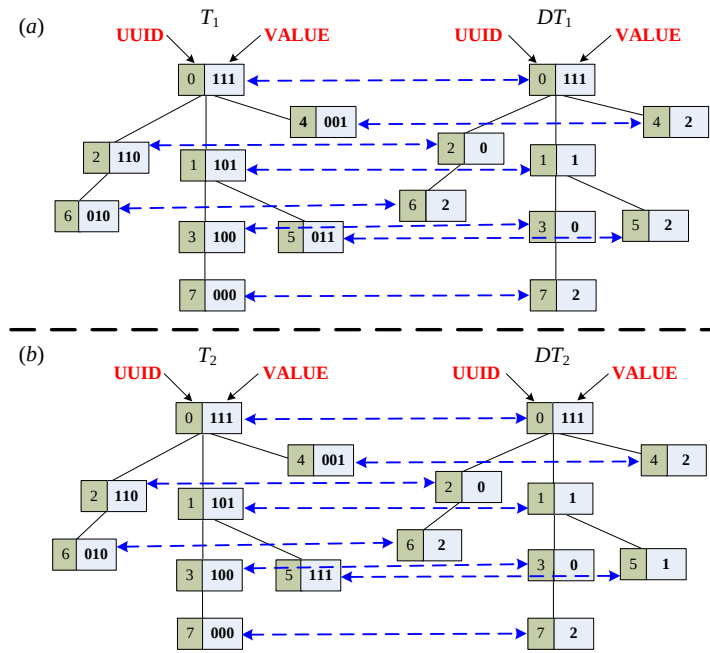


图 5.2 (a) 关于树 T_1 的维邻接关系树 DT_1 ; (b) 关于树 T_2 的维邻接关系树 DT_2

5.1.2 交叉立方体上顶点之间维邻接关系的重要性质

接下来, 我们将给出引理 5.2, 它给出了交叉立方体的一个顶点之间维邻接关系的重要性质——维扩散 (Dimensional Expanding) 性质, 该性质将被用于交叉立方体上以任一顶点为根的 IST 的构造过程中。

引理 5.2. 给定 CQ_n 上任意两条路径 $P: u, x^{(1)} = N(u, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$ 和 $P': u, y^{(1)} = N(u, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(t)} = N(y^{(t-1)}, h_t)$, 对于任意两个整数 $k, t, 1 \leq k \leq n$ 和 $1 \leq t \leq n$, 如果下列条件成立:

- (1) $0 \leq m_1 < m_2 < \dots < m_k \leq n-1$,
- (2) $0 \leq h_1 < h_2 < \dots < h_t = m_k$, 且
- (3) $m_1 \neq h_1$,

那么 $x^{(k)}$ 和 $y^{(t)}$ 具有一个最左不同位 $l \geq \min\{m_1, h_1\}$ 。

证明. 令 $A = \{m_1, m_2, \dots, m_k\}$, $B = \{h_1, h_2, \dots, h_t\}$ 和 $d = \max((A \cup B) \setminus (A \cap B))$ 。不失一般性, 我们假设 $u_d \in A$ 且 $u_d = m_\tau$, 这里 $1 \leq \tau \leq k$ 。那么, $m_{\tau+1} = h_{t-(k-\tau)+1}$, $m_{\tau+2} = h_{t-(k-\tau)+2}, \dots, m_k = h_t$ 。

我们有如下情形:

情形1. $d > \min\{m_1, h_1\}$ 。那么, $(d-1) \geq \min\{m_1, h_1\}$ 。我们有如下两种子情形。

情形1.1. d 是奇数。那么, 对于顶点 $x^{(\tau)}$ 中的子串 $x^{(\tau)}_d x^{(\tau)}_{d-1}$ 和顶点 $y^{(t-(k-\tau))}$ 中的子串 $y^{(t-(k-\tau))}_d y^{(t-(k-\tau))}_{d-1}$, 我们有 $x^{(\tau)}_d x^{(\tau)}_{d-1} = \overline{u}_d x^{(\tau)}_{d-1}$ 和 $y^{(t-(k-\tau))}_d y^{(t-(k-\tau))}_{d-1} = u_d y^{(t-(k-\tau))}_{d-1}$ 。不失一般性, 我们假设 $u_d = 0$ 。那么有如下情形:

情形1.1.1. $\overline{u}_d x^{(\tau)}_{d-1} = 10$ 且 $u_d y^{(t-(k-\tau))}_{d-1} = 00$ 。根据定义 2.4 和定义 2.5, $00 \sim 00$ 且 $10 \sim 10$ 。那么, $x^{(k)}_d \neq y^{(t)}_d$ 。

情形1.1.2. $\overline{u}_d x^{(\tau)}_{d-1} = 10$ 且 $u_d y^{(t-(k-\tau))}_{d-1} = 01$ 。根据定义 2.4 和定义 2.5, $00 \sim 00$, $11 \sim 01$, 且 $01 \sim 11$ 。那么, $x^{(k)}_{d-1} \neq y^{(t)}_{d-1}$ 。

情形1.1.3. $\overline{u}_d x^{(\tau)}_{d-1} = 11$ 且 $u_d y^{(t-(k-\tau))}_{d-1} = 00$ 。类似于情形1.1.2可以证明。

情形1.1.4. $\overline{u}_d x^{(\tau)}_{d-1} = 11$ 且 $u_d y^{(t-(k-\tau))}_{d-1} = 01$ 。类似于情形1.1.1可以证明。

情形1.2. d 是偶数。那么, 对于顶点 $x^{(\tau)}$ 中的子串 $x^{(\tau)}_{d+1} x^{(\tau)}_d$ 和顶点 $y^{(t-(k-\tau))}$ 中的子串 $y^{(t-(k-\tau))}_{d+1} y^{(t-(k-\tau))}_d$, 我们有 $x^{(\tau)}_{d+1} x^{(\tau)}_d = x^{(\tau)}_{d+1} \overline{u}_d$ 和 $y^{(t-(k-\tau))}_{d+1} y^{(t-(k-\tau))}_d = y^{(t-(k-\tau))}_{d+1} u_d$ 。不失一般性, 假设 $u_d = 0$ 。根据定义 2.4 和定义 2.5, 我们能够验证 $x^{(k)}_d = \overline{u}_d$ 和 $y^{(t)}_d = u_d$ 。即 $x^{(k)}_d \neq y^{(t)}_d$ 。

基于情形1中给出的讨论, 可以得出 $x^{(k)}$ 和 $y^{(t)}$ 具有一个最左不同位 $l \geq d-1 \geq \min\{m_1, h_1\}$ 。

情形2. $d = \min\{m_1, h_1\}$ 。不失一般性, 我们假设 $d = m_1 < h_1$ 和 $u_d = 0$ 。那么, 我们只需证明 $x^{(k)}_d \neq y^{(t)}_d$ 。

情形2.1. d 是奇数。考虑 $x^{(1)}$ 中的子串 $x^{(1)}_d x^{(1)}_{d-1}$ 和 u 中的 $u_d u_{d-1}$, 我们有 $x^{(1)}_d x^{(1)}_{d-1} = \overline{u}_d x^{(1)}_{d-1}$ 。令 $x^{(1)} = N(u, m_1) = N(u, d)$, 我们有如下两个子情形。

情形2.1.1. $u_d u_{d-1} = 00$ 。那么, 我们有 $x^{(1)}_d x^{(1)}_{d-1} = 10$ 。类似于情形1.1.1可以证明。

情形2.1.2. $u_d u_{d-1} = 01$ 。那么, 我们有 $x^{(1)}_d x^{(1)}_{d-1} = 11$ 。类似于情形1.1.1可以证明。

情形2.2. d 是偶数。类似于情形1.2可以证明。

依据情形2中给出的讨论, 我们得出 $x^{(k)}$ 和 $y^{(t)}$ 具有一个最左不同位 $l \geq d = \min\{m_1, h_1\}$ 。

综上所述, 引理成立。

备注 5.2. 引理 5.2 是一个重要的判断标准, 我们称其为维扩散性质, 其将用于下面的章节中。

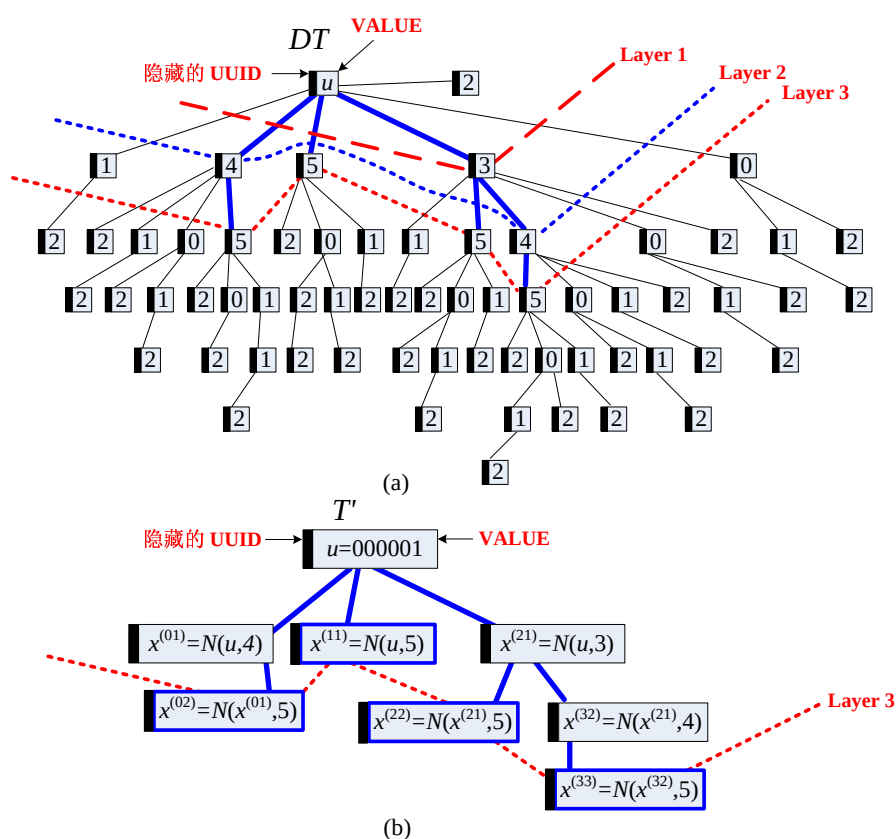


图 5.3 (a) 用于说明引理 5.2 的一个维邻接关系树; (b) 一个用于说明引理 5.2 的例子

备注 5.3. 一个退化情形是 $P: u, x^{(1)} = N(u, m_1), x^{(2)} = N(x^{(1)}, m_2)$ 和 $P': u, y^{(1)} = N(u, h_1)$, 这里 $0 \leq m_1 < m_2 \leq n-1$ 和 $0 \leq h_1 = m_2$ 。此时, 我们也可以令 $x^{(2)}$ 和 $y^{(1)}$ 具有一个最左不同位 $d \geq \min\{m_1, h_1\} = m_1$ 。

由于受图 5.3 中的空间限制，我们隐藏 `UUID` 成员属性。图 5.3(a) 是一棵关于 CQ_5 上以 u 为根的一棵生成树的维邻接关系树 DT 。树中在层 1 (Layer 1)、层 2 (Layer 2)、层 3 (Layer 3) 分别有一个、两个、四个顶点。

令 $u = 000001$ 。基于图 5.3(a) 中 DT ，我们能够得到一棵树 T' ，如图 5.3(b) 所示。我们给出四条路径 P_1, P_2, P_3 和 P_4 如下：

$$P_1: u = 000001, x^{(01)} = N(u, 4) = 010011, x^{(02)} = N(x^{(01)}, 5) = 110001.$$

$$P_2: u = 000001, x^{(11)} = N(u, 5) = 100011.$$

$$P_3: u = 000001, x^{(21)} = N(u, 3) = 001011, x^{(22)} = N(x^{(21)}, 5) = 101001.$$

$$P_4: u = 000001, x^{(21)} = N(u, 3) = 001011, x^{(32)} = N(x^{(21)}, 4) = 011001, x^{(33)} = N(x^{(32)}, 5) = 111011.$$

以 T' 中层3中的顶点为例, 我们能够验证集合 $\{x^{(02)}, x^{(11)}, x^{(22)}, x^{(33)}\}$ 中的任意两个顶点具有一个大于等于3的最左不同位。

5.2 基于顶点之间维邻接关系的维扩散性质构造IST

基于上节的结论, 我们给出一个算法, 称为SPT, 来构造 $CQ_n(u_{n-1}u_{n-2}\dots u_k)$ 上一棵以任一顶点 u 为根的生成树, 这里 $1 \leq k \leq n$ 。然后, 我们给出算法XCDAT来构造分别以顶点 $N(u, 0), N(u, 1), \dots, N(u, n-1)$ 为根的 n 棵维邻接关系树 $DT_0, DT_1, \dots, DT_{n-1}$ 。最后, 我们提出算法XCIST来构造 n 棵以 CQ_n 上任一顶点为根的IST。基于算法SPT, 我们给出引理 5.3, 其将被用于证明算法XCIST的正确性。算法XCDAT构造的 $DT_0, DT_1, \dots, DT_{n-1}$ 分别是对应于算法XCIST构造的 $T_0, T_1, \dots, T_{n-1}$ 的维邻接关系树。

5.2.1 一个构造 $CQ_n(u_{n-1}u_{n-2}\dots u_k)$ 上生成树的算法

现在我们给出一个算法, 称为SPT, 来构造 $CQ_n(u_{n-1}u_{n-2}\dots u_k)$ 上以任一顶点为根的生成树, 这里 $1 \leq k \leq n$ 。当 $k = n$ 时, $CQ_n(u_{n-1}u_{n-2}\dots u_k) = CQ_n$ 。

算法 8 SPT(Spanning Tree)

输入: 根顶点 $u = u_{n-1}u_{n-2}\dots u_0$ 和整数 k , 这里 $1 \leq k \leq n$;

输出: $CQ_n(u_{n-1}u_{n-2}\dots u_k)$ 上一棵以顶点 u 为根的树 T ;

开始

/*整型变量*iUUID* 是控制树 T 中顶点的UUID成员属性的变量。*/

1: $\text{int } iUUID = 0;$

2: $V(T) = [iUUID, u];$

3: $E(T) = \emptyset;$

/*整型变量*iLevel*被用来控制外循环。*/

4: **for** $iLevel = 0$ to $k - 1$ **do**

5: **for** 任一顶点 $x \in V(T)$ **do**

6: $iUUID++;$

7: $E(T) = E(T) \cup \{(x, [iUUID, N(x.VALUE, iLevel)])\};$

/* $x.VALUE$ 表示顶点 x 的VALUE成员属性。*/

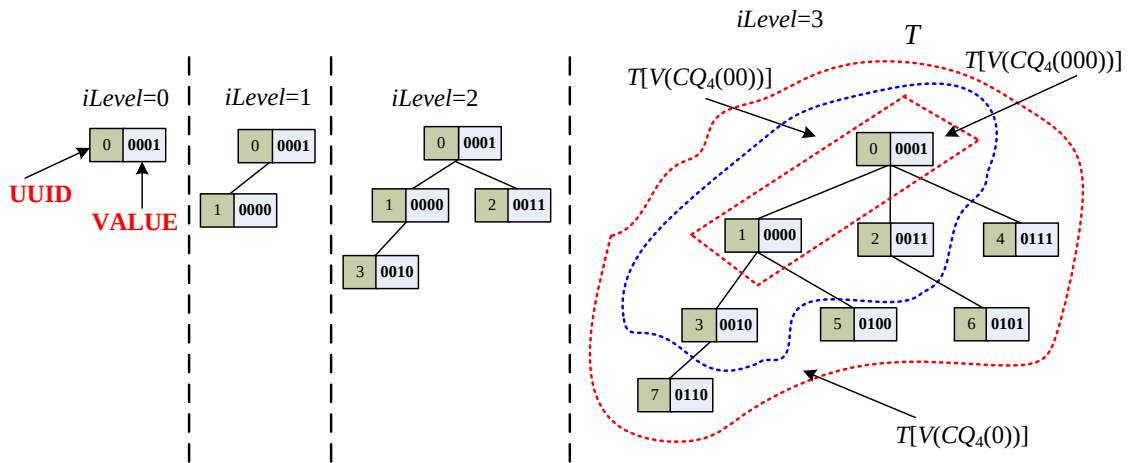
8: $V(T) = V(T) \cup \{[iUUID, N(x.VALUE, iLevel)]\};$

9: **end for**

10: **end for**

结束

图 5.4示出了 $CQ_4(0)$ 上以顶点0001为根的生成树 T 的构造过程。

图 5.4 $CQ_4(0)$ 上以顶点 0001 为根的生成树的构造过程

令 $u = 0001$ 和 $k = 3$ 。我们只需考虑树 T 中每个顶点的 VALUE 成员属性。初始时, $V(T)$ 中只有一个顶点 0001;

经过第一轮迭代 ($iLevel = 0$), $V(T)$ 中的任一顶点 v 连接顶点 $N(v, 0)$ 。此时, $V(T) = \{0001, 0000\}$, $E(T) = \{(0001, 0000)\}$, 树 T 是 $CQ_4(000)$ 的一棵生成树;

经过第二轮迭代 ($iLevel = 1$), $V(T)$ 中的任一顶点 v 连接顶点 $N(v, 1)$ 。因此, 边 $(0000, 0010)$ 和 $(0001, 0011)$ 被加入 T 中和 $V(T) = \{0000, 0001, 0010, 0011\}$ 。相应地, T 是 $CQ_4(00)$ 上的生成树;

经过第三轮迭代 ($iLevel = 2$), $V(T)$ 中的任一顶点 v 连接 $N(v, 3)$, T 是 $CQ_4(0)$ 上的一棵生成树;

现在我们提供如下引理, 其证明了由算法 SPT 构造出来的树是 $CQ_n(u_{n-1}u_{n-2}\dots u_k)$ 上的一棵生成树, 这里 $1 \leq k \leq n$ 。

引理 5.3. 给定任意整数 $n \geq 1$, 对于任意整数 $1 \leq k \leq n$, 令 $u = u_{n-1}u_{n-2}\dots u_0$, k 是算法 SPT 的输入, T 是算法 SPT 的输出。那么, T 是 $CQ_n(u_{n-1}u_{n-2}\dots u_k)$ 上一棵以 u 为根的生成树。

证明. 我们采用关于 k 的数学归纳法进行证明。

基础: 令 $k = 1$ 。 $N(u_{n-1}u_{n-2}\dots u_0, 0) = u_{n-1}u_{n-2}\dots u_1\bar{u}_0$ 。根据算法 SPT, 我们有 $V(T) = \{u_{n-1}u_{n-2}\dots u_0, u_{n-1}u_{n-2}\dots u_1\bar{u}_0\}$ 。 T 是 $CQ_n(u_{n-1}u_{n-2}\dots u_1)$ 上的一棵生成树。

归纳: 假设引理对于 $k = \tau$ ($1 \leq \tau < n$) 成立, 即:

给定任意整数 $n \geq 1$, 令 $u = u_{n-1}u_{n-2} \dots u_0$, τ 是算法SPT的输入, T' 是算法SPT的输出。那么, T' 是 $CQ_n(u_{n-1}u_{n-2} \dots u_\tau)$ 上以 u 为根的生成树。

我们将证明定理对于 $k = \tau + 1$ 成立:

根据算法SPT, $V(T) = V(T') \cup \{N(v, \tau) | v \in V(T')\} = V(CQ_n(u_{n-1}u_{n-2} \dots u_\tau)) \cup \{N(v, \tau) | v \in V(CQ_n(u_{n-1}u_{n-2} \dots u_\tau))\}$ 。根据引理 5.1, $N(v, \tau) \in V(CQ_n(u_{n-1}u_{n-2} \dots u_{\tau+1}))$, 对于给定的顶点 $v \in V(CQ_n(u_{n-1}u_{n-2} \dots u_\tau))$, 这意味着 $V(T) \subseteq V(CQ_n(u_{n-1}u_{n-2} \dots u_{\tau+1}))$ 。根据定义 2.5, 我们能够验证 $|V(T)| = |V(CQ_n(u_{n-1}u_{n-2} \dots u_\tau)) \cup \{N(v, \tau) | v \in V(CQ_n(u_{n-1}u_{n-2} \dots u_\tau))\}| = |V(CQ_n(u_{n-1}u_{n-2} \dots u_{\tau+1}))|$ 。因此, T 是 $CQ_n(u_{n-1}u_{n-2} \dots u_{\tau+1})$ 上的一棵生成树。

综上所述, 引理成立。

5.2.2 构造通用维邻接关系树的算法

首先, 我们给出 n 个向量, 这是维邻接关系树的基础。然后, 我们提供一个并行算法来构造 n 棵维邻接关系树, 其是 CQ_n 上以任一顶点为根的 n 棵IST的内在结构。基于这样一组维邻接关系树和任一顶点 u , 我们容易得到 CQ_n 上以 u 为根的 n 棵IST。

我们称一个向量 $\langle \gamma_1, \gamma_2, \dots, \gamma_n \rangle$ 为非负向量, 当 $\gamma_1, \gamma_2, \dots, \gamma_n \geq 0$ 。对于任意整数 $n \geq 1$, 令非负向量

$$A^{(0)} = \langle \alpha_1^{(0)}, \alpha_2^{(0)}, \dots, \alpha_n^{(0)} \rangle = \langle 1, 2, \dots, n-1, 0 \rangle.$$

...

$$A^{(k)} = \langle \alpha_1^{(k)}, \alpha_2^{(k)}, \dots, \alpha_n^{(k)} \rangle = \langle k+1, k+2, \dots, n-1, 0, 1, \dots, k \rangle \text{ 对于整数 } k, 1 \leq k \leq n-2.$$

...

$$A^{(n-1)} = \langle \alpha_1^{(n-1)}, \alpha_2^{(n-1)}, \dots, \alpha_n^{(n-1)} \rangle = \langle 0, 1, \dots, n-1 \rangle.$$

现在我们给出一个算法, 称为XCDAT, 分别基于 $A^{(0)}, A^{(1)}, \dots, A^{(n-1)}$ 来构造维邻接关系树 $DT_0, DT_1, \dots, DT_{n-1}$ 。

对于一个给定的整数 n , $DT_0, DT_1, \dots, DT_{n-1}$ 将被算法XCDAT唯一地构造。给定根顶点 u , 基于 $DT_0, DT_1, \dots, DT_{n-1}$, 我们能够容易地构造 CQ_n 上 n 棵以 u 为根的生成树。图 5.5(a) 示出了 CQ_4 上依据算法XCDAT的四棵维邻接关系树 DT_0, DT_1, DT_2 , 和 DT_3 的构造。以 DT_0 为例。首先, 在集合 $V(DT_0)$ 中仅仅有一个顶点 $[0, N(u, 0)]$ 。经过第一轮迭代 ($iLevel = 1$), $V(DT_0)$ 的顶点 $[0, N(u, 0)]$ 连接顶点 $[1, 1]$ 。从而 $V(DT_0) = \{[0, N(u, 0)], [1, 1]\}$ 和 $E(DT_0) = \{([0, N(u, 0)], [1, 1])\}$; 经过两轮迭代 ($iLevel = 2$), $V(DT_0)$ 中

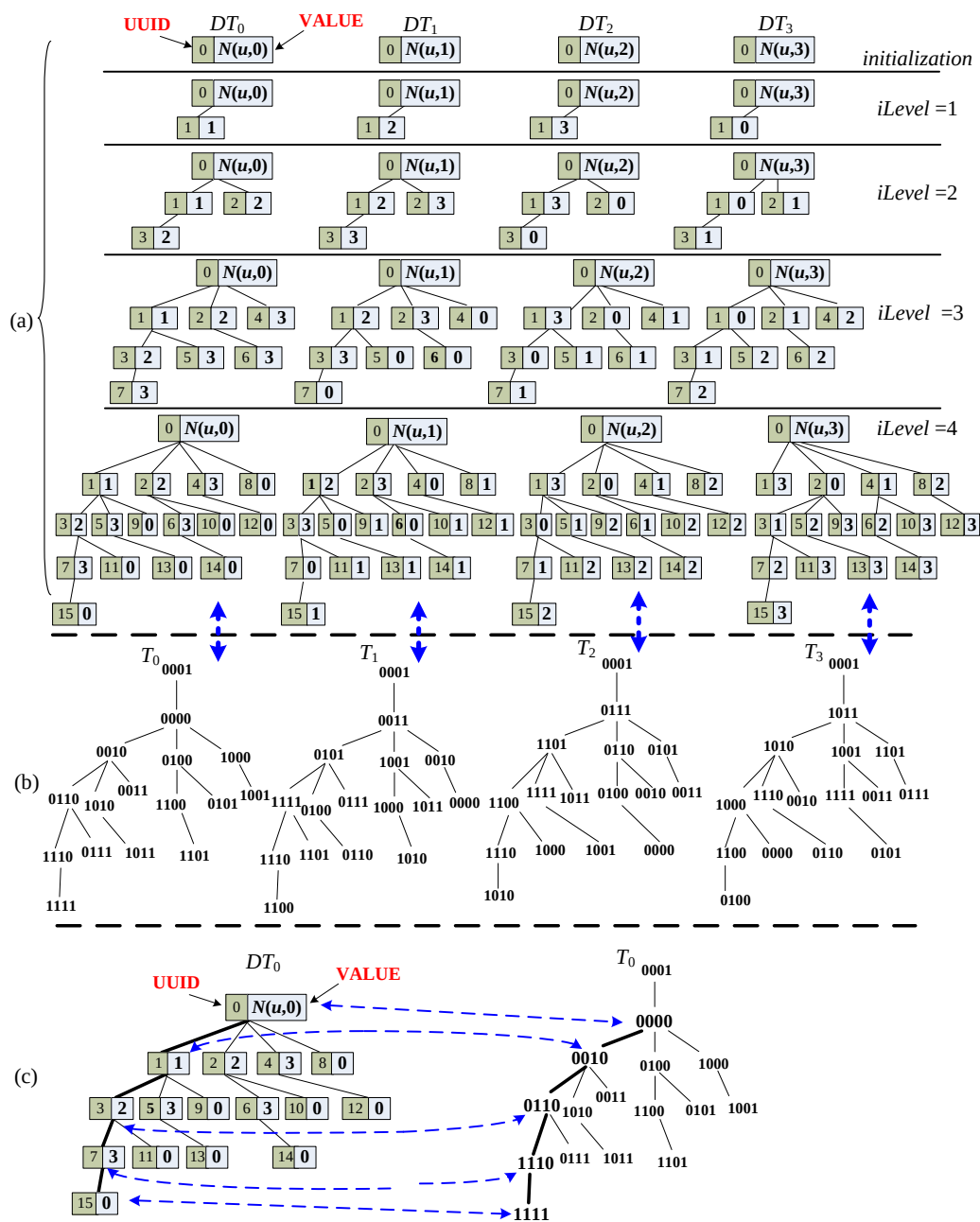


图 5.5 (a) 4 棵维邻接关系树的构造; (b) 4 棵以顶点 0001 为根的对应于 (a) 中 4 棵维邻接关系树的 IST 的构造; (c) DT_0 和 T_0 之间的关系

算法 9 XCDAT(Dimension-Adjacent Tree)**输入:** 顶点 u 和整数 $n \geq 1$;**输出:** n 维邻接关系树 $DT_0, DT_1, \dots, DT_{n-1}$ 分别以顶点 $N(u, 0), N(u, 1), \dots, N(u, n-1)$ 为根;**开始**1: **for** $i = 0$ to $n - 1$ **do**2: **并行模式**3: **int** $iUUID = 0$; /*这里, DT_i 中的每个顶点有两个成员属性, UUID成员属性和VALUE成员属性。整型变量 $iUUID$ 存储树 DT_i 中顶点的UUID成员属性的值。*/4: $V(DT_i) = [iUUID, N(u, i)]$;5: $E(DT_i) = \emptyset$; /*这里, 整型变量 $iLevel$ 是外层循环的控制变量。*/6: **for** $iLevel = 1$ to n **do**7: **for** 任一顶点 $x \in V(DT_i)$ **do**8: $iUUID++$;9: $y = (i + iLevel) \bmod n$; /* $y = (i - iLevel + n) \bmod n$ 是另外一种选择, y 表示两个相邻顶点之间的维邻接关系。*/10: $E(DT_i) = E(DT_i) \cup \{(x, [iUUID, y])\}$;11: $V(DT_i) = V(DT_i) \cup \{[iUUID, y]\}$;12: **end for**13: **end for** /*对于树 DT_i , 经过变量 $iLevel$ 的 n 次迭代, y 依次取整型值 $i + 1, i + 2, \dots, n - 1, 0, 1, \dots, i$ 。 如果我们令 $y = (i - iLevel + n) \bmod n$, 那么 y 依次取整型值 $i - 1, i - 2, \dots, 0, n - 1, n - 2, \dots, i$, 经过变量 $iLevel$ 的 n 次迭代。*/14: **end for****结束**

的每个顶点分别连接一个VALUE成员属性为2的新顶点。因此, 边 $([0, N(u, 0)], [2, 2])$ 和 $([1, 1], [3, 2])$ 被增加进 DT_0 且 $V(DT_0) = \{[0, N(u, 0)], [1, 1], [2, 2], [3, 2]\}$; 经过第三轮迭代($iLevel = 3$), $V(DT_0)$ 中的每个顶点连接一个VALUE成员属性为3的新顶点。相应地, $V(DT_0)$ 中顶点数量翻倍; 经过第四轮迭代($iLevel = 4$), $V(DT_0)$ 的每个顶点分别连接VALUE成员属性为0的新顶点。根据定义 5.2, DT_0 是 CQ_4 上的一棵维邻接关系树。 DT_1, DT_2 和 DT_3 的构造类似于 DT_0 。令根顶点 u 为0001。类似于图 5.3、图 5.5(b)中示出的构造过程, 图 5.5(a)中示出了 CQ_4 上以顶点0001为根的四棵维邻接关系树的构造过程。进一步, 以 DT_0 和 T_0 为例, 树中顶点之间的邻接关系如图 5.5(c)所

示。< 0000, 1111 >-路径在 T_0 中以粗线表示, 即 $0000=N(0001, 0)$, $0010=N(0000, 1)$, $0110=N(0010, 2)$, $1110=N(0110, 3)$, $1111=N(1110, 1)$ 。

关于 T_0 上< 0000, 1111 >-路径中的维邻接关系途径在 DT_0 中以粗线表示, 即:

$$N(u, 0) = N(0001, 0) = 0000, 1, 2, 3, 0.$$

容易验证图 5.5(b)示出的四棵树是 CQ_4 上以顶点0001为根的四棵IST。给定任意根顶点, CQ_4 上基于四棵维邻接关系树的四棵生成树将被唯一地构造, 如图 5.5(a)所示。

备注 5.4. 注意到在算法XC DAT中, 根顶点是任意的, 我们能够断言维邻接关系树独立于交叉立方体上顶点的地址。

5.2.3 基于顶点之间维邻接关系构造IST的一个算法

在本节中, 根据算法XC DAT, 我们进一步给出算法XC IST 来构造 CQ_n 上以任一顶点 u 为根的 n 棵IST如下, 如图 5.6所示。

接下来, 当讨论由算法XC IST构造的树中的顶点没有重复的VALUE成员属性的时候, 我们也能用VALUE成员属性来表示该顶点。算法XC IST中树的构造过程类似于算法XC DAT。这里, 我们仅仅考虑 T_0 中每个顶点的VALUE成员属性。以 CQ_4 上以0001为根的 T_0 为例。

初始时, $V(T_0)$ 中仅有一个顶点0000;

经过第一轮迭代($iLevel = 1$), $V(T_0)$ 中的顶点0000连接顶点 $N(0000, 1) = 0010$ 。因此, $V(T_0) = \{0000, 0010\}$ 且 $E(T_0) = \{(0000, 0010)\}$;

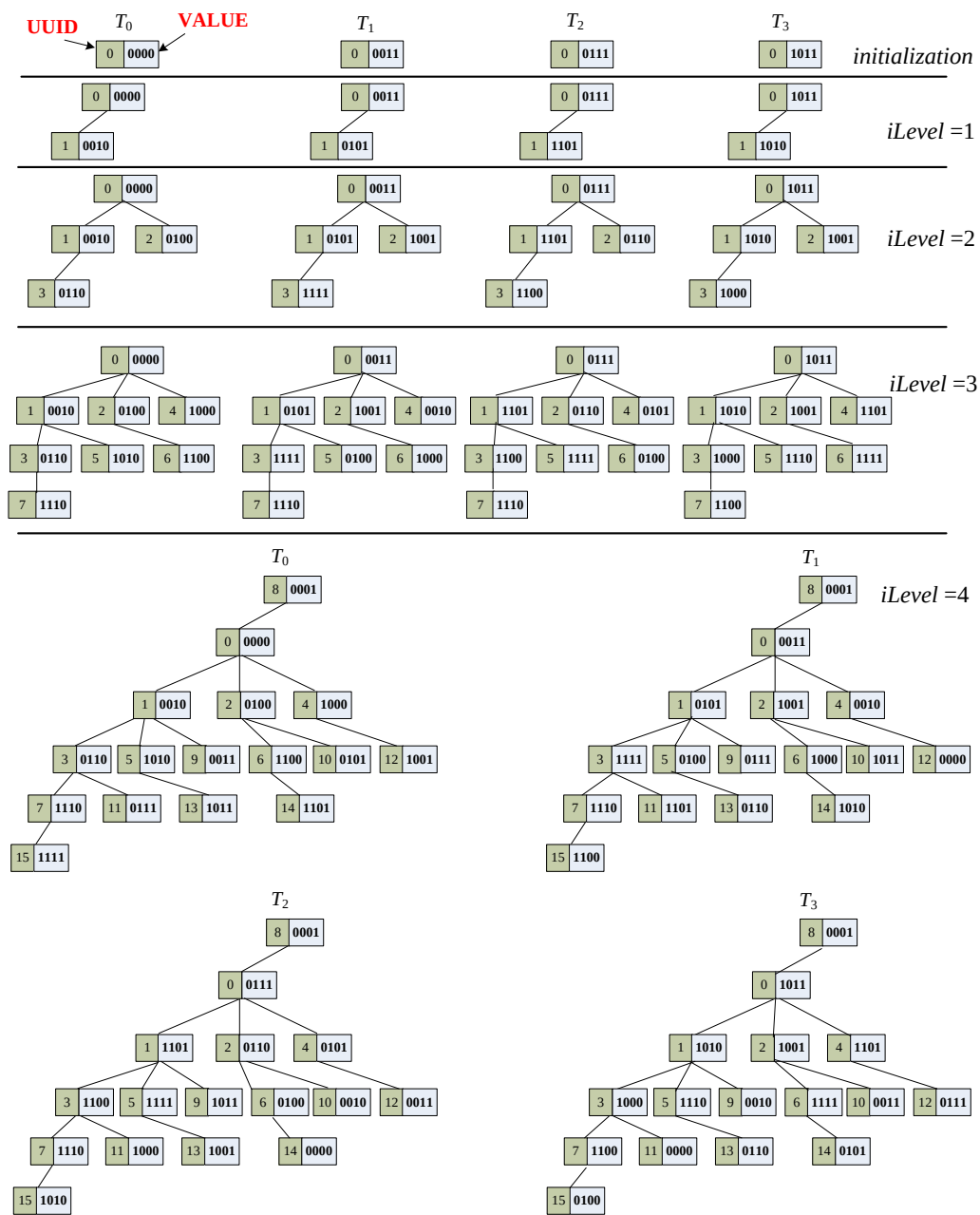
经过第二轮迭代($iLevel = 2$), $V(T_0)$ 中的每个顶点 v 连接顶点 $N(v, 2)$ 。因此, 边(0010, 0110) 和(0000, 0100) 增加进 T_0 , 此时 $V(T_0) = \{0000, 0010, 0100, 0110\}$;

经过第三轮迭代($iLevel = 3$), $V(T_0)$ 中的每个顶点 v 连接顶点 $N(v, 3)$ 。相应地, $V(T_0)$ 中顶点数目翻倍;

经过最后一轮迭代($iLevel = 4$), $V(T_0)$ 中的每个顶点 v 连接顶点 $N(v, 0)$ 。此时, T_0 是 CQ_4 上的一棵生成树。

5.3 基于顶点之间维邻接关系的IST的证明

算法XC DAT和算法XC IST的构造过程类似。给定根顶点 $N(u, i)$, 算法XC DAT构造的 DT_i 是算法XC IST构造的 T_i 的维邻接关系树, 对于 $i = 0, 1, \dots, n - 1$ (注意

图 5.6 CQ_4 上 4 棵以顶点 0001 为根的 IST

算法 10 XCIST(Independent Spanning Tree)**输入:** 根顶点 u 和整数 $n \geq 1$;**输出:** CQ_n 上以 u 为根的 n 棵树 $T_0, T_1, \dots, T_{n-1}$;**开始**1: **for** $i = 0$ to $n - 1$ **do**2: **并行模式**3: **int** $iUUID = 0$;/*这里, T_i 中的每个顶点具有两个成员, UUID 成员和VALUE成员。整型变量 $iUUID$ 被用来控制树 T_i 中顶点的UUID成员。*/4: $V(T_i) = [iUUID, N(u, i)]$;5: $E(T_i) = \emptyset$;/*这里, 整型变量 $iLevel$ 是外层循环的控制变量。*/6: **for** $iLevel = 1$ to n **do**7: **for** 任一顶点 $x \in V(T_i)$ **do**8: $iUUID++$;9: $y = N(x.VALUE, (i + iLevel) \bmod n)$;/*变量 y 是顶点 x 的 $((i + iLevel) \bmod n)$ 维邻接点, $y = N(x, (i - iLevel + n) \bmod n)$ 是另外一种选择。这里, $x.VALUE$ 表示顶点 x VALUE成员。*/10: $E(T_i) = E(T_i) \cup \{(x, [iUUID, y])\}$;11: $V(T_i) = V(T_i) \cup \{[iUUID, y]\}$;12: **end for**13: **end for**14: **end for****结束**

到 $N(u, i)$ 是树 T_i 中顶点 u 的唯一儿子, 我们也能说 T_i 是一棵以顶点 u 为根的树)。我们首先证明算法XCIST构造的树 $T_0, T_1, \dots, T_{n-1}$ 是 CQ_n 上以 u 为根的生成树, 然后证明 $T_0, T_1, \dots, T_{n-1}$ 相互独立。

5.3.1 路径和生成树的证明

根据交叉立方体的定义, 我们给出如下引理。

引理 5.4. 给定 CQ_n 上的途径 P : $x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$, 对于整数 $k, 2 \leq k \leq n$, 如果 $m_1, m_2, \dots, m_k$ 各不相同, $0 \leq m_i \leq n - 1, i = 1, 2, \dots, k$, 那么 P 是一条路径。

证明. 对于任意两个整数 $i, j, 0 \leq i < j \leq k$, 令 $l = \max\{m_{i+1}, m_{i+2}, \dots, m_j\}$ 。根据定义

2.5, 我们可以验证 $x^{(i)}$ 和 $x^{(j)}$ 有一个最左不同位 l , 这意味着 $x^{(i)} \neq x^{(j)}$, 引理成立。 $\square$

引理 5.5. 给定 CQ_n 上的一条途径 $P: x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k), x^{(k+1)} = N(x^{(k)}, m_1)$, 对于 $k, 2 \leq k \leq n$, 如果 $m_1, m_2, \dots, m_k$ 各不相同, 那么 P 是一条路径。

证明. 根据引理 5.4, $x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$ 是一条路径, 且 $x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k), x^{(k+1)} = N(x^{(k)}, m_1)$ 是一条路径。因此, 我们只需证明 $x^{(0)} \neq x^{(k+1)}$ 。令 $d = \max\{m_2, m_3, \dots, m_k\}$ 。基于假设, $d \neq m_1$ 。那么, 我们有如下两种情形。

情形1. $d > m_1$ 。根据引理 5.1, 我们有 $x^{(k+1)} \in V(CQ_n(u_{n-1}u_{n-2} \dots \overline{u_d}))$ 。既然 $x^{(0)} \in V(CQ_n(u_{n-1}u_{n-2} \dots u_d))$, 我们有 $x^{(0)} \neq x^{(k+1)}$ 。

情形2. $d < m_1$ 。根据定义 2.5, 我们能够验证 $x^{(0)} \neq x^{(k+1)}$ 。

因此, 引理成立。

基于引理 5.4 和定义 2.5, 我们有如下引理。

引理 5.6. 给定 CQ_n 上的两条路径 $P: x^{(0)} = N(u, i), x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$ 和 $P': x^{(0)} = N(u, i), y^{(1)} = N(u, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(t)} = N(y^{(t-1)}, h_t)$, 对于整数 i, k 和 $t, 0 \leq i \leq n-1$ 且 $1 \leq k, t \leq n$, 如果下面条件成立:

- (1) $0 \leq m_1 < m_2 < \dots < m_k$,
- (2) $0 \leq h_1 < h_2 < \dots < h_t$, 且
- (3) $m_1 \neq h_1$,

那么 $V(\text{path}(P, x^{(1)}, x^{(k)})) \cap V(\text{path}(P', y^{(1)}, y^{(t)})) = \emptyset$ 。

引理 5.7. 给定 CQ_n 上的两条路径 $P: x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$ 和 $P': y^{(0)}, y^{(1)} = N(y^{(0)}, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(t)} = N(y^{(t-1)}, h_t)$, 对于整数 $k, t, 1 \leq k, t \leq n$ 。如果下列条件成立:

- (1) $0 \leq m_1 < m_2 < \dots < m_k$,
- (2) $0 \leq h_1 < h_2 < \dots < h_t$, 且
- (3) $x^{(0)}$ 和 $y^{(0)}$ 具有一个最左不同位 $d > \max(m_k, h_t)$,

那么 $V(P) \cap V(P') = \emptyset$ 。

证明. 不失一般性, 我们假设 $x^{(0)}_d = 1$ 。那么, $y^{(0)}_d = 0$ 。根据条件(1), (3)和引理 5.1, 我们能够验证 $x^{(i)}$ 的第 d 位是 1, 对于 $i = 1, 2, \dots, k$ 。基于条件(2), (3) 和引理 5.1, 我们能够验证 $y^{(i)}$ 的第 d 位是 0, 对于 $i = 1, 2, \dots, t$ 。因此, $V(P) \cap V(P') = \emptyset$ 。

基于引理 5.4, 引理 5.5和算法XCIST, 我们能够得出如下引理。

引理 5.8. 假设 P 是算法XCIST构造的以 u 为根的 T_i 中一条从 u 到 v 任意的途径, 这里 v 为 T_i 上的一个任意叶子顶点, 对于 $i = 0, 1, \dots, n-1$ 。我们将 P 表示为 $u, x^{(1)} = N(u, m_1 = i), x^{(2)} = N(x^{(1)}, m_2), \dots, v = N(x^{(k-1)}, m_k)$, 这里 $k > 2$ 且 $m_k = i$ 。如下条件成立:

(1) P 是一条路径, 且

(2) $\langle \alpha^{(i)}_{i_1}, \alpha^{(i)}_{i_2}, \dots, \alpha^{(i)}_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$ 且 $m_k = i$ 。

引理 5.9. 对于任意整数 $i, 0 \leq i \leq n-1, |V(T_i)| = 2^n$ 。

证明. 显然, 经过 n 次迭代, $V(T_i)$ 中有 $1 + 2^0 + 2^1 + \dots + 2^{n-1} = 2^n$ 个顶点, 对于 $i, 0 \leq i \leq n-1$ (参见算法XCIST和算法XCIST)。

引理 5.10. $V(T_i) \subseteq V(CQ_n)$, 若我们仅考虑 T_i 中每个顶点的VALUE成员属性。

证明. 根据引理 5.1, 该引理成立。

引理 5.11. 由算法XCIST构造出的 T_i 是 $V(CQ_n)$ 上的一棵生成树, 对于整数 $i = 0, 1, \dots, n-1$ 。

证明. 基于算法XCIST, T_i 中的每个顶点具有一个唯一的UUID成员属性。根据引理 5.9, $|V(T_i)| = 2^n$ 。根据引理 5.10, $V(T_i) \subseteq V(CQ_n)$ 。那么, 我们只需证明任意两个具有不同UUID成员属性的顶点存储不同的VALUE成员属性。

假设 z 表示 T_i 上的任一叶子顶点, 对于 $i = 0, 1, \dots, n-1$ 。根据算法XCIST, 起点为 $N(u, i)$ 且终点为 z 的途径能够被表示为 CQ_n 上的途径 $P: x^{(0)} = N(u, i), x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k) = z$, 对于整数 $k, 2 \leq k \leq n$, 满足如下条件:

(1) $m_1, m_2, \dots, m_k$ 各不相同,

(2) $0 \leq m_i \leq n-1$ 对于 $i = 1, 2, \dots, k$, 且

(3) $\langle \alpha^{(i)}_{i_1}, \alpha^{(i)}_{i_2}, \dots, \alpha^{(i)}_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$ 。

根据引理 5.4, P 是一条路径。根据算法XCIST, 令 $S = \{N(u, i)\} \cup \{z | z \in V(T_i) \setminus \{N(u, i)\} \text{ 和 } \text{LDF}(\text{parent}(z), z) > i\}$ 。根据引理 5.6, 我们能够验证 S 中的任意两个顶点各不相同。删除集合 S 中任意两个顶点之间的所有边。我们获得 $1 + 2^0 + 2^1 + \dots + 2^{n-2-i}$ 棵不连通子树。给定任意一个顶点 $z \in S$, 类似于引理 5.3, 我们能够证明以 z 为根的子树是 $CQ_n(z_{n-1}z_{n-2} \dots z_i)$ 上的一棵生成树。

给定任意两个顶点 $v, w \in S$, 令 C_1 是 T_i 中 $\langle N(u, i), v \rangle$ -路径和 $\langle N(u, i), w \rangle$ -路径的起点为 $N(u, i)$ 的共同子路径部分。起点为 v 的子树和起点为 w 的子树被分别表示为 G_1 和 G_2 。显然 G_1 和 G_2 分别是 $CQ_n(v_{n-1}v_{n-2} \dots v_i)$ 和 $CQ_n(w_{n-1}w_{n-2} \dots w_i)$ 上的两棵生成树。我们也能将 T_i 上的 $\langle N(u, i), v \rangle$ -路径和 $\langle N(u, i), w \rangle$ -路径表示如下:

$\langle N(u, i), v \rangle$ -路径: $C_1, x^{(0)}, x^{(1)} = N(x^{(0)}, i), m_1, x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k) = v$ 和

$\langle N(u, i), w \rangle$ -路径: $C_1, x^{(0)}, y^{(1)} = N(x^{(0)}, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(t)} = N(y^{(t-1)}, h_t) = w$,

对于整数 $k, t, 1 \leq k, t \leq n - i - 1$ 。

进一步, 基于算法XCIST, 我们能够验证如下条件成立:

(1) $0 \leq m_1 < m_2 < \dots < m_k \leq n - 1$,

(2) $0 \leq h_1 < h_2 < \dots < h_t \leq n - 1$,

(3) $m_1 \neq h_1$, 且

(4) $\min\{m_1, h_1\} > i$ 。

我们有如下子情形:

情形1. $m_k \neq h_t$ 。不失一般性, 假设 $m_k > h_t$ 。

首先, 根据引理 5.6,

$$V(\text{path}(\langle N(u, i), v \rangle\text{-路径}, x^{(1)}, v)) \cap V(\text{path}(\langle N(u, i), w \rangle\text{-路径}, y^{(1)}, w)) = \emptyset. \quad (5.1)$$

其次, $V(\text{path}(\langle N(u, i), w \rangle\text{-路径}, y^{(1)}, w))$ 中的每个顶点和 $V(G_1)$ 中的每个顶点具有一个最左不同位 m_k 。我们有,

$$V(\text{路径}(\langle N(u, i), w \rangle\text{-路径}, y^{(1)}, w)) \cap V(G_1) = \emptyset. \quad (5.2)$$

现在我们证明 $V(\text{path}(\langle N(u, i), v \rangle\text{-路径}, x^{(1)}, v)) \cap V(G_2) = \emptyset$ 。

将 $\text{path}(< N(u, i), v > \text{-路径}, x^{(1)}, v)$: $x^{(1)} = N(x^{(0)}, m_1)$, $x^{(2)} = N(x^{(1)}, m_2)$, $\dots$, $x^{(k)} = N(x^{(k-1)}, m_k) = v$ 划分成路径 P_1 , P_2 和 P_3 。令 P_1 : $x^{(1)} = N(u, m_1)$, $x^{(2)} = N(x^{(1)}, m_2)$, $\dots$, $x^{(a)} = N(x^{(a-1)}, m_a)$, 这里 $m_a < h_t$ 和 $m_{a+1} \geq h_t$ 。

我们有如下两种子情形:

情形1.1. $m_{a+1} = h_t$ 。我们有

P_2 : $x^{(a+1)} = N(x^{(a)}, m_{a+1})$, 和

P_3 : $x^{(a+2)} = N(x^{(a+1)}, m_{a+2})$, $x^{(a+3)} = N(x^{(a+2)}, m_{a+3})$, $\dots$, $x^{(k)} = N(x^{(k-1)}, m_k)$ 。

基于算法XCIST, 我们能够验证 $V(P_1)$ 中每个顶点的第 h_t 位是 u_t , $V(G_2)$ 中每个顶点的第 h_t 位是 $\overline{u_t}$, 这意味着 $V(P_1) \cap V(G_2) = \emptyset$ 。基于引理 5.2, $x^{(a+1)}$ 和 w 有一个最左不同位 $d > i$ 。既然 G_2 是 $CQ_n(w_{n-1}w_{n-2} \dots w_i)$ 上以 w 为根的生成树, G_2 中每个顶点的第 d 位不同于 w 的对应位, 这意味着 $V(P_2) \cap V(G_2) = \emptyset$ 。根据算法XCIST, 任一顶点 $x^{(b)} \in V(P_3)$ 第 b 位是 $\overline{u_b}$, 集合 $V(G_2)$ 中任一顶点第 b 位是 u_b , 这意味着 $V(P_3) \cap V(G_2) = \emptyset$ 。

因此, 我们有

$$V(< N(u, i), v > \text{-路径}) \cap V(G_2) = \emptyset. \quad (5.3)$$

根据等式(5.1), (5.2) 和(5.3), $V(\text{path}(P, x^{(1)}, x^{(k)})) \cap V(\text{path}(P', y^{(1)}, y^{(t)})) = \emptyset$ 。

情形1.2. $m_{a+1} > h_t$ 。令 $V(P_2) = \emptyset$, $E(P_2) = \emptyset$ 和 P_3 : $x^{(a+1)} = N(x^{(a)}, m_{a+1})$, $x^{(a+2)} = N(x^{(a+1)}, m_{a+2})$, $\dots$, $x^{(k)} = N(x^{(k-1)}, m_k)$ 。类似于情形1.1, 我们可证明 $V(\text{path}(P, x^{(1)}, x^{(k)})) \cap V(\text{path}(P', y^{(1)}, y^{(t)})) = \emptyset$ 。

情形2. $m_k = h_t$ 。根据引理 5.2, 顶点 $x^{(k)}$ 和顶点 $y^{(t)}$ 具有一个最左不同位 $d > i$, 这意味着 $V(G_1) \cap V(G_2) = \emptyset$ 。

基于算法XCIST, $V(G_1)$ 中每个顶点的第 m_k 位是 $\overline{m_k}$, $V(< N(u, i), w > \text{-路径}) \setminus \{y^{(t)}\}$ 中每个顶点的第 m_k 位是 m_k , 这意味着 $(V(G_1) \cap V(< x^{(0)}, w > \text{-路径})) \setminus \{y^{(t)}\} = \emptyset$ 。

基于算法XCIST, $V(G_2)$ 中每个顶点的第 h_t 位是 $\overline{m_k}$, $V(< N(u, i), v > \text{-路径}) \setminus \{x^{(k)}\}$ 每个顶点的第 h_t 位是 m_k , 这意味着 $(V(G_2) \cap V(< x^{(0)}, v > \text{-路径})) \setminus \{x^{(k)}\} = \emptyset$ 。

那么, 我们有 $V(< x^{(0)}, v > \text{-路径}) \cap V(G_2) = \emptyset$ 。

基于如上讨论, 引理成立。

5.3.2 IST的证明

我们首先给出两个关于IST的基础结果。然后, 我们证明算法XCIST构造出

的 CQ_n 上以 u 为根的 n IST的正确性, 并且分析算法的时间复杂度。需要指出的是我们使用交叉立方体的顶点之间维邻接关系的重要性质来证明其正确性。

引理 5.12. 给定 CQ_n 上的两条路径 $P: u, x^{(0)} = N(u, q), x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k) = v$ 和 $P': u, y^{(0)} = N(u, r), y^{(1)} = N(y^{(0)}, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(t)} = N(y^{(t-1)}, h_t) = v$, 对于整数 $k, t, q, r, 1 \leq k, t \leq n$ 和 $0 \leq q < r \leq n-1$, 如果下面条件成立:

(1) $\langle \alpha_{i_1}^{(q)}, \alpha_{i_2}^{(q)}, \dots, \alpha_{i_k}^{(q)} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$ 且 $\alpha_{i_1}^{(q)} \neq q$, 且

(2) $\langle \alpha_{j_1}^{(r)}, \alpha_{j_2}^{(r)}, \dots, \alpha_{j_t}^{(r)} \rangle = \langle h_1, h_2, \dots, h_t \rangle$, 这里 $1 \leq j_1 < j_2 < \dots < j_t \leq n$, 和 $\alpha_{j_1}^{(r)} \neq r$,

那么, $\text{ancestor}(v, P) \cap \text{ancestor}(v, P') = \{u\}$ 且 $\text{ancestor}(v, P) \cup \text{ancestor}(v, P') \supset \{u\}$ 。

证明. 因 $q < r$, 故 $x^{(0)} \neq y^{(0)}$, 这意味着 $\text{ancestor}(v, P) \cup \text{ancestor}(v, P') \supset \{u\}$ 。令 $a = \max\{m_1, m_2, \dots, m_k\}$ 和 $b = \max\{h_1, h_2, \dots, h_t\}$ 。我们也能令

$$P_1: u, x^{(0)} = N(u, q), x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(a)} = N(x^{(a-1)}, m_a)。$$

$$P_2: x^{(a+1)} = N(x^{(a)}, m_{a+1}), x^{(a+2)} = N(x^{(a+1)}, m_{a+2}), \dots, x^{(k)} = N(x^{(k-1)}, m_k) = v。$$

$$P_1': u, y^{(0)} = N(u, r), y^{(1)} = N(y^{(0)}, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(b)} = N(y^{(b-1)}, h_b)。$$

$$P_2': y^{(b+1)} = N(y^{(b)}, h_{b+1}), y^{(b+2)} = N(y^{(b+1)}, h_{b+2}), \dots, y^{(t)} = N(y^{(t-1)}, h_t) = v。$$

我们有如下情形:

情形1. $a \neq b$ 和 $\max\{a, b\} > r$ 。不失一般性, 假设 $a > b$ 。那么, 路径 P 中 v 的第 a 位不同于路径 P' 中 v 的第 a 位, 这是一个矛盾。

情形2. $a = b > r$ 。那么, 我们能够验证 $V(P_2)$ 中的每个顶点和 $V(P_1')$ 中的每个顶点具有一个最左不同位 a , 这意味着

$$V(P_2) \cap V(P_1') = \emptyset. \quad (5.4)$$

类似地, 我们能够证明

$$V(P_1) \cap V(P_2') = \emptyset. \quad (5.5)$$

根据引理 5.2, $x^{(a)}$ 和 $y^{(b)}$ 具有一个最左不同位 $d \geq \min\{m_1, h_1\} \geq q$ 。根据定义 2.5, 我们能够验证 $d \in \{m_k, h_t\}$ 和 $m_k \neq h_t$ 。容易验证

$$V(P_2) \cap V(P_2') = \{v\}. \quad (5.6)$$

根据引理 5.6,

$$V(\text{path}(P_1, x^{(1)}, x^{(a)})) \cap V(\text{path}(P'_1, y^{(1)}, y^{(b)})) = \emptyset. \quad (5.7)$$

根据等式(5.4), (5.5), (5.6)和(5.7), 我们有 $\text{ancestor}(v, P) \cap \text{ancestor}(v, P') = \{u\}$ 。

情形3. $a = b$ 和 $\max\{a, b\} = r$ 。我们能够验证路径 P 中顶点 v 的第 r 位不同于路径 Q 中顶点 v 的第 r 位, 这是一个矛盾。

情形4. $a \neq b$ 和 $\max\{a, b\} = r$ 。那么, 我们有如下两种子情形。

情形4.1. $a = r > b$ 。那么, $V(P'_1) = \{u, y_1\}$ 。根据引理 5.6, 我们能够验证 y_1 和 $x^{(a)}$ 具有一个最左不同位 $d \geq q$ 。因此, 我们能够验证 $\max\{m_k, h_t\} = d$ 和 $m_k \neq h_t$ 。证明类似于情形2。

情形4.2. $b = r > a$ 。我们能够验证 $V(P') \setminus \{u, v\}$ 中的任一顶点和 $V(P) \setminus \{u, v\}$ 中的任一顶点具有一个最左不同位 r 。因此, $\text{ancestor}(v, P) \cap \text{ancestor}(v, P') = \{u\}$ 。

情形5. $\max\{a, b\} < r$ 。我们能够验证 P 中 v 的第 r 位不同于 Q 中 v 的第 r 位, 这是一个矛盾。

基于以上讨论, 引理成立。

定理 5.13. 对于整数 $n \geq 1$ 和 CQ_n 上的任一顶点 u , 由算法XCIST构造的 $T_0, T_1, \dots, T_{n-1}$ 是 CQ_n 上以顶点 u 为根的 n 棵IST。

证明. 当 $n = 1$ 时显然成立。现在我们考虑 $n \geq 2$ 。根据引理 5.11, 算法XCIST构造的 T_i 是 CQ_n 上的一棵生成树, 对于 $i = 0, 1, \dots, n-1$ 。现在我们证明 $T_0, T_1, \dots, T_{n-1}$ 是相互独立的。对于任意整数 i, j , $0 \leq i < j \leq n-1$ 和任一顶点 $v \in V(CQ_n)$, $v \neq u$, 令 P 表示 T_i 中的路径 $u, x^{(0)} = N(u, i), x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k) = v$ 且 P' 表示 T_j 中的路径 $u, y^{(0)} = N(u, j), y^{(1)} = N(y^{(0)}, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(t)} = N(y^{(t-1)}, h_t) = v$ 。根据算法XCIST, 我们容易验证 P 和 P' 满足引理 5.12中的条件, 因此, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) = \{u\}$ 且 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) \supset \{u\}$ 。根据引理 3.9, T_i 和 T_j 是相互独立的。

因此, 算法XCIST构造的 $T_0, T_1, \dots, T_{n-1}$ 是 CQ_n 上以任一顶点 u 为根的 n 棵IST。

基于算法XCIST、引理 5.9和定理 5.13, 我们有如下定理。

定理 5.14. 算法XCIST 在 $O(N)$ 时间内构造出 CQ_n 上以 u 为根的 n 棵IST, 其中 $N = 2^n$ 。

注意到算法XC DAT构造的 n 棵维邻接关系树独立于 CQ_n 的顶点地址。另外,我们容易验证算法XCIST构造的所有IST均同构于 n -级类二项树 BL_n 。

5.4 本章小结

本章讨论了交叉立方体上IST的并行构造问题。我们得到了三个主要结论:

- (1) 交叉立方体上存在一个顶点之间维邻接关系的重要性质——维扩散性质。
- (2) 对于任意整数 $n \geq 1$, 我们设计了一个时间复杂度为 $O(N)$ 的并行算法来构造 CQ_n 上以任一顶点为根的 n 棵IST $T_0, T_1, \dots, T_{n-1}$, 这里 $N = 2^n$ 。
- (3) 算法XC DAT构造的 n 棵维邻接关系树独立于 CQ_n 上的顶点地址, 且算法XCIST的输入是任意的。

本章关于在交叉立方体上IST的并行构造问题的相关结果已经发表在SCI源期刊《Journal of Parallel and Distributed Computing》上(第一作者)。

第六章 莫比乌斯立方体上IST的并行构造

基于交叉立方体、莫比乌斯立方体上IST的构造结果，交叉立方体上IST的并行构造结果，本章进一步研究了莫比乌斯立方体上IST的并行构造问题，主要取得如下结果：

(1) 从排列组合的角度，将圆排列的概念引入到IST的构造方法中。

(2) 首先证明了递减的圆排列能够用于并行构造莫比乌斯立方体上以任一顶点为根的IST。其次，进一步证明了任一圆排列能够用来并行构造莫比乌斯立方体和超立方体上以任一顶点为根的IST。

(3) 指出利用不同的圆排列构造的交叉立方体上的多组IST能够构造莫比乌斯两个顶点之间多组顶点不相交路径。

(4) 探讨了基于IST的高效诊断算法。

(5) 采用JUNG技术对IST的构造进行了模拟。

6.1 预备知识

n 个整数的序列称为排列 如果它包含从0 到 $n-1$ 的 n 个整数仅仅一次。一个圆排列 (Circular-Dimensional Permutation) 是将 n 个不同整数围成一个圆的排列 (顺时针方向)，如图 6.1所示。在一个圆中，所有的圆排列是等价的，例如，0123，1230，2301，3012是同样的圆排列。 n 个整数所能表示的圆排列的总数是 $n!/n = (n-1)!$ 个。例如，选择四个整数0,1,2,3的圆排列如图 6.1所示。

对于任意整数 k ， $1 \leq k \leq n-1$ ，和一个二进制串整数 $s \in \{0,1\}^k$ ，显然 $M_n(s)$ 表示 $V(M_n)$ 上所有具有共同前缀 s 的顶点在 M_n 中导出的子图。根据引理 4.1，如果 $x = x_{n-1}x_{n-2} \dots x_0$ 是 M_n 上的任一顶点，对于整数 $n \geq 2$ ，那么 x 属于 M_n 的子图 $M_n(x_{n-1}x_{n-2} \dots x_i)$ 。对于任意整数 i ， $1 \leq i \leq n-1$ ，显然 $M_n(x_{n-1}x_{n-2} \dots x_i)$ 同构于 $x_i-M_{n-|x_{n-1}x_{n-2} \dots x_i|}$ 。

6.2 维邻接关系途径和维邻接关系树

我们首先讨论维邻接关系途径和维邻接关系树的定义 (这里在存储表示上与前一章介绍的定义不一样，我们仍用相同名称，但表示法上有所区别)，其将用来构造 M_n 上以任一顶点为根的 n IST。

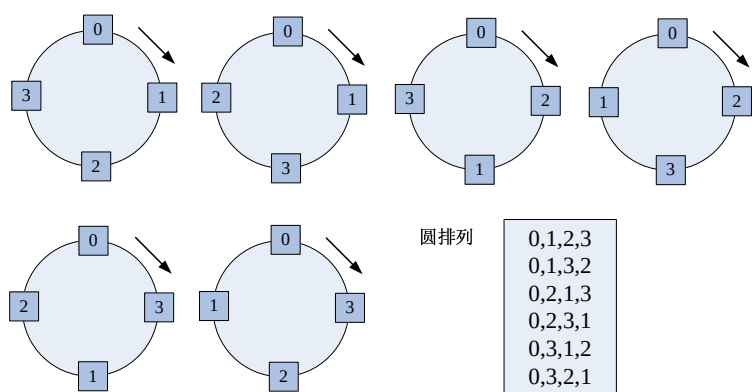
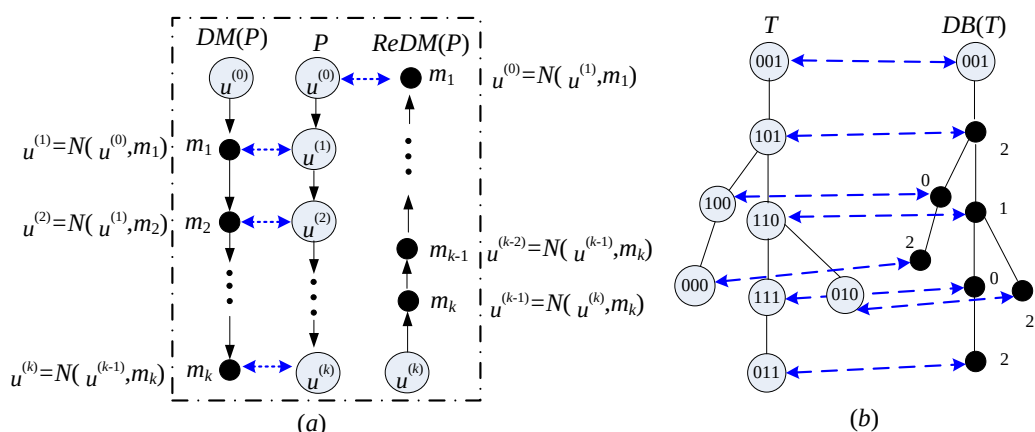


图 6.1 通过选择四个整数0,1,2,3得到的圆排列

图 6.2 (a) 维邻接关系途径和反向的维邻接关系途径; (b) $0-M_3$ 上以顶点001为根的树 T 和它的维邻接关系树以顶点001为根的 $DB(T)$

定义 6.1. 令 $P : u^{(0)}, u^{(1)}, \dots, u^{(k)}$ 为 M_n 上的一条路径, 对于 $n \geq 2$ 。对于任一顶点 $u^{(i)} = N(u^{(i-1)}, m_i) \in V(P) \setminus \{u^{(0)}\}$, 我们将 $u^{(i)}$ 替换为 m_i (显然, $0 \leq m_i \leq n-1$)。那么, 我们得到一条维邻接关系途径: $u^{(0)}, m_1, m_2, \dots, m_k$ 。令 $DM(P)$ 表示 P 的维邻接关系途径。

类似地, 反向维邻接关系途径被定义如下:

对于任一顶点 $u^{(i-1)} = N(u^{(i)}, m_i) \in V(P) \setminus \{u^{(0)}\}$, 我们将 $u^{(i-1)}$ 替换成 m_i 。那么, 我们得到一条反向维邻接关系途径: $u^{(k)}, m_k, m_{k-1}, \dots, m_1$ 。令 $ReDM(P)$ 表示 P 的反向维邻接关系途径。

图 6.2(a) 示出了维邻接关系途径和反向的维邻接关系途径的构造。基于定义 6.1, 我们进一步给出定义 6.2 如下。

定义 6.2. 假设 T 是 M_n 上一棵以 x 为根的树。对于任一顶点 $y \in V(T) \setminus \{x\}$, 如果 $\text{parent}(y, T) = N(y, m)$, 我们将 y 替换成 m 。那么, 我们得到一棵关于树 T 的维邻接关系树 $DB(T)$ 。

在图 6.2(b)中, T 是 $0-M_3$ 上以顶点001为根的一棵生成树, $DB(T)$ 是关于 T 的维邻接关系树。显然, 对于 M_n 上以顶点 x 为根的树 T , $DB(T)$ 将被唯一地构造, 反之亦然: 假设我们采用广度优先遍历。对于任一顶点 $y \in V(DB(T)) \setminus \{x\}$, 将树 $DB(T)$ 中的 y 替换成 $N(\text{parent}(y, T), y)$, 则也能得到树 T 。

注意到, 维邻接关系途径和维邻接关系树主要用来证明莫比乌斯立方体上以任一顶点为根的IST的正确性。

6.3 IST的构造

我们首先给出一个并行算法来构造 M_n 上以任一顶点为根的IST。然后我们证明这 n 棵IST的正确性并分析算法的时间复杂度。接着, 我们进一步给出高效的算法来构造 M_n 上以任一顶点为根的IST。我们总是用 $x = x_{n-1}x_{n-2} \dots x_0$ 来表示 M_n 上以任一顶点为根的IST。

6.3.1 基于一个圆排列来构造IST的算法

给定 n 个整数 $0, 1, \dots, n-1$, 我们有 $(n-1)!$ 圆排列, 其中递减的圆排列为 $n-1, n-2, \dots, 0$ 。我们首先设计一个并行算法, 称为PMCIIST, 基于递减的圆排列 $n-1, n-2, \dots, 0$ 来构造 M_n 上以任一顶点 x 为根的 n 棵IST。然后, 我们示出IST和维邻接关系树的构造过程, 分别如图 6.3(a) 和图 6.3(b)所示。

假设输入为 $n = 4$, $x = 1$ 和 $S = \{3, 2, 1, 0\}$ 。这里, $3, 2, 1, 0$ 是递减的圆排列。现在我们以 $0-M_4$ 上的 T_0 为例, 如图 6.3(a)所示。构造过程描述如下:

首先, 树 T_0 中仅有一个顶点0。经过第一轮迭代(level = 0), T_0 中顶点0连接其3维邻接点8。此时, $V(T_0) = \{0, 8\}$ 和 $E(T_0) = \{(0, 8)\}$ 。经过第二轮迭代(level = 1), 每个顶点 $y \in V(T_0)$ 连接其2维邻接点 $N(y, 2)$ 。边 $(0, 4)$ 和 $(8, 15)$ 被增加进 T_0 , 此时 $V(T_0) = \{0, 4, 8, 15\}$ 。经过第三轮迭代, 每个顶点 $y \in V(T_0)$ 连接其1维邻接点 $N(y, 1)$ 。此时, $V(T_0)$ 中顶点数目翻倍。在最后一轮迭代中, 每个顶点 $y \in V(T_0)$ 连接其0维邻接点 $N(y, 0)$ 。此时我们得到 $0-M_4$ 上一棵以顶点1为根的 T_0 。

图 6.3(a)中 $0-M_4$ 上 T_1 , T_2 和 T_3 的构造过程类似于 T_0 。图 6.3(b) 示出了分别对应于 T_0 , T_1 , T_2 和 T_3 的维邻接关系树 $DB(T_0)$, $DB(T_1)$, $DB(T_2)$ 和 $DB(T_3)$ 的构造。我们容

算法 11 PMCIST

输入: 一个整数 $n \geq 2$, 根顶点 x , 一个整型数组 $S = \{n-1, n-2, \dots, 0\}$;

输出: M_n 上以顶点 x 为根的 n 棵生成树 $T_0, T_1, \dots, T_{n-1}$;

开始

1: **for** $i = 0$ to $n-1$ **do**

2: 并行模式

3: $V(T_i) = N(x, i)$; /*初始时, 集合 $V(T_i)$ 中仅有一个顶点 x 的 i 维邻接点。*/

4: $E(T_i) = \emptyset$;

5: **for** level = 0 to $n-1$ **do**

6: **for** 任一顶点 $x \in V(T_i)$ **do**

7: $y = N(x, S[(\text{level} - i + n) \bmod n])$;

8: $E(T_i) = E(T_i) \cup \{(x, y)\}$;

9: $V(T_i) = V(T_i) \cup \{y\}$; /*这里, y 基于递减圆排列获得。*/

10: **end for**

11: **end for**

12: **end for**

结束

易验证图 6.3(a) 中 $0-M_4$ 上的四棵以顶点 1 为根的树是 IST。另一方面, $0-M_4$ 基于维邻接关系树以 1 为根的 IST 将被唯一构造, 如图 6.3(b) 所示。

6.3.2 算法 PMCIST 的正确性证明

首先我们给出一些关于莫比乌斯立方体上路径和途径的结论。然后, 我们证明算法 PMCIST 构造的 n 棵树是 M_n 上以顶点 x 为根的生成树。最后, 我们证明这 n 棵生成树是 M_n 上 n 棵以顶点 x 为根的 IST, 并且分析算法的时间复杂度 (假设 $n \geq 2$)。

基于莫比乌斯立方体的定义和引理 4.2, 我们引入如下两个引理。

若 $\alpha_1, \alpha_2, \dots, \alpha_n \geq 0$, 我们称向量 $\langle \alpha_1, \alpha_2, \dots, \alpha_n \rangle$ 为非负向量。我们总是令非负向量 $\langle \alpha_1, \alpha_2, \dots, \alpha_n \rangle = \langle n-1, n-2, \dots, 0 \rangle$ 。

基于引理 4.4 和引理 4.5, 我们给出如下三个关于不相交路径的引理。

引理 6.1. 假设路径 $P: u^{(0)}, u^{(1)} = N(u^{(0)}, m_1), u^{(2)} = N(u^{(1)}, m_2), \dots, u^{(k)} = N(u^{(k-1)}, m_k) = y$ 和 $P': v^{(0)}, v^{(1)} = N(v^{(0)}, m_1'), v^{(2)} = N(v^{(1)}, m_2'), \dots, v^{(t)} = N(v^{(t-1)}, m_t') = y$ 是 M_n 上的两条路径, 对于任意两个整数 $k, t, 1 \leq k, t \leq n$ 和 $m_k \neq m_t'$ 。如果下面条件成立:

(1) $\langle \alpha_{i_1}, \alpha_{i_2}, \dots, \alpha_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$, 且

(2) $\langle \alpha_{j_1}, \alpha_{j_2}, \dots, \alpha_{j_t} \rangle = \langle m_1', m_2', \dots, m_t' \rangle$, 这里 $1 \leq j_1 < j_2 < \dots < j_t \leq n$,

那么 $V(\text{path}(P, u^{(0)}, u^{(k)})) \cap V(\text{path}(P', v^{(0)}, v^{(t)})) = \{y\}$ 。

证明. 我们通过反证法证明该引理。假设引理不成立。根据定义 6.1, P 的反向维邻接关系途径为 $y, m_k, m_{k-1}, \dots, m_1$ 和 P' 的反向维邻接关系途径为 $y, m'_t, m'_{t-1}, \dots, m'_1$ 。那么, 存在 a 和 b , 使得 $u^{(a)} = v^{(b)}$, 这里 $0 \leq a \leq k-1$ 和 $0 \leq b \leq t-1$ 。令 $A = \{m_k, m_{k-1}, \dots, m_{a+1}\}$ 和 $B = \{m'_t, m'_{t-1}, \dots, m'_{b+1}\}$ 。既然 $m_k \neq m'_t$, 我们有 $A \neq B$ 。令 $l = \max((A \cup B) \setminus (A \cap B))$ 。根据引理 4.2 和引理 4.3, $u^{(a)}$ 的第 l 位不同于 $v^{(b)}$ 的对应位, 于是产生矛盾。因此, 引理成立。

引理 6.2. 假设路径 $P: u^{(0)}, u^{(1)} = N(u^{(0)}, m_1), u^{(2)} = N(u^{(1)}, m_2), \dots, u^{(k)} = N(u^{(k-1)}, m_k)$ 和 $P': u^{(0)}, v^{(1)} = N(u^{(0)}, m'_1), v^{(2)} = N(v^{(1)}, m'_2), \dots, v^{(t)} = N(v^{(t-1)}, m'_t)$ 是 M_n 上的两条路径, 对于任意两个整数 $k, t, 1 \leq k, t \leq n, m_1 \neq m'_1$ 。如果下面条件成立:

- (1) $\langle \alpha_{i_1}, \alpha_{i_2}, \dots, \alpha_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$, 且
- (2) $\langle \alpha_{j_1}, \alpha_{j_2}, \dots, \alpha_{j_t} \rangle = \langle m'_1, m'_2, \dots, m'_t \rangle$, 这里 $1 \leq j_1 < j_2 < \dots < j_t \leq n$,

这里 $V(\text{path}(P, u^{(1)}, u^{(k)})) \cap V(\text{path}(P', v^{(1)}, v^{(t)})) = \emptyset$ 。

证明. 根据引理中的条件, $u^{(1)}$ 是 $u^{(0)}$ 的第 m_1 维邻接点, $v^{(1)}$ 是 $v^{(0)}$ 的第 m'_1 维邻接点。显然, $u^{(1)} \in V(M_n(x_{n-1}x_{n-2} \dots x_{m_1+1} \overline{x_{m_1}}))$ 和 $v^{(1)} \in V(M_n(x_{n-1}x_{n-2} \dots x_{m'_1+1} \overline{x_{m'_1}}))$ 。类似地, 我们有 $\{u^{(2)}, u^{(3)} \dots u^{(k)}\} \subseteq V(M_n(x_{n-1}x_{n-2} \dots x_{m_1+1} \overline{x_{m_1}}))$ 和 $\{v^{(2)}, v^{(3)} \dots v^{(t)}\} \subseteq V(M_n(x_{n-1}x_{n-2} \dots x_{m'_1+1} \overline{x_{m'_1}}))$ 。

既然 $m_1 \neq m'_1$, 我们有 $V(M_n(x_{n-1}x_{n-2} \dots x_{m_1+1} \overline{x_{m_1}})) \cap V(M_n(x_{n-1}x_{n-2} \dots x_{m'_1+1} \overline{x_{m'_1}})) = \emptyset$, 这意味着 $\{u^{(1)}, u^{(2)}, u^{(3)} \dots u^{(k)}\} \cap \{v^{(1)}, v^{(2)}, v^{(3)} \dots v^{(t)}\} = \emptyset$ 。

因此, $V(\text{path}(P, u^{(1)}, u^{(k)})) \cap V(\text{path}(P', v^{(1)}, v^{(t)})) = \emptyset$ 。

接下来, 我们令非负向量 $\langle \beta_1, \beta_2, \dots, \beta_n \rangle = \langle q-1, \dots, 0, n-1, n-2, \dots, q \rangle$, 对于任意整数 $q, 0 \leq q \leq n-1$ 。

引理 6.3. 假设 $P: u^{(0)}, u^{(1)} = N(u^{(0)}, q), u^{(2)} = N(u^{(1)}, m_1), u^{(3)} = N(u^{(2)}, m_2), \dots, u^{(k+1)} = N(u^{(k)}, m_k)$ 和 $P': u^{(0)}, u^{(1)} = N(u^{(0)}, q), v^{(2)} = N(v^{(1)}, m'_1), v^{(3)} = N(v^{(2)}, m'_2), \dots, v^{(t+1)} = N(v^{(t)}, m'_t)$ 是 M_n 上的两条路径, 对于任意两个整数 $k, t, 2 \leq k, t \leq n, m_1 \neq m'_1$ 。如果下面条件成立:

- (1) $\langle \beta_{i_1}, \beta_{i_2}, \dots, \beta_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$, 且
- (2) $\langle \beta_{j_1}, \beta_{j_2}, \dots, \beta_{j_t} \rangle = \langle m'_1, m'_2, \dots, m'_t \rangle$, 这里 $1 \leq j_1 < j_2 < \dots < j_t \leq n$,

那么 $V(\text{path}(P, u^{(2)}, u^{(k+1)})) \cap V(\text{path}(P', v^{(2)}, v^{(t+1)})) = \emptyset$ 。

证明. 假设 $u^{(a)} = v^{(b)} \in V(\text{path}(P, u^{(2)}, u^{(k+1)})) \cap V(\text{path}(P', v^{(2)}, v^{(t+1)}))$ 。令 $A = \{m_1, m_2, \dots, m_{a-1}\}$ 和 $B = \{m_1', m_2', \dots, m_{b-1}'\}$ 。既然 $m_1 \neq m_1'$, 我们有 $A \neq B$ 。假设 $l = \max((A \cup B) \setminus (A \cap B))$ 。根据引理 4.2 和引理 4.3, $u^{(a)}$ 的第 l 位不同于 $v^{(b)}$ 的对应位。因此 $u^{(a)} \neq v^{(b)}$, 于是产生矛盾。

根据引理 6.3, 我们有如下推论:

推论 6.4. 假设 $P: u^{(0)}, u^{(1)} = N(u^{(0)}, q), u^{(2)} = N(u^{(1)}, m_1), u^{(3)} = N(u^{(2)}, m_2), \dots, u^{(j-1)} = N(u^{(j-2)}, m_{j-2}), u^{(j)} = N(u^{(j-1)}, m_{j-1}), u^{(j+1)} = N(u^{(j)}, m_j), u^{(j+2)} = N(u^{(j+1)}, m_{j+1}), \dots, u^{(k+1)} = N(u^{(k)}, m_k)$ 和 $P': u^{(0)}, u^{(1)} = N(u^{(0)}, q), u^{(2)} = N(u^{(1)}, m_1), u^{(3)} = N(u^{(2)}, m_2), \dots, u^{(j-1)} = N(u^{(j-2)}, m_{j-2}), u^{(j)} = N(u^{(j-1)}, m_{j-1}), v^{(j+1)} = N(u^{(j)}, m_j'), v^{(j+2)} = N(v^{(j+1)}, m_{j+1}'), \dots, v^{(t+1)} = N(v^{(t)}, m_t')$ 是 M_n 上的两条路径, 对于整数 $j, k, t, 2 \leq j \leq k \leq n, 2 \leq j \leq t \leq n$ 和 $m_j \neq m_j'$ 。如果下面条件成立:

(1) $\langle \beta_{i_1}, \beta_{i_2}, \dots, \beta_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$, 且

(2) $\langle \beta_{j_1}, \beta_{j_2}, \dots, \beta_{j_t} \rangle = \langle m_1, m_2, \dots, m_{j-2}, m_{j-1}, m_j', m_{j+1}', \dots, m_t' \rangle$, 这里 $1 \leq j_1 < j_2 < \dots < j_t \leq n$,

那么 $V(\text{path}(P, u^{(j+1)}, u^{(k+1)})) \cap V(\text{path}(P', v^{(j+1)}, v^{(t+1)})) = \emptyset$ 。

引理 6.5. $|V(T_i)| = 2^n$, 这里 $0 \leq i \leq n-1$ 且 T_i 是算法 *PMCIST* 构造的树。

证明. 经过算法 *PMCIST* 的 n -轮迭代, 显然, 在 $V(T_i)$ 中有

$$1 + 2^0 + 2^1 + \dots + 2^{n-1} = 2^n$$

个顶点, 对于任意整数 $i, 0 \leq i \leq n-1$ 。

备注 6.1. 假设在算法 *PMCIST* 构造出 M_n 上 n 棵以任一顶点 x 为根的树 $T_0, T_1, \dots, T_{n-1}$ 。那么 x 在树 T_i 中以 $N(x, i)$ (x 的 i 维邻接点) 为唯一孩子顶点, 对于 $i = 0, 1, \dots, n-1$ 。

图 6.3(a) 示出了基于算法 *PMCIST* 的 $0-M_4$ 上 4 棵以顶点 1 为根的树 T_0, T_1, T_2 和 T_3 的构造过程, 这里顶点 1 分别以 $N(1, 0) = 0, N(1, 1) = 3, N(1, 2) = 5$ 和 $N(1, 3) = 9$ 作为 T_0, T_1, T_2 和 T_3 中的唯一孩子顶点。

备注 6.2. 假设 P 是算法 *PMCIST* 构造的以顶点 x 为根的 T_i 中的任一 $\langle x, y \rangle$ -路径, 这里 y 表示 T_i 中的叶子顶点, 对于 $i = 0, 1, \dots, n-1$ 。我们将 P 表示为 $x, u^{(0)} = N(x, i)$,

$u^{(1)} = N(u^{(0)}, m_1)$, $u^{(2)} = N(u^{(1)}, m_2), \dots, y = N(u^{(k-1)}, m_k)$, 这里 $k > 2$ 且 $m_k = i$ 。那么, 我们有

$$\langle \beta_{i_1}, \beta_{i_2}, \dots, \beta_{i_k} \rangle = \langle m_1, m_2, m_3, \dots, m_k = i \rangle, \text{ 这里 } 1 \leq i_1 < i_2 < \dots < i_k \leq n,$$

例如, T_0 中路径 $\langle 1, 13 \rangle$ -路径的维邻接关系途径为序列 $1, 0, 3 = m_1, 2 = m_2, 1 = m_3, 0 = m_4$ 所示且该序列满足备注 6.2, 如图 6.3(b)所示。

根据引理 6.3, 引理 6.5 和算法 PMCIST, 我们有如下定理。

定理 6.6. 对于任意整数 i , $0 \leq i \leq n-1$, 算法 PMCIST 构造的 T_i 是 M_n 上一棵以顶点 x 为根的生成树。

证明. 根据算法 PMCIST, 我们得到 M_n 上以顶点 x 为根的 n 棵树 $T_0, T_1, \dots, T_{n-1}$ 。根据引理 6.3 和推论 6.4, T_i 中没有共同顶点, $i = 0, 1, \dots, n-1$ 。根据引理 4.3, $V(T_i) \subseteq V(M_n)$ 。根据引理 6.5, $|V(T_i)| = 2^n$ 。因此, 对于任意整数 i , $0 \leq i \leq n-1$, 算法 PMCIST 构造的 T_i 是 M_n 上以顶点 x 为根的生成树。

我们令非负向量 $\langle \gamma_1, \gamma_2, \dots, \gamma_n \rangle = \langle r-1, r-2, \dots, 0, n-1, n-2, \dots, r \rangle$, 对于整数 r , $0 \leq r \leq n-1$ 。现在, 我们给出如下引理, 其将用来证明算法 PMCIST 构造的 IST 的正确性。

引理 6.7. 假设 $P: u^{(0)}, u^{(1)} = N(u^{(0)}, q), u^{(2)} = N(u^{(1)}, m_1), u^{(3)} = N(u^{(2)}, m_2), \dots, u^{(k+1)} = N(u^{(k)}, m_k)$ 和 $P': v^{(0)}, v^{(1)} = N(v^{(0)}, r), v^{(2)} = N(v^{(1)}, m_1'), v^{(3)} = N(v^{(2)}, m_2'), \dots, v^{(t+1)} = N(v^{(t)}, m_t')$ 是 M_n 上的两条路径, 对于任意两个整数 k, t , $2 \leq k, t \leq n$ 和 $r \neq q$ 。如果下面条件成立:

- (1) $\langle \beta_{i_1}, \beta_{i_2}, \dots, \beta_{i_k} \rangle = \langle m_1, m_2, \dots, m_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$ 且 $\beta_{i_1} \neq q$,
- (2) $\langle \gamma_{j_1}, \gamma_{j_2}, \dots, \gamma_{j_t} \rangle = \langle m_1', m_2', \dots, m_t' \rangle$, 这里 $1 \leq j_1 < j_2 < \dots < j_t \leq n$ 且 $\gamma_{j_1} \neq r$, 且
- (3) $y \in V(P) \cap V(P') \setminus \{u^{(0)}\}$,

那么, $\text{ancestor}(y, P) \cap \text{ancestor}(y, P') = \{u^{(0)}\}$ 和 $\text{ancestor}(y, P) \cup \text{ancestor}(y, P') \supset \{u^{(0)}\}$ 。

证明. 不失一般性, 假设 $r > q$ 。既然 $r \neq q$, 我们有 $u^{(1)} \neq v^{(1)}$, 这意味着 $\text{ancestor}(y, P) \cup \text{ancestor}(y, P') \supset \{u^{(0)}\}$ 。如果 $y = u^{(1)}$ 或 $y = v^{(1)}$, 显然, $\text{ancestor}(y, P) \cap \text{ancestor}(y, P') = \{u^{(0)}\}$ 。因此, 接下来, 我们只需证明 $\text{ancestor}(y, P) \cap \text{ancestor}(y, P') = \{u^{(0)}\}$, 这里 $y =$

$u^{(a)} = v^{(b)} \in V(P) \cup V(P') \setminus \{u^{(0)}, u^{(1)}, v^{(1)}\}$ 。令 $A = \{m_1, m_2, \dots, m_{a-1}\}$ 和 $B = \{m_1', m_2', \dots, m_{b-1}'\}$, 这里 $1 \leq a \leq k+1$ 且 $1 \leq b \leq t+1$ 。假设 $l = \max((A \cup B) \setminus (A \cap B))$ 。我们有如下三种情形:

情形1. $l > r$ 。根据引理 4.2, $u^{(a)}$ 的第 l 位不同于 $v^{(b)}$ 的对应位。因此, $u^{(a)} \neq v^{(b)}$, 产生矛盾。

情形2. $l < r$ 。既然 $v^{(1)} = N(u^{(0)}, r)$, 根据引理 4.2, $u^{(a)}$ 的第 r 位不同于 $v^{(b)}$ 的对应位。那么, $u^{(a)} \neq v^{(b)}$, 这是一个矛盾。

情形3. $l = r$ 。我们将 A 划分成两个子集合 $A_1 = \{m_1, m_2, \dots, m_i\}$ 和 $A_2 = \{m_{i+1}, m_{i+2}, \dots, m_{a-1}\} \supseteq \{m_{a-1}\}$ 。将 B 划分成两个子集合 $B_1 = \{m_1', m_2', \dots, m_j'\}$ 和 $B_2 = \{m_{j+1}', m_{j+2}', \dots, m_{b-1}'\} \supseteq \{m_{b-1}'\}$, 使得如下四个序列

(1) $m_1, m_2, \dots, m_i,$

(2) $m_{i+1}, m_{i+2}, \dots, m_{a-1},$

(3) $m_1', m_2', \dots, m_j'$ 和

(4) $m_{j+1}', m_{j+2}', \dots, m_{b-1}'$

皆以递减顺序排列。我们有如下两种子情形:

情形3.1. $r \in B$ 和 $r \notin A$ 。根据引理中的条件, $\max(B_1) < r$ 且 $\min(B_2) \geq r$ 。因此, $m_{b-1}' = r$ 。既然 $r \notin A$, $m_{a-1} \neq r$ 。我们有如下三个子情形。

情形3.1.1. $m_{i+1} = m_{j+1}'$ 。根据引理 6.1,

$$V(\text{path}(P, u^{(i+2)}, u^{(k)})) \cap V(\text{path}(P', v^{(j+2)}, v^{(t)})) = \{y\}. \quad (6.1)$$

根据引理 6.2,

$$V(\text{path}(P, u^{(1)}, u^{(i+1)})) \cap V(\text{path}(P', v^{(1)}, v^{(j+1)})) = \emptyset. \quad (6.2)$$

显然, $m_{i+1} > r$ 。根据引理 4.2, $V(\text{path}(P, u^{(i+2)}, u^{(k)}))$ 中每个顶点的第 m_{i+1} 位不同于 $V(\text{path}(P', v^{(1)}, v^{(j+1)}))$ 中每个顶点的对应位。因此,

$$V(\text{path}(P, u^{(i+2)}, y)) \cap V(\text{path}(P', v^{(1)}, v^{(j+1)})) = \emptyset. \quad (6.3)$$

类似地,

$$V(\text{path}(P, u^{(1)}, u^{(i+1)})) \cap V(\text{path}(P', v^{(j+2)}, y)) = \emptyset. \quad (6.4)$$

根据等式(6.1), (6.2), (6.3)和(6.4), 我们有 $\text{ancestor}(y, P) \cap \text{ancestor}(y, P') = \{u^{(0)}\}$ 。

情形3.1.2. $m_{i+1} < m_{j+1}'$ 。我们有如下子情形:

情形3.1.2.1. $\{m_{j+1}', m_{j+2}', \dots, m_{b-1}'\} = \{m_{b-1}'\} = \{r\}$ 。根据引理 4.2, 路径 $V(\text{path}(P, u^{(1)}, y))$ 中每个顶点的第 r 位不同于路径 $V(\text{path}(P', v^{(1)}, v^{(b-1)}))$ 中每个顶点的第 r 位, 这意味着 $V(\text{path}(P, u^{(1)}, y)) \cap V(\text{path}(P', v^{(1)}, v^{(b-1)})) = \emptyset$ 。

因此, $\text{ancestor}(y, P) \cap \text{ancestor}(y, P') = \{u^{(0)}\}$ 。

情形3.1.2.2. $\{m_{j+1}', m_{j+2}', \dots, m_{b-1}'\} \supset \{r\}$ 。根据假设, $m_{j+1}' > r$ 。根据引理 4.2, P 中 y 的第 m_{j+1}' 位不同于 P' 中 y 的第 m_{j+1}' 位, 产生矛盾。

情形3.1.3. $m_{i+1} > m_{j+1}' \geq r$ 。根据引理 4.2, P 中 y 的第 m_{j+1}' 位不同于 P' 中 y 的第 m_{j+1}' 位, 产生矛盾。

情形3.2. $r \in A$ 且 $r \notin B$ 。那么, $q \leq m_{a-1} \leq r$ 且 $r \in A_2$ 。我们有如下子情形:

情形3.2.1. $m_{i+1} = m_{j+1}' > r$ 。我们能够验证 $m_{b-1}' > r$ 。因此, $m_{a-1} \neq m_{b-1}'$ 。证明类似于情形3.1.1。

情形3.2.2. $m_{i+1} \neq m_{j+1}'$ 。那么, 令 $l = \max\{m_{i+1}, m_{j+1}'\}$ 。进一步, 我们有如下情形:

情形3.2.2.1. $l > r$ 。那么, 我们有 $m_{b-1}' > r$ 。因此, $m_{a-1} \neq m_{b-1}'$ 。

情形3.2.2.2. $l = r$ 。那么, 我们有 $l = r$, $m_{i+1} = r$ 和 $m_{b-1}' < r$ 。因此, $m_{a-1} \neq m_{b-1}'$ 。

类似于情形3.1.1, 我们能够证明情形3.2.2.1 和情形3.2.2.2。

综上所述, 引理成立。

根据上述结果, 我们有如下定理, 其证明了莫比乌斯立方体上IST的正确性。

定理 6.8. 对于任意整数 $n \geq 2$ 和 M_n 上的任一顶点 x , 算法PMCIST构造的 $T_0, T_1, \dots, T_{n-1}$ 是 M_n 上 n 棵以顶点 x 为根的IST。

证明. 根据定理 6.6, 对于任意整数 i , $0 \leq i \leq n-1$, T_i 是 M_n 上一棵以顶点 x 为根的生长树。现在我们证明算法PMCIST构造的 $T_0, T_1, \dots, T_{n-1}$ 相互独立。对于任意两个整数 i 和 j , $0 \leq i < j \leq n-1$ 和任一顶点 $y \in V(M_n) \setminus \{x\}$, 令 P 表示 T_i 中的路径 $x, u^{(1)}, u^{(2)}, \dots, y$ 和 P' 表示 T_j 中的路径 $x, v^{(1)}, v^{(2)}, \dots, y$ 。我们可以验证 P 和 P' 满足引理 6.7 中的条件。因此, $\text{ancestor}(y, T_i) \cap \text{ancestor}(y, T_j) = \{x\}$ 和 $\text{ancestor}(y, T_i) \cup \text{ancestor}(y, T_j) \supset \{x\}$ 。根据引理 3.9, T_i 和 T_j 相互独立。

综上所述, 算法PMCIST构造的 $T_0, T_1, \dots, T_{n-1}$ 是 M_n 上 n 棵以顶点 x 为根的IST。

根据算法PMCIST, 引理 6.5 和定理 6.8, 我们容易验证如下定理。

定理 6.9. 算法PMCIST的时间复杂度为 $O(N)$, 这里 $N = 2^n$ 且 $n \geq 2$ 。

证明. 根据定理 6.8, 对于任意整数 $n \geq 2$ 和 M_n 上的任一顶点 x , 算法PMCIST构造的 $T_0, T_1, \dots, T_{n-1}$ 是 M_n 上 n 棵以顶点 x 为根的IST。根据引理 6.5, $|V(T_i)| = 2^n$, 这里 i , $0 \leq i \leq n-1$, T_i 是由算法PMCIST构造的。根据算法PMCIST, 所有IST都是并行构造的。因此, 定理成立。

6.3.3 基于圆排列的IST的构造算法

基于算法PMCIST, 我们设计一个并行算法, 称为OPMCIST。算法OPMCIST的第2行和第7行是控制语句。根据定理 6.9和算法OPMCIST, 我们容易得到如下定理。

算法 12 OPMCIST

输入: 整数 $n \geq 2$, 根顶点 x , 整型数组 $S = \{n-1, n-2, \dots, 0\}$;

输出: M_n 上以顶点 x 为根的 n 棵树 $T_0, T_1, \dots, T_{n-1}$;

开始

1: **for** $i = 0$ to $n-1$ **do**

2: 并行模式

3: $V(T_i) = N(x, i)$; /*初始时, $V(T_i)$ 中仅有 x 的 i 维邻接点一个顶点*/

4: $E(T_i) = \emptyset$;

5: **for** level= 0 to $n-1$ **do**

6: **for** 任一顶点 $x \in V(T_i)$ **do**

7: 并行模式

8: $y = N(x, S[(\text{level} - i + n) \bmod n])$;

9: $E(T_i) = E(T_i) \cup \{(x, y)\}$;

10: $V(T_i) = V(T_i) \cup \{y\}$; /*这里, y 基于递减圆排列得到。*/

11: **end for**

12: **end for**

13: **end for**

结束

定理 6.10. 算法OPMCIST在 $O(n)$ 时间内构造出 M_n 上以 x 为根的 n 棵IST, 这里 $n \geq 2$ 。

6.4 基于任一圆排列构造以任一顶点为根的IST

6.4.1 构造算法

首先, 我们修改算法PMCIST, 给出如下算法:

算法 13 BCIST

输入: 数组 $S = \{a_0, a_1, \dots, a_{n-1}\}$, 这里 $a_0, a_1, \dots, a_{n-1}$ 是一个CDP; X_n 中的任一顶点 u , 这里 $X_n \in \{Q_n, M_n\}$;

输出: X_n 上以 u 为根的 $T_0, T_1, \dots, T_{n-1}$;

开始

1: **for** $i = 0$ to $n - 1$ **do**

2: 并行模式

3: $V(T_i) = N(u, a_i)$; /* $N(u, a_i)$ 表示 u 的 a_i 维邻接点。 */

4: $E(T_i) = \emptyset$;

5: **for** $l = 0$ to $n - 1$ **do**

6: **for** 任一顶点 $v \in V(T_i)$ **do**

7: 并行模式

8: $d = S[(i + l + 1) \bmod n]$;

 /* S 中元素的下标从 0 到 $n - 1$ 计数, 这里 n 是数组 S 的长度。 */

9: $E(T_i) = E(T_i) \cup \{(v, N(v, d))\}$;

10: $V(T_i) = V(T_i) \cup \{N(v, d)\}$;

11: **end for**

12: **end for**

13: **end for**

结束

为便于证明, 我们令向量 $\langle \beta_1, \beta_2, \dots, \beta_n \rangle$, 使得集合 $\{\beta_1, \beta_2, \dots, \beta_n\}$ 等于集合 $\{0, 1, \dots, n - 1\}$ 。

引理 6.11. 假设 $P: u^{(0)}, u^{(1)} = N(u^{(0)}, a_1), u^{(2)} = N(u^{(1)}, a_2), \dots, u^{(k)} = N(u^{(k-1)}, a_k)$ 和 $P': v^{(0)}, v^{(1)} = N(v^{(0)}, a_1'), v^{(2)} = N(v^{(1)}, a_2'), \dots, v^{(t)} = N(v^{(t-1)}, a_t')$ 是 $X_n \in \{Q_n, M_n\}$ 上的两条路径, 对于任意两个整数 k, t , $1 \leq k, t \leq n$ 且 $a_1 \neq a_1'$ 。如果如下条件成立:

(1) $\langle \beta_{i_1}, \beta_{i_2}, \dots, \beta_{i_k} \rangle = \langle a_1, a_2, \dots, a_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$, 且

(2) $\langle \beta_{j_1}, \beta_{j_2}, \dots, \beta_{j_t} \rangle = \langle a_1', a_2', \dots, a_t' \rangle$, 这里 $1 \leq j_1 < j_2 < \dots < j_t \leq n$,

那么 $V(\text{path}(P, u^{(1)}, u^{(k)})) \cap V(\text{path}(P', v^{(1)}, v^{(t)})) = \emptyset$ 。

证明. 假设存在一个顶点 v , 使得 $v \in V(P) \cap V(P')$ 。假设 $v = u^{(i)}$ 和 $v = v^{(j)}$, $1 \leq i \leq k$, $1 \leq j \leq t$ 。令 $A = \{a_1, a_2, \dots, a_i\}$ 和 $B = \{a_1', a_2', \dots, a_j'\}$ 。我们有 $M_1 = \max((A \cup B) \setminus (A \cap B))$ 。

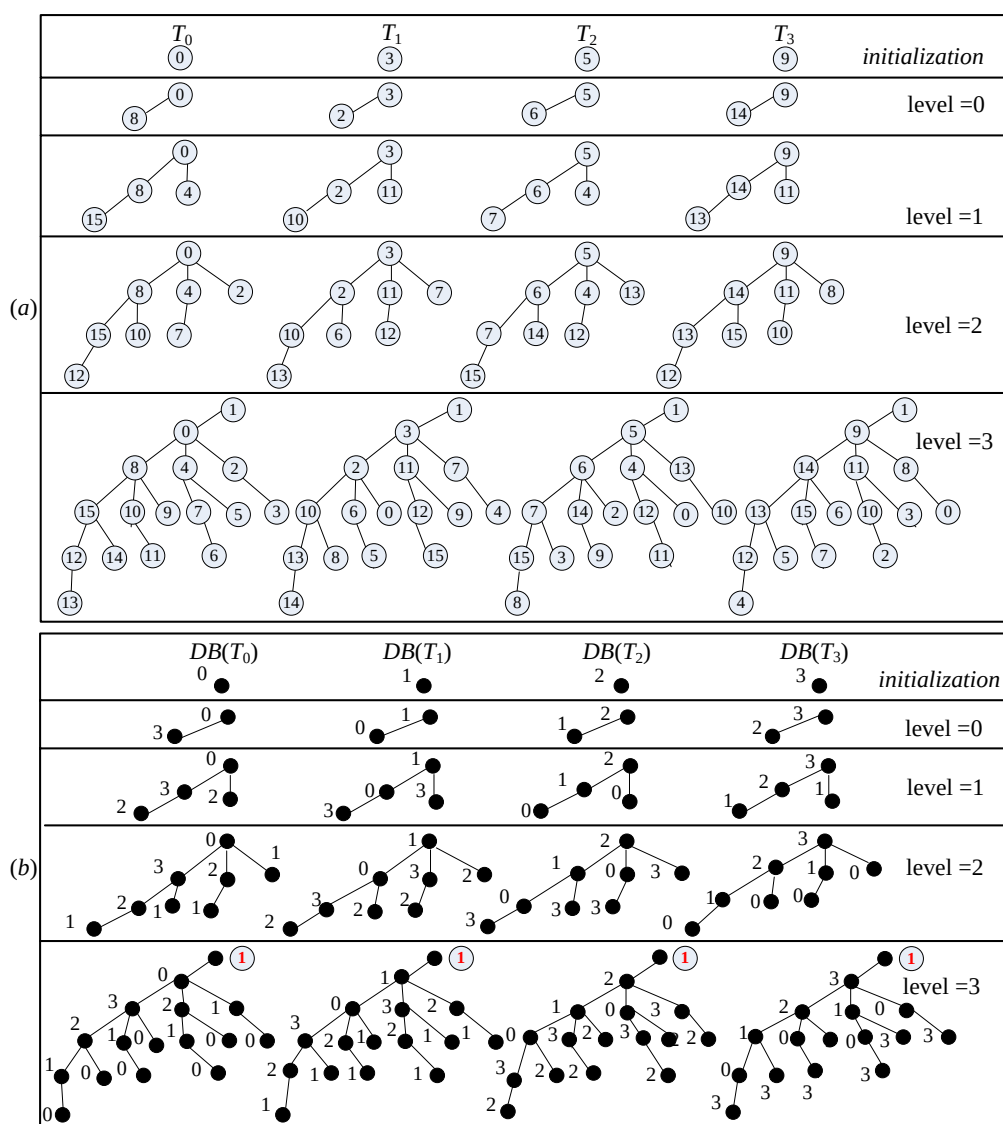
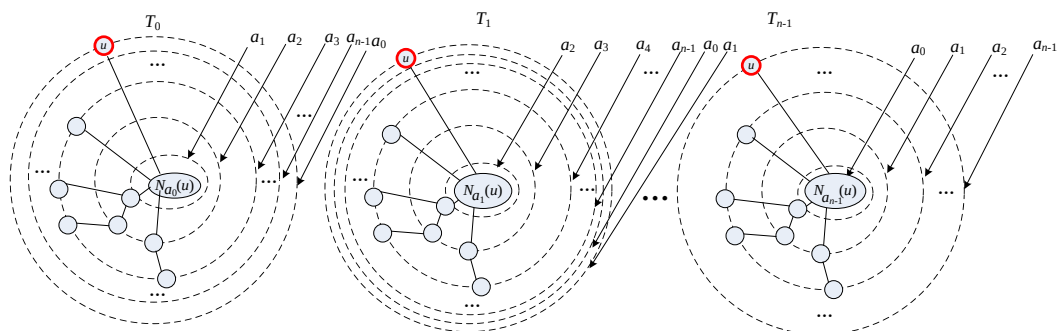
图 6.3 基于维邻接关系树构造 $0-M_4$ 以1为根的IST

图 6.4 基于算法BCIST的构造过程

$B))$ 。根据引理 4.3, 我们能够验证 P 中 v 的第 M_1 位不同于 P' 中 v 的对应位, 这是一个矛盾。

基于引理 6.11, 我们得到如下推论。

推论 6.12. 假设 $P: u^{(0)}, u^{(1)} = N(u^{(0)}, a_1), u^{(2)} = N(u^{(1)}, a_2), \dots, u^{(k)} = N(u^{(k-1)}, a_k) = v$ 和 $P': v^{(0)}, v^{(1)} = N(u^{(0)}, a_1'), v^{(2)} = N(v^{(1)}, a_2'), \dots, v^{(t)} = N(v^{(t-1)}, a_t') = v$ 是 $X_n \in \{Q_n, M_n\}$ 上的两条路径, 对于任意两个整数 $k, t, 1 \leq k, t \leq n$ 和 $a_k \neq a_t'$ 。如果如下条件成立:

(1) $\langle \beta_{i_1}, \beta_{i_2}, \dots, \beta_{i_k} \rangle = \langle a_1, a_2, \dots, a_k \rangle$, 这里 $1 \leq i_1 < i_2 < \dots < i_k \leq n$, 且

(2) $\langle \beta_{j_1}, \beta_{j_2}, \dots, \beta_{j_t} \rangle = \langle a_1', a_2', \dots, a_t' \rangle$, 这里 $1 \leq j_1 < j_2 < \dots < j_t \leq n$,

那么 $V(\text{path}(P, u^{(0)}, u^{(k)})) \cap V(\text{path}(P', v^{(0)}, v^{(t)})) = \{v\}$ 。

$T_0, T_1, \dots, T_{n-1}$ 的构造过程与前面的构造过程类似。以 T_0 为例, 构造过程描述如下, 如图 6.4 所示。

首先, 树 T_0 中仅有一个顶点 $N(u, a_0)$; 经过第一轮迭代 ($l=0$), 树 T_0 中的顶点 $N(u, a_0)$ 与其 a_1 维邻接点 $N(N(u, a_0), a_1)$ 相连。因此, $V(T_0) = \{N(u, a_0), N(N(u, a_0), a_1)\}$ 且 $E(T_0) = \{(N(u, a_0), N(N(u, a_0), a_1))\}$; 经过第二轮迭代 ($l=1$), T_0 中的任一顶点 v 与其 a_2 维邻接点 $N(v, a_2)$ 相连。因此, 边 $(N(N(u, a_0), a_1), N(N(N(u, a_0), a_1), a_2))$ 和 $(N(u, a_0), N(N(u, a_0), a_2))$ 被增加进 T_0 中且 $V(T_0) = \{N(u, a_0), N(N(u, a_0), a_2), N(N(u, a_0), a_1), N(N(N(u, a_0), a_1), a_2))\}$; 经过第三轮迭代, T_0 中的任一顶点 v 与其 a_3 维邻接点 $N(v, a_3)$ 相连。因此, T_0 中顶点数量增了一倍; 经过 l -轮迭代, T_0 中的任一顶点 v 与其 $S[l+1]$ 维邻接点 $N(v, S[l+1])$ 相连, 这里 $4 \leq l \leq n-2$; 在最后一轮迭代中, T_0 中的任一顶点 v 与其 a_0 维邻接点 $N(v, a_0)$ 相连。

引理 6.13. 令算法 BCIST 的输入 $a_0, a_1, \dots, a_{n-1}$ 是整数 $0, 1, \dots, n-1$ 的任一圆排列, $X_n \in \{Q_n, M_n\}$ 。算法 BCIST 构造的 T_i 是 X_n 上的一棵生成树, 这里 $i = 0, 1, \dots, n-1$ 。

证明. 不失一般性, 我们考虑算法 BCIST 构造的 T_0 。经过 n 轮迭代, 我们在树 T_0 中有 $1 + 2^0 + 2^1 + \dots + 2^{n-1} = 2^n$ 个顶点。从树 T_0 中选取任意两个顶点 $v^{(1)}$ 和 $v^{(2)}$, $\langle N(u, a_0), v^{(1)} \rangle$ -路径能够被表示成 $N(u, a_0), x^{(1)} = N(N(u, a_0), a_{i_1}), x^{(2)} = N(x^{(1)}, a_{i_2}), \dots, x^{(k)} = N(x^{(k-1)}, a_{i_k})$ 且 $\langle N(u, a_0), v^{(2)} \rangle$ -路径能够被表示成 $N(u, a_0), x^{(1)} = N(N(u, a_0), a_{j_1}), x^{(2)} = N(x^{(1)}, a_{j_2}), \dots, x^{(m)} = N(x^{(m-1)}, a_{j_m})$, 这里 $0 \leq i_1 < i_2 < \dots < i_k \leq n-1$ 和 $0 \leq j_1 < j_2 < \dots < j_m \leq n-1$ 。根据算法 BCIST, 容易验证 $\langle N(u, a_0), v^{(1)} \rangle$ -路径和 $\langle N(u, a_0), v^{(2)} \rangle$ -路径满足引理 6.11 中的条件, 这这意味着 $v^{(1)} \neq v^{(2)}$ 。

因此, T_0 是 $X_n \in \{Q_n, M_n\}$ 上以 $N(u, a_0)$ 为根的一棵生成树。注意到, 在树 T_0 中 u 既是 $N(u, a_0)$ 的孩子顶点, 它也是叶子顶点。因此, T_0 是以 u 为根的一棵生成树。

引理 6.14. 算法 BCIST 构造的 $T_0, T_1, \dots, T_{n-1}$ 是 $X_n \in \{Q_n, M_n\}$ 上的 n 棵 IST。

证明. 我们有如下两种情形:

情形1. X_n 是 M_n 。根据引理 6.13, 算法 BCIST 构造的 T_i 是 $V(M_n)$ 上的一棵生成树, 对于整数 $i = 0, 1, \dots, n-1$ 。我们只需证明 T_i 和 T_j 相互独立, 对于 $0 \leq i \leq j \leq n-1$ 。

当 $n = 1$ 和 $n = 2$ 时显然成立。我们考虑 $n \geq 3$ 。树 T_i 中的最长路径 P_1 和树 T_j 中的路径 P_2 可以被分别表示为

$P_1: u, x_0 = N(u, a_i), x_1 = N(x_0, a_{i+1}), \dots, x_{n-i-1} = N(x_{n-i-2}, a_{n-1}), x_{n-i} = N(x_{n-i-1}, a_0),$
 $x_{n-i+1} = N(x_{n-i}, a_1), \dots, x_n = N(x_{n-1}, a_i)$ 和

$P_2: u, y_0 = N(u, a_j), y_1 = N(y_0, a_{j+1}), \dots, y_{n-j-1} = N(y_{n-j-2}, a_{n-1}), y_{n-j} = N(y_{n-j-1}, a_0),$
 $y_{n-j+1} = N(y_{n-j}, a_1), \dots, y_n = N(y_{n-1}, a_j)$, 这里 $0 \leq i < j \leq n-1$ 。

令 $a_{c_0} = a_i$ 且 $a_{d_0} = a_j$ 。根据引理 3.9, 我们只需证明对于任一顶点 $v \in V(T_i) \cap V(T_j)$, $\text{ancestor}(v, T_i) \cap \text{ancestor}(v, T_j) = \{u\}$ 和 $\text{ancestor}(v, T_i) \cup \text{ancestor}(v, T_j) \supset \{u\}$ 。 T_i 中的任一路径和 T_j 中的任一路径可以被分别表示为

$P_3: u, x_0 = N(u, a_{c_0} = a_i), x_1' = N(x_0, a_{c_1}), \dots, x_k' = N(x_{k-1}', a_{c_k}), x_{k+1}' = N(x_k', a_{c_0})$
 和

$P_4: u, y_0 = N(u, a_{d_0} = a_j), y_1' = N(y_0, a_{d_1}), \dots, y_l' = N(y_{l-1}', a_{d_l}), y_{l+1}' = N(y_l', a_{d_0})$ 。

不失一般性, 假设 $a_j > a_i$ 和 $v \in V(P_3) \cap V(P_4)$ 。令 $v = a_{c_w} = a_{d_z}$, 这里 $1 \leq u \leq k$, $1 \leq z \leq m$ 。基于 P_1, P_2, P_3 和 P_4 , 我们定义途径 W_1, W_2, W_3 和 W_4 如下:

$W_1: a_i, a_{i+1}, \dots, a_{n-1}, a_0, a_1, \dots, a_i,$

$W_2: a_j, a_{j+1}, \dots, a_{n-1}, a_0, a_1, \dots, a_j,$

$W_3: a_{c_0} = a_i, a_{c_1}, \dots, a_{c_w}$, 和

$W_4: a_{d_0} = a_j, a_{d_1}, \dots, a_{d_z}$ 。

既然 $a_{c_0} \neq a_{d_0}$, 我们有 $x_0 \neq y_0$, 这意味着 $\text{ancestor}(v, P_3) \cup \text{ancestor}(v, P_4) \supset \{u\}$ 。我们只需证明 $\text{ancestor}(v, P_3) \cap \text{ancestor}(v, P_4) = \{u\}$ 。我们有如下子情形:

情形1.1. $\max(V(W_3)) \neq \max(V(W_4))$ 。既然 $\max(V(W_4)) \geq a_j$, 那么我们有如下子情形:

情形1.1.1. $\max(V(W_3)) < a_j$ 且 $\max(V(W_4)) = a_j$ 。那么, $V(P_4) \setminus \{y_{m+1}'\}$ 中的任一顶点和 $V(P_3)$ 中的任一顶点具有一个最左不同位 a_j 。那么, 我们有 $v = y_{m+1}'$ 和 $\text{ancestor}(v, P_3) \cap \text{ancestor}(v, P_4) = \{u\}$ 。

情形1.1.2. $\max(V(W_3 \cup W_4)) > a_j$ 。那么, P_3 中 v 的第 $\max(V(W_3 \cup W_4))$ 位不同于 P_4 中 v 的对应位。这是一个矛盾。

情形1.2. $\max(V(W_3)) = \max(V(W_4))$ 。那么, 我们有如下子情形:

情形1.2.1. $\max(V(W_3)) = \max(V(W_4)) = a_j$ 。容易验证, $v \neq y_{m+1}'$ 和 $a_j \notin \{a_{d_1}, a_{d_2}, \dots, a_{d_m}\}$ 。我们可以将路径 $\text{ancestor}(v, P_3)$ 划分为 P_{31} 和 P_{32} 。

$P_{31} : u, x_0 = N(u, a_{c_0}), x_1' = N(x_0, a_{c_1}), \dots, x_{f-1}' = N(x_{f-2}', a_{c_{f-1}})$ 和

$P_{32} : x_f' = N(x_{f-1}', a_{c_f} = a_j), x_{f+1}' = N(x_f', a_{c_{f+1}}), \dots, x_w' = N(x_{u-1}', a_{c_w})$ 。

根据引理 4.3, $V(P_{31})$ 中的任一顶点和 $V(P_4)$ 中的任一顶点具有一个最左不同位 a_j 。那么, $V(P_4) \cap V(P_{31}) = \emptyset$, 即 $\text{LDF}(x_f', y_0) = \max(\{a_{c_0} = a_i, a_{c_1}, \dots, a_{c_{f-1}}\})$ 。令 $A = \{a_{c_{f+1}}, a_{c_{f+2}}, \dots, a_{c_w}\}$ 和 $B = \{a_{d_1}, a_{d_2}, \dots, a_{d_z}\}$ 。进一步, 我们有如下子情形:

情形1.2.1.1. $\max((A \cup B) \setminus (A \cap B)) > \text{LDF}(x_f', y_0)$ 。那么, 根据引理 4.3, P_3 中 v 的第 $\max((A \cup B) \setminus (A \cap B))$ 位不同于 P_4 中 v 的对应位, 这是一个矛盾。

情形1.2.1.2. $\max((A \cup B) \setminus (A \cap B)) = \text{LDF}(x_f', y_0)$ 。根据算法 BCIST, T_i 和 T_j 是基于相同的 CDP 所得到的。那么, 我们容易验证 $a_{c_w} \neq a_{d_z}$ 。根据推论 6.12, $V(P_4) \cap V(P_{32}) = \{v\}$ 。

情形1.2.1.3. $\max((A \cup B) \setminus (A \cap B)) < \text{LDF}(x_f', y_0)$ 。那么, 根据引理 4.3, P_3 中 v 的第 $\text{LDF}(x_f', y_0)$ 位不同于 P_4 中 v 的对应位, 这是一个矛盾。

情形1.2.2. $\max(V(W_3)) = \max(V(W_4)) = M_1 > a_j$ 。我们将 $\langle u, v \rangle$ -路径 P_3 分割成 P_{31} 和 P_{32} 。

$P_{31} : u, x_0 = N(u, a_{c_0}), x_1' = N(x_0, a_{c_1}), \dots, x_{f-1}' = N(x_{f-2}', a_{c_{f-1}})$ 和

$P_{32} : x_f' = N(x_{f-1}', a_{c_f=M_1}), x_{f+1}' = N(x_f', a_{c_{f+1}}), \dots, x_w' = N(x_{u-1}', a_{c_w})$ 。

我们将 $\langle u, v \rangle$ -路径 P_4 划分为 P_{41} 和 P_{42} 。

$P_{41} : u, y_j = N(u, a_{d_0} = a_j), y_1' = N(y_i, a_{d_1}), \dots, y_{h-1}' = N(y_{h-2}', a_{d_{h-1}})$, 和

$P_{42} : y_h' = N(y_{h-1}', a_{d_h}), y_{h+1}' = N(y_h', a_{d_{h+1}}), \dots, y_z' = N(y_{z-1}', a_z)$,

根据引理 4.3, $V(P_{31}) \cap V(P_{41}) = \{u\}$ 。显然, $V(P_{42})$ 中任一顶点的第 M_1 位不同于 $V(P_{31})$ 任一顶点的对应位, 因此 $V(P_{42}) \cap V(P_{31}) = \emptyset$ 。类似地, 我们有 $V(P_{32}) \cap$

$V(P_{41}) = \emptyset$ 。我们只需证明 $V(P_{32}) \cap V(P_{42}) = \emptyset$ 。类似于情形1.2.1, 我们可以验证 $a_{c_w} \neq b_{d_c}$ 。根据推论 6.12, $V(P_{32}) \cap V(P_{42}) = \{v\}$ 。

因此, 我们有 $\text{ancestor}(v, P_3) \cap \text{ancestor}(v, P_4) = \{u\}$ 。

情形2. X_n 是 Q_n 。证明类似于情形1。

基于上述讨论, 根据引理 3.9, 引理成立。

既然对于 $0, 1, \dots, n-1$, 有 $(n-1)!$ 种 CDP, 基于引理 6.14, 我们有如下定理。

定理 6.15. 基于算法 BCIST, $X_n \in \{Q_n, M_n\}$ 上有 $(n-1)!$ 组以任一顶点为根的 IST。

6.4.2 基于IST的不相交路径

基于不同的 CDP, 我们能够得到多种 IST 组合。因此, 在任意两个顶点之间存在多种不相交路径组合。图 6.5(a)–(f) 示出了 $0-M_4$ 上以 1 为根的分别基于 6 种 CDP 的 IST 组合。显然, $0-M_4$ 上顶点 1 和 $v \in V(0-M_4) \setminus \{1\}$ 中的任一顶点之间有 6 种不相交路径组合, 这几种 4 条不相交路径的长度之和可能不一样。例如, 图 6.5(a) 示出了顶点 1 和 13 之间的 4 条路径

$1 \rightarrow 0 \rightarrow 8 \rightarrow 15 \rightarrow 12 \rightarrow 13,$

$1 \rightarrow 3 \rightarrow 2 \rightarrow 10 \rightarrow 13,$

$1 \rightarrow 5 \rightarrow 13,$ 和

$1 \rightarrow 9 \rightarrow 14 \rightarrow 13。$

图 6.5(c) 示出了顶点 1 和 13 之间的另外 4 条路径

$1 \rightarrow 0 \rightarrow 2 \rightarrow 10 \rightarrow 13,$

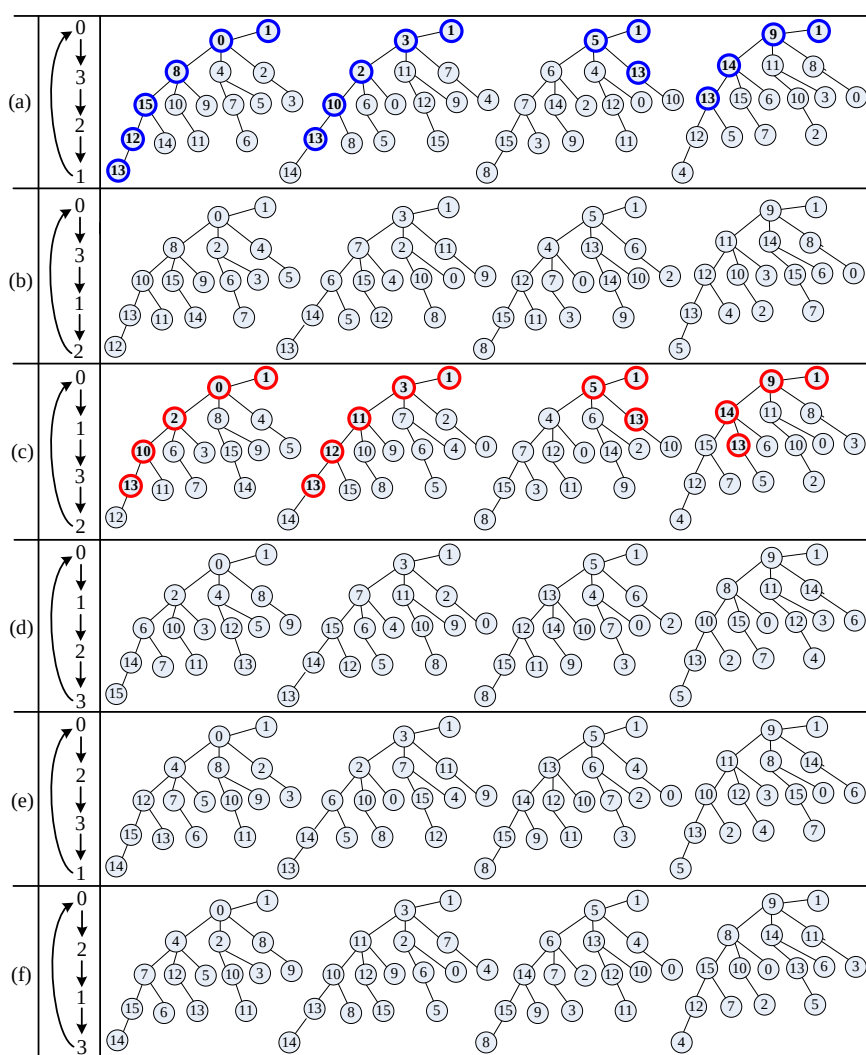
$1 \rightarrow 3 \rightarrow 11 \rightarrow 12 \rightarrow 13,$

$1 \rightarrow 5 \rightarrow 13,$ 和

$1 \rightarrow 9 \rightarrow 14 \rightarrow 13。$

图 6.5(c) 所示的顶点 1 和 13 之间的 4 条不相交路径长度之和小于图 6.5(a) 中相应顶点之间的长度之和。因此, 基于不同的 CDP, 对于任意两个顶点来说, 我们可能会得到其间性能较好的 n 条不相交路径。

进一步, 表 6.1 分别给出了对应于图 6.5(a) ~ (f) 的起点 1 到终点 $0, 2, 3, \dots, 15$ 的路径总长度。其中,

图 6.5 以1为根的IST的例子及 $0-M_4$ 上顶点1和13之间的不相交路径

- (1) 起点1—终点0、起点1—终点2、起点1—终点3、起点1—终点8、起点1—终点9、起点1—终点10、起点1—终点11的路径总长度没有变化。
- (2) 起点1—终点4的路径总长度最短为11，最长为12。
- (3) 起点1—终点5的路径总长度最短为12，最长为13。
- (4) 起点1—终点6的路径总长度最短为11，最长为12。
- (5) 起点1—终点7的路径总长度最短为12，最长为13。
- (6) 起点1—终点12的路径总长度最短为13，最长为15。
- (7) 起点1—终点13的路径总长度最短为13，最长为15。
- (8) 起点1—终点14的路径总长度最短为13，最长为15。

(9) 起点1—终点15的路径总长度最短为13, 最长为15。

表 6.1 对应于图 6.5(a) ~ (f)的起点1到终点0, 2, 3, ..., 15的路径总长度

	圆排列(a) 0321	(b) 0312	(c) 0132	(d) 0123	(e) 0231	(f) 0213
<1—0>-路径	10	10	10	10	10	10
<1—2>-路径	11	11	11	11	11	11
<1—3>-路径	11	11	11	11	11	11
<1—4>-路径	12	11	12	11	11	12
<1—5>-路径	12	13	12	13	13	12
<1—6>-路径	12	11	11	11	12	12
<1—7>-路径	12	13	13	13	12	12
<1—8>-路径	13	13	13	13	13	13
<1—9>-路径	11	11	11	11	11	11
<1—10>-路径	12	12	12	12	12	12
<1—11>-路径	12	12	12	12	12	12
<1—12>-路径	14	15	15	13	13	13
<1—13>-路径	14	15	13	15	15	13
<1—14>-路径	14	13	14	13	14	15
<1—15>-路径	14	13	14	15	14	15

6.5 IST在诊断上的应用

随着科学技术的快速发展, 采用大量处理器的并行计算系统层出不穷。为了维护这类系统的可靠性, 系统中故障处理器需要能够及时被替换成无故障的处理器。在替换之前, 故障处理器必须首先被诊断出来。故障处理器的鉴别 (Identify) 过程称为系统的诊断 (Diagnosis)。

众所周知, 系统级诊断是一种重要的诊断方法。一个称为PMC的诊断模型, 首先由Preparata等人 [129]提出, 接着基于该模型产生了大量研究成果 [47, 130–134]。在PMC诊断模型中, 两个相邻处理器互相测试。当一个处理器测试另外一个处理器, 若测试者是无故障的, 则结果是正确的, 但若测试者是有故障的, 则结果是不可靠的。一个系统是 t -诊断的当系统中所有故障处理器能够被鉴别, 假定故障处理器的个数不超过 t 的情况。文献 [135]给出了一个充分必要条件来验证一个系统在PMC模型下是 t -诊断的。文献 [136]提出了一个一般的时间复杂度为 $O(N^{2.5})$ 的诊断算法来鉴别

具有 N 个处理器的网络中的所有故障处理器。文献 [47]给出了一个PMC模型下的高效诊断算法, 其能够在 $O(N)$ 时间内鉴别 M_n 上的所有故障处理器, 当故障处理器个数不超过 n 的情况, 这里 $N = 2^n$ 且 $n \geq 4$ 。

在本节, 我们将给出 M_n 上基于IST的一个具有更高效率且简易的并行算法, 这里假定IST的根处理器无故障且 M_n 中故障处理器的个数不超过 n (既然在PMC模型下 M_n 是 n -诊断的 [137], 该假设是可靠的)。

我们称一个系统中故障处理器的集合为故障集合 (Faulty Set)。令 $\gamma(v, w)$ 表示 v 测试 w 的结果, F 为所有故障处理器的集合。对于任一 $(v, w) \in E(M_n)$ 和 $v \in V(M_n) \setminus F$, 若 $w \in V(M_n) \setminus F$, 则 $\gamma(v, w) = 0$; 否则 $\gamma(v, w) = 1$ 。

基于算法OPMCIST, 我们给出一个诊断算法, 称为DiagnosisIST。主要有如下两个变化:

- 1、我们删除了输入 $S = \{n-1, n-2, \dots, 0\}$ 和使用输出集合 F 来包含所有故障处理器。
- 2、我们将语句“ $y = N(x, S[(\text{level} - i + n) \bmod n])$ ”替换成“ $y = N(x, (i - \text{level} + n - 1) \bmod n)$ ”。

例如, 在图 6.6中, 假设故障处理器集合是 $\{4, 6, 9, 12\} \subset V(0-M_4)$ 。基于算法DiagnosisIST, 诊断过程描述为:

首先, 故障处理器9在树 T_3 中被诊断出来;

经过第一轮迭代, ($\text{level} = 0$), 故障处理器6在树 T_2 中被诊断出来;

经过第二轮迭代, ($\text{level} = 1$), 故障处理器4分别在 T_0 和 T_2 中被诊断出来;

经过第三轮迭代($\text{level} = 2$), 故障处理器6也能在树 T_1 中被诊断出来, 故障处理器12 分别在树 T_0 中 T_1 被诊断出来。此时, 集合 $\{4, 6, 9, 12\}$ 中的所有故障处理器皆被诊断出来。

在最后一轮迭代中, 故障处理器9也能分别在树 T_0 和 T_1 中被诊断出来。故障处理器4也能在树 T_1 中被诊断出来。

需要指出的是, 算法DiagnosisIST中的测试者永远是无故障的。基于算法OPMCIST, 在根顶点和莫比乌斯立方体上的任一其它顶点之间存在 n 条顶点不相交路径, 我们有如下定理。

定理 6.16. 假设故障处理器数量不超过 n 和根处理器是无故障的。那么, 基于算法DiagnosisIST, 所有的故障处理器能够在不超过 $n + 1$ 步的情况下被诊断出来。

算法 14 DiagnosisIST

输入: 根顶点 x (无故障) 和整数 n ;

输出: 被诊断为故障的处理器器的集合 F ;

开始

```

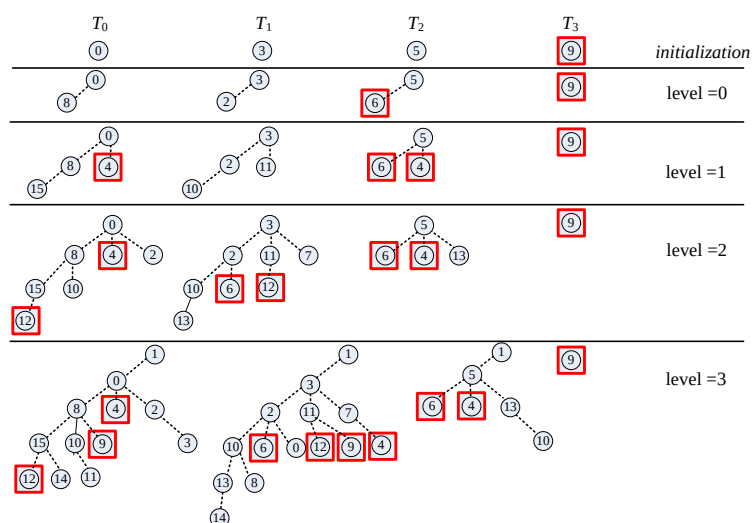
1:  $F = \emptyset$ ; /*第4到第9行表表明诊断步骤的第一步。*/
2: for  $i = 0$  to  $n - 1$  do
3:   并行模式
4:   if  $(\gamma(x, N(x, i)) = 0)$  then
5:      $V(T_i) = N(x, i)$ ; /*处理器 $N(x, i)$ 是无故障的。*/
6:   else
7:      $F = F \cup \{N(x, i)\}$ ;
8:   返回;
9:   end if
10:  for level= 0 to  $n - 1$  do
11:    for 任一处理器 $x \in V(T_i)$  do
12:      并行模式
13:       $y = N(x, (i - \text{level} + n - 1) \bmod n)$ ;
14:      if  $(\gamma(x, y) = 0)$  then
15:         $V(T_i) = V(T_i) \cup \{y\}$ ; /*处理器 $y$ 是无故障的。*/
16:      else
17:         $F = F \cup \{y\}$ ;
18:      返回;
19:      end if
20:    end for
21:  end for
22: end for

```

结束

证明. 假设输出集合为 $F = \emptyset$, 我们用算法DiagnosisIST中的测试关系作为树中的边。我们能够得到 M_n 上以顶点 x 为根的树 $T_0, T_1, \dots, T_{n-1}$, 与算法OPMCIST得到的树一样。根据定理 6.10, $T_0, T_1, \dots, T_{n-1}$ 是 M_n 上以顶点 x 为根的 n 棵IST, 这能够在 $O(n)$ 时间内得到。显然, 对于任一定点 $y \in V(M_n) \setminus \{x\}$, 有 n 条顶点不相交的 $\langle x, y \rangle$ -路径。

假设集合 $F \subset V(M_n)$ 中有 n 个故障处理器。对于任一顶点 $y \in F$, 有 n 条顶点不相交的 $\langle x, y \rangle$ -路径。然而, 在这 n 条顶点不相交的 $\langle x, y \rangle$ -路径中, 至多有 $|F \setminus \{x\}| = n - 1$ 故障处理器。那么, 存在一条 $\langle x, y \rangle$ -路径 P , 使得 $V(P) \setminus \{y\}$ 所有的顶点是无故障的。

图 6.6 $0-M_4$ 上的故障顶点诊断

因此，根据算法DiagnosisIST，故障顶点 y 能够在路径 P 中被准确地诊断出来。因此，定理成立。

如前所述，文献 [136]和 [47]中算法的时间复杂度分别为 $O(N^{2.5})$ 和 $O(N)$ ，这里 $N = 2^n$ 和 $n \geq 4$ 。根据定理 6.16，我们能够验证算法DiagnosisIST的时间复杂度为 $O(n)$ ，低于文献 [136]和 [47]中算法的时间复杂度。当然，前者是并行算法而后者是串行算法。

6.6 IST的模拟实验和IST在诊断上的应用

在本节，我们利用Java Universal Network/Graph Framework (JUNG)技术（结合多线程技术）来模拟莫比乌斯立方体上IST的构造及IST在诊断上的应用。JUNG是一个软件库，其提供了一个通用且可扩展的语言来对可以表示为一个图形或网络的数据进行建模、实现数据的可视化。基于JUNG的应用程序被允许构建Java API中的可扩展的内在组件，与Java类库的第三方插件一样。

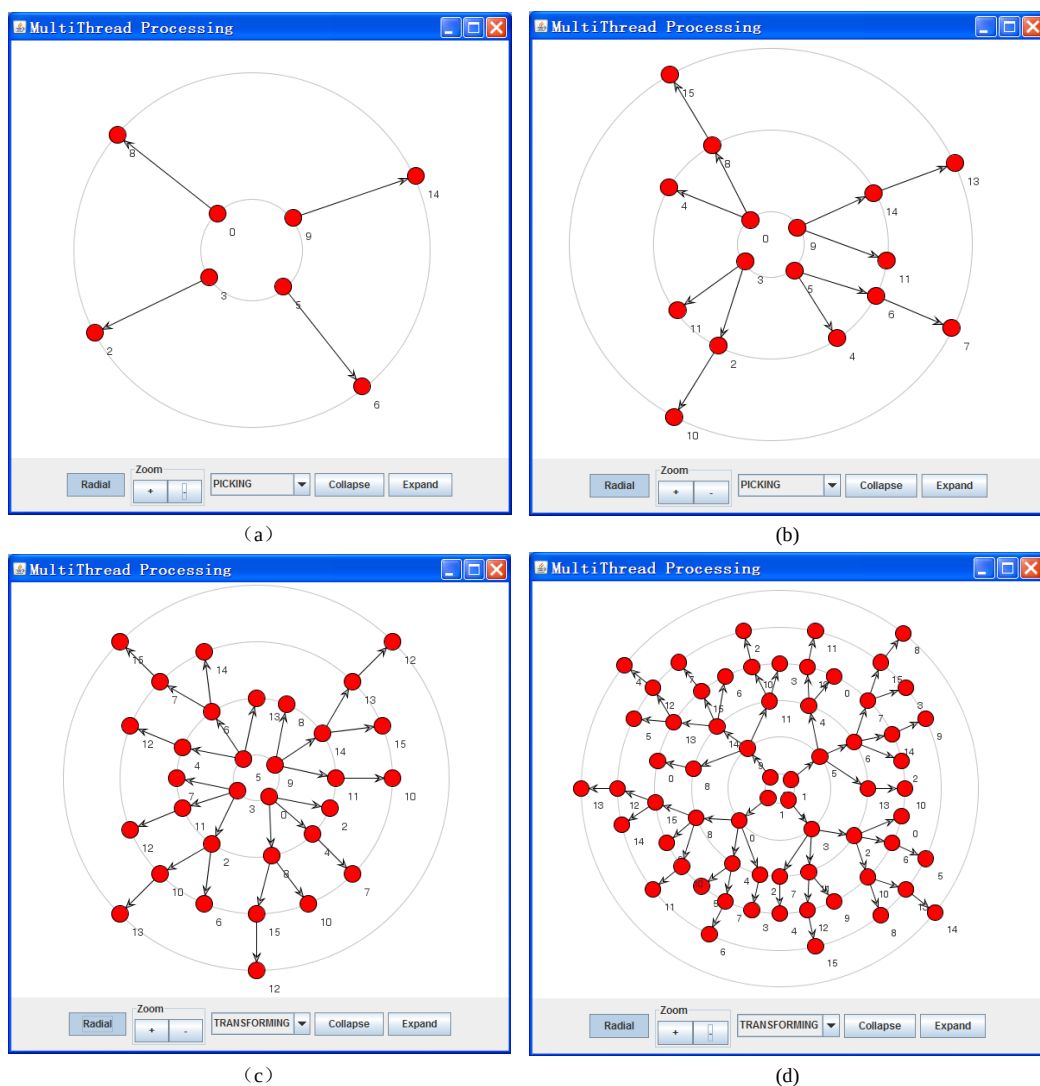
这里，主要存储类型如下：

```
Forest<String,Integer> graphMöbius;
```

```
Factory<DirectedGraph<String,Integer>> graphFactoryMöbius =
```

```
new Factory<DirectedGraph<String,Integer>>() {
```

```
    public DirectedGraph<String, Integer> create() {
```

图 6.7 以雷达方式示出的 $0-M_4$ 上IST的模拟实验(a) level=0; (b) level=1; (c) level=2; (d) level=3

```

return new DirectedSparseMultigraph<String,Integer>();
    }
}

Factory<Tree<String,Integer>> treeFactoryMöbius = new Factory<Tree<String, Integer>>(){

    public Tree<String, Integer> create() {

        return new DelegateTree<String, Integer>(graphFactoryMöbius);

    }
}

```

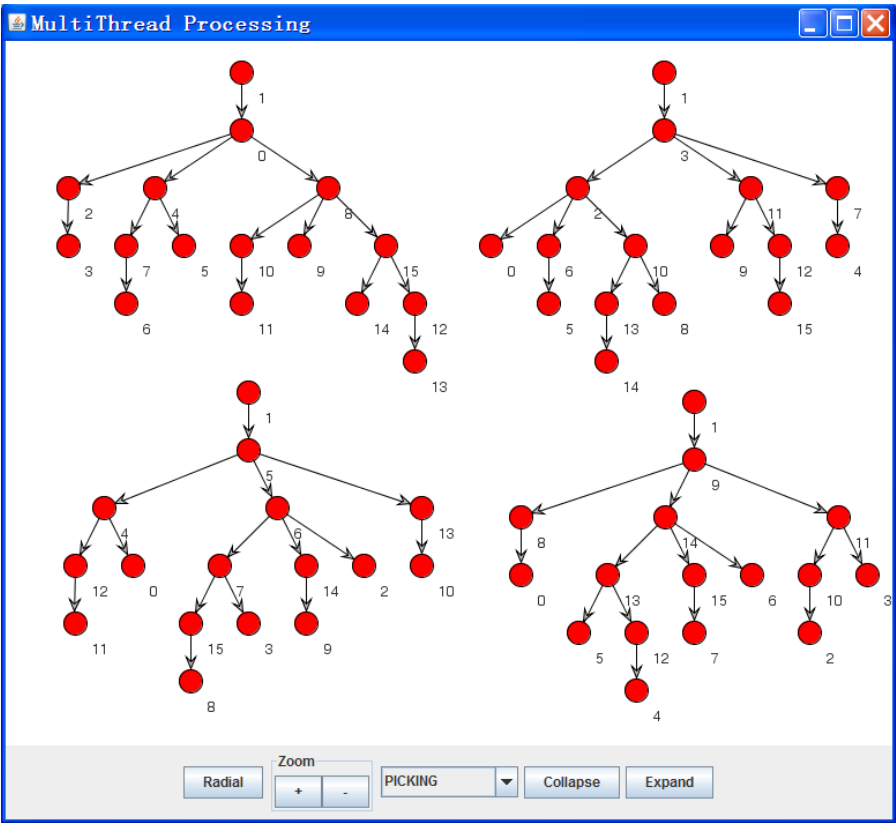


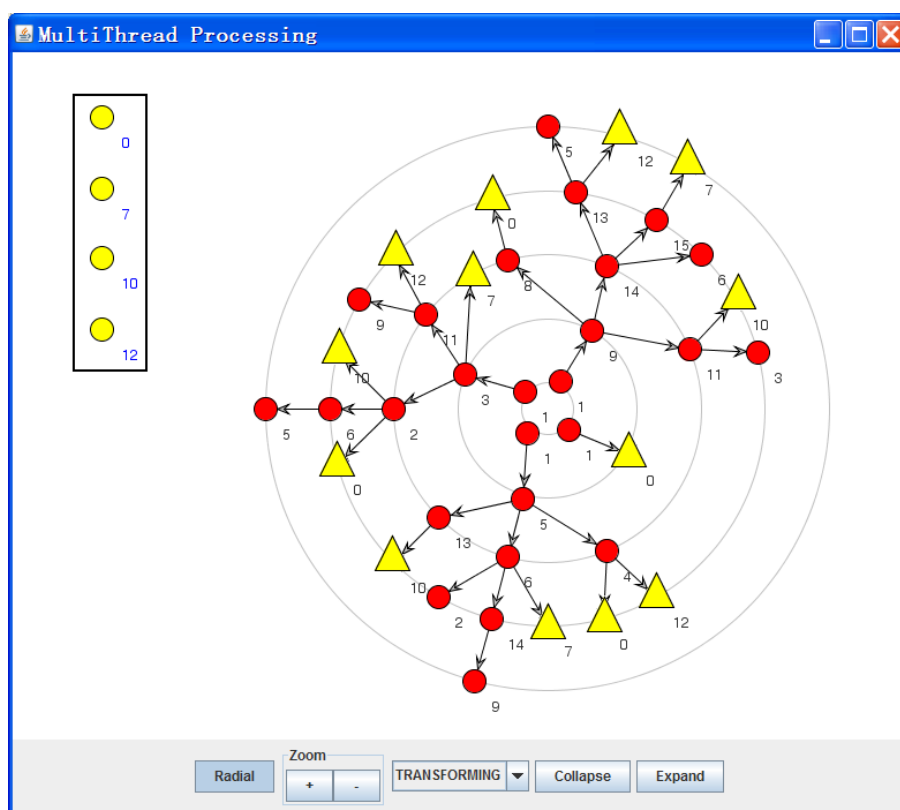
图 6.8 以平面图方式示出的 $0-M_4$ 上IST的模拟实验

```
}  
  
Factory<Integer> edgeFactoryMöbius = new Factory<Integer>() {  
    ...  
}  
  
Factory<String> vertexFactoryMöbius = new Factory<String>() {  
    ...  
}  
  
VisualizationViewer<String, Integer> vvMöbius;  
...  

```

假设 $0-M_4$ 网络上有16个处理器。基于算法OPMCIST，我们基于JUNG 技术设计程序。运行过程如下：

初始时，树 T_i 仅有一个 i 维邻接点处理器1，对于 $i=0,1,2,3$ ，也就是说， T_0, T_1 ,

图 6.9 采用JUNG技术对 $0-M_4$ 上的故障处理器进行诊断

T_2 和 T_3 中分别含有处理器0, 3, 5 和9, 如图 6.7(a)所示。对于集合 $\{0, 3, 5, 9\}$ 中的任一处理器, 我们存储整型数组 $S = \{3, 2, 1, 0\}$ 。

经过第 l 轮迭代, T_i 中的每个处理器连接它的 $S[(l - i + 4) \bmod 4]$ 维邻接点, 发送数组 S 给它的 $S[(l - i + 4) \bmod 4]$ 维邻接点, 对于 $l=1, 2, 3, 4$ 和 $i=0, 1, 2, 3$ 。在图 6.7中, (a), (b), (c)和(d) 示出了 $0-M_4$ 上采用JUNG技术以雷达方式构造IST的步骤。图 6.8 示出了 CQ_4 上采用JUNG技术以平面方式构造的以1为根的四棵IST。

假设在网络 $0-M_4$ 上有四个随机故障处理器(例如, 0, 7, 10, 12)和无故障的根处理器1。基于JUNG和算法DiagnosisIST, 我们模拟IST在诊断上的应用。在图 6.9中, 红色球表示正常处理器, 黄色三角形代表故障处理器。一旦处理器被检测为有故障, 处理器不再测试其它处理器。经过四轮测试, 所有故障处理器能被准确鉴别。

6.7 本章小结

在本章中, 我们基于圆排列的思想给出了一个 M_n 上以任一顶点为根的 n 棵IST的

并行构造。首先,我们给出了一个基于圆排列 $n-1, n-2, \dots, 0$ 的时间复杂度为 $O(N)$ 的并行算法来构造 M_n 上以任一顶点为根的 n 棵IST, 对于 $n \geq 2$ 。然后, 我们证明了算法的正确性。进一步, 我们给出了一个高效的诊断算法, 该算法能够在 $n+1$ 步内诊断出 M_n 上所有的故障处理器, 其中故障处理器数目不超过 n 。此外, 我们还基于JUNG技术设计了莫比乌斯立方体上独立生成树的构造及其在故障处理器诊断方面的应用的模拟实验。

本章关于莫比乌斯立方体上IST并行构造的结果已经发表在SCI源期刊《The Journal of Supercomputing》上(第一作者)。

第七章 BC网络上IST的构造

到目前为止,在特殊BC网络如超立方体、交叉立方体、局部扭立方体及莫比乌斯立方体等上IST的构造问题已经被解决。但是,特殊BC网络中IST的构造及其证明依赖于其特有的性质。因此,这些特殊BC网络如超立方体、交叉立方体、局部扭立方体、莫比乌斯立方体等上是否具有共同的性质,根据该性质是否能够设计一个一般性的算法来构造其上的IST?对于所有的BC网络,是否存在一个共同的方法来构造其上类似于二项树的生成树和类似于类二项树的IST?

本章研究了上述问题,取得了如下结果:

(1) 我们总结出一些特殊BC网络的共性,给出了条件BC网络的定义,我们证明了超立方体、交叉立方体、局部扭立方体、莫比乌斯立方体等均属于条件BC网络。进一步,我们给出了不属于条件BC网络的BC网络。

(2) 我们针对任一BC网络,提出了维的V排列。

(3) 我们证明了递增的圆排列能够用来在 $O(N)$ 时间内构造出 n 维条件BC网络 XC_n 上 n 棵以任一顶点为根的同构于 n -级类二项树的IST,这里 $n \geq 2$ 且 $N = 2^n$ 是 XC_n 的顶点个数。

(4) 我们证明了V排列能够用来构造任一 n 维BC网络 X_n 上以任一顶点为根的生成树且证明这些生成树均同构于 n -级二项树。进一步,我们指出,基于V排列能够构造 X_n 上有多组两两独立的生成树。

7.1 BC互连网络

BC网络定义参见定义2.10和定义2.11。根据其定义,我们有如下引理。

引理 7.1. 对于任意整数 $n \geq 1$,若 $(u_{n-1}u_{n-2} \dots u_0, v_{n-1}v_{n-2} \dots v_0)$ 是 n 维BC网络 X_n 中的一条边,则存在一个整数 l ,使得

$$(1) u_{n-1}u_{n-2} \dots u_l = v_{n-1}v_{n-2} \dots v_l,$$

$$(2) u_{l-1} \neq v_{l-1},$$

基于定义2.10,定义2.11和引理7.1,每个顶点 x 均有 n 个维邻接点,其与 x 的最左不同位依次为 $0, 1, \dots, n-1$ 。注意到, X_n 中的任一顶点地址被设置为0,我们能重新给

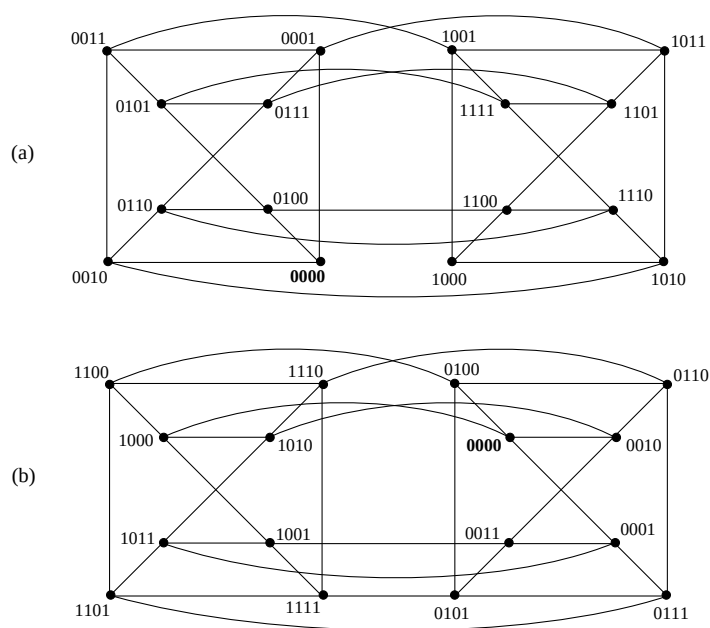


图 7.1 (a) 一个4维BC网络 X_4 ; (b) 重新编号后的4维BC网络 X_4

其它顶点设置编号, 使得它们仍然满足定义 2.11。如图 7.1 中, 将(a)中的顶点1111设置成0000, 我们重新进行编号得到的图(b)仍然满足BC网络的定义。

现在, 我们给出如下定义。

定义 7.1. 对于 $1 \leq m \leq n-1$ 和一个二进制串 $s \in \{0, 1\}^m$, $X_n(s)$ 被定义为由 X_n 中具有共同前缀 s 的顶点集合导出的子图。

基于定义 7.1, 对于 $n \geq 2$, 如果 $u = u_{n-1}u_{n-2} \dots u_0$ 是 X_n 中的任一顶点, 那么 u 属于 X_n 的子网络: $X_n(u_{n-1}u_{n-2} \dots u_{n-i})$, 这里 $1 \leq i \leq n-1$ 。显然, 下面的定理成立。

引理 7.2. 给定一个顶点 $u = u_{n-1}u_{n-2} \dots u_0 \in V(X_n)$, 顶点 $N(u, k) \in V(X_n(u_{n-1}u_{n-2} \dots u_{k+1}\overline{u_k}))$, 对于 $0 \leq k \leq n-1$ 。

多种并行计算机系统尤其是基于超立方体的计算机系统中, 二项树是一类重要的生成树结构 [138]。该结构被认为是一个理想的适合并行分治算法的计算结构 [139], 常用于前缀计算 [140]、数据累计和数据广播中。

7.2 条件BC网络

7.2.1 条件BC网络的定义

现在我们给出条件BC网络的定义。

条件 7.1. 给定 n 维BC网络 X_n 中的任意两条路径 $x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$ 和 $x^{(0)}, y^{(1)} = N(x^{(0)}, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(j)} = N(y^{(j-1)}, h_j)$, 对于 $1 \leq k, j \leq n$, 如果下面条件成立:

- (1) $0 \leq m_1 < m_2 < \dots < m_k$,
- (2) $0 \leq h_1 < h_2 < \dots < h_j = m_k$, 且
- (3) $m_1 \neq h_1$,

那么 $\text{LDF}(x^{(k)}, y^{(j)}) \geq \min(m_1, h_1)$ 。

定义 7.2. 令 G 是一个BC网络。若 G 满足条件 7.1, 那么 G 是一个条件BC网络 (Conditional BC Network)。我们用 XC_n 表示 n 维条件BC网络。 n 维条件BC网络族被定义为 $\mathcal{LC}_n = \{G | G \text{ 是一个 } n \text{ 维条件BC网络}\}$ 。

7.2.2 条件BC网络族

接下来, 我们证明 n 维条件BC网络族包括 $Q_n, LTQ_n, CQ_n, 0-M_n, 1-M_n$ 等。

引理 7.3. $LTQ_n \in \mathcal{LC}_n$ 。

证明. 给定 LTQ_n 上的任意两条路径 $x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$ 和 $x^{(0)}, y^{(1)} = N(x^{(0)}, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(j)} = N(y^{(j-1)}, h_j)$, 对于 $1 \leq k, j \leq n$, 如果下面条件成立:

- (1) $0 \leq m_1 < m_2 < \dots < m_k \leq n-1$,
- (2) $0 \leq h_1 < h_2 < \dots < h_j = m_k$, 且
- (3) $m_1 \neq h_1$,

令 $A = \{m_1, m_2, \dots, m_k\}$, $B = \{h_1, h_2, \dots, h_j\}$ 和 $d = \max((A \setminus B) \cup (B \setminus A))$ 。显然, $d \geq \min(m_1, h_1)$ 。根据条件 7.1, 我们只需验证 $\text{bit}(x^{(k)}, d) \neq \text{bit}(y^{(j)}, d)$ 。我们有如下情形:

情形1. $d = n-2$ 。不失一般性, 假设 $u_{d_i} = 0$ 。

情形1.1. $u_0 = 0$ 。根据定义 2.8, $u_d \oplus 0 = 0 \oplus 0 = 0$ 和 $\overline{u_d} \oplus 0 = 1 \oplus 0 = 1$ 。容易验证 $\text{bit}(x^{(k)}, d) \neq \text{bit}(y^{(j)}, d)$, 这意味着 $\text{LDF}(x^{(k)}, y^{(j)}) > \min(m_1, h_1)$ 。

情形1.2. $u_0 = 1$ 。根据定义 2.8, $u_d \oplus 1 = 0 \oplus 1 = 1$ 和 $\overline{u_d} \oplus 1 = 1 \oplus 1 = 0$ 。余下的证明类似与情形1.1。

情形2. $d \neq n-2$ 。根据定义 2.8, 容易验证 $\text{bit}(x^{(k)}, d) \neq \text{bit}(y^{(j)}, d)$, 这意味着 $\text{LDF}(x^{(k)}, y^{(j)}) \geq \min(m_1, h_1)$ 。

基于上述讨论, 容易验证 $\text{bit}(x^{(k)}, d) \neq \text{bit}(y^{(j)}, d)$ 。

综上所述, 引理成立。

根据 [141]中的成果, 我们有如下引理。

引理 7.4. $CQ_n \in \mathcal{LC}_n$ 。

根据引理 4.3, 定义 2.7和 Q_n 的定义, 类似于引理 7.3的证明, 我们能够验证如下引理成立。

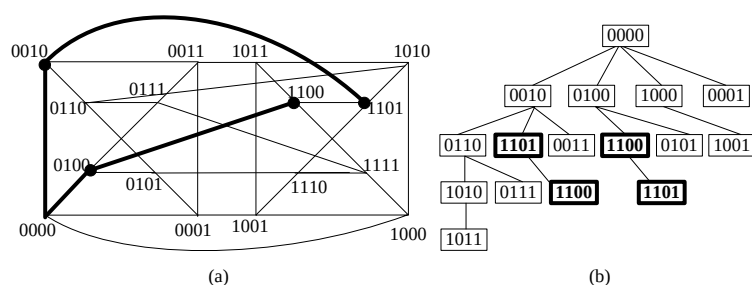
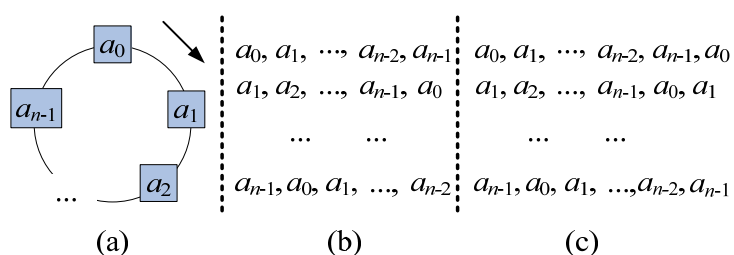
引理 7.5. $\{0-M_n, 1-M_n, Q_n\} \subseteq \mathcal{LC}_n$ 。

我们归纳引理 7.3, 引理 7.4和引理 7.5 为下述定理。

定理 7.6. $\mathcal{LC}_n \supseteq \{Q_n, CQ_n, LTQ_n, 0-M_n, 1-M_n\}$ 。

观察 7.1. 类似于引理 7.3, 验证其它的特殊BC网络如扭曲立方体、扭立方体是否满足条件 7.1是一件有意义的事情。

需要指出的是, 并非所有的BC网络都满足条件 7.1。假设顶点 u 和 v 是子网络 X_{n-1}^0 或 X_{n-1}^1 中的两个顶点, 对于 $n \geq 2$ 。令 $\text{LDF}(u, v) = l$ 。那么, $\text{LDF}(N(u, n-1), N(v, n-1))$ 可能比 l 小。图 7.2 示出了一个4维BC网络 X_4 。我们有 $\text{LDF}(N(0010, 0100)) = 2$ 和 $\text{LDF}(N(0010, 3), N(0100, 3)) = 0$ 。设输入为根顶点0001和一个数组 $S = \{1, 2, 3, 0\}$, 如果我们基于算法SpanningTree构造一个网络, 将有重复顶点在输出的图 T 中。图 7.2(b)显示了重复的顶点1100和1101。令路径 $P: u = 0000, x^{(1)} = N(u, m_1 = 1) = N(0000, 1) = 0010, x^{(2)} = N(x^{(1)}, m_2 = 3) = N(0010, 3) = 1101$ 和路径 $Q: u = 0000, y^{(1)} = N(u, h_1 = 2) = N(0000, 2) = 0100, y^{(2)} = N(y^{(1)}, h_2 = 3) = N(0100, 3) = 1100$ 。显然, $\text{LDF}(1100, 1101) = 0 < \min\{m_1, h_1\} = \min\{1, 2\} = 1$ 。因此, 图 7.2(a)不满足条件 7.1。这表明, 并非所有的BC网络都属于条件BC网络。

图 7.2 (a) 一个4维BC网络 X_4 ; (b) 图 T 中的重复顶点1100 和1101图 7.3 顶点集合 $\{a_0, a_1, \dots, a_{n-1}\}$ 中所有元素的CDP

7.2.3 圆排列和V排列

如第六章所述, 一个圆排列 (CDP) 是一种将 n 个不同整数按顺时针方向围成一个圆的排列。在一个圆中, 所有整数的循环排列是等价的, 基于一个CDP, 有 n 个循环排列。令 $\{a_0, a_1, \dots, a_{n-1}\} = \{0, 1, \dots, n-1\}$ 和 $n \geq 2$, 图 7.3(b)示出的CDP都是与图 7.3(a)相同的CDP。同时, 图 7.3 (c) 示出了另外一种表示法。我们称具有循环排列 $0, 1, \dots, n-1$ 的排列CDP为递增的圆排列。

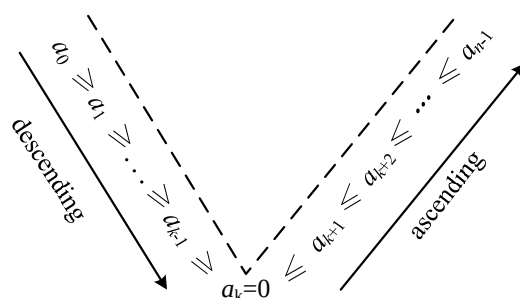
现在我们定义一个集合中所有元素的V排列如下。

定义 7.3. 令 $S = \{0, 1, \dots, n-1\}$, 对于 $n \geq 1$ 。集合 S 中所有元素的V排列 (V-Dimensional Permutation, 简称VDP) 是 n 个元素 $a_0, a_1, \dots, a_k, a_{k+1}, a_{k+2}, \dots, a_{n-1}$ 的有序排列, 这里 $0 \leq k \leq n-1$, 使得如下条件成立:

- (1) $\{a_0, a_1, \dots, a_{n-1}\} = \{0, 1, \dots, n-1\}$, 这里我们忽略数值的次序,
- (2) $a_0 \geq a_1 \geq \dots \geq a_{k-1} \geq a_k = 0$, 且
- (3) $0 \leq a_{k+1} \leq a_{k+2} \leq \dots \leq a_{n-1}$ 。

图 7.4示出了 $\{a_0, a_1, \dots, a_{k-1}, a_k, a_{k+1}, a_{k+2}, \dots, a_{n-1}\}$ 中所有元素的VDP。

如下定理给出了一个给定集合中的所有VDP的个数。

图 7.4 元素 $a_0, a_1, \dots, a_{k-1}, a_k, a_{k+1}, a_{k+2}, \dots, a_{n-1}$ 的 VDP

引理 7.7. 假设 $S = \{0, 1, \dots, n-1\}$, 对于 $n \geq 1$ 。那么, S 中的所有元素有 2^{n-1} 个 VDP。

证明. 令 $S_1 = \{a_0, a_1, \dots, a_k\} \subseteq S$, 这里 $a_0 \geq a_1 \geq \dots \geq a_k = 0$ 。构造 S 中所有元素的 VDP, 我们有 n 种选择 S_1 的方式。

情形1. $|S_1| = 1$ 。我们有 1 种方式。

情形2. $|S_1| = 2$ 。我们有 C_{n-1}^1 种方式。

情形3. $|S_1| = 3$ 。我们有 C_{n-1}^2 种方式。

...

情形 n . $|S_1| = n$ 。我们有 C_{n-1}^{n-1} 种方式。

因此, 总的选择方式为

$$1 + C_{n-1}^1 + C_{n-1}^2 + \dots + C_{n-1}^{n-1} = 2^{n-1}。$$

7.3 条件BC网络上IST构造

7.3.1 条件BC网络上IST的构造算法

在本节中, 依据上节的研究成果, 我们首先给出算法 **SpanningTree** 来构造任意条件BC网络 XC_n 上的生成树。然后, 我们提出一个算法 **MultiIST** 来构造 XC_n 上 n 棵以任一顶点为根的 IST。算法 **MultiIST** 基于递增的 CDP, 利用 n 个整数的循环排列来构造 XC_n 上的 n 棵 IST, 调用算法 **SpanningTree**, 通过一个循环排列来构造一棵 IST。接下来, 我们将证明算法 **MultiIST** 的正确性并总是使用 u 来表示 XC_n 中上生成树的根顶点。

例如: 假设 a_0, a_1, a_2, a_3 是 1, 2, 3, 0 的序列。基于算法 **MultiIST**, 以 $0-M_4$ 上根为 $u=1$ 的 T_0 为例, 如图 7.5 所示。这里, 顶点以十进制表示。初始时, $V(T_0)$ 中仅有一个顶点 0; 经过

算法 15 SpanningTree

输入: 根顶点 u , 条件BC网络 XC_n 和一个数组 $S = \{b_0, b_1, \dots, b_{n-1}\}$, 这里 $b_0, b_1, \dots, b_{n-1}$ 是一个基于递增CDP的整数循环排列;

输出: 一个图 T ;

开始

1: $V(T) = N(u, S[n-1]);$

2: $E(T) = \emptyset;$

//初始化 T 。

3: **for** $level = 0$ to $n-1$ **do**

4: **for** 每一个顶点 $x \in V(T)$ **do**

5: $y = N(x, S[level]);$

6: $E(T) = E(T) \cup \{(x, y)\};$

7: $V(T) = V(T) \cup \{y\};$

8: **end for**

9: **end for**

结束

第一轮迭代 ($level = 0$), $V(T_0)$ 中的顶点0 连接顶点 $N(0, 1) = 2$, 从而, $V(T_0) = \{0, 2\}$ 且 $E(T_0) = \{(0, 2)\}$; 经过第二轮迭代 ($level = 1$), $V(T_0)$ 中的每个顶点 v 连接顶点 $N(v, 2)$ 。因此, 边 $(2, 6)$ 和 $(0, 4)$ 被增加到 T_0 , 此时 $V(T_0) = \{0, 2, 4, 6\}$; 经过第三轮迭代 ($level = 2$), $V(T_0)$ 中的每个顶点 v 连接顶点 $N(v, 3)$ 。相应地, $V(T_0)$ 中顶点数目翻倍; 经过最后一轮迭代 ($level = 3$), $V(T_0)$ 中的每个顶点连接顶点 $N(v, 0)$ 。此时, $0-M_4$ 上一棵以1为根的IST T_0 被成功构造。

7.3.2 条件BC网络上IST的正确性证明

基于定义 2.11, 下面的定理能够应用于任一BC网络上。

引理 7.8. 给定 X_n 上的一条途径 P : $x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k), 1 \leq k \leq n$, 如果 $m_1, m_2, \dots, m_k$ 各不相同且 $0 \leq m_i \leq n-1, i = 1, 2, \dots, k$, 那么 P 是一条路径。

证明. 假设存在两个相同的顶点 $x^{(c)} = x^{(d)}$, 对于 $0 \leq c < d \leq k$ 。根据定义 2.11, 我们能够验证 $LDF(x^{(c)}, x^{(d)}) = \max(\{m_c, m_{c+1}, \dots, m_{d-1}\})$, 产生矛盾。因此, 引理成立。

现在我们给出下面的引理来讨论 n 维条件BC网络 XC_n 上的不相交路径, 并且将其作为一个关键定理来证明算法MultiIST构造出的树皆为 XC_n 上的生成树。

算法 16 MultiIST

输入: 根顶点 u , 条件BC网络 XC_n 和一个数组

SList[] = $\{\{a_0, a_1, \dots, a_{n-1}\},$
 $\{a_1, a_2, \dots, a_{n-1}, a_0\},$
 $\{a_2, a_3, \dots, a_{n-1}, a_0, a_1\},$
 $\dots$
 $\{a_{n-1}, a_0, a_1, \dots, a_{n-1}, a_{n-2}\}\}$

这里 $a_0, a_1, \dots, a_{n-1}$ 是一个递增型的CDP 和 $\{a_0, a_1, \dots, a_{n-1}\} = \{0, 1, \dots, n-1\}$;

输出: n 个图 $T_0, T_1, \dots, T_{n-1}$;

开始

1: **for** $i = 0$ to $n - 1$ **do**
 2: 并行模式
 3: 根据输入根顶点 u , XC_n 和SList[i]调用算法SpanningTree;
 4: 令 T_i 表示输出;
 5: **end for**

结束

引理 7.9. 给定 n 维条件BC网络 XC_n 上的两条路径 $P: x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$ 和 $P': x^{(0)}, y^{(1)} = N(x^{(0)}, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(j)} = N(y^{(j-1)}, h_j)$, 对于 $1 \leq k, j \leq n$, 如果下面条件成立:

- (1) $0 \leq m_1 < m_2 < \dots < m_k$,
- (2) $0 \leq h_1 < h_2 < \dots < h_j$, 且
- (3) $m_1 \neq h_1$,

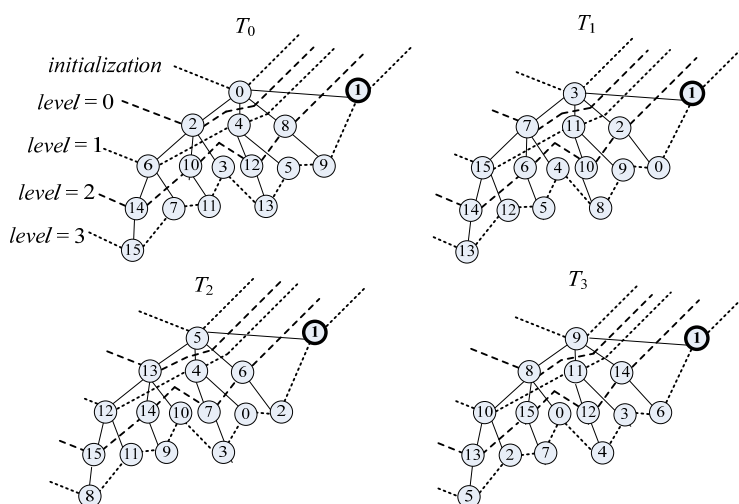
那么 $V(P) \cap V(P') = \{x^{(0)}\}$ 。

证明. 根据定义 2.11和定义 7.1, 引理成立。

根据引理 7.8, 引理 7.9和算法MultiIST, 我们有如下定理。

定理 7.10. 对于 $0 \leq i \leq n - 1$, 由算法MultiIST构造的 T_i 是 XC_n 上一棵以 u 为根高度为 $n+1$ 且同构于 BL_n 的生成树,

证明. 根据算法MultiIST, 经过 n 次迭代, 我们得到 n 个图 $T_0, T_1, \dots, T_{n-1}$ 。容易验证, $V(T_i)$ 中有 $1 + 2^0 + 2^1 + \dots + 2^{n-1} = 2^n$ 个顶点且 T_i 同构于 BL_n , 对于 $0 \leq i \leq n - 1$ 。既然 $V(XC_n) = 2^n$, T_i 中的初始顶点 $N(u, i)$ 有 n 个不同邻接点包括叶子顶点 u , 那么以 u 为根的 T_i 的高度为 $n + 1$ 。我们只需证明 T_i 中不存在相同顶点。假设 T_i 中存

图 7.5 $0-M_4$ 上以 1 为根的 IST 的构造

在两个共同顶点 v 。 T_i 中两条起点为 $N(u, i)$ 终点为 v 的路径能够表示为 P : $N(u, i)$, $x^{(1)} = N(x^{(0)}, m_1)$, $x^{(2)} = N(x^{(1)}, m_2)$, $\dots$, $x^{(k)} = N(x^{(k-1)}, m_k) = v$ 和 Q : $N(u, i)$, $y^{(1)} = N(x^{(0)}, h_1)$, $y^{(2)} = N(y^{(1)}, h_2)$, $\dots$, $y^{(j)} = N(y^{(j-1)}, h_j) = v$ 。令 $d_1 = \max\{m_1, m_2, \dots, m_k\}$ 和 $d_2 = \max\{h_1, h_2, \dots, h_j\}$ 。我们有如下两种情形:

情形1. $d_1 \neq d_2$ 。那么, 路径 P 和路径 Q 中 v 的第 $\max\{d_1, d_2\}$ 位不相同, 这是一个矛盾。

情形2. $d_1 = d_2$ 。令 $m_a = h_b$ 。既然算法 MultiIST 的输入 $a_0, a_1, \dots, a_{n-1}$ 是一个递增的 CDP, 我们有 $m_1 < m_2 < \dots < m_a$, $m_{a+1} < m_{a+2} < \dots < m_k$, $h_1 < h_2 < \dots < h_b$ 和 $h_{b+1} < h_{b+2} < \dots < h_j$ 。根据定义 7.2, $\text{LDF}(x^{(a)}, y^{(b)}) \geq \min\{m_1, h_1\}$ 。令 $S = \{m_{a+1}, m_{a+2}, \dots, m_k, h_{b+1}, h_{b+2}, \dots, h_j\}$ 。进一步, 根据算法 MultiIST, 我们有如下两种子情形。

情形2.1. $S = \emptyset$ 。那么, $v = x^{(a)}$, $v = y^{(b)}$, 且 $x^{(a)} \neq y^{(b)}$, 这是一个矛盾。

情形2.2. $S \neq \emptyset$ 。那么根据算法 MultiIST 的输入, 我们能够验证 $\max(S) < \min\{m_1, h_1\}$ 。根据定义 2.11 和定义 7.2, 我们能够验证 P 中 v 的第 $\text{LDF}(x^{(a)}, y^{(b)})$ 位不同于 Q 中 v 的第 $\text{LDF}(x^{(a)}, y^{(b)})$ 位, 这是一个矛盾。

因此, 定理成立。

根据定义 7.3, 我们有如下引理:

引理 7.11. 假设 T_i 和 T_j 是算法 MultiIST 的输出且 $v \in V(XC_n) \setminus \{u\}$, 对于 $0 \leq i < j \leq$

$n-1$ 。令 P 和 P' 分别表示路径 $path(T_i, u, v)$ 和 $path(T_j, u, v)$ 。那么, $ancestor(v, P) \cap ancestor(v, P') = \{u\}$ 和 $ancestor(v, P) \cup ancestor(v, P') \supset \{u\}$ 。

证明. XC_n 上的路径 P 能够表示为 $u, x^{(1)} = N(u, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$, 路径 P' 能够表示为 $u, y^{(1)} = N(u, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(j)} = N(y^{(j-1)}, h_j)$, 对于 $1 \leq k, j \leq n$ 。根据算法 MultiIST, 如下三个条件成立:

- (1) $\langle m_1, m_2, \dots, m_k \rangle$ 是序列 $\langle m_1, m_1 + 1, \dots, n-1, 0, 1, m_1 - 1 \rangle$ 的一个子序列,
- (2) $h_1, h_2, \dots, h_j$ 是序列 $\langle h_1, h_1 + 1, \dots, n-1, 0, 1, h_1 - 1 \rangle$ 的一个子序列, 且
- (3) $m_1 \neq h_1$ 。

不失一般性, 假设 $m_1 < h_1$ 。那么, 我们有 $x^{(1)} = N(u, m_1) \neq y^{(1)} = N(u, h_1)$, 这意味着 $ancestor(v, P) \cup ancestor(v, P') \supset \{u\}$ 。现在我们证明 $ancestor(v, P) \cap ancestor(v, P') = \{u\}$ 。令 $A = \{m_1, m_2, \dots, m_k\}$ 且 $B = \{h_1, h_2, \dots, h_j\}$ 。既然 $\max(B) \geq h_1$, 我们有如下情形:

情形1. $\max(A) \neq \max(B)$ 和 $\max(A \cup B) > h_1$ 。那么, P 中 v 的第 $\max(A \cup B)$ 位不同于 P' 中 v 的第 $\max(A \cup B)$ 位, 这是一个矛盾。

情形2. $\max(A) \neq \max(B)$ 和 $\max(A \cup B) = h_1$ 。那么, $\max(A) < h_1$ 且 $V(P) \setminus \{u, v\}$ 中任一顶点的第 h_1 位不同于 $V(P') \setminus \{u, v\}$ 中任一顶点的第 h_1 位。

情形3. $\max(A) = \max(B)$ 。那么, P 能够表示为 $u, x^{(1)} = N(u, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(c)} = N(x^{(c-1)}, m_c), x^{(c+1)} = N(x^{(c)}, m_{c+1}), x^{(c+2)} = N(x^{(c+1)}, m_{c+2}), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$, 这里 $m_c = \max(A)$ 。 P' 能够表示为 $u, y^{(1)} = N(u, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(e)} = N(y^{(e-1)}, h_e), y^{(e+1)} = N(y^{(e)}, h_{e+1}), y^{(e+2)} = N(y^{(e+1)}, h_{e+2}), \dots, y^{(j)} = N(y^{(j-1)}, h_j)$, 这里 $h_e = \max(B)$ 。

根据条件 7.1, 我们能够验证 $LDF(x^{(c)}, y^{(e)}) \geq \min(m_1, h_1)$ 。令 $d = LDF(x^{(c)}, y^{(e)})$, 我们能够验证 $\max\{m_k, h_j\} = d$ 和 $m_k \neq h_j$ 。令 $U_1 = \{u, x^{(1)}, x^{(2)}, \dots, x^{(c-1)}\}$, $U_2 = \{x^{(c)}, x^{(c+1)}, \dots, x^{(k-1)}\}$, $W_1 = \{u, y^{(1)}, y^{(2)}, \dots, y^{(e-1)}\}$, 和 $W_2 = \{y^{(e)}, y^{(e+1)}, \dots, y^{(j-1)}\}$ 。我们能够验证 U_2 中的每一个顶点和 W_2 中的每一个顶点具有一个最左不同位 d , 这意味着 $U_2 \cap W_2 = \emptyset$ 。类似地, $U_1 \cap W_2 = \emptyset$ 和 $U_2 \cap W_1 = \emptyset$ 。根据引理 7.9, $U_1 \cap W_1 = \{u\}$ 。

因此, $ancestor(v, P) \cap ancestor(v, P') = \{u\}$ 。

综上所述, 引理成立。

定理 7.12. 对于 $n \geq 2$ 和 XC_n 上的任一顶点 u , 如果 $a_0, a_1, \dots, a_{n-1}$ 是 $0, 1, \dots, n-1$ 的递增的 CDP, 由算法 MultiIST 构造出的 $T_0, T_1, \dots, T_{n-1}$ 是 XC_n 上 n 棵高度为 $n+1$ 以顶点 u 为根的同构于 BL_n 的 IST。

证明. 根据定理 7.10, 对于 $0 \leq i \leq n-1$, T_i 是 XC_n 上 n 棵高度为 $n+1$ 以顶点 u 为根的同构于 BL_n 的生成树。现在, 我们证明 T_k 和 T_l 相互独立, 对于 $0 \leq k < l \leq n-1$ 。根据引理 7.11, $\text{ancestor}(v, T_k) \cap \text{ancestor}(v, T_l) = \{u\}$ 和 $\text{ancestor}(v, T_k) \cup \text{ancestor}(v, T_l) \supset \{u\}$, 对于任一顶点 $v \in V(XC_n) \setminus \{u\}$ 。根据引理 3.9, T_k 和 T_l 相互独立。因此, 定理成立。

根据定理 7.12 和算法 MultiIST, 容易验证如下定理成立。

定理 7.13. 算法 MultiIST 在 $O(N)$ 时间内构造出 XC_n 上 n 棵以顶点 u 为根的 IST, 这里 $N = 2^n$ 是 XC_n 上顶点的个数。

引理 7.14. 根据算法 MultiIST, 如果递增的 CDP 能够被用来构造 n 维 BC 网络 X_n 上以任一顶点 u 为根的 n 棵 IST $T_0, T_1, \dots, T_{n-1}$, 那么 X_n 满足条件 7.1。

证明. 假设递增的 CDP 能够用来构造 n 维条件 BC 网络 X_n 上以任一顶点为根的 n 棵 IST $T_0, T_1, \dots, T_{n-1}$ 且 X_n 不满足条件 7.1。即存在 X_n 上的两条路径 $x^{(0)}, x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$ 和 $x^{(0)}, y^{(1)} = N(x^{(0)}, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(j)} = N(y^{(j-1)}, h_j)$, 这里 $1 \leq k, j \leq n$, 使得如下条件成立:

- (1) $0 \leq m_1 < m_2 < \dots < m_k \leq n-1$,
- (2) $0 \leq h_1 < h_2 < \dots < h_j = m_k$, 和
- (3) $m_1 \neq h_1$ 。

那么 $\text{LDF}(x^{(k)}, y^{(j)}) < \min(m_1, h_1)$ 。

不失一般性, 我们假设 $m_1 < h_1$ 。因 $\text{LDF}(x^{(k)}, y^{(j)}) \geq 0$, 故 $0 < m_1 < h_1$ 。

显然 n 个整数的循环排列在圆中是等价的, 且基于递增 CDP 有 n 个循环排列, 令 $a_0, a_1, \dots, a_{n-1}$ 为序列 $m_1, m_1+1, \dots, n-1, 0, 1, m_1-1$ 。根据算法 MultiIST, $N(u, m_1-1)$ 是 T_{m_1-1} 中顶点 u 的唯一孩子顶点, 故 T_{m_1-1} 也是一棵以顶点 $N(u, m_1-1)$ 为根的树。

根据算法 MultiIST, 经过 $n-m_1$ 次迭代, 在树 T_{m_1} 中存在两条路径 $x^{(0)} = N(u, m_1-1), x^{(1)} = N(x^{(0)}, m_1), x^{(2)} = N(x^{(1)}, m_2), \dots, x^{(k)} = N(x^{(k-1)}, m_k)$ 和 $x^{(0)} = N(u, m_1-1), y^{(1)} = N(x^{(0)}, h_1), y^{(2)} = N(y^{(1)}, h_2), \dots, y^{(j)} = N(y^{(j-1)}, h_j)$ 。现在我们仅以顶点 $x^{(k)}$ 和 $y^{(j)}$ 作为新的顶点。令 $x^{(k)} = v_{n-1}v_{n-2}\dots v_0$ 和 $y^{(j)} = w_{n-1}w_{n-2}\dots w_0$ 。经过剩下的 m_1-1 次迭代, 得到分别以顶点 $x^{(k)}$ 和 $y^{(j)}$ 为根的子树 T_1' 和 T_2' 。这两棵子树都有 2^{m_1-1} 个顶点。根据定义 2.11 和算法 MultiIST, 容易验证 T_1' 是 $X_n(v_{n-1}v_{n-2}\dots v_{m_1-1})$ 上的一棵生成树,

T_2' 是 $X_n(w_{n-1}w_{n-2}\dots w_{m_1-1})$ 上的一棵生成树。因为 $\text{LDF}(x^{(k)}, y^{(j)}) < \min(m_1, h_1) \leq m_1 - 1$, 所以 $V(T_1') \cap V(T_2') \neq \emptyset$ 。因此, T_{m_1-1} 不是一棵生成树, 于是产生矛盾。

综上所述, 引理成立。

观察 7.2. 条件 7.1 基于递增的CDP。类似地, 我们能够定义其它CDP。既然IST没有性能上的改进如高度、结构等, 我们将不讨论这些CDP。

7.4 任意BC网络上的生成树和IST

对于任一BC网络 X_n 和一个根顶点 u , X_n 上的一棵生成树能够被该网络上除根顶点之外的 $2^n - 1$ 个处理器并行构造。因此, 我们给出如下的高效算法, 称为 **EfficientTree**, 来构造一棵同构于 B_n 的生成树。在算法 **EfficientTree** 中, 每一个顶点 $x \in V(X_n) \setminus \{u\}$ 能够根据其地址与根顶点地址的区别决定它的邻接点。

算法 17 EfficientTree

输入: 根顶点 u , 一个BC网络 X_n ;

输出: 一个图 T ;

开始

1: $V(T) = u$;

2: $E(T) = \emptyset$;

3: **for** 每一个顶点 $x \in V(X_n) \setminus \{u\}$ **do**

4: **并行模式**

5: $y = N(x, \text{LDF}(x, u))$;

6: $E(T) = E(T) \cup \{(x, y)\}$;

7: $V(T) = V(T) \cup \{y\}$;

8: **end for**

结束

基于算法 **EfficientTree**, 我们有如下定理:

定理 7.15. 令输入为 X_n 上的任一顶点 u 和任一BC网络 X_n 。那么, 由算法 **EfficientTree** 获得的图 T 是 X_n 上以 u 为根的同构于 B_n 的树。

在图 7.6 中, (a) 和 (b) 示出了 $0-M_4$ 上一棵以顶点 11 为根的生成树的构造, 分别以二进制和十进制表示。事实上, 该生成树也可基于维排列 $0, 1, \dots, n-1$ 通过 n 次迭代构造而出。以图 7.6(b) 中的生成树为例, 顶点 10 是顶点 11 的 0 维邻接点; 顶点 8 和 9 分

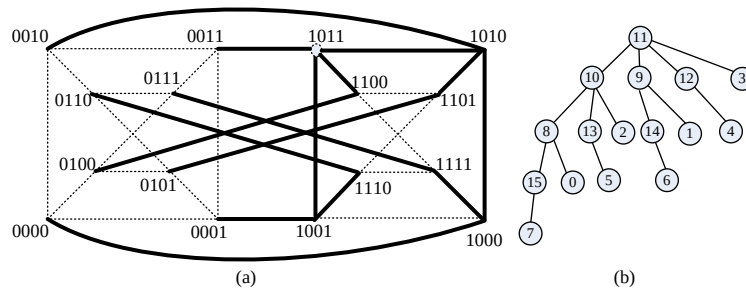


图 7.6 $0-M_4$ 上以顶点 1011 为根的一棵生成树的构造(a)二进制表示; (b) 十进制表示

别是顶点 10 和 11 的 1 维邻接点; 顶点 12, 13, 14, 15 分别是顶点 11, 10, 9, 8 的 2 维邻接点; 顶点 0, 1, ..., 7 分别是顶点 8, 9, ..., 15 的 3 维邻接点。

现在我们将证明整数 $0, 1, \dots, n-1$ 构成的所有 VDP 能够用来构造 X_n 上的生成树。接下来, 我们将总是讨论修改后的算法 *SpanningTree*: 输入 S 由一个集合替换, 满足 $b_0, b_1, \dots, b_{n-1}$ 是一个 VDP 和第一条语句“ $V(T) = N(u, S[n-1]);$ ”被替换为“ $V(T) = u;$ ”。

引理 7.16. 令算法 *SpanningTree* 的输入为根顶点 u 和 $S = \{b_0, b_1, \dots, b_{n-1}\}$, 这里 $b_0, b_1, \dots, b_{n-1}$ 是整数 $0, 1, \dots, n-1$ 的任一 VDP。那么, 输出 T 是任一 BC 网络 X_n 上以任一顶点为根的同构于 B_n 的生成树。

证明. 根据算法 *SpanningTree*, 经过 n 次迭代, 我们在树 T 中有 $1 + 2^0 + 2^1 + \dots + 2^{n-1} = 2^n$ 个顶点。从树 T 中选择任意两个顶点 $v^{(1)}$ 和 $v^{(2)}$, $\text{path}(T, u, v^{(1)})$ 能够表示为 $u, x^{(1)} = N(u, a_{i_1}), x^{(2)} = N(x^{(1)}, a_{i_2}), \dots, x^{(k)} = N(x^{(k-1)}, a_{i_k})$ 和 $\text{path}(T, u, v^{(2)})$ 能够表示为 $u, x^{(1)} = N(u, a_{j_1}), x^{(2)} = N(x^{(1)}, a_{j_2}), \dots, x^{(m)} = N(x^{(m-1)}, a_{j_m})$, 这里 $0 \leq i_1 < i_2 < \dots < i_k \leq n-1$ 和 $0 \leq j_1 < j_2 < \dots < j_m \leq n-1$ 。令 $S^{(1)} = \{a_{i_1}, a_{i_2}, \dots, a_{i_k}\}$ 和 $S^{(2)} = \{a_{j_1}, a_{j_2}, \dots, a_{j_m}\}$ 。根据算法 *SpanningTree*, $S^{(1)} \neq S^{(2)}$ 。现在我们证明 $v^{(1)} \neq v^{(2)}$ 。假设 $l = \max((S^{(1)} \cup S^{(2)}) \setminus (S^{(1)} \cap S^{(2)}))$ 。根据引理 7.8, 引理 7.9, 和定义 2.11, 我们能够验证 $\text{bit}(v^{(1)}, l) \neq \text{bit}(v^{(2)}, l)$ 。因此, $v^{(1)} \neq v^{(2)}$, 这意味着 T 是 X_n 上一棵以 u 为根的生成树。显然, 经过 n 次迭代后, T 同构于 B_n 。

基于上述讨论, 引理成立。

依据引理 4.3 和引理 7.16, 我们有如下定理:

定理 7.17. 基于整数 $0, 1, \dots, n-1$ 的 VDP 和算法 *SpanningTree*, X_n 上有 2^{n-1} 棵以顶点 u 为根的同构于 B_n 的生成树。

引理 7.18. 假设 $b_0, b_1, \dots, b_{n-1} = n-1$ 和 $b'_0, b'_1, \dots, b'_{n-1} = n-1$ 是整数 $0, 1, \dots, n-1$ 的VDP。如果下列两个条件成立:

- (1) $b_0, b_1, \dots, b_{n-1}$ 和顶点 u 是输入, T 是算法 *SpanningTree* 的输出, 且
 - (2) $b'_0, b'_1, \dots, b'_{n-1}$ 和顶点 $N(u, n-1)$ 是输入, T' 是算法 *SpanningTree* 的输出,
- 那么以顶点 u 为根的 T 和 T' 是两棵IST。

基于定理 7.17 和引理 7.18, 有 2^{n-2} 种途径分别来构造 T 和 T' 。进一步, 我们可以验证如下定理。

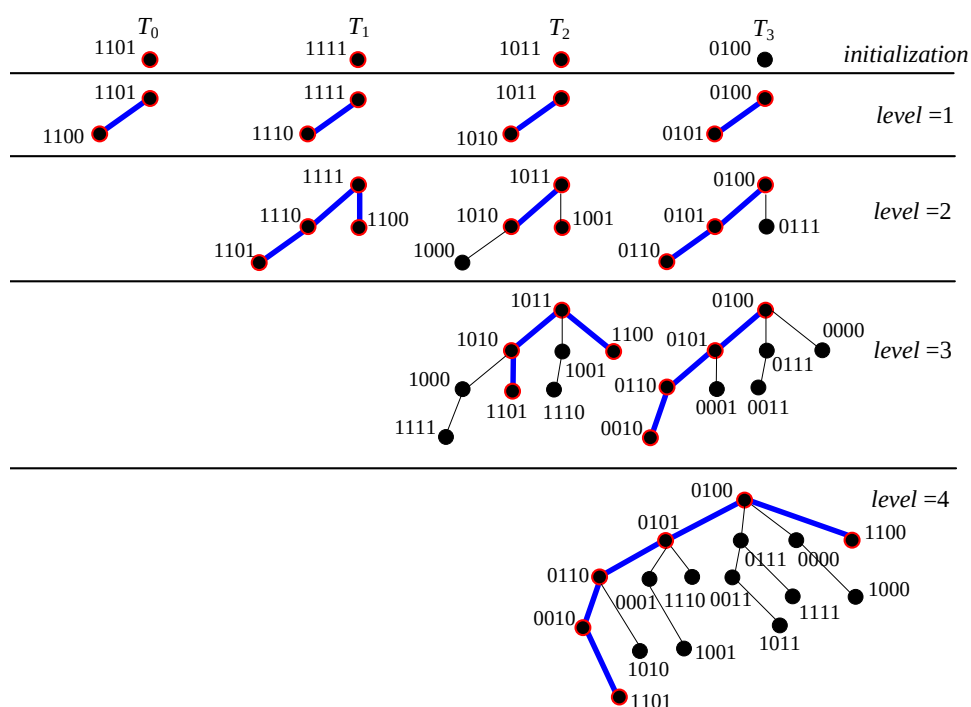
定理 7.19. 令集合 S 中的 $b_0, b_1, \dots, b_{n-1}$ 是整数 $0, 1, \dots, n-1$ 的VDP, 根顶点依次取 $N(u, 0), N(u, 1), \dots, N(u, n-1), T_0, T_1, \dots, T_{n-1}$ 是由算法 *SpanningTree* 输出的 n 棵树。那么, 如下结论成立:

- (1) T_i 是一棵以 u 为根的同构于 BL_n 的生成树, $0 \leq i \leq n-1$ 。
- (2) 对于顶点 $v = N(u, 0), \text{path}(T_0, u, v), \text{path}(T_1, u, v), \dots, \text{path}(T_{n-1}, u, v)$ 是在 X_n 上由算法 *SpanningTree* 构造出的 n 条不相交路径, 其最大长度不超过 $n+1$ 。
- (3) 对于任一顶点 $v \in V(X_n) \setminus \{u, N(u, 0)\}$, 如果 $LDF(u, v) = d$, 那么 $\text{path}(T_d, u, v), \text{path}(T_{d+1}, u, v), \dots, \text{path}(T_{n-1}, u, v)$ 是由算法 *SpanningTree* 构造出的 X_n 上 $n-d$ 条顶点不相交路径, 其最大长度不超过 $n+1$ 。

例如, 对于图 7.2(a) 所表示的BC网络 X_4 和集合 $\{0, 1, 2, 3\}$ 。图 7.7 示出了 $0-M_4$ 上顶点 1100 和 1101 之间 4 条不相交路径的构造过程 (T_0, T_1, T_2 和 T_3 中各一条), 其长度分别为 1, 3, 3 和 5。

7.5 本章小结

本章研究了 n 维条件BC网络 XC_n 上以任一顶点为根的 n 棵IST的并行构造。首先, 我们提出了条件BC网络和V排列的定义。然后, 我们证明了条件BC网络包含超立方体、交叉立方体、局部扭立方体、莫比乌斯立方体等, 且给出了一个在任一条件BC网络上构造以任一顶点为根的 n 棵同构于类二项树 BL_n 的IST 的高效算法。此外, 我们证明了所有的V排列能够用来构造 n 维BC网络 X_n 上同构于 n -级二项树并讨论了 X_n 上的IST构造问题。相关研究结果表明可以找到一个统一的方法来构造一类BC网络——条件BC网络上以任一顶点为根的IST。我们已经证明, 并非所有

图 7.7 $0-M_4$ 上顶点1100和1101之间的顶点不相交路径

的BC网络都属于条件BC网络，那么，是否存在一个不采用V排列的更一般的方法来构造任意BC网络上的IST？这仍然是一个值得我们研究的问题。

本章关于BC网络上IST构造的结果已经被SCI源期刊《IEEE Transactions on Parallel and Distributed Systems》录用（第一作者）。

第八章 总结与展望

8.1 总结

迄今为止,互连网络已经被广泛地研究,作为并行计算系统研究中提出的一类互连网络——BC互连网络,其包含众所周知的超立方体及其变型交叉立方体、莫比乌斯立方体、扭立方体等。本文采用从特殊到一般的方法,研究了BC网络族上IST的构造,包括特殊BC网络上IST的串行构造、特殊BC网络上IST的并行构造、条件BC网络上及任意BC网络上IST的构造等内容。

第三章研究了特殊BC网络交叉立方体上以任一顶点为根的独立生成树的串行构造方法及其算法。

(1) 证明了 n 维交叉立方体 CQ_n 满足一个重要的地址变换性质,依据该地址变换性质给出了 CQ_{n-1}^0 和 CQ_{n-1}^1 上同构树的构造方法。

(2) 给出了 CQ_n 上以任一顶点为根的IST的串行构造方法,并设计了时间复杂度为 $O(N\log^2 N)$ 的构造算法。

第四章研究了特殊BC网络莫比乌斯立方体上以任一顶点为根的独立生成树的串行构造方法及其算法。

与交叉立方体不同的是, n 维莫比乌斯立方体 M_n 是由两个不同构的子莫比乌斯立方体 M_{n-1}^0 和 M_{n-1}^1 组成,从而交叉立方体上地址变换的方法不再适用于在莫比乌斯立方体上构造IST。进一步,我们从顶点之间维邻接关系的角度研究,取得了如下成果:

(1) 结合 M_n 的构造规律,提出了在 M_{n-1}^0 和 M_{n-1}^1 上基于顶点之间的维邻接关系构造同构树的方法。

(2) 从顶点之间维邻接关系的角度,我们设计了 M_n 上独立生成树在构造过程中扩展顶点集合和扩展顶点集合划分的选择方法,并引入非负向量来证明其上IST的正确性。

(3) 设计了时间复杂度为 $O(N\log N)$ 的递归构造算法。

M_n 上基于顶点之间的维邻接关系构造 M_{n-1}^0 和 M_{n-1}^1 中同构树的方法同样适合在 CQ_{n-1}^0 和 CQ_{n-1}^1 中构造同构树,是 CQ_n 上地址变换方法的改进。在 M_n 上所设计

的IST的递归构造算法的效率比 CQ_n 上的有所提高。

第五章研究了交叉立方体上独立生成树的并行构造方法及其算法。

从交叉立方体和莫比乌斯立方体上IST的串行构造方法来看,其对应算法的时间复杂度高且IST在构造过程中第 n 棵树的构造依赖于前面 $n-1$ 棵树的构造。我们研究交叉立方体的新的性质并基于其设计独立生成树的并行构造方法,取得了如下研究结果:

(1) 证明了交叉立方体上满足一个重要性质——维扩散性质。

(2) 提出一个具有UUID成员和VALUE成员的树存储结构,确保树中顶点的UUID成员总是具有唯一性。

(3) 基于交叉立方体上维扩散性质给出了其上IST的并行构造算法。

第六章研究了莫比乌斯立方体上独立生成树的并行构造方法及其算法。

在总结交叉立方体上IST的串行和并行构造方法、莫比乌斯立方体上IST的串行构造方法的基础上,我们进一步研究了莫比乌斯立方体上IST的并行构造问题,取得了如下研究结果:

(1) 从排列组合的角度,将圆排列的概念引入到IST的构造方法中。

(2) 首先证明了递减的圆排列能够用于并行构造莫比乌斯立方体上以任一顶点为根的IST。其次,进一步证明了任一圆排列能够用来并行构造莫比乌斯立方体上以任一顶点为根的IST。

(3) 指出能够利用不同的圆排列构造的多组IST构造莫比乌斯立方体上任意两个顶点之间多组顶点不相交路径。

(4) 研究了基于IST的高效诊断算法。

第七章研究了BC网络上IST的构造方法及其算法。

特殊BC网络如超立方体、交叉立方体、局部扭立方体、莫比乌斯立方体等中IST的构造及其证明依赖于其特有的性质。我们研究了基于它们共同的性质来设计一个一般性的算法来构造其上的IST,取得如下结果:

(1) 我们给出了条件BC网络的定义,证明了超立方体、交叉立方体、局部扭立方体、莫比乌斯立方体等均属于条件BC网络。同时,我们指出了存在不属于条件BC网络的BC网络。

(2) 我们证明了递增的圆排列能够用来构造在 n 维条件BC网络 XC_n 上以任一顶点为根的同构于 n -级类二项树的IST,这里 $n \geq 2$ 。

(3) 针对任一BC网络, 我们提出了维的 V 排列。我们证明了 V 排列能够用来构造任一 n 维BC网络 X_n 上以任一顶点为根的生成树且证明这些生成树均同构于 n 级二项树。我们指出, 基于 V 排列, 能够构造 X_n 上有多组两两独立的生成树。

8.2 展望

本文研究了BC网络上IST构造方法及其算法, 取得了一系列创新性研究成果。但是在研究的过程中难免有些不足, 今后我们对BC网络及一般网络上IST的构造, 可以做进一步的研究和探索。主要体现在以下几个方面:

(1) 交叉立方体和莫比乌斯立方体的直径均接近于超立方体的一半。那么, 在这两种立方体上是否存在最大高度较小的IST, 如其最大高度是其直径加上一个常数, 该问题值得进一步的研究。

(2) 1维BC网络只有一种结构, 2维BC网络只有1种结构, 3维BC网络只有两种结构, 而对于4维以上的BC网络包含多少种结构目前没有研究结果。对于任意整数 $n \geq 5$, 是否能找到适合于所有BC网络 X_n 的以任一顶点为根的 n 棵IST的通用构造算法, 这一问题还需要我们进一步的研究。

(3) 依据Gopalan等的结论 [75], 顶点猜想和边猜想之间的转换关系仍是一个未解决的问题。

(4) 对于任意整数 $n \geq 5$, 任意 n -边连通图上的EIST存在问题和 n -顶点连通图上的IST存在问题仍是两个具有重要的理论和实际意义的难题。

参考文献

- [1] B. Parhami. Algorithms and Architectures[M]. New York: Kluwer Academic Publishers, 2002.
- [2] 董强. 几类规则互连网络的嵌入与容错嵌入研究[D]. 重庆大学, 2010.
- [3] 陈国良. 并行计算—结构·算法·编程(第3版) [M]. 北京: 高等教育出版社, 2011.
- [4] 陈国良等. 并行计算机体系结构[M]. 北京: 高等教育出版社, 2002.
- [5] 喻昕, 吴敏, 王国军. 一种新的交叉立方体最短路径路由算法[J]. 计算机学报, 2007, 30(3): 615–621.
- [6] S.-Y. Hsieh and Y.-R. Cian. Conditional edge-fault Hamiltonicity of augmented cubes[J]. Information Sciences, 2010, 180(13): 2596–2617.
- [7] D. Wang. On embedding Hamiltonian cycles in crossed cubes[J]. IEEE Transactions on Parallel and Distributed Systems, 2008, 19(3): 334–346.
- [8] C.-J. Lai and C.-H. Tsai. Embedding a family of meshes into twisted cubes[J]. Information Processing Letters, 2008, 108(5): 326–330.
- [9] P.-L. Lai and H.-C. Hsu. Constructing the nearly shortest path in crossed cubes[J]. Information Sciences, 2009, 179(14): 2487–2493.
- [10] I. A. Stewart and Y. Xiang. Bipanconnectivity and bipancyclicity in k -ary n -cubes[J]. IEEE Transactions on Parallel and Distributed Systems, 2009, 20(1): 25–33.
- [11] Y. Xiang and I. A. Stewart. Augmented k -ary n -cubes[J]. Information Sciences, 2011, 181(1): 239–256.
- [12] Q. Dong and X. Yang. Fault-tolerant embedding of meshes/tori in twisted cubes[J]. International Journal of Computer Mathematics, 2011, 88(8): 1595–1602.

- [13] C.-H. Tsai. Embedding of meshes in Möbius cubes[J]. Theoretical Computer Science, 2008, 401(1-3): 181–190.
- [14] J. Fan and X. Jia. Embedding meshes into crossed cubes[J]. Information Sciences, 2007, 177(15): 3151–3160.
- [15] J. Fan, X. Jia, X. Liu, S. Zhang and J. Yu. Efficient unicast in bijective connection networks with the restricted faulty node set[J]. Information Sciences, 2011, 181(11): 2303–2315.
- [16] X.-B. Chen. Paired many-to-many disjoint path covers of hypercubes with faulty edges[J]. Information Processing Letters, 2012, 112(3): 61–66.
- [17] X. Wang, J. Fan, X. Jia, S. Zhang and J. Yu. Embedding meshes into twisted-cubes[J]. Information Sciences, 2011, 181(14): 3085–3099.
- [18] W.-S. Hong and S.-Y. Hsieh. Strong diagnosability and conditional diagnosability of augmented cubes under the comparison diagnosis model[J]. IEEE Transactions on Reliability, 2012, 61(1): 140–148.
- [19] P.-L. Lai. A systematic algorithm for identifying faults on hypercube-like networks under the comparison model[J]. IEEE Transactions on Reliability, 2012, 61(2): 452–459.
- [20] F. Harary, J. P. Hayes and H. J. Wu. A survey of the theory of hypercube graphs[J]. Computers & Mathematics with Applications, 1988, 15(4): 277–289.
- [21] Y.-C. Tseng and T.-H. Lai. On the embedding of a class of regular graphs in a faulty hypercube[C]. Proc. 1994 Int’l Conf. on Parallel and Distributed Systems, 1994: 488–495.
- [22] Y.-C. Tseng. Embedding a ring in a hypercube with both faulty links and faulty nodes[J]. Information Processing Letters, 1996, 59(4): 217–222.
- [23] S. L. Bezrukov. Embedding complete trees into the hypercube[J]. Discrete Applied Mathematics, 2001, 110(2-3): 101–119.

- [24] S.-Y. Hsieh. Embedding of cycles in the faulty hypercube[C]. Asia-Pacific Computer Systems Architecture Conference, 2005: 229–235.
- [25] C.-H. Tsai and S.-Y. Jiang. Path bipancyclicity of hypercubes[J]. Information Processing Letters, 2007, 101(3): 93–97.
- [26] C.-H. Tsai and Y.-C. Lai. Conditional edge-fault-tolerant edge-bipancyclicity of hypercubes[J]. Information Sciences, 2007, 177(24): 5590–5597.
- [27] X.-B. Chen. Edge-fault-tolerant diameter and bipanconnectivity of hypercubes[J]. Information Processing Letters, 2010, 110(24): 1088–1092.
- [28] M. Adler, E. Halperin, R. M. Karp and V. V. Vazirani. A stochastic process on the hypercube with applications to peer-to-peer networks[C]. STOC, 2003: 575–584.
- [29] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek and H. Balakrishnan. Chord: A scalable peer-to-peer lookup service for internet applications[C]. SIGCOMM, 2001: 149–160.
- [30] S. Rhea, B. Godfrey, B. Karp, J. Kubiatowicz, S. Ratnasamy, S. Shenker, I. Stoica and H. Yu. OpenDHT: a public DHT service and its uses[C]. SIGCOMM, 2005: 73–84.
- [31] I. Stoica, R. Morris, D. Liben-Nowell, D. R. Karger, M. F. Kaashoek, F. Dabek and H. Balakrishnan. Chord: a scalable peer-to-peer lookup protocol for Internet applications[J]. IEEE/ACM Transactions on Networking, February 2003, 11(1): 17–32.
- [32] M. Castro, P. Druschel, A.-M. Kermarrec and A. I. T. Rowstron. Scribe: A large-scale and decentralized application-level multicast infrastructure[J]. IEEE Journal on Selected Areas in Communications, 2002, 20(8): 1489–1499.
- [33] A. H. Mohajerzadeh, M. H. Yaghmaee, Z. Eskandari and H. Deldari. Energy efficient and congestion aware routing algorithms for Wireless Sensor Networks connected as Hypercube[C]. International Symposium on Telecommunications, 2008: 324–329.
- [34] H. Zhao and R. Liu. Hypercube-based key management in wireless sensor network[C]. Wireless Communications, Networking and Mobile Computing, 2009: 1–4.

- [35] Z. Yang and J. J. Garcia-Luna-Aceves. Hop-reservation multiple access (HRMA) for ad-hoc networks[C]. INFOCOM, 1999: 194–201.
- [36] S. C. Lee and L. R. Hook. Logic and computer design in nanospace[J]. IEEE Transactions on Computers, 2008, 57(7): 965–977.
- [37] J. Wu and Y. Wang. Hypercube-based multi-path social feature routing in human contact networks[J]. IEEE Transactions on Parallel and Distributed Systems, 2012, 99: 1–14.
- [38] S. Abraham and K. Padmanabhan. The twisted cube topology for multiprocessors: a study in networks asymmetry[J]. Journal of Parallel and Distributed Computing, 1991, 13(1): 104–110.
- [39] K. Efe. The crossed cube architecture for parallel computation[J]. IEEE Transactions on Parallel and Distributed Systems, 1992, 3(5): 513–524.
- [40] P. Cull and S. M. Larson. The Möbius cubes[J]. IEEE Transactions on Computers, 1995, 44(5): 647–659.
- [41] X. Yang, D. J. Evans and G. M. Megson. The locally twisted cubes[J]. International Journal of Computer Mathematics, 2005, 82(4): 401–413.
- [42] A. S. Vaidya, P. S. N. Rao and S. R. Shankar. A class of hypercube-like networks[C]. Proc. 5th IEEE Symp. Parallel and Distributed Processing, 1993: 800–803.
- [43] 樊建席, 何力勤. BC互连网络及其性质[J]. 计算机学报, 2003, 26(1): 84–90.
- [44] X. Tan, S.-Z. Yu and J. H. Park. A note about some properties of BC graphs[J]. Information Processing Letters, 2008, 108(6): 398–401.
- [45] J. Fan, X. Jia, B. Cheng and J. Yu. An efficient fault-tolerant routing algorithm in bijective connection networks with restricted faulty edges[J]. Theoretical Computer Science, 2011, 412(29): 3440–3450.
- [46] W. Zhou, J. Fan, X. Jia and S. Zhang. The spined cube: A new hypercube variant with smaller diameter[J]. Information Processing Letters, 2011, 111(12): 561–567.

- [47] T.-K. Li, C.-H. Tsai and H.-C. Hsu. A fast fault-identification algorithm for bijective connection graphs using the PMC model[J]. Information Sciences, 2012, 187: 291–297.
- [48] Q. Zhu. On conditional diagnosability and reliability of the BC networks[J]. The Journal of Supercomputing, 2008, 45(2): 173–184.
- [49] Q. Zhu, X.-K. Wang and G. Cheng. Reliability evaluation of BC networks[J]. IEEE Transactions on Computers, 2013, 62(11): 2337–2340.
- [50] S.-Y. Hsieh and P.-Y. Yu. Cycle embedding on twisted cubes[C]. Proc. 7th Int’l Conf. on Parallel and Distributed Computing, Applications and Technologies, 2006: 102–104.
- [51] J. Fan, X. Lin and X. Jia. Optimal path embedding in crossed cubes[J]. IEEE Transactions on Parallel and Distributed Systems, 2005, 16(12): 1190–1200.
- [52] P. Kulasinghe and S. Bettayeb. Embedding binary trees into crossed cubes[J]. IEEE Transactions on Computers, 1995, 44(7): 923–929.
- [53] V. Auletta, A. A. Rescigno and V. Scarano. Embedding graphs onto the supercube[J]. IEEE Transactions on Computers, 1995, 44(4): 593–597.
- [54] X. Yang, G. M. Megson and D. J. Evans. Locally twisted cubes are 4-pancyclic[J]. Applied Mathematics Letters, 2004, 17(8): 919–925.
- [55] J. Fan, X. Jia and X. Lin. Optimal embeddings of paths with various lengths in twisted cubes[J]. IEEE Transactions on Parallel and Distributed Systems, 2007, 18(4): 511–521.
- [56] J. Fan and X. Jia. Edge-pancyclicity and path-embeddability of bijective connection graphs[J]. Information Sciences, 2008, 178(2): 340–351.
- [57] J.-M. Chang and J.-S. Yang. Fault-tolerant cycle-embedding in alternating group graphs[J]. Applied Mathematics and Computation, 2008, 197(2): 760–767.

- [58] J.-S. Yang, J.-M. Chang, S.-M. Tang and Y.-L. Wang. On the independent spanning trees of recursive circulant graphs $G(cd^m, d)$ with $d > 2$ [J]. Theoretical Computer Science, 2009, 410(21-23): 2001–2010.
- [59] Q. Dong, X. Yang and D. Wang. Embedding paths and cycles in 3-ary n -cubes with faulty nodes and links[J]. Information Sciences, 2010, 180(1): 198–208.
- [60] J.-S. Yang and J.-M. Chang. Independent spanning trees on folded hyper-stars[J]. Networks, 2010, 56(4): 272–281.
- [61] S.-M. Tang, J.-S. Yang, Y.-L. Wang and J.-M. Chang. Independent spanning trees on multidimensional torus networks[J]. IEEE Transactions on Computers, 2010, 59(1): 93–102.
- [62] H.-C. Chen, T.-L. Kung and L.-H. Hsu. Embedding a hamiltonian cycle in the crossed cube with two required vertices in the fixed positions[J]. Applied Mathematics and Computation, 2011, 217(24): 10058–10065.
- [63] T.-K. Li, C.-J. Lai and C.-H. Tsai. A novel algorithm to embed a multi-dimensional torus into a locally twisted cube[J]. Theoretical Computer Science, 2011, 412(22): 2418–2424.
- [64] X.-B. Chen. Parallel construction of optimal independent spanning trees on Cartesian product of complete graphs[J]. Information Processing Letters, 2011, 111(5): 235–238.
- [65] J.-S. Kim, H.-O. Lee, E. Cheng and L. Lipták. Optimal independent spanning trees on odd graphs[J]. The Journal of Supercomputing, 2011, 56(2): 212–225.
- [66] J.-S. Kim, H.-O. Lee, E. Cheng and L. Lipták. Independent spanning trees on even networks[J]. Information Sciences, 2011, 181(13): 2892–2905.
- [67] X. Yang, L. Wang and L. Yang. Optimal broadcasting for locally twisted cubes[J]. Information Processing Letters, 2012, 112(4): 129–134.
- [68] Q. Dong, J. Zhou, Y. Fu and X. Yang. Embedding a mesh of trees in the crossed cube[J]. Information Processing Letters, 2012, 112(14-15): 599–603.

- [69] F. Bao, Y. Igarashi and S. R. Öhring. Reliable broadcasting in product networks[J]. Discrete Applied Mathematics, 1998, 83(1-3): 3–20.
- [70] F. Bao, Y. Funyu, Y. Hamada and Y. Igarashi. Reliable broadcasting and secure distributing in channel networks[J]. IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences, 1998, 81(5): 796–806.
- [71] B. Cheng, J. Fan, X. Jia and J. Jia. Parallel construction of independent spanning trees and an application in diagnosis on Möbius cubes[J]. The Journal of Supercomputing, 2013, 65(3): 1279–1301.
- [72] A. Itai and M. Rodeh. The multi-tree approach to reliability in distributed networks[J]. Information and Computation, 1988, 79(1): 43–59.
- [73] A. Zehavi and A. Itai. Three tree-paths[J]. Journal of Graph Theory, 1989, 13(2): 175–188.
- [74] S. Khuller and B. Schieber. On independent spanning trees[J]. Information Processing Letters, 1992, 42(6): 321–323.
- [75] A. Gopalan and S. Ramasubramanian. A counterexample for the proof of implication conjecture on independent spanning trees[J]. Information Processing Letters, 2013, 113(14–16): 522–526.
- [76] J. Cheriyan and S. N. Maheshwari. Finding nonseparating induced cycles and independent spanning trees in 3-connected graphs[J]. Journal of Algorithms, 1988, 9(4): 507–537.
- [77] S. Curran, O. Lee and X. Yu. Finding four independent trees[J]. SIAM Journal on Computing, 2006, 35(5): 1023–1058.
- [78] A. Gopalan and S. Ramasubramanian. On constructing three edge independent spanning trees[J]. Technical Report. [Online]Available: <http://srini.ca/p/3trees.pdf>, 2011: 1–22.
- [79] A. Huck. Independent trees in graphs[J]. Graphs and Combinatorics, 1994, 10(1): 29–45.

- [80] A. Huck. Independent trees in planar graphs[J]. Graphs and Combinatorics, 1999, 15(1): 29–77.
- [81] K. Miura, D. Takahashi, S.-I. Nakano and T. Nishizeki. A linear-time algorithm to find four independent spanning trees in four connected planar graphs[J]. International Journal of Foundations of Computer Science, 1999, 10(2): 195–210.
- [82] S. Nagai and S. Nakano. A linear-time algorithm to find independent spanning trees in maximal planar graphs[J]. IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences, 2001, 84(5): 1102–1109.
- [83] K. Obokata, Y. Iwasaki, F. Bao and Y. Igarashi. Independent spanning trees of product graphs and their construction[J]. IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences, 1996, 79(11): 1894–1903.
- [84] J.-S. Yang and J.-M. Chang. Optimal independent spanning trees on Cartesian product of hybrid graphs[J]. The Computer Journal, 2014, 57(1): 93–99.
- [85] Y. Iwasaki, Y. Kajiwara, K. Obokata and Y. Igarashi. Independent spanning trees of chordal rings[J]. Information Processing Letters, 1999, 69(3): 155–160.
- [86] J.-S. Yang, J.-M. Chang, S.-M. Tang and Y.-L. Wang. Reducing the height of independent spanning trees in chordal rings[J]. IEEE Transactions on Parallel and Distributed Systems, 2007, 18(5): 644–657.
- [87] Z. Ge and S. L. Hakimi. Disjoint rooted spanning trees with small depths in de Bruijn and Kautz graphs[J]. SIAM Journal on Computing, 1997, 26(1): 79–92.
- [88] S.-M. Tang, Y.-L. Wang and Y.-H. Leu. Optimal independent spanning trees on hypercubes[J]. Journal of Information Science and Engineering, 2004, 20(1): 143–156.
- [89] S. Lakshmivarahan and S. K. Dhall. Ring, torus and hypercube architectures/algorithms for parallel computing[J]. Parallel Computing, 1999, 25(13): 1877–1906.
- [90] J.-S. Yang, S.-M. Tang, J.-M. Chang and Y.-L. Wang. Parallel construction of optimal independent spanning trees on hypercubes[J]. Parallel Computing, 2007, 33(1): 73–79.

- [91] J. Werapun, S. Intakosum and V. Boonjing. An efficient parallel construction of optimal independent spanning trees on hypercubes[J]. *Journal of Parallel and Distributed Computing*, 2012, 72(12): 1713–1724.
- [92] S.-Y. Hsieh and C.-J. Tu. Constructing edge-disjoint spanning trees in locally twisted cubes[J]. *Theoretical Computer Science*, 2009, 410(8–10): 926–932.
- [93] J.-C. Lin, J.-S. Yang, C.-C. Hsu and J.-M. Chang. Independent spanning trees vs. edge-disjoint spanning trees in locally twisted cubes[J]. *Information Processing Letters*, 2010, 110(10): 414–419.
- [94] Y. Liu, J. K. Lan, W. Y. Chou and C. Chen. Constructing independent spanning trees for locally twisted cubes[J]. *Theoretical Computer Science*, 2011, 412: 2237–2252.
- [95] M.-C. Yang. Constructing edge-disjoint spanning trees in twisted cubes[J]. *Information Sciences*, 2010, 180(20): 4075–4083.
- [96] Y. Wang, J. Fan, G. Zhou and X. Jia. Independent spanning trees on twisted cubes[J]. *Journal of Parallel and Distributed Computing*, 2012, 72(1): 58–69.
- [97] Y. Wang, J. Fan, X. Jia and H. Huang. An algorithm to construct independent spanning trees on parity cubes[J]. *Theoretical Computer Science*, 2012, 465: 61–72.
- [98] 徐俊明. 图论及其应用[M]. 合肥: 中国科学技术大学出版社, 2010.
- [99] F. Harary. *Graph Theory*[M]. Massachusetts: Addison-Wesley, 1969.
- [100] R. Diestel. *Graph Theory*[M]. New York: Springer-Verlag, 2005.
- [101] Y. Saad and M. H. Schultz. Topological properties of hypercube[J]. *IEEE Transactions on Computers*, 1988, 37(7): 867–872.
- [102] P. Cull and S. M. Larson. On generalized twisted cubes[J]. *Information Processing Letters*, 1995, 55(1): 53–55.
- [103] K. Efe. A variation on the hypercube with lower diameter[J]. *IEEE Transactions on Computers*, 1991, 40(11): 1312–1316.

- [104] P. A. J. Hilbers, M. R. J. Koopman and J. L. A. Van de Snepscheut. The twisted cube[C]. Proc. Volume I: Parallel architectures on PARLE: Parallel Architectures and Languages Europe, 1987: 152–159.
- [105] J. Fan, X. Lin, X. Jia and R. W. H. Lau. Edge-pancyclicity of twisted cubes[J]. Lecture Notes in Computer Science(ISAAC 2005), 2005, LNCS 3827: 1090–1099.
- [106] D. Wang and L. Zhao. The twisted-cube connected networks[J]. Journal of Computer Science and Technology, 1999, 14(2): 181–187.
- [107] C.-D. Park and K.-Y. Chwa. Hamiltonian properties on the class of hypercube-like networks[J]. Information Processing Letters, 2004, 91(1): 11–17.
- [108] S.-Y. Hsieh, C.-Y. Wu and C.-W. Lee. Fault-free Hamiltonian cycles in locally twisted cubes under conditional edge faults[C]. Proc. 13th Int'l Conf. on Parallel and Distributed Systems, 2007: 1–8.
- [109] J.-H. Park, H.-C. Kim and H.-S. Lim. Fault-Hamiltonicity of hypercube-like interconnection networks[C]. IPDPS, 2005: 1–10.
- [110] J.-M. Xu, M. Ma and M. Lü. Paths in Möbius cubes and crossed cubes[J]. Information Processing Letters, 2006, 97(3): 94–97.
- [111] J. Fan, X. Jia and X. Lin. Complete path embeddings in crossed cubes[J]. Information Sciences, 2006, 176(22): 3332–3346.
- [112] J. Fan, X. Lin, Y. Pan and X. Jia. Optimal fault-tolerant embedding of paths in twisted cubes[J]. Journal of Parallel and Distributed Computing, 2007, 67(2): 205–214.
- [113] M. Ma and J.-M. Xu. Panconnectivity of locally twisted cubes[J]. Applied Mathematics Letters, 2006, 19(7): 673–677.
- [114] K. S. Hu, S.-S. Yeoh, C. Chen and L.-H. Hsu. Node-pancyclicity and edge-pancyclicity of hypercube variants[J]. Information Processing Letters, 2007, 102(1): 1–7.

- [115] E. Abuelrub and S. Bettayeb. Embedding of complete binary trees in twisted hypercubes[C]. Proc. Int'l Conf. Computer Applications in Design, Simulation, and Analysis, 1993: 1–4.
- [116] A. Y. Wu. Embedding of tree networks into hypercubes[J]. Journal of Parallel and Distributed Computing, 1985, 2(3): 238–249.
- [117] Y. Han, J. Fan, L. You and Y. Wang. Embedding complete binary trees into locally twisted cubes[C]. Advanced Engineering Forum, 2012: 70–75.
- [118] C.-P. Chang, T.-Y. Sung and L.-H. Hsu. Edge congestion and topological properties of crossed cubes[J]. IEEE Transactions on Parallel and Distributed Systems, 2000, 11(1): 64–80.
- [119] J.-M. Xu and Z.-G. Deng. Wide diameter of Möbius cubes[J]. Journal of Interconnection Networks, 2005, 6(1): 51–62.
- [120] W. Zhou, J. Fan, B. Cheng, Y. Han and Y. Wang. Broadcast in bijective connection networks with the restricted faulty node set[C]. Proc. 5th Int'l Conf. on Computer Science and Education, 2010: 230–235.
- [121] C.-P. Chang and C.-C. Wu. Conditional fault diameter of crossed cubes[J]. Journal of Parallel and Distributed Computing, 2009, 69(1): 91–99.
- [122] D. Wang. Hamiltonian embedding in crossed cubes with failed links[J]. IEEE Transactions on Parallel and Distributed Systems, 2012, 23(11): 2117–2124.
- [123] D. Wang. Constructing optimal subnetworks for the crossed cube network[J]. Networks, 2012, 60(2): 86–93.
- [124] X. Yang, Q. Dong and Y. Y. Tang. Embedding meshes/tori in faulty crossed cubes[J]. Information Processing Letters, 2010, 110(14): 559–564.
- [125] C.-J. Lai, C.-H. Tsai, H.-C. Hsu and T.-K. Li. A dynamic programming algorithm for simulation of a multi-dimensional torus in a crossed cube[J]. Information Sciences, 2010, 180(24): 5090–5100.

- [126] P. Kulasinghe and S. Bettayeb. Multiply-twisted hypercube with five or more dimensions is not vertex-transitive[J]. Information Processing Letters, 1995, 53(1): 33–36.
- [127] J. Huang and J. Xu. Multiply-twisted hypercube with four or less dimensions is vertex-transitive[J]. Chinese Quarterly Journal of Mathematics, 2005, 20(4): 430–434.
- [128] P. Kulasinghe and S. Bettayeb. Symmetry of the multiply-twisted hypercube[J]. Dept. of Computer Science TR, 1994: 94–004.
- [129] F. P. Preparata, G. Metze and R. T. Chien. On the connection assignment problem of diagnosable systems[J]. IEEE Transactions on Electronic Computers, 1967, 16(6): 848–854.
- [130] G.-Y. Chang. Conditional (t, k) -diagnosis under the PMC model[J]. IEEE Transactions on Parallel and Distributed Systems, 2011, 22(11): 1797–1803.
- [131] N.-W. Chang and S.-Y. Hsieh. Conditional diagnosability of augmented cubes under the PMC model[J]. IEEE Transactions on Dependable and Secure Computing, 2012, 9(1): 46–60.
- [132] N.-W. Chang, T.-Y. Lin and S.-Y. Hsieh. Conditional diagnosability of k -ary n -cubes under the PMC model[J]. ACM Transactions on Design Automation of Electronic Systems, 2012, 17(4): 46.
- [133] C.-L. Kuo, M.-J. Yang, Y.-M. Chang and Y.-M. Yeh. High diagnosability of a sequential diagnosis algorithm in hypercubes under the PMC model[J]. The Journal of Supercomputing, 2012, 61(3): 1116–1134.
- [134] M.-C. Yang. Conditional diagnosability of balanced hypercubes under the PMC model[J]. Information Sciences, 2013, 222: 754–760.
- [135] A. T. Dahbura and G. M. Masson. An $O(N^{2.5})$ fault identification algorithm for diagnosable systems[J]. IEEE Transactions on Computers, 1984, 33(6): 486–492.
- [136] S. L. Hakimi and A. T. Amin. Characterization of connection assignment of diagnosable systems[J]. IEEE Transactions on Computers, 1974, 100(1): 86–88.

- [137] J. Fan. Diagnosability of the Möbius cubes[J]. IEEE Transactions on Parallel and Distributed Systems, 1998, 9(9): 923–928.
- [138] H. Sullivan and T. R. Bashkow. A large scale, homogeneous, fully distributed parallel machine, I[C]. ACM SIGARCH computer architecture news, 1977: 105–117.
- [139] V. M. Lo, S. V. Rajopadhye, S. Gupta, D. Keldsen, M. A. Mohamed and J. A. Telle. Mapping divide-and-conquer algorithms to parallel architectures[C]. Proc. 1990 Int'l Conf. on Parallel Processing, 1990: 128–135.
- [140] W.-J. Hsu, C. V. Page and J.-S. Liu. Computing prefixes on a large family of interconnection topologies[C]. Proc. 1992 Int'l Conf. on Parallel Processing, 1992: 153–159.
- [141] B. Cheng, J. Fan, X. Jia and J. Wang. Dimension-adjacent trees and parallel construction of independent spanning trees on crossed cubes[J]. Journal of Parallel and Distributed Computing, 2013, 73(5): 641–652.

攻读博士学位期间取得的科研成果及参与的学术交流情况

作者在攻读博士学位期间发表的论文

[1] **Baolei Cheng**, Jianxi Fan and Xiaohua Jia. Dimensional-permutation-based independent spanning trees in bijective connection networks[J]. IEEE Transactions on Parallel and Distributed Systems, doi: 10.1109/TPDS.2014.2307871. (SCI源期刊, 中国计算机学会推荐A类期刊)

[2] **Baolei Cheng**, Jianxi Fan, Xiaohua Jia and Shukui Zhang. Independent spanning trees in crossed cubes[J]. Information Sciences, 2013, 233(1): 276–289. (SCI二区源期刊, 中国计算机学会推荐B类期刊)

[3] **Baolei Cheng**, Jianxi Fan, Xiaohua Jia and Jin Wang. Dimension-adjacent trees and parallel construction of independent spanning trees on crossed cubes[J]. Journal of Parallel and Distributed Computing, 2013, 73(5): 641–652. (SCI源期刊, 中国计算机学会推荐B类期刊)

[4] **Baolei Cheng**, Jianxi Fan, Xiaohua Jia, Shukui Zhang and Bangrui Chen. Constructive algorithm of independent spanning trees on Möbius cubes[J]. The Computer Journal, 2013, 56 (11): 1347-1362. (SCI源期刊, 中国计算机学会推荐B类期刊)

[5] **Baolei Cheng**, Jianxi Fan, Xiaohua Jia and Juncheng Jia. Parallel construction of independent spanning trees and an application in diagnosis on Möbius cubes[J]. The Journal of Supercomputing, 2013, 65(3): 1279–1301. (SCI源期刊, 中国计算机学会推荐C类期刊)

[6] **Baolei Cheng**, Jianxi Fan, Jiwen Yang and Xi Wang. Circular dimensional-permutations and reliable broadcasting for hypercubes and Möbius cubes[C]. 10th IFIP International Conference on Network and Parallel Computing (NPC 2013), Lecture Notes in Computer Science, 8147: 232-244, 2013. (EI收录, 中国计算机学会推荐C类会议)

[7] **Baolei Cheng**, Jianxi Fan, Shukui Zhang, Yan Wang, Xi Wang and Wenjun Liu. Towards the Independent spanning trees in conditional BC networks[C]. 2013 International

Conference on Advanced Computer Science and Electronics Information(ICACSEI 2013), 25-26 July, pp. 79-82, Beijing, China, 2013. (EI收录)

[8] **Baolei Cheng**, Jianxi Fan, Jiwen Yang and Yuejuan Han. An algorithm to construct independent spanning trees on crossed cubes[C]. Proc. 2nd International Conference on Information Science and Engineering (ICISE2010), 4-6 Dec. 2010, pp 961-964, Hangzhou, China, 2010. (EI收录)

[9] Jianxi Fan, Xiaohua Jia, **Baolei Cheng** and Jia Yu. An efficient fault-tolerant routing algorithm in bijective connection networks with restricted faulty edges[J]. Theoretical Computer Science, 2011, 412(29): 3440–3450. (SCI源期刊, 中国计算机学会推荐B类期刊)

[10] Jianxi Fan, Kenli Li, Shukui Zhang, Wujun Zhou, and **Baolei Cheng**. One-to-one communication in twisted cubes under restricted connectivity[J]. Frontiers of Computer Science in China, 2010, 4(4): 489–499. (SCIE源期刊)

[11] Jin Wang, Kejie Lu, Shukui Zhang, Jianxi Fan, Yanqin Zhu and **Baolei Cheng**. An efficient communication relay placement algorithm for content-centric wireless mesh networks[J]. International Journal of Communication Systems, <http://dx.doi.org/10.1002/dac.2666>. (SCIE源期刊)

[12] Haotian Xu, Jianxi Fan, **Baolei Cheng** and Chen Shen. Independent spanning trees on LHL cubes[J/OL]. Transactions of Nanjing University of Aeronautics & Astronautics, <http://www.cnki.net/kcms/detail/32.1389.V.20140515.2230.001.html>.

[13] 程宝雷, 樊建席. BC网络上相邻顶点间独立路径的构造算法[OL]. [2011-12-09]. 中国科技论文在线, <http://www.paper.edu.cn/releasepaper/content/201112-267>.

[14] 程宝雷, 樊建席. 关于莫比乌斯立方体上独立生成树的构造[OL]. [2011-04-08]. 中国科技论文在线, <http://www.paper.edu.cn/releasepaper/content/201104-205>.

[15] Xi Wang, Jianxi Fan, **Baolei Cheng**, Wenjun Liu and Yan Wang. One-to-one disjoint path covers in Dcell [C]. 10th IFIP International Conference on Network and Parallel Computing (NPC 2013), Lecture Notes in Computer Science, 8147: 61-70, 2013. (EI收录, 中国计算机学会推荐C类会议)

[16] Yuejuan Han, Jianxi Fan, Lantao You, **Baolei Cheng** and Yan Wang, Adaptive diagnosis in the locally twisted cube[C]. International Symposium on Parallel Architec-

tures, Algorithms and Programming (PAAP'12), pp 106-110, Taipei, December 17-20, 2012.
(EI收录)

作者在攻读博士学位期间撰写的软件著作权

- [1] 程宝雷, 樊建席. 超立方体网络性质演示软件V1.0. 登记号: 2011SR075580.
- [2] 樊建席, 程宝雷. 莫比乌斯立方体网络生成树构建系统V1.0. 登记号: 2012SR122770.
- [3] 程宝雷, 樊建席. BC互连网络性质演示系统V1.0. 登记号: 2013SR100055.
- [4] 程宝雷, 樊建席. Crossed Cube结构城市公交网络性质演示系统V1.0. 登记号: 2013SR099766.
- [5] 朱丽华, 程宝雷, 何云成. Android手机模拟小键盘系统V1.0. 登记号: 2013SR109289.

作者在攻读博士学位期间主持和参与的科研项目

- [1] 主持江苏省普通高校研究生科研创新计划项目“交叉立方体网络上独立生成树构造问题的研究”, 项目编号: CXZZ11_0100. 验收结果为优秀。
- [2] 参与江苏省信息处理重点实验室开放课题“局部扭立方体上若干性质的研究”, 项目编号: KJS1129.
- [3] 参与国家自然科学基金项目“条件BC网络上独立生成树及其性质的研究”, 项目编号: 61170021.
- [4] 参与博士点基金项目“无线传感器网络中基于多路径数据传送的容错检测研究”, 项目编号: 20103201110018.
- [5] 参与苏州市工业基础研究计划项目“WSN容错数据收集协议研究及其在智能交通中的应用”, 项目编号: SYG201240.

作者在攻读博士学位期间的学术交流情况

- [1] 2010年7月2日至4日在北京参加中国计算机学会主办的“多核技术”学科前沿讲习班, 取得结业证书。

[2] 2011 年10月25日至27日在山东济南参加中国计算机学会主办的“高性能计算算法与应用”学科前沿讲习班，取得结业证书。

[3] 2012年12月17-20在台湾科技大学参加第五届国际并行体系结构、算法与程序设计会议（PAAP）。

[4] 2013年9月19-21，在贵州贵阳会议NPC 2013（The 10th IFIP International Conference on Network and Parallel Computing）上交流学术论文。

致 谢

弹指一挥间，四年的博士生生活即将结束。在此，怀着一颗感恩的心，我要向所有给予我指导、关心、支持和帮助的老师、领导和同学们表示我最真挚的谢意。

衷心感谢我的导师樊建席教授四年来对我学习和工作的悉心关怀和指导。在进入互连网络的研究领域时，导师从未在意我知识的缺乏，总是耐心地剖析，纠正我对于所研究问题的错误理解，传授正确的学习方法。我还清楚地记得您对我的多篇小论文无数次的修改，圈出来密密麻麻的问题，让我清晰地认识到自身的不足。从您那里，我不仅学到了扎实的专业知识，进行科学研究的方法，更学到了做人的道理。您严谨治学的态度、渊博的知识、无私的奉献精神、高尚的学术道德使我受益终生。感谢您在我整个博士的学习研究期间对我的耐心指导和帮助，每当我们在理论研究和证明上遇到不能解决的问题时，您总能给予有力的帮助。每当我们进行每周工作汇报时，您总能看出研究工作中存在的问题，提出有益的建议。在此，我要衷心地道一声：谢谢您！

衷心感谢我的老师和领导杨季文教授对我无微不至的关怀以及对我学习与工作的肯定。您对工作的情感和严谨的工作作风一直深深地影响着我。您对我的谆谆教导无不透露着对我的殷切期望。您给我指明了研究方向，关心着我成长的每一步，对如何进行科学研究提出了很多有益的建议。而我，也努力按照您的期望，在学习、工作和生活上的担任好我的每一个角色。

衷心感谢陈国良院士对我学习和工作的悉心关怀和指导。您给我们做过多次科学研究的精彩报告，对如何进行科学研究提出了很多有益的建议。每次在给您汇报完工作后，您都能指出我们研究工作中存在的问题，并提出改进建议。而我们，也努力按照您的期望，争取下一次汇报时能有所进步。

衷心感谢美国蒙特克莱尔州立大学的Dajin Wang教授，您为我们提供了许多有价值的资料和研究建议，您在论文的写作上给我们给予了细致的指导。您利用几个暑假时间来对我们进行言传身教，使我们获益良多。

衷心感谢纵横汉字信息技术研究所的吕强教授、赵雷老师、李培峰老师在学习、生活和工作方面对我的关心和帮助，你们知识渊博、思路灵活、工作勤奋，一直是我学习的好榜样。

衷心感谢计算机科学与技术学院为我们搭建好一个科研学习的平台，感谢所有的领导和同事对我工作和学习的支持和帮助。

感谢课题组的林政宽老师、韩月娟老师、王岩老师、刘钊老师，师弟刘文军、王喜、李勇、周吴军、闫少华、王成、李硕、王仁喜、胥大成、刘源潮、陈浩、沈海飞、王永伟、张标、马良，师妹鲁登月、储燕和张丽丽，在互连网络领域和图论知识方面，我们一直进行理论上的深入讨论，一起学习、共同进步。

程宝雷

二〇一四年三月一日

苏州大学 博士学位论文

苏州大学研究生部统一印制